Reinforcement Learning

From Bandits to RLHF

First Edition

Based on lectures by K. K. Pchelin
Moscow, Russia

This book was typeset using LaTeX software.

Copyright © 2026 Based on lectures by K. K. Pchelin
License: Creative Commons Zero 1.0 Universal

Preface

This book grew out of a conviction that the two great traditions of reinforcement learning — the classical mathematical theory and the modern practice of aligning large language models — deserve to be told as a single, coherent story. In recent years, ideas that spent decades confined to the pages of control theory and dynamic programming have burst into the centre of AI development, powering the training of systems that write, reason, and converse. That journey is the subject of the present volume.

A note on inspiration and originality. Every textbook builds on earlier work, and this one is no exception. We wish to acknowledge two books that have shaped the broader field and, inevitably, our own perspective—while clarifying what distinguishes the present volume.

The first is *Reinforcement Learning: An Introduction* by Richard S. Sutton and Andrew G. Barto, the standard reference for classical RL. Sutton and Barto established the pedagogical progression—from bandits through MDPs to temporal-difference methods—that has guided virtually every subsequent course. Our treatment in Parts I and II follows the same logical arc, but departs in emphasis and presentation: we include tighter convergence analyses informed by recent results, introduce Generalised Advantage Estimation and PPO within the core narrative rather than as afterthoughts, and motivate every classical idea with an eye toward its eventual role in language-model training. These choices reflect lessons learned from designing and delivering university-level RL courses as well as from applying these algorithms in production settings at scale.

The second is *The RLHF Book* by Nathan Lambert, the most thorough practical guide to reinforcement learning from human feedback for language models yet written. Lambert demonstrated that the algorithmic ideas behind ChatGPT, Claude, and their successors could be explained precisely and accessibly. Parts III and IV of our book cover overlapping territory—reward modelling, preference optimisation, PPO for LLMs—but extend the discussion in directions shaped by our own research and engineering experience: we provide a unified formal treatment connecting DPO, IPO, and KTO through the KL-constrained RL objective, and cover ORPO as a distinct reference-model-free approach; dedicate full chapters to GRPO and reinforcement learning with verifiable rewards (RLVR), topics that postdate Lambert's initial treatment; and include a substantial chapter on multi-step agentic systems that bridges RL theory with the emerging practice of tool-using language agents.

More broadly, this book is not a synthesis of the two works above but an attempt to tell a *new* story: one continuous narrative that begins with Bellman's principle of optimality and ends with the training pipelines behind modern reasoning models. That narrative is informed by hands-on experience with large-scale RLHF training, reward-model debugging, and the practical pitfalls that no purely theoretical account can capture. Where we succeed in making this bridge feel natural, the credit is shared with the community whose open research made it possible. Where we fall short, the responsibility is entirely our own.

What this book is trying to do. The central goal of this volume is to build a single, unbroken bridge between the mathematical foundations laid by Bellman, Sutton, and Barto and the practical methods that now govern how large language models are trained and aligned. The path from Bellman's principle of optimality to a working RLHF pipeline is shorter than it might appear, but it requires genuine fluency in both worlds. A practitioner who understands only the engineering recipes misses the theoretical guarantees and the diagnostic intuition that theory provides. A theorist who has not engaged with the realities of language-model training — the instability of reward models, the sensitivity of PPO hyperparameters, the challenge of specifying human preferences — is ill-equipped to contribute to the most important application of RL today.

Parts I and II of this book (covering bandits, MDPs, dynamic programming, Monte Carlo methods, temporal-difference learning, function approximation, policy gradients, and actor-critic methods) are inspired by Sutton and Barto's rigorous treatment and can be read entirely independently as a self-contained classical RL textbook. Parts III and IV build on that foundation to cover reward modelling, RLHF, Direct Preference Optimisation, Group Relative Policy Optimisation, multi-step agentic systems, and open research frontiers, drawing substantially on Lambert's practical insights.

Who this book is for. We have written with three overlapping audiences in mind.

Graduate students in machine learning and AI who want a mathematically honest account of reinforcement learning that does not stop at the classical results but carries them all the way to the frontier of language-model alignment.

ML engineers and practitioners who have encountered RLHF in the context of language models and want to understand the theoretical machinery underneath the implementation, so that they can debug, extend, and improve it with confidence.

Researchers transitioning from classical supervised or unsupervised learning to reinforcement learning, including those whose primary interest is alignment, safety, or evaluation, who need a solid grounding in the underlying concepts before engaging with the research literature.

The book is not aimed at complete beginners to machine learning; some prior exposure to the ideas of the field is assumed. It is, however, self-contained with respect to reinforcement learning itself: no prior knowledge of RL is required.

Prerequisites. The reader is assumed to be comfortable with basic probability theory (random variables, expectations, conditional probability, Bayes' rule), linear algebra (vectors, matrices, eigenvalues), and single- and multi-variable calculus (derivatives, gradients, the chain rule). Familiarity with the core concepts of supervised machine learning — loss functions, gradient descent, neural networks, overfitting — will make Parts III and IV significantly more accessible, though we recall the key ideas where they are needed. Python fluency is useful for the implementation-oriented material in the appendices but is not necessary to follow the theoretical development.

How to read this book. The four parts of the book form a natural progression, but readers with different backgrounds may wish to enter at different points.

Part I (Foundations) develops the classical theory from scratch: multi-armed bandits, Markov decision processes, dynamic programming, Monte Carlo methods, and temporal-difference learning. A reader with no prior exposure to RL should work through these chapters in order.

Part II (Function Approximation and Deep RL) moves to settings where tabular methods are no longer feasible. It covers linear and nonlinear value-function approximation, policy gradient methods, and the actor-critic family, culminating in Proximal Policy Optimisation.

Together with Part I, this material constitutes a complete introduction to classical and deep RL that stands on its own.

Part III (RL for Language Models, Chapters 10–14) applies everything developed in Parts I and II to the modern challenge of training and aligning large language models. Readers who are already fluent in classical RL may proceed directly here. These chapters cover reward shaping and reward models (Chapter 10), the language-model training pipeline from pre-training through SFT (Chapter 11), Direct Preference Optimisation and its variants (Chapter 12), Group Relative Policy Optimisation and online RL with verifiable rewards (Chapter 13), and the practical engineering of RLHF at scale (Chapter 14).

Part IV (Frontiers, Chapters 15–17) explores the cutting edge of the field: reasoning, tool use, and reinforcement finetuning (Chapter 15); multi-agent systems (Chapter 16); and a survey of open problems and emerging directions (Chapter 17).

Each chapter closes with a set of exercises that range from straightforward verifications to more involved proofs and design questions. We recommend attempting at least a few exercises per chapter: reinforcement learning is, appropriately, a subject best understood by doing.

Companion code and notebooks. This book is accompanied by a public GitHub repository containing practical Jupyter notebooks, one for each chapter. These notebooks provide runnable implementations of every major algorithm discussed in the text — from ε-greedy bandits and Q-learning to DQN, PPO, DPO, and GRPO — together with visualisations and guided experiments. All notebooks are designed to run on free-tier platforms such as Google Colab or Kaggle, requiring no specialised hardware. The repository also includes detailed exercise solutions with full proofs and derivations, as well as a framework for community translations into other languages.

The companion repository is available at:

https://github.com/pyshka501/rl-textbook

We encourage readers to work through the notebooks alongside the relevant chapters. The combination of mathematical rigour in the text and hands-on experimentation in the code is, in our experience, the most effective way to develop genuine fluency in reinforcement learning.

We hope this book serves you well, wherever on the path from classical theory to modern alignment practice you happen to be.

Moscow, 2026

Contents

Preface iv

Notation xix

I Foundations 1

1 Introduction to Reinforcement Learning 2
1.1 What Is Reinforcement Learning? . . . 2
1.2 Key Challenges in Reinforcement Learning . . . 3
1.3 A Brief History of Reinforcement Learning . . . 3
1.4 Reinforcement Learning and Other Machine Learning Paradigms . . . 4
1.5 Applications of Reinforcement Learning . . . 5
1.6 The Agent–Environment Interface . . . 5
1.7 Overview of the Book . . . 6
Exercises . . . 6

2 Multi-Armed Bandits 8
2.1 Problem Formulation . . . 8
2.2 Action Values, Optimal Actions, and Regret . . . 9
2.3 Greedy and ε-Greedy Strategies . . . 10
2.4 Sample Averages and Incremental Updates . . . 10
2.5 Upper Confidence Bounds (UCB) . . . 11
2.6 Thompson Sampling . . . 14
2.7 The Lai–Robbins Lower Bound . . . 17
2.8 Contextual Bandits . . . 18
2.9 Empirical Comparison . . . 20
2.10 Summary . . . 22
Exercises . . . 23

3 MDPs and the Bellman Equation 25
3.1 The Markov Property . . . 25
3.2 Formal Definition of an MDP . . . 27
3.3 Episodes, Returns, and the Discount Factor . . . 28
3.4 Policies . . . 29
3.5 Value Functions . . . 30
3.6 The Bellman Equation . . . 31
3.7 Optimal Value Functions and the Bellman Optimality Equation . . . 33

3.8 The Bellman Operator . . . 34
3.9 Worked Example: The Student MDP . . . 35
3.10 Summary . . . 37
Exercises . . . 37

4 Dynamic Programming 39
4.1 Policy Evaluation (Prediction) . . . 39
4.2 Policy Improvement . . . 42
4.3 Policy Iteration . . . 43
4.4 Value Iteration . . . 44
4.5 Generalized Policy Iteration . . . 46
4.6 Asynchronous Dynamic Programming . . . 47
4.7 Efficiency and Limitations of Dynamic Programming . . . 47
4.8 Worked Example: Jack's Car Rental . . . 48
4.9 Summary . . . 49
Exercises . . . 49

5 Monte Carlo Methods 51
5.1 Monte Carlo Prediction . . . 51
5.2 Worked Example: Blackjack . . . 54
5.3 Monte Carlo Estimation of Action Values . . . 56
5.4 Monte Carlo Control with Exploring Starts . . . 57
5.5 On-Policy Monte Carlo Control . . . 58
5.6 Off-Policy Monte Carlo via Importance Sampling . . . 59
5.7 Bias–Variance Trade-off in Monte Carlo . . . 62
5.8 Summary . . . 63
Exercises . . . 63

6 Temporal-Difference Learning 65
6.1 TD(0) Prediction . . . 65
6.1.1 From Monte Carlo to Temporal Difference . . . 65
6.1.2 The TD Error . . . 66
6.1.3 Zero-Mean Property of the TD Error . . . 66
6.1.4 TD(0) Algorithm . . . 67
6.1.5 Convergence of Tabular TD(0) . . . 67
6.1.6 Worked Example: Random Walk . . . 68
6.2 Advantages of TD Methods . . . 68
6.3 SARSA: On-Policy TD Control . . . 69
6.3.1 From Prediction to Control . . . 69
6.3.2 The SARSA Update . . . 70
6.3.3 Convergence of SARSA . . . 70
6.3.4 Worked Example: Cliff Walking . . . 71
6.4 Q-Learning: Off-Policy TD Control . . . 71
6.4.1 The Q-Learning Update . . . 71
6.4.2 Convergence of Q-Learning . . . 71
6.4.3 Maximisation Bias . . . 72
6.5 Expected SARSA . . . 72
6.6 Double Q-Learning . . . 73
6.7 Comparison: SARSA, Q-Learning, and Expected SARSA . . . 73
6.8 n-Step TD Methods . . . 74

6.8.1 The n-Step Return . . . 74
6.8.2 Bias–Variance Trade-Off in n-Step TD . . . 75
6.8.3 n-Step SARSA . . . 75
6.9 TD(λ) and Eligibility Traces . . . 75
6.9.1 The λ-Return . . . 76
6.9.2 Forward View of TD(λ) . . . 76
6.9.3 Eligibility Traces . . . 77
6.9.4 Backward View of TD(λ): The Online Algorithm . . . 77
6.9.5 Equivalence of Forward and Backward Views . . . 77
6.9.6 Practical Considerations . . . 78
6.9.7 Worked Example: Random Walk with TD(λ) . . . 79
6.10 Generalised Advantage Estimation (GAE) . . . 79
6.10.1 The Advantage Function . . . 79
6.10.2 n-Step Advantage Estimates . . . 80
6.10.3 GAE: The λ-Weighted Advantage . . . 80
6.10.4 Special Cases and Bias–Variance Trade-Off . . . 81
6.10.5 Connection to TD(λ) . . . 81
6.11 Bias, Variance, and Convergence Analysis . . . 82
6.11.1 Formal Bias Analysis . . . 82
6.11.2 Variance Analysis . . . 82
6.11.3 Mean Squared Error Decomposition . . . 82
6.11.4 Comparison Table . . . 83
6.11.5 The Deadly Triad . . . 83
6.11.6 Convergence Under the Robbins–Monro Schedule . . . 83
6.12 Summary . . . 83
Exercises . . . 84

II Function Approximation and Deep RL 87

7 Value Function Approximation 88
7.1 Why Tabular Methods Fail . . . 89
7.1.1 The Curse of Dimensionality . . . 89
7.1.2 The Generalisation Argument . . . 90
7.2 Linear Value Function Approximation . . . 90
7.2.1 Feature Vectors and the Linear Ansatz . . . 90
7.2.2 Tile Coding . . . 91
7.2.3 Linear TD(0) . . . 91
7.3 The Projected Bellman Equation . . . 91
7.3.1 Geometry of Function Approximation . . . 91
7.3.2 Existence of the TD Fixed Point . . . 92
7.3.3 Approximation Error at the TD Fixed Point . . . 93
7.4 Semi-Gradient Methods . . . 93
7.4.1 True Gradient Descent for Prediction . . . 93
7.4.2 Why TD Breaks True Gradient Descent . . . 93
7.4.3 Semi-Gradient TD Algorithm . . . 94
7.4.4 Convergence of Semi-Gradient TD . . . 94
7.5 The Deadly Triad . . . 95
7.5.1 Three Ingredients for Instability . . . 95
7.5.2 Baird's Counterexample . . . 96

7.6 Deep Q-Networks . . . 96
7.6.1 From Linear to Nonlinear Approximation . . . 96
7.6.2 Experience Replay . . . 97
7.6.3 Target Networks . . . 97
7.6.4 DQN Algorithm . . . 98
7.6.5 Architecture and Atari Results . . . 98
7.7 Extensions of DQN . . . 98
7.7.1 Double DQN . . . 98
7.7.2 Dueling DQN . . . 99
7.7.3 Prioritised Experience Replay . . . 100
7.8 Convergence Analysis . . . 100
7.8.1 What We Can Prove . . . 100
7.8.2 Gradient-TD Methods . . . 100
7.8.3 The Nonlinear Case . . . 101
7.9 Applications and Practical Considerations . . . 101
7.9.1 Feature Engineering vs. End-to-End Learning . . . 101
7.9.2 Hyperparameter Sensitivity . . . 102
7.9.3 Practical Recommendations . . . 102
7.9.4 Historical Notes . . . 102
Exercises . . . 103

8 Policy Gradient Methods 105
8.1 Why Optimise the Policy Directly? . . . 106
8.1.1 Limitations of the Value-Based Approach . . . 106
8.1.2 Stochastic Policies Are Sometimes Necessary . . . 106
8.1.3 The Objective Function . . . 106
8.2 The Policy Gradient Theorem . . . 107
8.2.1 The Likelihood-Ratio Trick . . . 107
8.2.2 The Causality Lemma and Reward-to-Go . . . 107
8.2.3 The Policy Gradient Theorem: Compact Form . . . 108
8.3 REINFORCE . . . 109
8.3.1 The Algorithm . . . 109
8.3.2 A Worked Example . . . 110
8.4 Variance Reduction: Baselines . . . 111
8.4.1 The Baseline Theorem . . . 111
8.4.2 The Optimal Baseline . . . 111
8.4.3 The State-Value Baseline . . . 111
8.5 REINFORCE with Learned Baseline . . . 112
8.6 RLOO: REINFORCE Leave-One-Out . . . 112
8.6.1 Motivation: Policy Gradient in the Language Model Setting . . . 112
8.6.2 The RLOO Estimator . . . 113
8.6.3 Unbiasedness of RLOO . . . 113
8.6.4 Variance Reduction of RLOO . . . 114
8.6.5 Connection to RLHF . . . 115
8.7 Natural Policy Gradient . . . 115
8.7.1 The Problem with the Ordinary Gradient . . . 115
8.7.2 The Fisher Information Matrix . . . 115
8.7.3 Practical Considerations . . . 116
8.8 Convergence Analysis . . . 116
8.8.1 Convergence of REINFORCE . . . 116

8.8.2 Convergence Rate . . . 117
8.8.3 The Bias-Variance Tradeoff . . . 117
8.9 Comparison of Policy Gradient Estimators . . . 117
Exercises . . . 118

9 Actor-Critic Methods and PPO 121
9.1 The Advantage Function . . . 122
9.1.1 Definition and Basic Properties . . . 122
9.1.2 Why the Advantage Reduces Variance . . . 122
9.2 The Actor-Critic Architecture . . . 123
9.2.1 Two-Component Design . . . 123
9.2.2 Shared versus Separate Networks . . . 124
9.3 TD Error as Advantage Estimate . . . 124
9.3.1 The One-Step Estimate . . . 124
9.3.2 One-Step Actor-Critic Algorithm . . . 125
9.4 Generalized Advantage Estimation . . . 125
9.4.1 Multi-Step Returns and the n-Step Advantage . . . 125
9.4.2 GAE Definition and Derivation . . . 126
9.4.3 The λ Parameter: Bias-Variance Tradeoff . . . 127
9.4.4 The GAE Target for the Critic . . . 128
9.5 A2C and A3C . . . 128
9.5.1 Parallelism as a Variance Reduction Tool . . . 128
9.5.2 A2C: the Synchronous Variant . . . 128
9.6 Trust Region Methods . . . 129
9.6.1 Off-Policy Reuse via Importance Sampling . . . 129
9.6.2 The Danger of Large Updates . . . 130
9.6.3 Trust Region Policy Optimization (TRPO) . . . 130
9.7 Proximal Policy Optimization . . . 130
9.7.1 From Trust Regions to Clipping . . . 130
9.7.2 Case Analysis of the Clipping Mechanism . . . 131
9.7.3 Full PPO Objective . . . 132
9.7.4 The PPO Algorithm . . . 132
9.8 PPO for Language Models . . . 133
9.8.1 Language Models as RL Policies . . . 133
9.8.2 Reward Modeling and the KL Penalty . . . 134
9.8.3 The Four-Model RLHF Architecture . . . 135
9.8.4 Practical Considerations . . . 135
9.9 Convergence and Practical Considerations . . . 136
9.9.1 Convergence Properties . . . 136
9.9.2 Comparison of Policy Gradient Algorithms . . . 137
9.9.3 Hyperparameter Sensitivity and Implementation Tips . . . 137
Exercises . . . 138

III RL for Language Models 140

10 Reward Shaping and Reward Models 141
10.1 Potential-Based Reward Shaping . . . 142
10.1.1 The Sparse-Reward Problem . . . 142
10.1.2 The Shaping Theorem . . . 142

10.1.3 Designing Potential Functions: Examples . . . 143
10.2 From Hand-Crafted to Learned Rewards . . . 144
10.2.1 The Limits of Manual Reward Design . . . 144
10.2.2 Reward Hacking in Language Models . . . 144
10.3 The Bradley–Terry Model . . . 145
10.3.1 Preference Data . . . 145
10.3.2 The Bradley–Terry Preference Model . . . 145
10.3.3 Connection to Elo Ratings . . . 146
10.4 Preference Reward Models . . . 147
10.4.1 Maximum Likelihood Estimation . . . 147
10.4.2 Architecture Choices . . . 147
10.4.3 Practical Training Procedure . . . 148
10.5 Outcome Reward Models (ORM) . . . 148
10.5.1 Sparse Terminal Rewards . . . 148
10.5.2 When to Use ORMs . . . 149
10.6 Process Reward Models (PRM) . . . 149
10.6.1 Step-Level Supervision . . . 149
10.6.2 Training Process Reward Models . . . 150
10.6.3 Using PRMs for Inference-Time Search . . . 150
10.6.4 ORM vs. PRM: A Comparison . . . 151
10.7 Training Objectives and Practical Considerations . . . 151
10.7.1 Margin-Based Loss Variants . . . 151
10.7.2 Label Smoothing . . . 152
10.7.3 Data Quality and Annotation Protocols . . . 152
10.7.4 Evaluation Metrics . . . 152
10.8 Reward Overoptimization . . . 152
10.8.1 Goodhart's Law in Reinforcement Learning . . . 152
10.8.2 KL Penalty as Regularisation . . . 153
10.9 Summary and Connections . . . 154
Exercises . . . 155

11 Language Models and Post-Training 158
11.1 The Transformer Architecture . . . 158
11.1.1 Self-Attention . . . 159
11.1.2 Positional Encoding . . . 159
11.1.3 Feed-Forward Blocks and Layer Normalisation . . . 159
11.2 Autoregressive Language Modelling . . . 161
11.2.1 Training Objective: Cross-Entropy Loss . . . 161
11.2.2 Scaling Laws . . . 162
11.3 Language Generation as a Markov Decision Process . . . 162
11.4 Pre-Training . . . 164
11.4.1 Data . . . 164
11.4.2 The Pre-Training Loss and Emergent Capabilities . . . 164
11.4.3 Base Models and Their Limitations . . . 165
11.5 Supervised Fine-Tuning . . . 165
11.5.1 Data Format and Instruction Templates . . . 165
11.5.2 SFT Loss Function . . . 165
11.5.3 Datasets and Examples . . . 165
11.5.4 Exposure Bias and the Ceiling of SFT . . . 166
11.6 The Post-Training Pipeline . . . 166

11.6.1 Stage Interactions . . . 166
11.6.2 What Each Stage Optimises . . . 167
11.7 Synthetic Data for Post-Training . . . 167
11.7.1 Why Synthetic Data Is Necessary . . . 167
11.7.2 Self-Instruct . . . 168
11.7.3 Distillation from Teacher Models . . . 168
11.7.4 Evol-Instruct and Complexity Escalation . . . 168
11.7.5 Persona-Based Generation . . . 169
11.7.6 Preference Data Synthesis . . . 169
11.7.7 Data Filtering and Quality Control . . . 170
11.7.8 Model Collapse and Mitigations . . . 170
11.7.9 Key Synthetic Datasets . . . 171
11.8 Evaluation of Language Models . . . 171
11.8.1 Automated Benchmarks . . . 171
11.8.2 Human Evaluation . . . 172
11.8.3 LLM-as-Judge . . . 172
11.8.4 Reward Model Evaluation . . . 172
11.9 Summary . . . 173
Exercises . . . 173

12 DPO and Alignment Methods 177
12.1 From RLHF to Direct Alignment . . . 178
12.1.1 The Cost of the Reward Model . . . 178
12.1.2 Simplicity, Memory, and Stability . . . 178
12.2 Deriving DPO . . . 179
12.2.1 The KL-Constrained Objective . . . 179
12.2.2 The Optimal Policy Reparametrisation . . . 179
12.2.3 Substituting into the Bradley–Terry Model . . . 179
12.2.4 The DPO Loss . . . 180
12.3 The DPO Algorithm . . . 181
12.3.1 Training Procedure . . . 181
12.3.2 Data Requirements . . . 181
12.3.3 Implementation Details . . . 182
12.3.4 The DPO vs. PPO Pipeline: A Diagram . . . 182
12.4 Variants of DPO . . . 182
12.4.1 IPO: Identity Preference Optimisation . . . 182
12.4.2 KTO: Kahneman-Tversky Optimisation . . . 183
12.4.3 cDPO: Conservative DPO . . . 184
12.4.4 ORPO: Odds Ratio Preference Optimisation . . . 184
12.5 Limitations of Direct Alignment . . . 185
12.5.1 The Distribution Shift Problem . . . 185
12.5.2 No Online Exploration . . . 185
12.5.3 Mode-Seeking vs. Mode-Covering . . . 185
12.5.4 When DPO Fails . . . 186
12.6 Online RL in Production . . . 186
12.6.1 Why Online RL Matters . . . 186
12.6.2 On-Policy vs. Off-Policy Considerations . . . 187
12.6.3 Catastrophic Forgetting Mitigation . . . 187
12.6.4 Safety Constraints in Online RL . . . 187
12.6.5 PPO vs. DPO in the Online Setting . . . 188

12.6.6 Practical Deployment Patterns . . . 188
12.7 Comprehensive Comparison of Alignment Methods . . . 189
12.7.1 Method Overview . . . 189
12.7.2 Choosing an Alignment Method by Goal . . . 189
12.8 Summary . . . 191
Exercises . . . 192

13 GRPO and Online RL 195
13.1 Motivation: Beyond PPO and DPO . . . 196
13.1.1 The Cost of the Critic in PPO . . . 196
13.1.2 The Distribution Shift Problem in DPO . . . 196
13.1.3 GRPO: The Best of Both Worlds . . . 196
13.2 The GRPO Objective . . . 197
13.2.1 Setup and Notation . . . 197
13.2.2 Group-Relative Advantage . . . 197
13.2.3 The Clipped Surrogate Objective . . . 198
13.2.4 The KL Penalty . . . 198
13.2.5 Full GRPO Objective . . . 199
13.3 The GRPO Algorithm . . . 199
13.3.1 Key Hyperparameters . . . 199
13.3.2 Worked Example . . . 201
13.4 Theoretical Analysis . . . 201
13.4.1 Connection to REINFORCE with Baseline . . . 201
13.4.2 Comparison with RLOO . . . 202
13.4.3 Variance Reduction from Group Normalisation . . . 202
13.4.4 Effect of Group Size G . . . 202
13.5 RLVR: Reinforcement Learning with Verifiable Rewards . . . 203
13.5.1 Definition and Motivation . . . 203
13.5.2 Formal Setup . . . 204
13.5.3 Connection to GRPO . . . 204
13.5.4 Domains for RLVR . . . 204
13.6 Rejection Sampling and Best-of-N . . . 205
13.6.1 Algorithm . . . 205
13.6.2 Rejection Sampling Fine-Tuning . . . 205
13.6.3 Comparison with GRPO . . . 205
13.7 Online RL Training at Scale . . . 206
13.7.1 The Two-Phase Architecture . . . 206
13.7.2 Handling Variable-Length Completions . . . 206
13.7.3 Batch Construction . . . 207
13.7.4 Synchronous vs. Asynchronous Design . . . 207
13.7.5 Experience Replay for GRPO . . . 207
13.7.6 Monitoring and Diagnostics . . . 207
13.8 Comparison: PPO vs GRPO vs DPO . . . 208
13.8.1 When to Use Each Method . . . 208
13.9 Summary . . . 209
Exercises . . . 209

14 Practical RLHF **211**
14.1 The RLHF Pipeline End-to-End . . . 211
14.2 Collecting Preference Data . . . 212
14.2.1 Annotation Formats . . . 212
14.2.2 Inter-Annotator Agreement . . . 213
14.2.3 Data Quality versus Quantity . . . 213
14.2.4 Common Pitfalls . . . 213
14.3 Training Reward Models in Practice . . . 214
14.3.1 Architecture Choices . . . 214
14.3.2 Training Stability and Overfitting . . . 214
14.3.3 Calibration and Evaluation Metrics . . . 214
14.3.4 When to Retrain . . . 215
14.4 PPO for Language Models: Implementation Details . . . 215
14.4.1 The Four-Model Setup . . . 215
14.4.2 KL Penalty Implementation . . . 216
14.4.3 Advantage Estimation with GAE . . . 216
14.4.4 Mini-Batch Construction and Gradient Accumulation . . . 216
14.4.5 Mixed Precision Training . . . 217
14.4.6 Hyperparameters . . . 217
14.5 Debugging RLHF . . . 217
14.5.1 Reward Hacking . . . 217
14.5.2 KL Divergence Explosion . . . 218
14.5.3 Mode Collapse . . . 218
14.5.4 Training Instability . . . 218
14.5.5 Diagnostic Metrics to Monitor . . . 219
14.6 Scaling RLHF . . . 219
14.6.1 Generation versus Training . . . 219
14.6.2 Parallelism Strategies . . . 219
14.6.3 Compute Budget Estimates . . . 220
14.7 Safety and Alignment Constraints . . . 220
14.7.1 Red-Teaming . . . 220
14.7.2 Constitutional AI Integration . . . 220
14.7.3 Helpfulness versus Harmlessness . . . 221
14.7.4 When to Stop Training . . . 221
14.8 Case Studies . . . 221
14.8.1 InstructGPT (Ouyang et al., 2022) . . . 221
14.8.2 Llama 2 (Touvron et al., 2023) . . . 221
14.8.3 Anthropic's Approach (Bai et al., 2022) . . . 222
14.8.4 DeepSeek's Approach with GRPO (DeepSeek-AI, 2024) . . . 222
14.9 Summary . . . 222
Exercises . . . 223

IV Frontiers **225**

15 Reasoning and Reinforcement Finetuning **226**
15.1 From RLHF to Reinforcement Learning with Verifiable Rewards . . . 227
15.1.1 The Verification Insight . . . 227
15.1.2 Verifiable Domains . . . 227
15.1.3 Formal Comparison with the RLHF Objective . . . 228

15.2 Chain-of-Thought Reasoning . . . 229
15.2.1 The Basic Idea . . . 229
15.2.2 Why Longer Reasoning Helps . . . 229
15.2.3 Thinking Tokens and Format . . . 230
15.2.4 Connection to Inference-Time Compute . . . 230
15.3 Training Reasoning Models . . . 230
15.3.1 The Two-Stage Recipe . . . 230
15.3.2 Difficulty Filtering . . . 231
15.3.3 Removing the KL Penalty . . . 231
15.3.4 Format Rewards and Length Penalties . . . 231
15.3.5 Loss Normalisation and the GRPO Bias . . . 232
15.3.6 Self-Taught Reasoning and Distillation . . . 232
15.4 Inference-Time Scaling . . . 232
15.4.1 The Test-Time Compute Hypothesis . . . 232
15.4.2 Majority Voting . . . 233
15.4.3 Value-Guided Sampling . . . 234
15.4.4 Compute-Optimal Inference . . . 234
15.5 Tool Use and Function Calling . . . 234
15.5.1 Motivation . . . 234
15.5.2 Function Calling Format . . . 234
15.5.3 Training Tool Use . . . 235
15.5.4 SWiRL: Step-Wise RL for Tool Use . . . 235
15.6 Multi-Step Agents and Agentic RL . . . 236
15.6.1 The Agent Loop . . . 236
15.6.2 Agentic Tasks and Environments . . . 237
15.6.3 Credit Assignment in Multi-Step Trajectories . . . 237
15.6.4 Exploration in Agentic RL . . . 237
15.7 Practical Considerations . . . 238
15.7.1 Algorithm Choices . . . 238
15.7.2 Online vs. Offline RL . . . 239
15.7.3 Infrastructure at Scale . . . 239
15.7.4 Open-Source Tooling . . . 239
15.8 Summary and Open Questions . . . 240
Exercises . . . 241

16 Multi-Agent Systems and RL 243
16.1 Multi-Agent Markov Games . . . 244
16.1.1 From MDPs to Markov Games . . . 244
16.1.2 Nash Equilibrium . . . 244
16.1.3 Partial Observability and the Dec-POMDP . . . 245
16.2 Cooperative MARL . . . 246
16.2.1 The Cooperative Setting . . . 246
16.2.2 Centralised Training with Decentralised Execution . . . 246
16.2.3 QMIX . . . 246
16.2.4 MAPPO . . . 247
16.3 Competitive and Mixed Settings . . . 248
16.3.1 Zero-Sum Games and Self-Play . . . 248
16.3.2 AlphaGo and AlphaZero . . . 248
16.3.3 Competitive Training Instability and Non-Transitive Strategies . . . 249
16.4 Communication and Emergent Behaviour . . . 249

16.4.1 Learned Communication Protocols . . . 249
16.4.2 Emergent Communication . . . 249
16.4.3 Social Dilemmas in Multi-Agent Learning . . . 250
16.5 Multi-Agent RL for LLMs . . . 250
16.5.1 Debate . . . 250
16.5.2 Multi-Agent Code Generation . . . 251
16.5.3 LLM Agents Cooperating on Tasks . . . 251
16.5.4 Agentic Workflows with Specialised Agents . . . 251
16.6 Challenges in Multi-Agent RL . . . 252
16.6.1 Non-Stationarity . . . 252
16.6.2 Credit Assignment in Teams . . . 252
16.6.3 Scalability . . . 252
16.6.4 Reward Shaping for Cooperation . . . 253
16.6.5 Sample Efficiency . . . 253
16.7 Summary . . . 253
Exercises . . . 254

17 Open Problems and Emerging Directions 257
17.1 Sample Efficiency . . . 257
17.1.1 Model-Based RL and World Models . . . 257
17.1.2 Transfer and Few-Shot RL . . . 258
17.2 Reward Specification and Alignment . . . 258
17.2.1 Scalable Oversight . . . 259
17.2.2 The Fundamental Challenge . . . 259
17.3 Safety and Robustness . . . 259
17.3.1 Safe Exploration . . . 259
17.3.2 Distributional Robustness and Adversarial Attacks . . . 259
17.3.3 Formal Verification . . . 260
17.4 Offline and Batch RL . . . 260
17.4.1 The Distribution-Shift Problem . . . 260
17.4.2 Sequence Modelling and Decision Transformers . . . 260
17.4.3 Offline-to-Online . . . 261
17.5 Multimodal and Embodied RL . . . 261
17.5.1 Sim-to-Real Transfer . . . 261
17.5.2 Vision-Language-Action Models . . . 261
17.5.3 Continuous Control at Scale . . . 261
17.6 RL and Foundation Models . . . 262
17.6.1 RL for Language and Reasoning . . . 262
17.6.2 RL for Visual Generation . . . 262
17.6.3 RL for Scientific Discovery . . . 262
17.7 Theoretical Frontiers . . . 262
17.7.1 Regret and Sample Complexity for Deep RL . . . 262
17.7.2 Convergence of Nonlinear Function Approximation . . . 263
17.7.3 Statistical Complexity of RLHF . . . 263
17.7.4 Why Does RL Work for LLMs? . . . 263
17.8 The Road Ahead . . . 263

A Mathematical Background **265**
A.1 Linear Algebra Essentials 265
A.2 Calculus and Optimisation 266
A.3 Probability Theory 267
A.4 Sampling Methods 269
A.5 Information Theory 269
A.6 Operators and Contraction Mappings 270

Afterword **273**

Index **275**

Notation

The following tables collect the symbols used consistently throughout the book. Where a symbol carries a meaning specific to a single chapter, that chapter introduces it explicitly; the entries here cover the globally recurring conventions.

General Mathematics

Symbol	**Meaning**
$\mathbb{R}$, $\mathbb{R}^n$	Real numbers; n-dimensional real vector space
$\mathbb{N}$	Natural numbers $\{0, 1, 2, \ldots\}$
$\mathbb{Z}$	Integers
$[n]$	Index set $\{1, 2, \ldots, n\}$
$\lvert S \rvert$	Cardinality of set S
$\langle x, y \rangle$	Inner product of vectors x and y
$\lVert x \rVert$, $\lVert x \rVert_p$	Euclidean (2-)norm; ℓ^p norm
$\lVert A \rVert_F$	Frobenius norm of matrix A
$A^\top$	Transpose of matrix A
A^{-1}	Inverse of square matrix A
$\nabla_\theta f$	Gradient of f with respect to θ
$\nabla_\theta^2 f$	Hessian of f with respect to θ
$\partial_x f$	Partial derivative of f with respect to x
$f \circ g$	Function composition: $(f \circ g)(x) = f(g(x))$
$\propto$	Proportional to (up to a normalising constant)
$\mathbb{1}\{A\}$	Indicator function: 1 if event A holds, else 0
$\log$	Natural logarithm (base e)
$\log_2$	Binary logarithm (base 2)
$\exp(x)$	Exponential function e^x
$[x]_+$	Positive part: $\max(x, 0)$
$\lfloor x \rfloor$, $\lceil x \rceil$	Floor and ceiling functions

Probability and Statistics

Symbol	**Meaning**
$\mathbb{P}(A)$	Probability of event A
$\mathbb{P}(A \mid B)$	Conditional probability of A given B
$\mathbb{E}[X]$	Expected value of random variable X
$\mathbb{E}[X \mid Y]$	Conditional expectation of X given Y
$\mathrm{Var}[X]$	Variance of X: $\mathbb{E}[X^2] - (\mathbb{E}[X])^2$
$\mathrm{Cov}[X, Y]$	Covariance of X and Y
$p(x)$	Probability density or mass function
$p(x \mid y)$	Conditional density/mass of x given y
$X \sim p$	X is distributed according to p
$X \perp Y$	X and Y are independent
$\mathcal{N}(\mu, \sigma^2)$	Normal distribution with mean μ, variance σ^2
$\mathrm{Bernoulli}(p)$	Bernoulli distribution with success probability p
$\mathrm{Cat}(\mathbf{p})$	Categorical distribution with probability vector $\mathbf{p}$
$D_{\mathrm{KL}}(p \parallel q)$	Kullback-Leibler divergence from q to p
$H(p)$	Shannon entropy of distribution p
$H(p, q)$	Cross-entropy between p and q
$I(X; Y)$	Mutual information between X and Y
$\sigma(x)$	Sigmoid function $1/(1 + e^{-x})$
$\mathrm{softmax}(\mathbf{z})_i$	$e^{z_i} / \sum_j e^{z_j}$

Reinforcement Learning Core

Symbol	Meaning
$\mathcal{S}$	State space
$\mathcal{A}$	Action space
$\mathcal{R}$	Reward space (also used for regret)
$\mathcal{M}$	MDP tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$
$P(s' \mid s, a)$	Transition probability function
$R(s, a)$, $r(s, a)$	Expected reward; reward signal
s_t, a_t, r_t	State, action, reward at time step t
τ	Trajectory (episode): $(s_0, a_0, r_0, s_1, \ldots)$
T	Horizon (episode length); may be ∞
$\gamma \in [0, 1)$	Discount factor
G_t	Return from time t: $\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$
$\pi(a \mid s)$	Stochastic policy
$\pi(s)$	Deterministic policy (maps s to an action)
π^*	Optimal policy
$V^\pi(s)$	State-value function under policy π
$Q^\pi(s, a)$	Action-value (Q-)function under policy π
$V^*(s)$, $Q^*(s, a)$	Optimal value functions
$A^\pi(s, a)$	Advantage function $Q^\pi(s, a) - V^\pi(s)$
δ_t	TD error at time t
α	Learning rate / step size
ϵ	Exploration rate (e.g. ϵ-greedy)
λ	Eligibility-trace decay (TD(λ))
n	n-step look-ahead
θ, ϕ, ψ	Parameter vectors (policy, value, reward model)
π_θ	Policy parameterised by θ
V_ϕ, Q_ϕ	Value function parameterised by ϕ
$\mathcal{T}^\pi$	Bellman operator for policy π
$\mathcal{T}^*$	Bellman optimality operator
$d^\pi(s)$	Stationary state distribution under π
$J(\theta)$	Policy objective (expected return)
$\hat{A}_t$	Estimated advantage at time t
$\hat{g}$	Stochastic policy-gradient estimate
L^{CLIP}	PPO clipped surrogate objective
$\rho_t(\theta)$	Probability ratio $\pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$

Language Model Notation

Symbol	Meaning
$\mathcal{V}$	Vocabulary (set of tokens)
x	Prompt / input sequence
y	Response / output sequence
y_t	t-th token of response y
$y_{<t}$	Response tokens before position t
$\lvert y \rvert$	Length of response y (in tokens)
eos	End-of-sequence token
$\pi_\theta(y \mid x)$	Language model policy (autoregressive)
$\pi_{\text{ref}}(y \mid x)$	Reference (supervised) policy
$r_\psi(x, y)$	Reward model with parameters ψ
β	KL penalty coefficient
$\mathcal{D}_{\text{pref}}$	Human preference dataset
y^+, y^-	Preferred and rejected response
$p^*(y^+ \succ y^-)$	Human preference probability
$Z(x)$	Partition function for optimal LM policy
ℓ_{DPO}	DPO training loss
g_i	i-th sampled completion (GRPO)
$\hat{r}_i$	Normalised reward for group i (GRPO)

Common Abbreviations

Abbreviation	Full Form
RL	Reinforcement Learning
MDP	Markov Decision Process
POMDP	Partially Observable MDP
TD	Temporal Difference
MC	Monte Carlo
DP	Dynamic Programming
GPI	Generalised Policy Iteration
VFA	Value Function Approximation
DQN	Deep Q-Network
DDPG	Deep Deterministic Policy Gradient
A2C/A3C	Advantage Actor-Critic / Asynchronous A3C
PPO	Proximal Policy Optimisation
TRPO	Trust Region Policy Optimisation
GAE	Generalised Advantage Estimation
SGD	Stochastic Gradient Descent
SFT	Supervised Fine-Tuning
RLHF	Reinforcement Learning from Human Feedback
DPO	Direct Preference Optimisation
GRPO	Group Relative Policy Optimisation
RLVR	RL with Verifiable Rewards
CoT	Chain-of-Thought
LLM	Large Language Model
KL	Kullback-Leibler (divergence)
i.i.d.	Independent and Identically Distributed
w.r.t.	With Respect To

Part I

Foundations

Chapter 1

Introduction to Reinforcement Learning

An agent learns not by being told the correct action, but by discovering — through trial and interaction — which actions yield the greatest long-term reward.

Reinforcement learning (RL) is the branch of machine learning concerned with *sequential decision-making under uncertainty.* Unlike supervised learning, where a fixed dataset of labelled examples is provided in advance, an RL agent must interact with an *environment*, choose *actions*, observe their consequences, and use the resulting *rewards* to improve its behaviour over time.

This chapter introduces the core ideas, motivations, and challenges of reinforcement learning, and situates it among the broader landscape of machine learning paradigms.

1.1 What Is Reinforcement Learning?

Definition 1.1 (Reinforcement Learning)**. Reinforcement learning** is a computational approach to learning from interaction, in which an *agent* takes actions in an *environment*, receives *rewards*, and seeks to maximise the cumulative reward over a horizon of interaction.

The defining characteristic of RL is that the agent never receives explicit instruction about which action is "correct." Instead, it receives only *evaluative feedback*: a scalar signal indicating how good (or bad) the outcome of its behaviour turned out to be. The agent's task is therefore not to imitate a given labelling, but to *discover a strategy* — called a *policy* — that maximises long-term performance.

To formalise the quality of an agent's behaviour, we introduce the *return* (also called *cumulative discounted reward*), which aggregates future rewards starting from time step t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \tag{1.1}$$

where $\gamma \in [0, 1)$ is the *discount factor* and R_{t+k+1} is the reward received $k + 1$ steps after time t. Following the standard convention, at time step t the agent selects action A_t, and the environment then returns reward R_{t+1} and next state S_{t+1}.

If the agent's policy is denoted by π, the standard RL objective is

$$J(\pi) = \mathbb{E}_\pi[G_0]. \tag{1.2}$$

In words, we seek a policy π that maximises the expected return (1.2).

Remark 1.1. In *episodic tasks*, the sum in (1.1) terminates at a terminal state. In *continuing tasks*, the horizon is infinite and the discount factor $\gamma < 1$ ensures that the sum remains finite.

1.2 Key Challenges in Reinforcement Learning

Even in the simplest RL settings, several fundamental challenges arise:

- **Delayed reward.** The consequence of an action may not become apparent until many time steps later. A chess move that seems innocuous in the opening may determine the outcome of the game fifty moves later.
- **Exploration versus exploitation.** The agent must balance *exploration* — trying unfamiliar actions to discover potentially better strategies — with *exploitation* — repeating actions that already appear rewarding. This tension is the central theme of Chapter 2.
- **Distribution shift induced by the policy.** Unlike supervised learning, where the training distribution is typically fixed, in RL the agent's own policy determines which states and transitions it will observe next. Changing the policy changes the data.
- **Correlated data.** Successive observations along a trajectory are temporally correlated and not independently drawn, which complicates the application of standard statistical learning guarantees.

Remark 1.2. It is precisely the dependence of data on the current policy that makes RL substantially more sensitive to errors during early training. A poor initial strategy may prevent the agent from ever visiting regions of the state–action space where superior behaviour could be discovered.

1.3 A Brief History of Reinforcement Learning

The intellectual roots of RL lie at the intersection of several fields:

Psychology (1900s–1950s) Edward Thorndike's *Law of Effect* (1911) stated that actions followed by satisfying consequences tend to be repeated. This idea of learning from reward signals is the conceptual ancestor of RL.

Optimal control (1950s) Richard Bellman formulated the theory of *dynamic programming* and introduced the *Bellman equation*, which remains the backbone of RL algorithms to this day.

Stochastic approximation Robbins and Monro (1951) developed iterative methods for finding roots of stochastic equations, providing the mathematical foundation for incremental learning rules.

Temporal-difference learning (1980s–1990s) Sutton's TD(λ) algorithm (1988) unified Monte Carlo and dynamic programming ideas. Watkins introduced Q-learning (1989), and Tesauro's TD-Gammon (1992) demonstrated that RL could achieve expert-level play in backgammon.

Deep RL (2013–present) The combination of deep neural networks with RL led to a series of breakthroughs: DQN playing Atari games at human level (Mnih et al., 2013, 2015), AlphaGo defeating the world champion in Go (Silver et al., 2016, 2017), and continuous-control algorithms such as DDPG, SAC, and PPO enabling robotic manipulation.

RL for language models (2020–present) The application of RL techniques — particularly PPO-based RLHF — to align large language models (Ouyang et al., 2022; OpenAI, 2023) has made RL a central tool in modern AI. Methods such as DPO (Rafailov et al., 2023) and GRPO (Shao et al., 2024) continue to expand the RL–LLM frontier.

1.4 Reinforcement Learning and Other Machine Learning Paradigms

To appreciate the distinctive features of RL, it is helpful to compare it with other core paradigms of machine learning.

Paradigm	**Data**	**Objective**	**Key Challenge**
Supervised Learning	input–label pairs (x, y)	minimise prediction error	generalisation, label noise, distribution shift
Unsupervised Learning	unlabelled data x	discover structure, density, or representations	choice of quality metric; interpretation
Imitation Learning	expert demonstrations	reproduce the expert's policy	covariate shift; compounding errors along trajectories
Reinforcement Learning	trajectories of interaction with an environment	maximise expected return	exploration, delayed reward, training instability

Table 1.1: Comparison of reinforcement learning with other major machine learning paradigms.

The most salient distinction is that in RL the training distribution is *not fixed*: the agent's own policy determines which data it will collect next. When the policy changes, so does the distribution of observations.

From a practical standpoint, RL is often more demanding than supervised learning for several reasons:

- Data collection is expensive: it requires simulation, interaction with a physical system, or a large number of episodes.

- The reward signal is frequently noisy, sparse, or poorly designed.
- Using neural-network function approximators introduces instabilities that require specialised stabilisation techniques (see the "deadly triad" in Chapter 7).

1.5 Applications of Reinforcement Learning

Reinforcement learning has been successfully applied in a remarkable variety of domains:

Game playing Board games (Go, chess, shogi via AlphaZero), video games (Atari, StarCraft II, Dota 2), and card games (poker via Libratus, Pluribus).

Robotics Locomotion, dexterous manipulation, and sim-to-real transfer using PPO, SAC, and model-based methods.

Recommendation systems Optimising long-term user engagement rather than immediate click-through rates.

Autonomous driving End-to-end learning of driving policies from simulation and real-world interaction.

Operations research Inventory management, dynamic pricing, resource allocation, and combinatorial optimisation.

Healthcare Personalised treatment policies, adaptive clinical trials, and drug dosing optimisation.

Language model alignment RLHF for instruction-following, safety, and helpfulness; DPO, GRPO, and related methods for aligning LLMs with human preferences. This application is the focus of Part III of this book.

1.6 The Agent–Environment Interface

At the highest level, every RL problem involves two entities:

1. The **agent**: the learner and decision-maker.
2. The **environment**: everything the agent interacts with.

At each discrete time step $t = 0, 1, 2, \ldots$, the following cycle repeats:

1. The agent observes the current state S_t (or observation O_t in partially observable settings).
2. The agent selects an action A_t according to its policy π.
3. The environment transitions to a new state S_{t+1} and emits a scalar reward R_{t+1}.

This interaction produces a *trajectory*:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots \tag{1.3}$$

The formal mathematical framework for this interaction — the *Markov Decision Process* (MDP) — is the subject of Chapter 3.

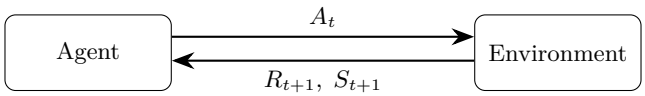


Figure 1.1: The agent–environment interaction loop. At each step the agent sends an action A_t and receives a reward R_{t+1} and the next state S_{t+1}.

1.7 Overview of the Book

The remainder of this book is organised into four parts.

Part I: Foundations (Chapters 2–6). We begin with the multi-armed bandit problem (Chapter 2), which introduces the exploration–exploitation trade-off in its purest form. We then formalise the full RL setting via Markov Decision Processes (Chapter 3), develop exact solution methods based on dynamic programming (Chapter 4), and introduce two families of sample-based algorithms: Monte Carlo methods (Chapter 5) and temporal-difference learning (Chapter 6).

Part II: Function Approximation and Deep RL (Chapters 7–9). We address the challenge of scaling RL to large state and action spaces through function approximation (Chapter 7), policy gradient methods (Chapter 8), and actor-critic architectures culminating in PPO (Chapter 9).

Part III: RL for Language Models (Chapters 10–14). We apply the RL toolkit to the training and alignment of large language models, beginning with reward shaping and reward models (Chapter 10), followed by the LLM training pipeline (Chapter 11), DPO and direct alignment methods (Chapter 12), GRPO and online RL with verifiable rewards (Chapter 13), and practical RLHF engineering (Chapter 14).

Part IV: Frontiers (Chapters 15–17). We explore reasoning and reinforcement finetuning (Chapter 15), multi-agent systems (Chapter 16), and conclude with a survey of open problems and emerging research directions (Chapter 17).

Exercises

Exercise 1.1. Identify three real-world problems that are naturally formulated as reinforcement learning tasks. For each, describe the agent, the environment, the action space, and the reward signal.

Exercise 1.2. Consider the task of teaching a robot to walk. Explain why a supervised-learning approach (e.g. learning from pre-recorded trajectories of a walking expert) may fail, and why an RL approach may be more appropriate. What role does the exploration–exploitation trade-off play?

Exercise 1.3. Show that for $\gamma \in [0, 1)$ and bounded rewards $|R_t| \leq R_{\max}$ for all t, the return (1.1) satisfies

$$|G_t| \leq \frac{R_{\max}}{1-\gamma}.$$

Exercise 1.4. For each of the following tasks, determine which machine learning paradigm (supervised, unsupervised, imitation, or reinforcement learning) is most natural, and justify your answer:

(a) Classifying emails as spam or not spam.

(b) Training a chatbot to be helpful and harmless using human preference data.

(c) Discovering customer segments from purchase histories.

(d) Teaching a self-driving car to navigate using recordings of expert drivers.

Chapter 2

Multi-Armed Bandits

The simplest setting in which the exploration–exploitation dilemma already appears in full force is the multi-armed bandit problem.

In this chapter we study the *multi-armed bandit* problem — a sequential decision-making task without states, where the agent repeatedly chooses one of k actions and observes a stochastic reward. Despite its simplicity, the bandit setting already exposes the core tension between gathering information (exploration) and maximising immediate payoff (exploitation). We begin with the basic formulation, analyse ε-greedy strategies, and then develop more principled approaches: *Upper Confidence Bounds* (UCB), *Thompson Sampling*, and the information-theoretic lower bound of Lai and Robbins. The chapter closes with a discussion of contextual bandits — a bridge toward the full reinforcement learning problem — and an empirical comparison of the algorithms studied.

2.1 Problem Formulation

Definition 2.1 (k-armed bandit)**.** A **k-armed bandit** is a sequential decision problem in which, at each time step $t = 1, 2, \ldots$, the agent selects an action A_t from a finite set

$$\mathcal{A} = \{1, 2, \ldots, k\},$$

and receives a stochastic reward $R_t \in \mathbb{R}$.

Remark 2.1 (Notation convention)*.* In the bandit setting, we write R_t for the reward received at time t in response to action A_t. This differs from the MDP convention introduced in Chapter 3, where the reward is denoted R_{t+1} (received after transitioning from state S_t with action A_t). The distinction arises because bandits have no state transitions; the simpler indexing R_t is standard in the bandit literature.

For each action $a \in \mathcal{A}$ there is a reward distribution ν_a. When action $A_t = a$ is chosen, the reward is drawn independently from ν_a. In the *stationary* setting these distributions do not change over time.

Unlike the general RL problem studied later in this book, the bandit setting has:

- no notion of state;
- immediate (non-delayed) rewards;

- no transition dynamics — each round is independent.

Nevertheless, the fundamental challenge is already present: to learn which action is best, the agent must try different actions, yet to maximise reward it should favour actions that already seem good.

2.2 Action Values, Optimal Actions, and Regret

Definition 2.2 (Action value)**.** The **true value** (or *expected reward*) of action $a \in \mathcal{A}$ is

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a]. \tag{2.1}$$

If the values (2.1) were known, the optimal strategy would simply be to always play

$$a^* \in \operatorname*{argmax}_{a \in \mathcal{A}} \; q_*(a). \tag{2.2}$$

Let

$$q^* = \max_{a \in \mathcal{A}} \; q_*(a) \tag{2.3}$$

denote the value of the best action.

Since the values are unknown, we measure performance by comparison with the omniscient strategy.

Definition 2.3 (Pseudo-regret)**.** The **pseudo-regret** after T steps is

$$\mathcal{R}_T = T\, q^* - \sum_{t=1}^{T} q_*(A_t). \tag{2.4}$$

Its expectation equals

$$\mathbb{E}[\mathcal{R}_T] = T\, q^* - \sum_{t=1}^{T} \mathbb{E}[q_*(A_t)].$$

Intuitively, $\mathcal{R}_T$ measures how much reward the algorithm "left on the table" compared to always playing the optimal arm.

Define the *sub-optimality gap* of action a as

$$\Delta_a = q^* - q_*(a) \geq 0, \tag{2.5}$$

and let $N_T(a) = \sum_{t=1}^{T} \mathbb{1}\{A_t = a\}$ count the number of times action a has been played.

Proposition 2.1 (Regret decomposition)**.** *In the stationary bandit setting (where the reward distributions do not change over time),*

$$\mathbb{E}[\mathcal{R}_T] = \sum_{a \in \mathcal{A}} \Delta_a \; \mathbb{E}[N_T(a)]. \tag{2.6}$$

Proof. Because the setting is stationary, $q_*(a)$ is constant across rounds. Since $\sum_{a \in \mathcal{A}} N_T(a) = T$ and $\sum_{t=1}^{T} q_*(A_t) = \sum_{a \in \mathcal{A}} q_*(a)\, N_T(a)$ (grouping the sum by which arm was played), we have

$$\mathcal{R}_T = T\, q^* - \sum_{a \in \mathcal{A}} q_*(a)\, N_T(a) = \sum_{a \in \mathcal{A}} (q^* - q_*(a))\, N_T(a) = \sum_{a \in \mathcal{A}} \Delta_a\, N_T(a).$$

Taking expectations on both sides yields (2.6). □

Formula (2.6) is fundamental: minimising regret is equivalent to limiting the number of times sub-optimal actions are played.

Remark 2.2. If $\mathbb{E}[\mathcal{R}_T] = O(\log T)$, then the average per-step regret decays as $O(\log T/T) \to 0$, meaning the agent chooses sub-optimal actions with vanishing frequency.

2.3 Greedy and ε-Greedy Strategies

Since true action values are unknown, the agent must estimate them from data.

Definition 2.4 (Action-value estimate)**.** The **action-value estimate** $Q_t(a)$ is a quantity computed from all rewards observed before step t for action a.

Definition 2.5 (Greedy action)**.** An action a is **greedy** at step t if

$$a \in \operatorname*{argmax}_{b \in \mathcal{A}} Q_t(b). \tag{2.7}$$

The simplest strategy is *pure greedy*: always choose the greedy action. While intuitive, this can fail catastrophically: early noisy observations may cause the agent to underestimate the best arm and never return to it.

Example 2.1. Consider a 2-armed bandit with $q_*(1) = 1$ and $q_*(2) = 0.9$. If the first observation from arm 1 happens to be unusually low and the first from arm 2 is unusually high, a greedy agent may lock onto arm 2 and never re-explore arm 1.

To escape such traps, we introduce random exploration.

Definition 2.6 (ε-greedy strategy)**.** The **ε-greedy strategy** selects actions by the rule

$$A_t = \begin{cases} \operatorname*{argmax}_{a \in \mathcal{A}} Q_t(a), & \text{with probability } 1-\varepsilon, \\ \text{uniform random from } \mathcal{A}, & \text{with probability } \varepsilon, \end{cases} \tag{2.8}$$

where $\varepsilon \in [0, 1]$.

Remark 2.3. If the optimal action is correctly identified as greedy, then it is selected with probability at least $1 - \varepsilon + \varepsilon/k$. However, with a fixed $\varepsilon > 0$, the expected number of random (potentially sub-optimal) choices grows linearly in T, yielding $\mathbb{E}[\mathcal{R}_T] = \Omega(T)$. That is, fixed-$\varepsilon$ strategies have *linear* regret.

2.4 Sample Averages and Incremental Updates

The most natural estimate of $q_*(a)$ is the *sample average*:

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i \, \mathbb{1}\{A_i = a\}}{N_t(a)}, \qquad N_t(a) > 0. \tag{2.9}$$

For actions not yet tried, we initialise $Q_1(a) = 0$.

Proposition 2.2. *If the rewards for action a are i.i.d. with mean $q_*(a)$ and $N_T(a) \to \infty$, then $Q_T(a) \to q_*(a)$ almost surely, by the strong law of large numbers.*

Computing (2.9) from scratch at every step is wasteful. Instead, we use the *incremental update*:

$$Q_{n+1}(a) = Q_n(a) + \frac{1}{n+1}\big(R_{n+1}(a) - Q_n(a)\big), \tag{2.10}$$

where $n = N_t(a)$ and $R_{n+1}(a)$ is the newest reward for action a.

Remark 2.4 (General update rule). Equation (2.10) is a special case of

$$Q \leftarrow Q + \alpha\,(R - Q), \tag{2.11}$$

with $\alpha = 1/(n+1)$. For *non-stationary* problems a constant step size $\alpha \in (0,1)$ is often preferred, giving exponentially decaying weights to older observations:

$$Q_{n+1} = (1-\alpha)^n Q_1 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i.$$

The ε-greedy bandit agent can now be stated concisely:

Algorithm 1: ε-Greedy Bandit Agent

Input: Number of arms k, exploration rate ε, horizon T

Initialise $Q(a) \leftarrow 0$, $N(a) \leftarrow 0$ for all $a \in \mathcal{A}$;
for $t = 1, 2, \ldots, T$ **do**
  With probability $1-\varepsilon$: $A_t \leftarrow \operatorname{argmax}_a Q(a)$;
  With probability ε: $A_t \leftarrow \text{Uniform}(\mathcal{A})$;
  Observe reward R_t;
  $N(A_t) \leftarrow N(A_t) + 1$;
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$;

2.5 Upper Confidence Bounds (UCB)

The ε-greedy strategy explores *uniformly at random*, without regard for which arms are more uncertain. A more principled approach is to explore preferentially toward actions about which we know the least. This is the idea behind *optimism in the face of uncertainty*.

The Principle of Optimism

The intuition behind optimism is elegant: when we are uncertain about the value of an arm, we should *assume the best*. If that optimistic assumption turns out to be correct, we are rewarded handsomely. If it turns out to be wrong — the arm is not as good as we hoped — then we have gained useful information and will assign that arm a lower confidence bound in the future. Either way, optimism is productive.

More concretely, suppose we maintain for each arm a an upper confidence bound $U_t(a)$ such that $q_*(a) \le U_t(a)$ holds with high probability. Selecting the arm with the largest $U_t(a)$ simultaneously exploits arms that are genuinely good and explores arms whose true value is still uncertain enough that their upper bound is elevated. As an arm is pulled more, observations accumulate and the bound $U_t(a)$ tightens around the empirical mean $Q_t(a)$; an arm that turns out to be sub-optimal will eventually have a low upper bound and be abandoned.

Definition 2.7 (UCB1)**.** The **UCB1** algorithm (Auer, Cesa-Bianchi, and Fischer, 2002) selects the action

$$A_t = \operatorname*{argmax}_{a \in \mathcal{A}} \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right], \tag{2.12}$$

where $c > 0$ is an exploration parameter (typically $c = \sqrt{2}$).

The quantity $Q_t(a) + c\sqrt{(\ln t)/N_t(a)}$ serves as an *upper confidence bound* on the true value $q_*(a)$. The second term — the *exploration bonus* — is large when action a has been tried infrequently, encouraging the agent to explore it. The factor $\ln t$ in the numerator grows slowly, ensuring that the agent continues to revisit uncertain arms as the horizon grows, while the $N_t(a)$ in the denominator ensures that the bonus shrinks as an arm is sampled more.

Derivation from Hoeffding's inequality. For bounded rewards $R \in [0, 1]$, Hoeffding's inequality gives

$$\mathbb{P}\big(q_*(a) > Q_t(a) + u\big) \le e^{-2N_t(a)\,u^2}.$$

Setting the right-hand side equal to t^{-4} (so that the failure probability is summable) and solving for u yields

$$u = \sqrt{\frac{2\ln t}{N_t(a)}},$$

which motivates the form (2.12) with $c = \sqrt{2}$.

Worked Numerical Example

To see UCB1 in action, consider a simple 3-armed bandit with unknown true values. We initialise each arm with one forced pull (a common initialisation strategy to avoid division by zero).

Example 2.2 (UCB1 arm-selection trace)**.** Suppose $k = 3$ and after the three forced initial pulls ($t = 1, 2, 3$) we have observed rewards $R_1 = 0.8$ from arm 1, $R_2 = 0.4$ from arm 2, and $R_3 = 0.6$ from arm 3. Thus $Q_3(1) = 0.8$, $Q_3(2) = 0.4$, $Q_3(3) = 0.6$, and $N_3(a) = 1$ for all a. Set $c = \sqrt{2}$.

At $t = 4$, the UCB scores are:

$$\begin{aligned}
U_4(1) &= 0.8 + \sqrt{\tfrac{2\ln 4}{1}} \approx 0.8 + 1.664 = 2.464,\\
U_4(2) &= 0.4 + \sqrt{\tfrac{2\ln 4}{1}} \approx 0.4 + 1.664 = 2.064,\\
U_4(3) &= 0.6 + \sqrt{\tfrac{2\ln 4}{1}} \approx 0.6 + 1.664 = 2.264.
\end{aligned}$$

Arm 1 is selected. Suppose we observe $R_4 = 0.3$. Now $Q(1) = (0.8 + 0.3)/2 = 0.55$ and $N(1) = 2$.

At $t = 5$:

$$\begin{aligned}
U_5(1) &= 0.55 + \sqrt{\tfrac{2\ln 5}{2}} \approx 0.55 + 1.270 = 1.820,\\
U_5(2) &= 0.4 + \sqrt{\tfrac{2\ln 5}{1}} \approx 0.4 + 1.794 = 2.194,\\
U_5(3) &= 0.6 + \sqrt{\tfrac{2\ln 5}{1}} \approx 0.6 + 1.794 = 2.394.
\end{aligned}$$

Arm 3 is selected. The key observation is that arm 1, despite having the highest initial reward, was "punished" by the outcome $R_4 = 0.3$, and arms 2 and 3 — which have not been pulled again — have maintained their large exploration bonuses. The algorithm will continue cycling until all arms have been sampled enough that the bonuses shrink and the empirical means dominate.

The figure below illustrates how confidence intervals shrink as an arm is pulled more, with the UCB score converging toward the true value.

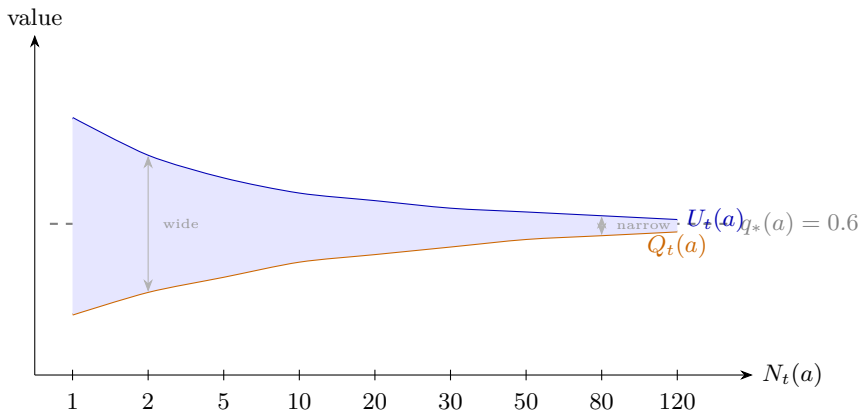


Figure 2.1: Schematic showing UCB confidence bounds shrinking as arm a is pulled more often. The blue shaded band represents the confidence interval $[Q_t(a), U_t(a)]$. As $N_t(a)$ grows, the exploration bonus $c\sqrt{\ln t / N_t(a)}$ decays and the UCB score converges toward the true value $q_*(a)$ (dashed line).

Regret Bound and Proof Sketch

Theorem 2.3 (Regret bound for UCB1)**.** *For a k-armed bandit with rewards in $[0, 1]$, the UCB1 algorithm with $c = \sqrt{2}$ satisfies*

$$\mathbb{E}[\mathcal{R}_T] \leq \sum_{a:\Delta_a>0} \left(\frac{8 \ln T}{\Delta_a}\right) + \left(1 + \frac{\pi^2}{3}\right) \sum_{a=1}^{k} \Delta_a. \tag{2.13}$$

In particular, $\mathbb{E}[\mathcal{R}_T] = O\left(\sqrt{kT \ln T}\right)$ in the worst case over gaps Δ_a.

The proof of Theorem 2.3 is instructive because it illustrates the key technique of *counting bad events*. We sketch the main argument here.

Proof sketch. The regret decomposition (2.6) tells us that $\mathbb{E}[\mathcal{R}_T] = \sum_{a:\Delta_a>0} \Delta_a \, \mathbb{E}[N_T(a)]$. It therefore suffices to bound $\mathbb{E}[N_T(a)]$ for each sub-optimal arm a.

Fix a sub-optimal arm a with gap $\Delta_a > 0$ and let $\ell = \lceil 8 \ln T / \Delta_a^2 \rceil$. Arm a is pulled at time t only if its UCB score exceeds that of the optimal arm a^*:

$$Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \geq Q_t(a^*) + c\sqrt{\frac{\ln t}{N_t(a^*)}}.$$

For this to happen when $N_t(a) \geq \ell$, either the empirical mean of a^* must be far below its true value, or the empirical mean of a must be far above its true value. By Hoeffding's inequality (applied with a union bound over all t), the probability of either event occurring after ℓ pulls is at most $2t^{-4}$, which is summable. A careful calculation then gives

$$\mathbb{E}[N_T(a)] \leq \ell + \sum_{t=1}^{T} \mathbb{P}(\text{bad event at } t) \leq \frac{8 \ln T}{\Delta_a^2} + 1 + \frac{\pi^2}{3}.$$

Multiplying by Δ_a and summing over sub-optimal arms yields (2.13). □

Remark 2.5. The key insight is that UCB achieves $O(\log T)$ regret — dramatically better than the $\Omega(T)$ linear regret of ε-greedy with fixed ε. Moreover, the exploration is directed rather than uniform: arms that are more uncertain or have higher empirical means receive more attention. The $\pi^2/3$ constant in (2.13) arises as follows: the Hoeffding failure probability per round is $2t^{-4}$, but summing over all possible values of the two sample sizes (each up to t) introduces a factor of t^2, yielding a per-round contribution of $t^2 \cdot 2t^{-4} = 2t^{-2}$. Summing gives $2\sum_{t=1}^{\infty} t^{-2} = 2 \cdot \pi^2/6 = \pi^2/3$.

2.6 Thompson Sampling

An alternative approach to the exploration–exploitation trade-off is to maintain a *Bayesian posterior* over the action values and use it to guide exploration. Rather than constructing an explicit optimistic bound, we let the posterior's uncertainty do the work automatically.

The Posterior-Sampling Intuition

Thompson Sampling — introduced by William R. Thompson in 1933, long before the term "bandit problem" existed — is based on a beautifully simple idea: act as if the world were as good as it might plausibly be. At each step, we draw a single sample from the posterior distribution of each arm's value and then act greedily with respect to those samples.

Why does this work? Consider two arms, one well-explored (narrow posterior) and one poorly explored (wide posterior). For the well-explored arm, the sample will almost surely fall close to the true mean. For the poorly explored arm, the sample has a non-negligible chance of being much higher than the true mean. This means uncertain arms are explored with higher probability, naturally implementing the optimism principle — but in a randomised, Bayesian fashion rather than a worst-case frequentist one.

Crucially, the probability that arm a is selected equals the posterior probability that a is optimal:

$$\mathbb{P}(A_t = a) = \mathbb{P}\big(\tilde{q}(a) = \max_b \tilde{q}(b)\big) \approx \mathbb{P}\big(q_*(a) = \max_b q_*(b) \mid \text{data}\big).$$

This is the *probability matching* property: the agent samples each arm in proportion to how likely it is to be optimal given everything observed so far. As evidence accumulates, these probabilities concentrate on the true best arm.

Definition 2.8 (Thompson Sampling)**. Thompson Sampling** (Thompson, 1933) proceeds as follows at each step:

1. For each arm a, sample $\tilde{q}(a)$ from the posterior distribution of $q_*(a)$.
2. Play the arm with the highest sample: $A_t = \operatorname{argmax}_a \tilde{q}(a)$.
3. Update the posterior using the observed reward.

Bernoulli bandits with Beta priors. Consider the case where rewards are Bernoulli with unknown success probability θ_a for each arm. If we place a $\text{Beta}(\alpha_a, \beta_a)$ prior on θ_a, then after observing s_a successes and f_a failures, the posterior is

$$\theta_a \mid \text{data} \sim \text{Beta}(\alpha_a + s_a,\ \beta_a + f_a).$$

Thompson Sampling then draws $\tilde{\theta}_a \sim \text{Beta}(\alpha_a + s_a, \beta_a + f_a)$ for each arm and plays $A_t = \text{argmax}_a\, \tilde{\theta}_a$.

The Beta distribution is conjugate to the Bernoulli likelihood — meaning posterior updates have a simple closed form. This makes the algorithm extremely efficient: no numerical integration is needed, and each update requires only incrementing two counters.

Algorithm 2: Thompson Sampling for Bernoulli Bandits

```
Input: Number of arms k, prior parameters α_0, β_0 (e.g. α_0 = β_0 = 1)
Initialise S(a) ← 0, F(a) ← 0 for all a;
for t = 1, 2, ..., T do
    for a = 1, ..., k do
        Sample θ̃_a ~ Beta(α_0 + S(a), β_0 + F(a));
    A_t ← argmax_a θ̃_a;
    Observe reward R_t ∈ {0, 1};
    if R_t = 1 then
        S(A_t) ← S(A_t) + 1
    else
        F(A_t) ← F(A_t) + 1
```

Step-by-Step Worked Example

Example 2.3 (Thompson Sampling with three Bernoulli arms)**.** Consider a 3-armed Bernoulli bandit with true success probabilities $\theta_1 = 0.3$, $\theta_2 = 0.5$, $\theta_3 = 0.7$. We begin with uniform priors $\text{Beta}(1, 1)$ on each arm.

Round 1. We draw one sample from each $\text{Beta}(1, 1) = \text{Uniform}(0, 1)$ prior. Suppose the samples are $\tilde{\theta}_1 = 0.62$, $\tilde{\theta}_2 = 0.29$, $\tilde{\theta}_3 = 0.45$. We select arm 1 (highest sample). The outcome is a failure ($R = 0$), so we update: arm 1 posterior becomes $\text{Beta}(1, 2)$.

Round 2. We draw from $\text{Beta}(1, 2)$, $\text{Beta}(1, 1)$, $\text{Beta}(1, 1)$. $\text{Beta}(1, 2)$ is skewed toward smaller values (mean $1/3$), so arm 1 is now less likely to win. Suppose $\tilde{\theta}_1 = 0.21$, $\tilde{\theta}_2 = 0.74$, $\tilde{\theta}_3 = 0.38$. Arm 2 is selected. Suppose success ($R = 1$): arm 2 becomes $\text{Beta}(2, 1)$.

Round 3. Now arm 3 has never been tried and retains its $\text{Beta}(1, 1)$ prior. Arm 2 has a $\text{Beta}(2, 1)$ posterior (mean $2/3$, skewed upward). Suppose the samples are $\tilde{\theta}_1 = 0.18$, $\tilde{\theta}_2 = 0.55$, $\tilde{\theta}_3 = 0.82$, so arm 3 is selected. Suppose success: arm 3 becomes $\text{Beta}(2, 1)$.

After many rounds, arms with higher true probabilities accumulate more successes, their Beta posteriors shift rightward, and their samples are stochastically larger. The posteriors concentrate, and arm 3 is eventually selected almost exclusively.

Figure 2.2 illustrates how the Beta posteriors evolve for each arm as data accumulates.

Gaussian Bandits Variant

Thompson Sampling extends naturally to Gaussian reward distributions. Suppose $R_t \mid A_t = a \sim \mathcal{N}(\mu_a, \sigma^2)$ with known variance σ^2 and unknown mean μ_a. A conjugate prior is

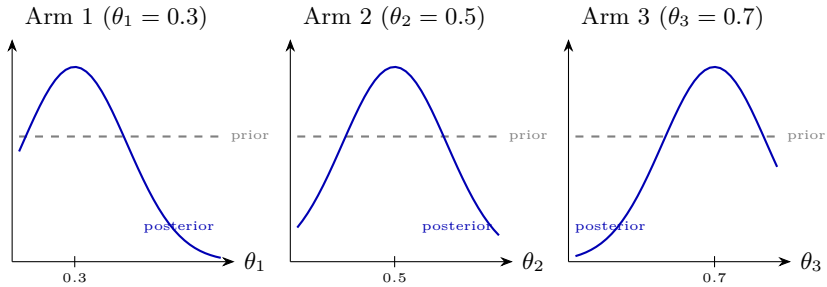


Figure 2.2: Schematic evolution of Beta posteriors for a 3-armed Bernoulli bandit. Dashed lines show the flat Beta$(1, 1)$ prior; solid curves show the posterior after $T = 50$ rounds. Each posterior concentrates near the true success probability of its arm. The posterior for arm 3 (the best arm) is the tallest and narrowest, reflecting the most accumulated evidence.

$\mu_a \sim \mathcal{N}(\mu_0, \sigma_0^2)$. After n observations with sample mean $\bar{r}_a$, the posterior is

$$\mu_a \mid \text{data} \sim \mathcal{N}\left(\frac{\sigma^{-2} \cdot n\,\bar{r}_a + \sigma_0^{-2}\mu_0}{\sigma^{-2} \cdot n + \sigma_0^{-2}},\ \frac{1}{\sigma^{-2} \cdot n + \sigma_0^{-2}}\right).$$

At each step, we sample $\tilde{\mu}_a$ from this posterior for each arm and select $A_t = \operatorname{argmax}_a \tilde{\mu}_a$. The Gaussian case provides a continuous analogue of the Beta-Bernoulli model and is convenient when rewards are unbounded.

Thompson Sampling vs. UCB

Both algorithms achieve $O(\log T)$ regret, but they differ in character. UCB is a *deterministic* algorithm (given the observations, it always makes the same choice) and its optimism is explicit via the confidence term. Thompson Sampling is *randomised*: even if the current samples happen not to select the best arm, the probability that this happens decreases as evidence accumulates.

In practice, Thompson Sampling often achieves lower regret than UCB1, particularly in the early phase when priors are informative. Its main limitation is the need to specify a prior and to sample from the posterior, which can be computationally expensive for complex models. UCB, by contrast, requires no prior and is straightforward to implement in any setting where empirical averages are meaningful.

Remark 2.6. Thompson Sampling has several attractive properties:

- It naturally balances exploration and exploitation: uncertain arms are explored because their posterior is wide, producing high samples with non-negligible probability.
- Empirically, it often matches or outperforms UCB.
- It achieves $O(\log T)$ Bayesian regret (Kaufmann, Korda, and Munos, 2012) and $O(\sqrt{kT \ln k})$ frequentist regret in many settings.
- The algorithm is modular: changing the reward model requires only changing the likelihood and the conjugate prior, leaving the high-level loop unchanged.

2.7 The Lai–Robbins Lower Bound

We have seen that UCB1 and Thompson Sampling both achieve $O(\log T)$ regret. A natural question arises: is $O(\log T)$ the best possible, or could a cleverer algorithm do even better? The landmark result of Lai and Robbins (1985) answers this question definitively — no consistent algorithm can do better than $\Omega(\log T)$.

Why $\log T$ Is a Fundamental Limit

The intuitive reason for the $\log T$ barrier is that any algorithm that eventually learns which arm is best must *continuously test* the hypothesis that a given arm is sub-optimal. Distinguishing arm a (with mean μ_a) from the optimal arm (with mean $\mu^* > \mu_a$) is a statistical testing problem. By standard information-theoretic arguments, if we have seen only n samples from arm a, we can distinguish μ_a from μ^* only if $n \gtrsim 1/\mathrm{KL}(\nu_a \| \nu_{a^*})$, where the KL divergence quantifies how different the two distributions look per sample. For the algorithm to be *consistent* — meaning it eventually identifies the best arm on every problem instance — it must pull arm a at least $\Omega(\log T/\mathrm{KL}(\nu_a \| \nu_{a^*}))$ times. Multiplying by the gap Δ_a gives the lower bound (2.14) below.

Definition 2.9 (Consistent algorithm)**.** A bandit algorithm is **consistent** if, for every problem instance and every sub-optimal arm a,

$$\mathbb{E}[N_T(a)] = o(T^\alpha) \quad \text{for all } \alpha \in (0, 1).$$

In words, the expected number of pulls of any sub-optimal arm grows sub-polynomially in T, so the fraction of time spent on it tends to zero. Consistency is a minimal rationality requirement: a consistent algorithm eventually stops wasting time on clearly inferior actions. Note that any algorithm with $O(\log T)$ regret (such as UCB1 or Thompson Sampling) is automatically consistent.

Theorem 2.4 (Lai and Robbins, 1985)**.** *For any consistent bandit algorithm and any problem instance with sub-optimality gaps $\Delta_a > 0$,*

$$\liminf_{T\to\infty} \frac{\mathbb{E}[\mathcal{R}_T]}{\ln T} \geq \sum_{a:\Delta_a>0} \frac{\Delta_a}{\mathrm{KL}(\nu_a \| \nu_{a^*})}, \tag{2.14}$$

where $\mathrm{KL}(\nu_a \| \nu_{a^*})$ denotes the Kullback–Leibler divergence between the reward distributions of arm a and the optimal arm a^*.

Information-Theoretic Argument Sketch

The proof of Theorem 2.4 can be understood through a *change-of-measure* argument. Suppose we have a bandit problem $\mathcal{P}$ and we consider an alternative problem $\mathcal{P}'$ that differs only in the reward distribution of arm a: in $\mathcal{P}'$, arm a is actually the best arm. Any consistent algorithm must behave differently on $\mathcal{P}$ and $\mathcal{P}'$, pulling arm a less often in $\mathcal{P}$ (where it is sub-optimal) than in $\mathcal{P}'$ (where it is optimal). The divergence $\mathrm{KL}(\nu_a \| \nu_{a^*})$ controls how quickly the algorithm can distinguish $\mathcal{P}$ from $\mathcal{P}'$ given samples from arm a. A data-processing inequality then establishes that the number of pulls of arm a in $\mathcal{P}$ must grow at least as $\ln T/\mathrm{KL}(\nu_a \| \nu_{a^*})$ for the algorithm to reliably distinguish the two problems.

Worked Example: Gaussian Case

For Gaussian rewards with unit variance, $\mathrm{KL}(\nu_a \| \nu_{a^*}) = \Delta_a^2/2$, and the bound becomes

$$\liminf_{T\to\infty} \frac{\mathbb{E}[\mathcal{R}_T]}{\ln T} \geq \sum_{a:\Delta_a>0} \frac{2}{\Delta_a}.$$

Example 2.4 (Gaussian lower bound, two arms)**.** Consider a 2-armed Gaussian bandit with $q_*(1) = 0$ and $q_*(2) = \Delta > 0$ (unit variances). The lower bound gives

$$\liminf_{T\to\infty} \frac{\mathbb{E}[\mathcal{R}_T]}{\ln T} \geq \frac{2}{\Delta}.$$

Notice that when Δ is small — the two arms are hard to distinguish — the bound is large: the algorithm must pull the sub-optimal arm many times to be sure it is not actually the best. Conversely, when Δ is large, the arms are easy to tell apart and only $O(1/\Delta)$ pulls are needed, incurring proportionally smaller regret per forced exploration. This formalises the intuition that bandit problems with nearly-equal arm values are intrinsically harder than those with clearly separated values.

Remark 2.7. The Lai–Robbins bound implies that $\Omega(\log T)$ regret is unavoidable for any algorithm that eventually learns the optimal action. Both UCB and Thompson Sampling achieve regret of order $O(\log T)$, so they are *asymptotically optimal* in the instance-dependent sense. The KL-UCB algorithm (Garivier and Cappé, 2011) and Thompson Sampling with conjugate priors are known to achieve the exact Lai–Robbins constant, not just the correct rate.

2.8 Contextual Bandits

The bandit formulation studied so far is *context-free*: the agent observes only the rewards, not any side information about the environment. In most real-world applications, however, rich context is available before each decision. A recommendation system knows the user's browsing history; a clinical trial knows the patient's age and biomarkers; an ad-placement system knows the content of the webpage being viewed. Ignoring this context leads to a one-size-fits-all policy, whereas incorporating it enables personalisation. This motivates the *contextual bandit* problem.

Definition 2.10 (Contextual bandit)**.** At each step t:

1. A context $x_t \in \mathcal{X}$ is revealed to the agent.
2. The agent selects $A_t \in \mathcal{A}$.
3. A reward R_t is observed, drawn from a distribution that depends on both x_t and A_t.

The goal is to learn a policy $\pi : \mathcal{X} \to \mathcal{A}$ that maximises the expected cumulative reward.

The key difference from the standard bandit: the optimal action may vary with the context. In a drug trial, a treatment that is best for elderly patients may be harmful for young ones. A good contextual-bandit algorithm must learn this dependence from data.

LinUCB: Upper Confidence Bounds with Linear Models

A widely used contextual-bandit algorithm is **LinUCB** (Li et al., 2010), which assumes that the expected reward is linear in the context:

$$\mathbb{E}[R_t \mid x_t, A_t = a] = x_t^\top \theta_a^*, \tag{2.15}$$

where $\theta_a^* \in \mathbb{R}^d$ is an unknown parameter vector for arm a. Given observations, the least-squares estimate $\hat{\theta}_a$ can be computed, and ridge regression provides a confidence ellipsoid around it.

Definition 2.11 (LinUCB decision rule)**.** At each step t, LinUCB selects

$$A_t = \operatorname*{argmax}_{a \in \mathcal{A}} \left[x_t^\top \hat{\theta}_a(t) + \alpha \sqrt{x_t^\top A_a(t)^{-1} x_t} \right], \tag{2.16}$$

where $A_a(t) = \lambda I + \sum_{s<t:\, A_s = a} x_s x_s^\top$ is the regularised design matrix for arm a and $\alpha > 0$ is an exploration parameter.

The term $\sqrt{x_t^\top A_a(t)^{-1} x_t}$ is the *ellipsoidal exploration bonus*: it is large when the context x_t lies in a direction that has not been well-explored for arm a, and it shrinks as more diverse contexts are observed. LinUCB enjoys a regret bound of $O(d\sqrt{T \ln T})$ under the linearity assumption (2.15), where d is the context dimension.

Practical Applications

Contextual bandits arise naturally in a number of high-stakes domains.

Online advertising. When a user visits a webpage, the system must decide which advertisement to display. The context x_t includes user features (demographics, browsing history) and page features (topic, time of day). Each arm corresponds to an ad, and the reward is a click indicator. Because user preferences vary enormously, a personalised policy can significantly outperform one that displays the same ad to everyone. LinUCB was deployed by Yahoo! for news article recommendation (Li et al., 2010), yielding substantial click-through-rate improvements over non-contextual approaches.

Clinical trials and personalised medicine. In adaptive clinical trials, a patient arrives with a covariate vector x_t (age, weight, genetic markers) and the clinician must assign a treatment from a set $\mathcal{A}$. The outcome R_t is a health measure such as reduction in tumour size or blood pressure. A contextual bandit policy learns, over the course of the trial, which treatment is best for which patient subgroup. Compared to a classical randomised controlled trial (which assigns treatments without using patient covariates), a contextual bandit can reduce the number of patients assigned to inferior treatments while still generating enough data to reach valid conclusions.

News recommendation. A news website must decide which article to show a user on the front page. The context encodes user history and article features; the reward is reading time or engagement. The exploration–exploitation trade-off is especially delicate here: showing only articles the user already prefers sacrifices the chance of discovering a new topic they might love.

From Contextual Bandits to Full RL

Contextual bandits occupy a natural middle ground between the stateless bandit problem and full reinforcement learning.

Stateless bandits	No context; one global optimal action.
Contextual bandits	Context x_t observed; optimal action depends on x_t; but actions do not influence future contexts.
Full RL (MDP)	State s_t observed; optimal action depends on s_t; and actions influence the distribution of future states.

The critical distinction is that in the contextual bandit setting, actions do not affect the distribution of future contexts — pulling an arm now has no bearing on what context appears next. In full RL (studied from Chapter 3 onward), actions shape the future, requiring the agent to reason about long-horizon consequences. Techniques developed for contextual bandits — especially the UCB-style exploration bonus and posterior-sampling approaches — have inspired algorithms for RL exploration such as UCRL (Jaksch, Ortner, and Auer, 2010) and RLSVI (Osband et al., 2016).

2.9 Empirical Comparison

To ground the theoretical discussion, consider the standard *10-armed Gaussian testbed* introduced by Sutton and Barto:

Example 2.5 (Gaussian 10-armed testbed)**.** Each arm $a = 1, \ldots, 10$ has a true value $q_*(a) \sim \mathcal{N}(0, 1)$, sampled once and then fixed. Rewards are generated as $R_t \mid A_t = a \sim \mathcal{N}(q_*(a), 1)$.

Results are averaged over many independently sampled problem instances to reduce the influence of any particular configuration of arm values.

Regret Curves

Figure 2.3 shows schematic cumulative regret curves for the algorithms studied in this chapter. Several qualitative features are worth noting.

Pure greedy. Because pure greedy locks onto an arm early, it accrues regret linearly. The slope is steep: the algorithm has a substantial probability of being stuck on a sub-optimal arm for the entire horizon.

ε-greedy. With $\varepsilon = 0.1$, ten percent of all steps are purely random, giving the algorithm ample opportunity to discover better arms. However, this randomness continues indefinitely, so the regret still grows linearly at rate $\varepsilon\,\bar{\Delta}/k$ (where $\bar{\Delta}$ is the average sub-optimality gap), just with a smaller coefficient than pure greedy.

UCB1. The logarithmic regret is clearly visible as the curve bending toward horizontal. The initial transient is somewhat expensive because UCB must try each arm at least once, and the early exploration bonuses are all large. Once every arm has been sampled, the algorithm settles into a nearly-greedy regime, exploring only when genuine uncertainty remains.

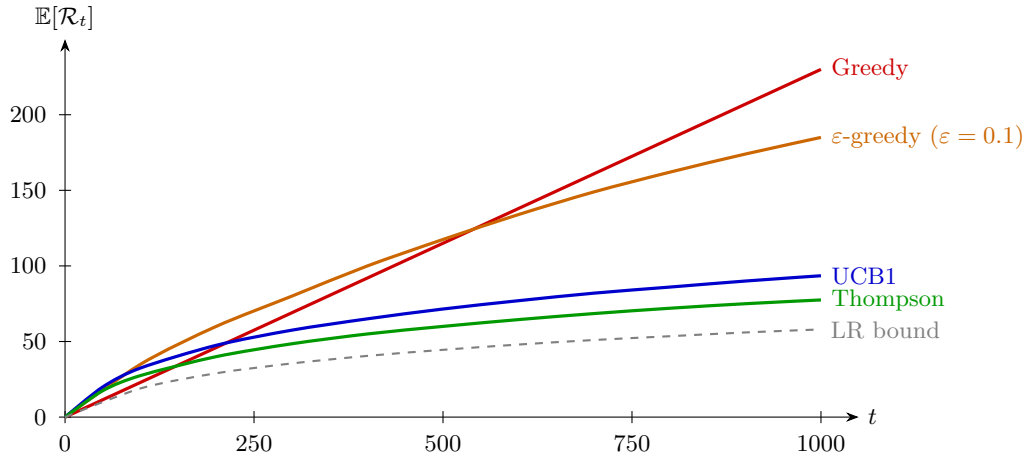


Figure 2.3: Schematic cumulative regret $\mathbb{E}[\mathcal{R}_t]$ versus time t for the algorithms studied in this chapter on the 10-armed Gaussian testbed. Pure greedy exhibits linear regret, while ε-greedy achieves lower but still linear regret. UCB1 and Thompson Sampling achieve logarithmic regret (concave curves), with Thompson Sampling tracking slightly closer to the Lai–Robbins (LR) lower bound. Actual performance varies with problem instance and parameter settings.

Thompson Sampling. Thompson Sampling typically tracks slightly below UCB1 in terms of cumulative regret, sometimes approaching the Lai–Robbins lower bound closely. This advantage is most pronounced when the prior is well-specified; misspecified priors can degrade performance.

Parameter Sensitivity

Both UCB1 and ε-greedy expose a parameter (c and ε, respectively) that controls the exploration rate.

- For UCB1, setting c too small causes under-exploration: sub-optimal arms are abandoned before their values are reliably estimated. Setting c too large causes over-exploration: the agent spends too much time on unpromising arms. The theoretically motivated choice $c = \sqrt{2}$ works well across many benchmarks, but practitioners sometimes tune c on held-out data.
- For ε-greedy, a common heuristic is to decay $\varepsilon_t = \min(1, \varepsilon_0/t)$, which achieves sub-linear (though not logarithmic) regret. Even with decay, the optimal schedule depends on problem-specific constants that are unknown in advance.
- Thompson Sampling is comparatively robust to prior misspecification when the prior is uninformative (e.g., $\text{Beta}(1, 1)$), and it does not expose an explicit exploration parameter. This makes it more "plug-and-play" in practice.

Table 2.1 summarises the key properties of all algorithms discussed in this chapter.

Table 2.1: Comparison of multi-armed bandit algorithms.

Algorithm	Regret	Parameter	Prior	Notes
Pure greedy	$\Theta(T)$	—	No	Can lock onto sub-optimal arm
ε-greedy (fixed)	$\Theta(\varepsilon T)$	ε	No	Simple; linear regret
ε-greedy (decay)	$O(\sqrt{T})$	schedule	No	Suboptimal; schedule-dependent
UCB1	$O(\log T)$	c	No	Deterministic; worst-case optimal
Thompson Sampling	$O(\log T)$	—	Yes	Randomised; often best empirically

2.10 Summary

The multi-armed bandit is one of the most thoroughly studied problems in sequential decision-making, and for good reason: it is simple enough to be analysed rigorously, yet rich enough to capture the core tension between exploration and exploitation that pervades all of reinforcement learning.

We began with the basic formulation in Section 2.1 and established the regret decomposition $\mathbb{E}[\mathcal{R}_T] = \sum_a \Delta_a \, \mathbb{E}[N_T(a)]$ in Section 2.2. This identity reframes the problem: controlling regret means controlling how often sub-optimal arms are pulled, which requires the algorithm to learn the arm ranking from noisy observations.

The ε-greedy family (Section 2.3) provides a simple baseline. By reserving a fixed fraction of steps for random exploration, the algorithm prevents locking onto a bad arm but pays the price of perpetual random exploration, yielding linear regret. The incremental update rule (Section 2.4) shows that maintaining accurate value estimates can be done in constant time per step, an important practical consideration.

Upper Confidence Bounds (Section 2.5) offer a principled solution. By adding an exploration bonus proportional to $\sqrt{\ln t / N_t(a)}$, UCB1 directs exploration toward arms that are either genuinely good or insufficiently sampled. The algorithm achieves $O(\log T)$ regret, which Theorem 2.3 makes precise through an instance-dependent bound. The regret proof illustrates a widely applicable technique: decompose bad events by arm and time, bound each event's probability via concentration inequalities, then sum the pieces.

Thompson Sampling (Section 2.6) takes a Bayesian perspective, maintaining a posterior distribution over each arm's value and sampling from it to make decisions. In the Beta-Bernoulli model, posterior updates are trivially cheap (increment two counters), and the probability-matching property ensures that exploration is calibrated to the actual posterior uncertainty. Thompson Sampling matches UCB1's $O(\log T)$ regret guarantee while often outperforming it empirically, and it extends cleanly to Gaussian bandits and other exponential-family reward models.

The Lai–Robbins lower bound (Section 2.7) closes the circle. No consistent algorithm can achieve regret growing slower than $c \ln T$ for a problem-instance-specific constant c determined by the KL divergences between arm distributions. Both UCB and Thompson Sampling achieve this rate, making them *asymptotically optimal*. The proof idea — that distinguishing a sub-optimal arm from the best requires a minimum number of observations dictated by information theory — is a prototype for lower-bound arguments throughout reinforcement learning theory.

Contextual bandits (Section 2.8) show how the bandit framework scales to real-world complexity. By conditioning decisions on an observed context, algorithms like LinUCB achieve personalised policies that outperform any context-free approach, at the cost of a higher-dimensional parameter estimation problem. Applications span online advertising,

clinical trials, and content recommendation, and the key ideas — optimism under uncertainty, posterior sampling — carry over directly from the non-contextual setting.

Looking forward, the bandit problem is not just a simplified warm-up; it is a building block. Every RL algorithm must, at its core, solve a local version of the exploration–exploitation problem at each state. The tools developed here — confidence bounds, Bayesian posteriors, regret analysis, lower bounds via information theory — will reappear in the analysis of temporal-difference learning, policy gradient methods, and modern deep RL exploration strategies in later chapters.

Exercises

Exercise 2.1. Prove that an ε-greedy strategy with fixed $\varepsilon > 0$ and at least one strictly sub-optimal action satisfies $\mathbb{E}[\mathcal{R}_T] = \Omega(T)$.

Exercise 2.2. Suppose the optimal action has the highest current estimate. Find:

(a) the probability of selecting the optimal action under ε-greedy;

(b) the probability of selecting any particular sub-optimal action;

(c) the expected number of exploratory steps in T rounds.

Exercise 2.3. Let $R_1, \ldots, R_n$ be i.i.d. rewards with mean $q_*(a)$ and variance σ_a^2. Prove that the sample average $Q_n(a) = \frac{1}{n}\sum_{i=1}^n R_i$ is unbiased and has variance σ_a^2/n.

Exercise 2.4. Derive the explicit form of the constant-step-size update:

$$Q_{n+1} = (1-\alpha)^n Q_1 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i.$$

Discuss how the weights on past observations behave as n grows.

Exercise 2.5. Construct a specific two-armed bandit instance in which the pure greedy algorithm, starting from $Q_1(1) = Q_1(2) = 0$, locks onto the sub-optimal arm forever with positive probability. Specify the reward distributions explicitly.

Exercise 2.6. Show that any algorithm satisfying $\mathbb{P}(A_t = a) \geq \delta$ for all a, all t, and some $\delta > 0$ must have $\mathbb{E}[\mathcal{R}_T] \geq cT$ for some constant $c > 0$ (assuming at least one arm is strictly sub-optimal).

Exercise 2.7 (UCB exploration bonus)**.** Starting from Hoeffding's inequality for bounded random variables, derive the UCB1 exploration bonus $\sqrt{2\ln t/N_t(a)}$. *Hint*: set the failure probability to t^{-4} and take a union bound over arms.

Exercise 2.8 (Thompson Sampling simulation)**.** Consider a 3-armed Bernoulli bandit with success probabilities $\theta_1 = 0.3$, $\theta_2 = 0.5$, $\theta_3 = 0.7$. Using Beta$(1,1)$ priors, simulate Thompson Sampling for $T = 1000$ steps. Plot the posterior distributions of $\theta_1, \theta_2, \theta_3$ at $t = 10$, $t = 100$, and $t = 1000$. Observe how the posteriors concentrate around the true values.

Exercise 2.9. For a Gaussian bandit with unit variance, the KL divergence is $\mathrm{KL}(\mathcal{N}(\mu_a, 1)\|\mathcal{N}(\mu^*, 1)) = \Delta_a^2/2$. Substitute into the Lai–Robbins bound (2.14) and interpret: how does the difficulty of distinguishing arm a from the optimum depend on Δ_a?

Exercise 2.10 (LinUCB derivation)**.** Assume rewards satisfy $R_t = x_t^\top \theta^* + \eta_t$ where $\eta_t \sim \mathcal{N}(0, \sigma^2)$ and all arms share a single parameter $\theta^* \in \mathbb{R}^d$. Show that the ridge regression estimate $\hat{\theta}_t = A_t^{-1} b_t$ (where $A_t = \lambda I + \sum_{s \le t} x_{A_s,s} x_{A_s,s}^\top$ and $b_t = \sum_{s \le t} R_s x_{A_s,s}$) is the maximum-a-posteriori estimate under a Gaussian prior $\theta^* \sim \mathcal{N}(0, \sigma^2 \lambda^{-1} I)$. Verify that the exploration bonus $\sqrt{x_t^\top A_t^{-1} x_t}$ is the posterior standard deviation of $x_t^\top \theta^*$.

Chapter 3

Markov Decision Processes and the Bellman Equation

The theory of dynamic programming rests on a single observation: the future is independent of the past, given the present.

In Chapter 2 we studied the multi-armed bandit problem, where actions have no consequence beyond the immediate reward. Real decision-making is richer: choosing to study hard today affects tomorrow's exam score, which in turn affects next year's career prospects. The agent's choices shape the world it subsequently inhabits. To reason about such problems rigorously, we need a mathematical framework that captures the interplay between states, actions, rewards, and transitions — a framework capacious enough to encode chess, robot locomotion, dialogue systems, and language-model alignment alike.

That framework is the *Markov Decision Process* (MDP). Introduced by Richard Bellman in the 1950s and formalised by Howard (1960) and Puterman (1994), the MDP has become the standard model for sequential decision-making under uncertainty. Virtually every algorithm studied in this book — dynamic programming, Monte Carlo methods, temporal-difference learning, and policy gradient methods — can be understood as a strategy for solving, or approximately solving, an MDP.

This chapter builds the MDP framework from the ground up. We begin with the Markov property, which is both a mathematical convenience and a deep claim about the structure of the world. We then define MDPs formally, introduce policies and value functions, derive the celebrated Bellman equations, and study their theoretical properties. The chapter closes with a detailed worked example that ties all concepts together.

3.1 The Markov Property

Motivation

Imagine a robot navigating a building. At any moment the robot has a position, orientation, and velocity. To decide where to move next, must it remember every step it has taken since it was switched on? Intuitively, no — knowing the current position and velocity should be enough to plan the next move. This intuition is the Markov property.

More formally, consider a sequence of random variables $S_0, A_0, R_1, S_1, A_1, R_2, \ldots$ representing the state, action, and reward at each time step. The full history up to time t is the tuple

$$H_t = (S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{t-1}, A_{t-1}, R_t, S_t).$$

In principle, the next state S_{t+1} might depend on this entire history. The Markov property asserts that only the present state S_t matters.

Definition 3.1 (Markov Property)**.** A stochastic process $(S_t)_{t\geq 0}$ satisfies the **Markov property** if, for all $t \geq 0$ and all states $s, s' \in \mathcal{S}$ and actions $a \in \mathcal{A}$,

$$\mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a, S_{t-1}, A_{t-1}, \ldots, S_0, A_0) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a). \quad (3.1)$$

When (3.1) holds, the current state S_t is said to be a **sufficient statistic** for the history H_t.

Remark 3.1. In the RL setting we additionally require the joint distribution of the next state *and* the reward to satisfy the Markov property:

$$\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid H_t) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t, A_t).$$

This is the form used throughout this book.

Examples

Example 3.1 (Weather model)**.** Suppose tomorrow's weather depends only on today's weather, not on last week's. If we encode the state as the current weather (sunny, cloudy, or rainy), the Markov property holds and the dynamics are captured by a 3×3 transition matrix. Adding more granularity — temperature, humidity, pressure — may produce an even better Markov state.

Example 3.2 (Robot navigation)**.** A mobile robot's next position depends on its current position and the velocity command it issues. If we represent the state as (x, y, θ, v)—position, heading, speed — the dynamics are approximately Markov. Crucially, a purely positional state (x, y) would *not* be Markov, because future motion also depends on the current velocity.

When does the Markov property hold?

For many natural choices of state representation the Markov property is a good approximation, not an exact truth. Weather patterns have long-range dependencies; a robot's position is influenced by joint wear. In practice, we choose the state representation to be as informative as possible while remaining computationally tractable.

When the Markov property fails — that is, when the agent can only observe a *partial* view of the true state — we enter the realm of *Partially Observable MDPs* (POMDPs). POMDPs are substantially harder to solve, and we return to them briefly in Chapter 16. For now, we assume the Markov property holds exactly.

Why does the Markov property matter?

The Markov property has a profound consequence: any optimal decision rule can be stated as a function of the current state alone. Without it, an optimal policy might need to consult an ever-growing history. This would make both analysis and computation intractable. With it, we can write down concise optimality conditions — the Bellman equations — and design efficient algorithms to find or approximate optimal behaviour.

3.2 Formal Definition of an MDP

Definition 3.2 (Markov Decision Process)**.** A **(discounted, finite) Markov Decision Process** is a tuple

$$\mathcal{M} = (\mathcal{S},\, \mathcal{A},\, p,\, \gamma), \tag{3.2}$$

where:

$\mathcal{S}$	is a finite set of **states**;
$\mathcal{A}$	is a finite set of **actions** (we write $\mathcal{A}(s)$ for the actions available in state s);
$p(s', r \mid s, a)$	is the **transition-reward kernel**: the probability that taking action a in state s leads to next state s' and reward $r \in \mathcal{R} \subset \mathbb{R}$;
$\gamma \in [0, 1)$	is the **discount factor**.

The reward set $\mathcal{R}$ is assumed finite (or countable) and bounded: there exists $R_{\max} > 0$ such that $|r| \leq R_{\max}$ for all $r \in \mathcal{R}$. The kernel p satisfies the normalisation condition

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1 \qquad \forall s \in \mathcal{S},\ \forall a \in \mathcal{A}(s). \tag{3.3}$$

The function p is the single most important object in an MDP: it encodes everything the environment can do. At each time step t, the agent observes $S_t = s$, selects $A_t = a$, and the environment samples $(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot \mid s, a)$.

Derived quantities

Working directly with the joint kernel $p(s', r \mid s, a)$ is sometimes cumbersome. Three derived quantities are more convenient.

Proposition 3.1 (Derived quantities)**.** *For any $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, $s' \in \mathcal{S}$:*

$$p(s' \mid s, a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a), \tag{3.4}$$

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} r\, p(s', r \mid s, a), \tag{3.5}$$

$$r(s, a, s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] = \sum_{r \in \mathcal{R}} r\, \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}, \quad p(s' \mid s, a) > 0. \tag{3.6}$$

Proof. **Equation** (3.4)**.** The marginal probability of transitioning to s' is obtained by summing the joint distribution over all reward values:

$$\begin{aligned} p(s' \mid s, a) &= \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a) \\ &= \sum_{r \in \mathcal{R}} \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a). \end{aligned}$$

Equation (3.5)**.** By the law of total expectation,

$$\mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r\, \mathbb{P}(R_{t+1} = r \mid S_t = s, A_t = a).$$

Marginalising over s':

$$\mathbb{P}(R_{t+1} = r \mid S_t = s, A_t = a) = \sum_{s' \in \mathcal{S}} p(s', r \mid s, a).$$

Substituting gives (3.5).

Equation (3.6)**.** By definition of conditional expectation,

$$\mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] = \sum_{r \in \mathcal{R}} r\, \mathbb{P}(R_{t+1} = r \mid S_t = s, A_t = a, S_{t+1} = s').$$

Applying Bayes' theorem,

$$\begin{aligned}&\mathbb{P}(R_{t+1} = r \mid S_t = s, A_t = a, S_{t+1} = s') \\ &\quad = \frac{\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)}{\mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a)} = \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}.\end{aligned}$$

Substituting gives (3.6). □

3.3 Episodes, Returns, and the Discount Factor

Episodic and continuing tasks

RL tasks come in two flavours. In an **episodic task**, the agent–environment interaction breaks naturally into finite segments called *episodes*: a game of chess, a robot arm reaching for an object, a conversation with a user. Each episode ends in a **terminal state**, after which the environment resets. In a **continuing task**, the interaction goes on indefinitely: a trading algorithm, a power grid controller, a recommendation engine.

The return

Whatever the task type, the agent's goal is to maximise the total reward it accumulates. We formalise this with the *return*.

Definition 3.3 (Return)**.** The **return** G_t at time t is the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots. \tag{3.7}$$

For an episodic task terminating at step T, rewards beyond T are zero, so (3.7) reduces to $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$.

The recursive property and its proof

The return satisfies a simple but crucial recursion.

Theorem 3.2 (Recursive return)**.** *For all t,*

$$G_t = R_{t+1} + \gamma G_{t+1}. \tag{3.8}$$

Proof. Starting from definition (3.7):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = R_{t+1} + \gamma\big(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots\big) = R_{t+1} + \gamma G_{t+1}.$$

□

This one-line proof is deceptively important. It says that the value of being in a state right now equals the immediate reward plus the discounted value of the next state — the backbone of every Bellman equation and temporal-difference algorithm.

Boundedness of the discounted return

Proposition 3.3 (Bounded return)**.** *If $|R_t| \leq R_{\max}$ for all t and $\gamma \in [0, 1)$, then*

$$|G_t| \leq \frac{R_{\max}}{1-\gamma}. \tag{3.9}$$

Proof. By the triangle inequality and the geometric series formula:

$$|G_t| \leq \sum_{k=0}^{\infty} \gamma^k |R_{t+k+1}| \leq R_{\max} \sum_{k=0}^{\infty} \gamma^k = \frac{R_{\max}}{1-\gamma}. \qquad \square$$

The role of the discount factor

The parameter γ interpolates between two extremes. When $\gamma = 0$, the agent is *myopic*: it cares only about the immediate reward R_{t+1}. When $\gamma \to 1$, the agent is *far-sighted*: rewards in the distant future matter almost as much as immediate ones. In practice γ is chosen close to 1 (e.g. 0.99) for tasks requiring long-term planning, and closer to 0 for tasks where recent rewards are most informative.

Discounting also has a financial interpretation: γ is the per-step interest factor, so γ^k is the present value of a reward received k steps in the future.

Example 3.3 (Worked return calculation)**.** Suppose an agent observes rewards $R_1 = 1,\ R_2 = 2,\ R_3 = -1,\ R_4 = 3$, followed by a terminal state (all subsequent rewards zero), and the discount factor is $\gamma = 0.9$.

Computing backwards from $t = 4$:

$$\begin{aligned}
G_4 &= 0 \quad \text{(terminal)},\\
G_3 &= R_4 + \gamma G_4 = 3 + 0.9 \times 0 = 3.000,\\
G_2 &= R_3 + \gamma G_3 = -1 + 0.9 \times 3 = -1 + 2.7 = 1.700,\\
G_1 &= R_2 + \gamma G_2 = 2 + 0.9 \times 1.7 = 2 + 1.53 = 3.530,\\
G_0 &= R_1 + \gamma G_1 = 1 + 0.9 \times 3.53 = 1 + 3.177 = 4.177.
\end{aligned}$$

Notice that computing backwards via the recursion (3.8) is efficient: a single pass through the trajectory suffices.

3.4 Policies

A *policy* specifies how the agent behaves. Formally, it maps states to distributions over actions.

Definition 3.4 (Policy)**.** A **stochastic policy** π is a family of probability distributions over actions, indexed by states:

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s), \qquad s \in \mathcal{S},\ a \in \mathcal{A}(s). \tag{3.10}$$

A **deterministic policy** is a special case in which there exists a function $\pi : \mathcal{S} \to \mathcal{A}$ such that $\pi(a \mid s) = \mathbb{1}\{a = \pi(s)\}$.

Remark 3.2. We use the same symbol π for both the stochastic and deterministic cases; context will always clarify which is meant. All deterministic policies are stochastic policies (with point-mass distributions), but not vice versa.

Example 3.4 (Policies in a 2×2 gridworld)**.** Consider a four-cell grid with cells $\{(0,0),(0,1),(1,0),(1,1)\}$ and actions {Up, Down, Left, Right}. A deterministic policy might map $(0,0) \mapsto$ Right, $(0,1) \mapsto$ Up, $(1,0) \mapsto$ Right, $(1,1) \mapsto$ Up. A stochastic policy might assign equal probability 0.25 to each action in every state — a uniform random walk.

Under a fixed policy π, the MDP reduces to a Markov reward process: the state transitions are governed by

$$p_\pi(s' \mid s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)\, p(s' \mid s, a),$$

and the expected reward from state s is $r_\pi(s) = \sum_a \pi(a \mid s)\, r(s,a)$.

3.5 Value Functions

Value functions measure how good it is to be in a given state, or to take a given action in a given state, when following policy π.

Definition 3.5 (State-value and action-value functions)**.** Let π be a policy. The **state-value function** $V_\pi : \mathcal{S} \to \mathbb{R}$ is

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \tag{3.11}$$

and the **action-value function** $Q_\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is

$$Q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s,\, A_t = a]. \tag{3.12}$$

Here $\mathbb{E}_\pi[\,\cdot\,]$ denotes expectation under the distribution over trajectories induced by π and p.

Relationship between V_π and Q_π

Proposition 3.4. *For any policy π and state $s \in \mathcal{S}$,*

$$V_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)\, Q_\pi(s,a). \tag{3.13}$$

Proof. Conditioning on the action taken at time t and applying the law of total expectation:

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \sum_{a \in \mathcal{A}(s)} \mathbb{P}_\pi(A_t = a \mid S_t = s)\; \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \\ &= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)\, Q_\pi(s,a). \end{aligned}$$

□

Intuitively, $V_\pi(s)$ is the policy-weighted average of the action values: the agent's expected return is the average of what each action would give, weighted by the probability the policy assigns to each.

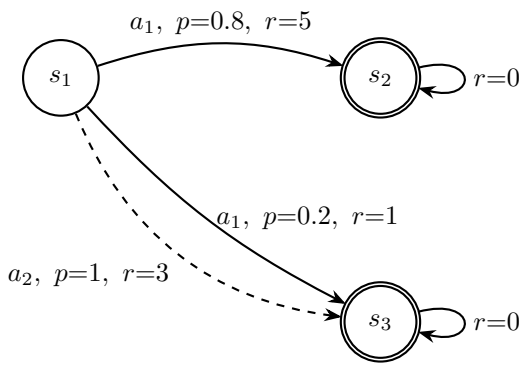


Figure 3.1: A three-state MDP. Action a_1 (solid) leads to s_2 with probability 0.8 and reward 5, or to s_3 with probability 0.2 and reward 1. Action a_2 (dashed) leads deterministically to s_3 with reward 3. States s_2 and s_3 are absorbing.

Worked example: three-state MDP

Example 3.5 (Value computation)**.** Consider the MDP shown in Figure 3.1 with three states $\{s_1, s_2, s_3\}$, two actions $\{a_1, a_2\}$ available only in s_1 (states s_2 and s_3 are absorbing with zero reward), and discount factor $\gamma = 0.5$.

Because s_2 and s_3 are absorbing with zero future reward, $V_\pi(s_2) = V_\pi(s_3) = 0$ for any policy.

For s_1, action values are computed directly from definition (3.12):

$$\begin{aligned} Q_\pi(s_1, a_1) &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s_1, A_t = a_1] \\ &= 0.8 \cdot (5 + 0.5 \cdot 0) + 0.2 \cdot (1 + 0.5 \cdot 0) = 4.0 + 0.2 = 4.2, \\ Q_\pi(s_1, a_2) &= 1.0 \cdot (3 + 0.5 \cdot 0) = 3.0. \end{aligned}$$

Suppose the policy is $\pi(a_1 \mid s_1) = 0.6$, $\pi(a_2 \mid s_1) = 0.4$. Then by (3.13):

$$V_\pi(s_1) = 0.6 \times 4.2 + 0.4 \times 3.0 = 2.52 + 1.20 = 3.72.$$

Under the greedy policy ($\pi(a_1 \mid s_1) = 1$), we get $V_\pi(s_1) = 4.2$, which is larger.

3.6 The Bellman Equation

The Bellman equation is the central identity of RL theory. It expresses the value function as a consistency condition: the value of a state must equal the expected immediate reward plus the discounted value of the successor state.

Derivation for V_π

Theorem 3.5 (Bellman expectation equation for V_π)**.** *For any policy π and any state $s \in \mathcal{S}$,*

$$V_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \big[r + \gamma V_\pi(s')\big]. \tag{3.14}$$

Proof. Begin with the definition (3.11) and apply the recursive return (3.8):

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s].$$

By linearity of expectation:

$$V_\pi(s) = \mathbb{E}_\pi[R_{t+1} \mid S_t = s] + \gamma\, \mathbb{E}_\pi[G_{t+1} \mid S_t = s].$$

Apply the tower property to the second term, conditioning on S_{t+1}:

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s] = \mathbb{E}_\pi\big[\mathbb{E}_\pi[G_{t+1} \mid S_{t+1}] \mid S_t = s\big] = \mathbb{E}_\pi[V_\pi(S_{t+1}) \mid S_t = s].$$

Therefore $V_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s]$. Expanding the expectation explicitly over actions and transitions:

$$\begin{aligned} V_\pi(s) &= \sum_a \pi(a \mid s)\, \mathbb{E}\big[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s, A_t = a\big] \\ &= \sum_a \pi(a \mid s) \sum_{s'} \sum_r p(s', r \mid s, a)\big[r + \gamma V_\pi(s')\big]. \end{aligned}$$

□

Bellman equation for Q_π

Proposition 3.6 (Bellman equation for Q_π)**.**

$$Q_\pi(s,a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a)\Big[r + \gamma \sum_{a' \in \mathcal{A}(s')} \pi(a' \mid s')\, Q_\pi(s', a')\Big]. \tag{3.15}$$

Proof. Starting from $Q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$ and applying the tower property conditioned on (S_{t+1}, R_{t+1}), then using $\mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s'] = V_\pi(s') = \sum_{a'} \pi(a' \mid s') Q_\pi(s', a')$. □

Backup diagram

The Bellman equation has a natural graphical representation known as a *backup diagram*, shown in Figure 3.2. Open circles represent states; filled circles represent state-action pairs. The root is the state s; the diagram fans out to actions under π, then further to successor states under p.

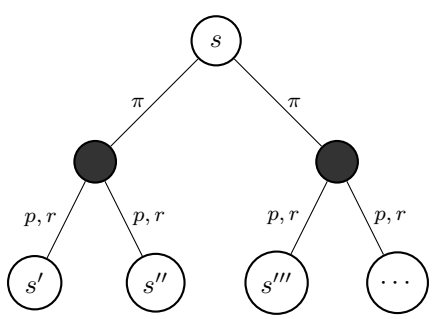


Figure 3.2: Backup diagram for the Bellman equation for $V_\pi(s)$. Open circles are states; filled circles are state-action pairs. Edges labelled π show action selection; edges labelled p, r show environment transitions with associated reward.

Matrix form for finite MDPs

When $\mathcal{S}$ is finite with $n = |\mathcal{S}|$ states, equation (3.14) can be written in matrix-vector form. Define:

- $\mathbf{v}_\pi \in \mathbb{R}^n$ with $[\mathbf{v}_\pi]_s = V_\pi(s)$;
- $\mathbf{r}_\pi \in \mathbb{R}^n$ with $[\mathbf{r}_\pi]_s = r_\pi(s) = \sum_a \pi(a \mid s) r(s, a)$;
- $\mathbf{P}_\pi \in \mathbb{R}^{n \times n}$ with $[\mathbf{P}_\pi]_{s,s'} = \sum_a \pi(a \mid s) p(s' \mid s, a)$.

Then the Bellman equation becomes the linear system

$$\mathbf{v}_\pi = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_\pi, \tag{3.16}$$

with unique solution

$$\mathbf{v}_\pi = (I - \gamma \mathbf{P}_\pi)^{-1} \mathbf{r}_\pi. \tag{3.17}$$

The matrix $(I - \gamma \mathbf{P}_\pi)$ is invertible because $\mathbf{P}_\pi$ is a stochastic matrix (spectral radius 1), so all eigenvalues of $\gamma \mathbf{P}_\pi$ have modulus at most $\gamma < 1$, making all eigenvalues of $(I - \gamma \mathbf{P}_\pi)$ bounded away from zero.

Example 3.6 (Solving the Bellman system)**.** Return to Example 3.5 but now suppose both states s_2 and s_3 loop back to s_1 with reward 0, and s_1 uses the policy $\pi(a_1|s_1) = 0.6$, $\pi(a_2|s_1) = 0.4$, $\gamma = 0.5$.

The transition matrix under π from s_1 is:

$$p_\pi(s_2 \mid s_1) = 0.6 \times 0.8 = 0.48, \quad p_\pi(s_3 \mid s_1) = 0.6 \times 0.2 + 0.4 \times 1 = 0.52.$$

Expected reward: $r_\pi(s_1) = 0.6 \times 4.2 + 0.4 \times 3.0 = 3.72$ (as before). For simplicity set $V(s_2) = V(s_3) = 0$ (absorbing). The Bellman equation for s_1 is then:

$$V_\pi(s_1) = 3.72 + 0.5(0.48 \cdot 0 + 0.52 \cdot 0) = 3.72.$$

This matches the direct calculation in Example 3.5.

3.7 Optimal Value Functions and the Bellman Optimality Equation

Optimal value functions

Definition 3.6 (Optimal value functions)**.** The **optimal state-value function** is

$$V_*(s) = \sup_\pi V_\pi(s), \qquad s \in \mathcal{S}, \tag{3.18}$$

and the **optimal action-value function** is

$$Q_*(s, a) = \sup_\pi Q_\pi(s, a), \qquad s \in \mathcal{S},\ a \in \mathcal{A}(s). \tag{3.19}$$

A policy π_* is **optimal** if $V_{\pi_*}(s) = V_*(s)$ for all $s \in \mathcal{S}$.

Policy ordering and existence of an optimal policy

Definition 3.7 (Policy ordering)**.** We write $\pi' \succeq \pi$ and say π' is **at least as good as** π if

$$V_{\pi'}(s) \geq V_\pi(s) \qquad \forall s \in \mathcal{S}.$$

Theorem 3.7 (Existence of an optimal policy)**.** *For any finite MDP there exists at least one deterministic optimal policy π_* such that $V_{\pi_*}(s) = V_*(s)$ for all $s \in \mathcal{S}$.*

Proof sketch. The key observation is that a policy is optimal if and only if, at every state s, it acts greedily with respect to V_*:

$$\pi_*(a \mid s) > 0 \implies a \in \operatorname*{argmax}_{a' \in \mathcal{A}(s)} Q_*(s, a').$$

Such a deterministic policy always exists (simply break ties arbitrarily). That $V_{\pi_*} = V_*$ then follows from the Bellman optimality equations derived below. □

The optimal value functions satisfy their own Bellman equations, obtained by replacing the policy-weighted sum over actions with a maximum.

Theorem 3.8 (Bellman optimality equations)**.**

$$V_*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a)\big[r + \gamma V_*(s')\big], \tag{3.20}$$

$$Q_*(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a)\Big[r + \gamma \max_{a' \in \mathcal{A}(s')} Q_*(s', a')\Big]. \tag{3.21}$$

Proof. For (3.20): since $V_*(s) = \max_\pi V_\pi(s)$, the Bellman equation (3.14) for the optimal policy π_* gives

$$V_*(s) = \sum_a \pi_*(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V_*(s')].$$

Because π_* is greedy w.r.t. V_*, the weighted sum over a equals the maximum over a, yielding (3.20). Equation (3.21) follows analogously from the Q-version of the Bellman equation. □

Remark 3.3. The Bellman optimality equations are *non-linear* in V_* (due to the max operator) and cannot be solved by simple matrix inversion. Dynamic programming and related algorithms are needed; see Chapter 4.

3.8 The Bellman Operator

The Bellman equations can be studied as fixed-point problems of linear operators, a perspective that explains why algorithms for solving them converge.

Definition 3.8 (Bellman operators)**.** Let $\mathcal{B}(\mathcal{S})$ denote the Banach space of bounded functions $V : \mathcal{S} \to \mathbb{R}$ equipped with the sup-norm $\|V\|_\infty = \max_{s \in \mathcal{S}} |V(s)|$.

The **Bellman policy operator** $\mathcal{T}_\pi : \mathcal{B}(\mathcal{S}) \to \mathcal{B}(\mathcal{S})$ is

$$(\mathcal{T}_\pi V)(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]. \tag{3.22}$$

The **Bellman optimality operator** $\mathcal{T}_* : \mathcal{B}(\mathcal{S}) \to \mathcal{B}(\mathcal{S})$ is

$$(\mathcal{T}_* V)(s) = \max_{a \in \mathcal{A}(s)} \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]. \tag{3.23}$$

With this notation, the Bellman equations read $V_\pi = \mathcal{T}_\pi V_\pi$ and $V_* = \mathcal{T}_* V_*$: value functions are *fixed points* of their respective Bellman operators.

Theorem 3.9 (γ-contraction)**.** *Both $\mathcal{T}_\pi$ and $\mathcal{T}_*$ are γ-contractions in the sup-norm: for any $V, W \in \mathcal{B}(\mathcal{S})$,*

$$\|\mathcal{T}_\pi V - \mathcal{T}_\pi W\|_\infty \le \gamma \|V - W\|_\infty, \tag{3.24}$$

$$\|\mathcal{T}_* V - \mathcal{T}_* W\|_\infty \le \gamma \|V - W\|_\infty. \tag{3.25}$$

Proof. We prove (3.24); the proof of (3.25) is analogous. Fix $s \in \mathcal{S}$:

$$\begin{aligned}
|(\mathcal{T}_\pi V)(s) - (\mathcal{T}_\pi W)(s)| &= \Big| \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\, \gamma \big(V(s') - W(s')\big) \Big| \\
&\le \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\, \gamma\, |V(s') - W(s')| \\
&\le \gamma\, \|V - W\|_\infty \underbrace{\sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)}_{=1}.
\end{aligned}$$

Taking the maximum over s gives (3.24).

For (3.25), use the inequality $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$ and proceed identically. □

Corollary 3.10 (Banach fixed-point theorem)**.** *Since $\mathcal{B}(\mathcal{S})$ is a complete metric space (it is finite-dimensional) and $\mathcal{T}_\pi$, $\mathcal{T}_*$ are contractions ($\gamma < 1$), the Banach fixed-point theorem guarantees:*

1. *Each operator has a* unique *fixed point: V_π and V_* respectively.*
2. *Iterating the operator from any initial $V^{(0)} \in \mathcal{B}(\mathcal{S})$:*
$$V^{(k+1)} = \mathcal{T} V^{(k)}$$
converges to the fixed point at a geometric rate:
$$\|V^{(k)} - V_\pi\|_\infty \le \gamma^k \|V^{(0)} - V_\pi\|_\infty \to 0.$$

This corollary is the theoretical backbone of *policy evaluation* in dynamic programming (Chapter 4) and of TD(0) learning (Chapter 6): both can be understood as iterative application of (an approximation to) the Bellman operator.

3.9 Worked Example: The Student MDP

We now work through a complete example inspired by the student MDP of Sutton and Barto (2018).

Example 3.7 (Student MDP)**.** A student can be in one of four states:

- s_1: **Class** — attending lecture.
- s_2: **Facebook** — browsing social media.
- s_3: **Study** — studying for an exam.
- s_4: **Sleep** — sleeping (terminal / absorbing state).

The actions and transition dynamics are given in Table 3.1, and the MDP is depicted in Figure 3.3. We use discount factor $\gamma = 0.9$.

Table 3.1: Transition dynamics of the Student MDP. Each row gives the action taken in the current state, the next state, its probability, and the reward received.

State	Action	Next state	Prob. p	Reward r
s_1 (Class)	attend	s_3 (Study)	1.0	-2
s_1 (Class)	distract	s_2 (Facebook)	1.0	-1
s_2 (Facebook)	quit	s_1 (Class)	1.0	-1
s_2 (Facebook)	keep	s_2 (Facebook)	1.0	-1
s_3 (Study)	pass	s_4 (Sleep)	1.0	$+10$
s_3 (Study)	study more	s_3 (Study)	1.0	-2

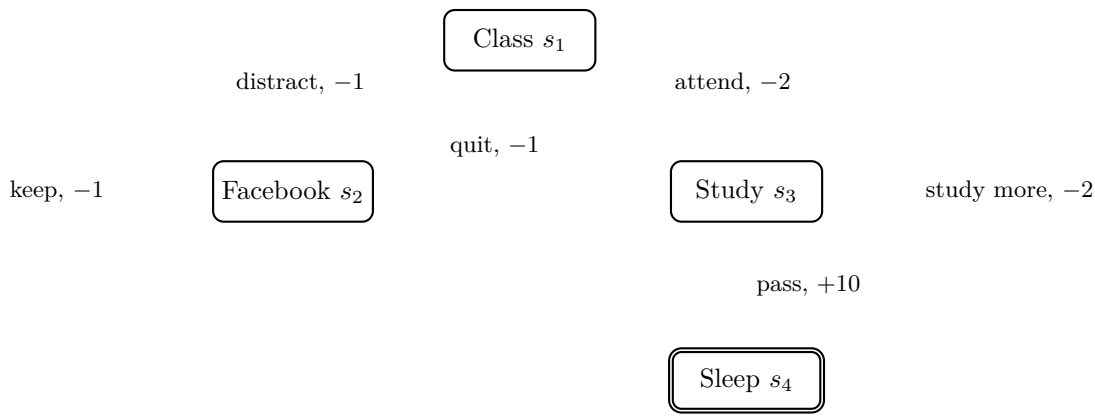


Figure 3.3: The Student MDP. Edges are labelled with the action and the immediate reward. The terminal state Sleep (s_4, double border) is absorbing.

Computing value functions under the optimal policy. The optimal strategy is clearly to attend class and then pass the exam: attend s_1, reach s_3, immediately pass to s_4.

Under this policy π_*:

$$\begin{aligned}
V_*(s_4) &= 0 \quad \text{(absorbing)}, \\
V_*(s_3) &= r(\text{pass}) + \gamma V_*(s_4) = 10 + 0.9 \times 0 = 10.0, \\
V_*(s_1) &= r(\text{attend}) + \gamma V_*(s_3) = -2 + 0.9 \times 10 = -2 + 9 = 7.0.
\end{aligned}$$

So the optimal state value of Class is 7.0.

Computing action values. At s_1:

$$\begin{aligned}
Q_*(s_1, \text{attend}) &= -2 + 0.9 \times 10 = 7.0, \\
Q_*(s_1, \text{distract}) &= -1 + 0.9 \times V_*(s_2).
\end{aligned}$$

For $V_*(s_2)$ we note that from Facebook the optimal action is *quit* (returning to Class, which has value 7.0), so:

$$V_*(s_2) = -1 + 0.9 \times 7.0 = -1 + 6.3 = 5.3.$$

Therefore:

$$Q_*(s_1, \text{distract}) = -1 + 0.9 \times 5.3 = -1 + 4.77 = 3.77.$$

Since $Q_*(s_1, \text{attend}) = 7.0 > 3.77 = Q_*(s_1, \text{distract})$, the greedy policy indeed chooses "attend", confirming our earlier reasoning.

Verifying the Bellman optimality equation at s_2. The Bellman optimality equation (3.20) says:

$$V_*(s_2) = \max\{Q_*(s_2, \text{quit}),\ Q_*(s_2, \text{keep})\}.$$

We have:

$$\begin{aligned} Q_*(s_2, \text{quit}) &= -1 + 0.9 \times 7.0 = 5.3, \\ Q_*(s_2, \text{keep}) &= -1 + 0.9 \times V_*(s_2) = -1 + 0.9 \times 5.3 = 3.77. \end{aligned}$$

Indeed $\max\{5.3, 3.77\} = 5.3 = V_*(s_2)$. ✓

3.10 Summary

- The **Markov property** asserts that the future is independent of the past given the present state. It is the foundation that makes RL tractable.
- A **Markov Decision Process** $(\mathcal{S}, \mathcal{A}, p, \gamma)$ is the standard framework for sequential decision-making. The kernel $p(s', r \mid s, a)$ encodes all dynamics.
- The **return** $G_t = \sum_k \gamma^k R_{t+k+1}$ captures cumulative discounted future reward. Its recursive property $G_t = R_{t+1} + \gamma G_{t+1}$ is the seed of the Bellman equations.
- **Value functions** $V_\pi(s)$ and $Q_\pi(s, a)$ measure expected return under policy π. They are related by $V_\pi(s) = \sum_a \pi(a|s) Q_\pi(s, a)$.
- The **Bellman expectation equations** express value functions as self-consistent fixed points. They can be solved exactly via linear algebra for small MDPs.
- The **Bellman optimality equations** characterise V_* and Q_* via a max operator. Their solutions yield the optimal policy directly.
- The **Bellman operators** $\mathcal{T}_\pi$ and $\mathcal{T}_*$ are γ-contractions in the sup-norm. By the Banach fixed-point theorem, iterating either operator from any starting point converges geometrically to the unique fixed point. This is the theoretical bedrock of dynamic programming and TD learning.

Exercises

Exercise 3.1. Give an example of a task where the raw sensor observation (e.g. a single pixel) is *not* a Markov state, but an augmented state (e.g. the last k frames) is. Explain why the augmented state satisfies the Markov property.

Exercise 3.2. Consider a continuing task in which the agent always receives reward $c > 0$ at every step, for some constant c, and $\gamma \in [0, 1)$.

(a) Compute G_t in closed form.

(b) Show that $G_t = c/(1 - \gamma)$.

(c) Explain why the condition $\gamma < 1$ is essential for the return to be finite.

(d) What happens when $\gamma = 1$?

Exercise 3.3. Using the definitions in Proposition 3.1, verify that

$$r(s,a) = \sum_{s' \in \mathcal{S}} p(s' \mid s,a)\, r(s,a,s').$$

This shows that the three-argument expected reward $r(s,a,s')$ is more informative than $r(s,a)$.

Exercise 3.4. In the Student MDP (Example 3.7), suppose the student follows a policy where, in state s_3, they study more with probability 0.5 and pass with probability 0.5. Use $\gamma = 0.9$.

(a) Write the Bellman equation for $V_\pi(s_3)$ under this mixed policy.

(b) Solve for $V_\pi(s_3)$.

(c) Compare with $V_*(s_3) = 10$ and explain the difference.

Exercise 3.5. Prove the Bellman equation for Q_π (Proposition 3.6) in full detail, starting from the definition $Q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$ and the recursive return (3.8).

Exercise 3.6. Consider a two-state MDP with $\mathcal{S} = \{s_1, s_2\}$, a single action, transition matrix

$$P_\pi = \begin{pmatrix} 0.3 & 0.7 \\ 0.6 & 0.4 \end{pmatrix},$$

expected reward vector $\mathbf{r}_\pi = (2,5)^\top$, and $\gamma = 0.8$.

(a) Write the matrix Bellman equation (3.16).

(b) Compute $(I - \gamma P_\pi)$ explicitly.

(c) Solve for $\mathbf{v}_\pi$ using the formula (3.17).

Exercise 3.7. Complete the proof of Theorem 3.9 for the optimality operator $\mathcal{T}_*$. *Hint*: use the identity $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$.

Exercise 3.8. Starting from $V^{(0)} \equiv 0$ and applying $\mathcal{T}_*$ iteratively with $\gamma = 0.9$ and $R_{\max} = 1$:

(a) Give an upper bound on $\|V^{(k)} - V_*\|_\infty$ in terms of k and γ.

(b) How many iterations are needed to guarantee $\|V^{(k)} - V_*\|_\infty \le 0.01$?

Exercise 3.9. Give an example of a finite MDP that has infinitely many optimal policies but a unique optimal value function V_*. Verify that all optimal policies achieve V_*.

Exercise 3.10. Consider a 1×3 grid (cells 1, 2, 3). The agent can move left or right. Cell 3 is the goal (reward $+1$, terminal). The agent does *not* observe its current cell; it only receives a signal indicating whether it just bumped into a wall.

(a) Explain why the observation is not a Markov state.

(b) Define an augmented state (using the observation history) that restores the Markov property.

(c) Describe an optimal policy in terms of this augmented state.

Chapter 4

Dynamic Programming

Give me a place to stand on, and I will move the Earth. — Archimedes

Dynamic programming (DP) occupies a unique position in reinforcement learning: it solves the central problem of sequential decision-making *exactly*, but only when the agent possesses a perfect mathematical model of the environment. In this chapter we treat DP both as a collection of practical algorithms and as the theoretical bedrock upon which the rest of this book rests. Every algorithm we encounter in subsequent chapters — Monte Carlo methods, temporal-difference learning, policy gradient — can be understood as an approximation to DP that replaces exact model-based computation with sampling or function approximation.

Formally, we assume throughout this chapter that the environment is a finite Markov decision process $(\mathcal{S}, \mathcal{A}, p, \gamma)$ as defined in Chapter 3. The transition kernel $p(s', r \mid s, a)$ is known in full, the state and action spaces are finite, and the discount factor satisfies $\gamma \in [0, 1)$. Under these conditions DP finds the optimal value function and an optimal policy in time polynomial in $|\mathcal{S}|$ and $|\mathcal{A}|$.

The fundamental insight underlying all DP algorithms is the *Bellman equation*: optimal value functions satisfy a recursive consistency condition, and DP exploits this structure by iterating a carefully defined operator until convergence. We present four main algorithms — iterative policy evaluation, policy improvement, policy iteration, and value iteration — and unify them under the banner of *Generalized Policy Iteration* (GPI).

4.1 Policy Evaluation (Prediction)

The first and most basic task is *policy evaluation*: given a fixed policy π, compute its state-value function v^π. This is a prediction problem — we are not yet trying to find a *good* policy, only to understand how well the given one performs.

The Bellman Operator for a Fixed Policy

Recall from Chapter 3 that v^π is the unique solution to the *Bellman expectation equations*

$$v^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r} p(s', r \mid s, a) \big[r + \gamma\, v^\pi(s') \big], \qquad s \in \mathcal{S}. \tag{4.1}$$

This is a linear system of $|\mathcal{S}|$ equations in $|\mathcal{S}|$ unknowns. In principle one could solve it directly by Gaussian elimination, but for the large state spaces common in RL that is computationally prohibitive. Instead, DP solves (4.1) by *successive approximation*: define the **Bellman policy operator**

$$(T^\pi v)(s) = \sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{s'\in\mathcal{S}} \sum_{r} p(s', r \mid s, a)\big[r + \gamma\, v(s')\big], \tag{4.2}$$

and iterate $v_{k+1} = T^\pi v_k$ starting from any bounded initialisation v_0.

Theorem 4.1 (Contraction of T^π)**.** *The operator T^π is a contraction on the space of bounded functions $\mathcal{S} \to \mathbb{R}$ equipped with the sup-norm $\|v\|_\infty = \max_{s\in\mathcal{S}} |v(s)|$. Specifically, for any two bounded functions $u, v\colon \mathcal{S} \to \mathbb{R}$,*

$$\|T^\pi u - T^\pi v\|_\infty \le \gamma \|u - v\|_\infty.$$

Consequently, T^π has a unique fixed point v^π, and the iterates $v_{k+1} = T^\pi v_k$ converge to v^π from any starting point v_0.

Proof. Fix any state $s \in \mathcal{S}$. Then

$$\begin{aligned}
\big|(T^\pi u)(s) - (T^\pi v)(s)\big| &= \Big|\sum_{a} \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\, \gamma\big[u(s') - v(s')\big]\Big| \\
&\le \sum_{a} \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\, \gamma\, \|u - v\|_\infty \\
&= \gamma \|u - v\|_\infty,
\end{aligned}$$

where the inequality uses the triangle inequality and the fact that the weights $\pi(a \mid s)\, p(s', r \mid s, a)$ are non-negative and sum to one. Taking the supremum over s gives $\|T^\pi u - T^\pi v\|_\infty \le \gamma \|u - v\|_\infty$. Since $\gamma < 1$, the operator is a strict contraction. By the Banach fixed-point theorem it has a unique fixed point, and the iterates converge at a geometric rate with ratio γ. □

Remark 4.1. The error after k iterations satisfies $\|v_k - v^\pi\|_\infty \le \gamma^k \|v_0 - v^\pi\|_\infty$. For $\gamma = 0.9$ and a desired accuracy of $\varepsilon = 10^{-3}$, one needs roughly $k \ge \log(10^{-3}/\|v_0 - v^\pi\|_\infty)/\log(0.9)$ iterations — independent of $|\mathcal{S}|$.

Algorithm and Stopping Criterion

In practice, one sweeps through all states and applies the update simultaneously (synchronous DP) or in sequence (see Section 4.6). The algorithm halts when the maximum change across all states falls below a threshold θ.

Stopping criterion. The threshold θ controls the trade-off between accuracy and computation. A common practical choice is $\theta = 10^{-4}$ or $\theta = 10^{-6}$. The *a posteriori* error bound is

$$\|V - v^\pi\|_\infty \le \frac{\theta\, \gamma}{1 - \gamma},$$

which follows from the geometric series argument applied to the contraction.

Algorithm 3: Iterative Policy Evaluation

Input: Policy π, MDP dynamics p, discount γ, threshold $\theta > 0$

Initialise $V(s) \leftarrow 0$ for all $s \in \mathcal{S}$; set $V(\text{terminal}) \leftarrow 0$;
repeat
  $\Delta \leftarrow 0$;
  for *each* $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$;
    $V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$;
    $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$;
until $\Delta < \theta$;

Output: $V \approx v^\pi$

Worked Example: Gridworld

Example 4.1 (4×4 Gridworld)**.** Consider the standard 4×4 gridworld with 16 states arranged in a grid. States s_1 (top-left) and s_{16} (bottom-right) are terminal. From each non-terminal state the agent may move UP, DOWN, LEFT, or RIGHT; attempted moves off the grid leave the state unchanged. Each transition yields reward -1 (the agent wants to reach a terminal state as quickly as possible). We evaluate the *equiprobable random policy* $\pi(a \mid s) = 0.25$ for all a and $\gamma = 1$ (undiscounted episodic task).

The following table shows V after several sweeps.

Iteration	V values (row by row, terminal states shown as 0)			
$k = 0$	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
$k = 1$	0	-1	-1	-1
	-1	-1	-1	-1
	-1	-1	-1	-1
	-1	-1	-1	0
$k = 2$	0	-1.75	-2	-2
	-1.75	-2	-2	-2
	-2	-2	-2	-1.75
	-2	-2	-1.75	0
$k \to \infty$	0	-14	-20	-22
	-14	-18	-20	-20
	-20	-20	-18	-14
	-22	-20	-14	0

The converged values reflect the expected number of steps to reach a terminal state under the random policy. A state like the one diagonally opposite both terminals (-22) is hardest to escape; states adjacent to terminals have $v^\pi \approx -14$. This example illustrates how policy evaluation propagates information *backwards* from terminal states through the grid.

Converged v^π (random policy, $\gamma = 1$)

0	−14	−20	−22
−14	−18	−20	−20
−20	−20	−18	−14
−22	−20	−14	0

Figure 4.1: State-value function v^π for the equiprobable random policy on the 4×4 gridworld after convergence of iterative policy evaluation. Shaded cells are terminal states with value 0. Values represent the expected cumulative (undiscounted) reward, i.e. the negative of the expected number of steps to reach a terminal.

4.2 Policy Improvement

Once we have computed v^π for some policy π, a natural question is: can we construct a strictly better policy? The *policy improvement theorem* answers this definitively.

The Action-Value Function

Given v^π, the *action-value function* $q^\pi(s, a)$ captures the value of taking action a in state s and then following π:

$$q^\pi(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r} p(s', r \mid s, a)\big[r + \gamma\, v^\pi(s')\big]. \tag{4.3}$$

In the model-based setting, q^π can be computed directly from v^π using the one-step lookahead in (4.3). The key observation is that $q^\pi(s, a) \geq v^\pi(s)$ means that taking action a *once* in state s and then reverting to π is at least as good as following π from the outset.

Theorem 4.2 (Policy Improvement Theorem)**.** *Let π and π' be two deterministic policies such that for all $s \in \mathcal{S}$,*

$$q^\pi\big(s, \pi'(s)\big) \geq v^\pi(s). \tag{4.4}$$

Then π' is at least as good as π:

$$v^{\pi'}(s) \geq v^\pi(s), \qquad \forall s \in \mathcal{S}.$$

Moreover, if the inequality in (4.4) is strict for at least one state, then $v^{\pi'}(s) > v^\pi(s)$ for at least one state.

Proof. From (4.4) and the definition of q^π:

$$v^\pi(s) \leq q^\pi\big(s, \pi'(s)\big) = \sum_{s', r} p(s', r \mid s, \pi'(s))\big[r + \gamma\, v^\pi(s')\big] = (T^{\pi'} v^\pi)(s).$$

Hence $v^\pi \leq T^{\pi'} v^\pi$ pointwise. The operator $T^{\pi'}$ is *monotone*: if $u \leq v$ pointwise then $(T^{\pi'} u)(s) \leq (T^{\pi'} v)(s)$ for all s, as the weights $p(s', r \mid s, \pi'(s))$ are non-negative. Applying $T^{\pi'}$ repeatedly and using monotonicity,

$$v^\pi \leq T^{\pi'} v^\pi \leq (T^{\pi'})^2 v^\pi \leq \cdots \leq (T^{\pi'})^n v^\pi \qquad \forall n \geq 1.$$

By Theorem 4.1 (with π' in place of π), $(T^{\pi'})^n v^\pi \to v^{\pi'}$ as $n \to \infty$. Since the inequality holds for every finite n, it is preserved in the limit:

$$v^\pi(s) \le v^{\pi'}(s), \qquad \forall s \in \mathcal{S}. \qquad \square$$

Greedy Policy Construction

The *greedy policy* π' with respect to v^π satisfies the improvement condition (4.4) maximally:

$$\pi'(s) \in \operatorname*{argmax}_{a \in \mathcal{A}} q^\pi(s,a) = \operatorname*{argmax}_{a \in \mathcal{A}} \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v^\pi(s')\big]. \tag{4.5}$$

By construction, $q^\pi(s, \pi'(s)) = \max_a q^\pi(s,a) \ge v^\pi(s)$ for all s, so the theorem applies.

When improvement stops. Suppose the greedy improvement produces $\pi' = \pi$—that is, the current policy is already greedy with respect to its own value function. Then $v^\pi(s) = \max_a q^\pi(s,a)$ for all s, which is precisely the *Bellman optimality equation*:

$$v^\pi(s) = \max_{a \in \mathcal{A}} \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v^\pi(s')\big] = v^*(s).$$

Hence π is an optimal policy. This observation drives the policy iteration algorithm.

4.3 Policy Iteration

Policy iteration alternates between the two steps developed above: evaluate the current policy to convergence, then greedily improve it.

Finite Convergence

Theorem 4.3 (Finite convergence of policy iteration)**.** *Policy iteration terminates in a finite number of outer iterations.*

Proof. Each policy π is a deterministic mapping from $\mathcal{S}$ to $\mathcal{A}$. There are at most $|\mathcal{A}|^{|\mathcal{S}|}$ such mappings — a finite number. The policy improvement theorem guarantees that whenever $\pi' \ne \pi$ is produced, $v^{\pi'} \ge v^\pi$ with strict inequality somewhere. Therefore the sequence of policies generated by policy iteration is strictly improving in value (until it stabilises), which prevents any policy from being visited twice. Since there are finitely many policies, the algorithm must terminate. $\square$

Remark 4.2. In practice, policy iteration converges in very few iterations — often fewer than ten — even for large MDPs. Each iteration requires solving a linear system (or running iterative evaluation to convergence), which dominates the computation.

Example 4.2 (Policy Iteration on the Gridworld)**.** Returning to the 4×4 gridworld of Example 4.1, we start policy iteration from the equiprobable random policy.

- **Iteration 0**: Evaluate the random policy (as computed above). The greedy improvement immediately produces a much better policy: in every state, move toward the nearest terminal.

Algorithm 4: Policy Iteration

Input: MDP dynamics p, discount γ, threshold $\theta > 0$

Initialise $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \leftarrow 0$;
repeat
  `// Policy Evaluation`
  repeat
    $\Delta \leftarrow 0$;
    for *each* $s \in \mathcal{S}$ **do**
      $v \leftarrow V(s)$;
      $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))\big[r + \gamma V(s')\big]$;
      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$;
  until $\Delta < \theta$;
  `// Policy Improvement`
  POLICY-STABLE $\leftarrow$ `true`;
  for *each* $s \in \mathcal{S}$ **do**
    $a_{\text{old}} \leftarrow \pi(s)$;
    $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$;
    if $a_{old} \neq \pi(s)$ **then**
      POLICY-STABLE $\leftarrow$ `false`
until *POLICY-STABLE* = `true`;

Output: $V \approx v^*$, $\pi \approx \pi^*$

- **Iteration 1**: The improved policy already looks near-optimal. Re-evaluation reveals that it achieves $v^{\pi_1}(s) = -$ (Manhattan distance to nearest terminal) for all s, exactly.
- **Iteration 2**: The greedy policy with respect to v^{π_1} is identical to π_1. Policy iteration halts.

Thus, policy iteration converges in only 2 outer iterations on this small problem, even though iterative evaluation at each step requires many inner sweeps.

4.4 Value Iteration

Policy iteration can be expensive if policy evaluation requires many sweeps to converge before each improvement step. *Value iteration* avoids this by interleaving a single evaluation step with an improvement step, effectively applying the *Bellman optimality operator* directly.

The Bellman Optimality Operator

Define T^* by

$$(T^* v)(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} \sum_{r} p(s', r \mid s, a)\big[r + \gamma\, v(s')\big]. \tag{4.6}$$

The optimal value function v^* is the unique fixed point of T^*: $T^* v^* = v^*$.

Theorem 4.4 (Contraction of T^*)**.** *T^* is a γ-contraction on $(B(\mathcal{S}), \|\cdot\|_\infty)$:*

$$\|T^*u - T^*v\|_\infty \le \gamma \, \|u - v\|_\infty.$$

Consequently, the iterates $v_{k+1} = T^* v_k$ converge to v^* from any v_0.

Proof. For any $s \in \mathcal{S}$,

$$\begin{aligned}
(T^*u)(s) - (T^*v)(s) &= \max_a \sum_{s',r} p(s', r \mid s, a)\, \gamma\, u(s') - \max_a \sum_{s',r} p(s', r \mid s, a)\, \gamma\, v(s') \\
&\le \max_a \sum_{s',r} p(s', r \mid s, a)\, \gamma \big[u(s') - v(s')\big] \\
&\le \gamma \, \|u - v\|_\infty,
\end{aligned}$$

using $\max f - \max g \le \max(f - g)$ and non-negativity of the transition weights. Symmetrically, $(T^*v)(s) - (T^*u)(s) \le \gamma \|u - v\|_\infty$. Taking the sup norm gives the result. □

Algorithm

Algorithm 5: Value Iteration

Input: MDP dynamics p, discount γ, threshold $\theta > 0$

Initialise $V(s) \leftarrow 0$ for all $s \in \mathcal{S}$; $V(\text{terminal}) \leftarrow 0$;
repeat
  $\Delta \leftarrow 0$;
  for *each* $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$;
    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, V(s')\big]$;
    $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$;
until $\Delta < \theta$;
Output a deterministic greedy policy π w.r.t. V;

Output: $V \approx v^*$, $\pi \approx \pi^*$

Convergence rate. Since T^* is a γ-contraction, the error satisfies $\|v_k - v^*\|_\infty \le \gamma^k \|v_0 - v^*\|_\infty$. For a desired accuracy ε, the number of iterations required is $k \ge \log(\varepsilon / \|v_0 - v^*\|_\infty) / \log \gamma$, which grows as $O\big((1-\gamma)^{-1} \log(1/\varepsilon)\big)$. Smaller γ means faster convergence but heavier discounting of future rewards.

Relationship to Policy Iteration

Value iteration can be viewed as *policy iteration with truncated evaluation*: instead of running evaluation to convergence, perform exactly one sweep of T^π before improving. Since $T_k^\pi = T^*$ when π_k is greedy with respect to v_k (by definition of the max in (4.6)), each value-iteration sweep is simultaneously one step of both evaluation and improvement.

More generally, if policy evaluation is stopped after m sweeps (rather than 1 or ∞), we obtain a family of algorithms that interpolate between pure value iteration ($m = 1$) and policy iteration ($m \to \infty$).

Example 4.3 (Value Iteration on the Gridworld)**.** On the same 4×4 gridworld, we apply value iteration starting from $V \equiv 0$.

After each sweep the value function rapidly reflects the structure of optimal behaviour. After approximately 10–15 sweeps with threshold $\theta = 10^{-4}$, the algorithm converges and the greedy policy extracted from V is identical to the optimal policy found by policy iteration. Value iteration requires fewer total sweeps than policy iteration (which needs many inner sweeps per outer iteration) when the number of states is small, but the advantage reverses for larger MDPs because policy iteration converges in very few outer iterations.

4.5 Generalized Policy Iteration

Policy evaluation and policy improvement are not rigidly sequential steps — they can be interleaved in any proportion. The term *Generalized Policy Iteration* (GPI) refers to the general idea of maintaining a value function V and a policy π, and repeatedly applying some form of evaluation (making V more consistent with π) and some form of improvement (making π more greedy with respect to V).

Two competing processes. The two processes pull in opposite directions: evaluation pushes V toward v^π for the *current* π, while improvement changes π to be greedy with respect to the current V, which immediately makes V an inaccurate estimate for the *new* π. Despite this tension, the two processes cooperate in the following sense: each makes progress toward the same goal. The interaction can be visualised as two lines in value-policy space being pulled toward each other (see Figure 4.2).

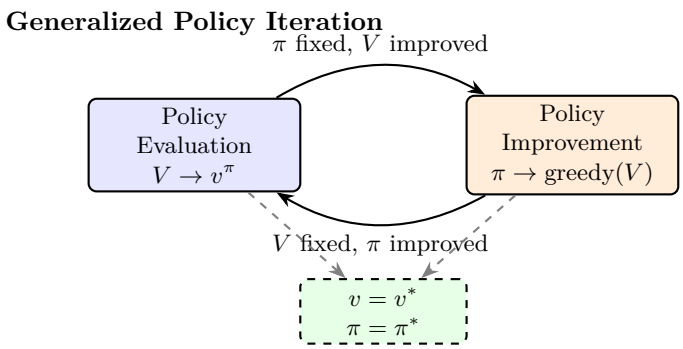


Figure 4.2: The GPI framework. Policy evaluation and policy improvement interact repeatedly. Each step moves toward the stable fixed point where $V = v^*$ and $\pi = \pi^*$ simultaneously. In policy iteration, evaluation runs to completion before each improvement step. In value iteration, a single sweep serves as both. All model-free algorithms in subsequent chapters can be understood as approximate GPI.

GPI as the universal framework. Every major RL algorithm fits the GPI template. In Monte Carlo control (Chapter 5), evaluation is performed by averaging sample returns, and improvement is an ε-greedy step. In temporal-difference learning (Chapter 6), evaluation uses bootstrapped one-step estimates. In actor-critic methods (Chapter 9), the actor performs improvement and the critic performs evaluation.

On-policy vs. off-policy GPI. An important distinction that becomes crucial in model-free RL is whether the data used for evaluation comes from the same policy being evaluated (*on-policy*) or from a different *behaviour policy* (*off-policy*). In pure DP, this distinction is moot because the model p provides exact expectations; no data is collected at all. However, once we move to sampling-based methods in Chapters 5–6, the distinction becomes central.

4.6 Asynchronous Dynamic Programming

The algorithms described so far perform *synchronous* sweeps: every state is updated exactly once per iteration. *Asynchronous* DP relaxes this requirement, updating states in any order, any number of times, without waiting for a complete sweep.

In-place updates. In the synchronous version, we maintain two arrays V (current) and V' (next), and write $V'(s) = T^\pi V(s)$ using only values from V. In the *in-place* variant, we maintain a single array and immediately overwrite $V(s)$ with the new value. Because the update of $V(s)$ may use already-updated values $V(s')$ for $s' \neq s$, the in-place version is not synchronous, yet it typically converges faster in practice: an update in state s' propagates immediately to any state s that is updated later in the same sweep.

Prioritized sweeping. Not all states are equally important to update. *Prioritized sweeping* maintains a priority queue and updates the state s whose Bellman error $|v - T^\pi V(s)|$ is largest. When a state is updated, the priorities of all its predecessors (states from which s is reachable) are recomputed. This focuses computation on the most informative updates and can dramatically reduce the number of updates needed for convergence.

Real-time DP. In *real-time* DP (Barto, Bradtke, and Singh, 1995), the agent interacts with the environment in real time and updates only the states it actually visits. This is the conceptual bridge between model-based DP and model-free TD learning: if the model is available, visited states can be updated using exact Bellman backups; if not, the backups are replaced by sampled transitions.

Remark 4.3. Asynchronous DP converges to v^* (or v^π) as long as every state continues to be updated infinitely often — that is, no state is permanently ignored. This *sufficient updates* condition is easy to satisfy by visiting states repeatedly in any order.

4.7 Efficiency and Limitations of Dynamic Programming

Polynomial complexity. DP algorithms are polynomial in $|\mathcal{S}|$ and $|\mathcal{A}|$. Each sweep of policy evaluation or value iteration requires $O(|\mathcal{S}|^2|\mathcal{A}|)$ operations (for each of $|\mathcal{S}|$ states, the update sums over $|\mathcal{A}|$ actions and $|\mathcal{S}|$ next states). Policy iteration performs at most $|\mathcal{A}|^{|\mathcal{S}|}$ outer iterations, but in practice converges in very few (typically $O(\log|\mathcal{S}|)$). Compare this to brute-force policy enumeration, which requires evaluating all $|\mathcal{A}|^{|\mathcal{S}|}$ policies.

Curse of dimensionality. Despite polynomial complexity in $|\mathcal{S}|$, DP is often impractical because $|\mathcal{S}|$ itself is astronomically large. A robot with n continuous joint angles discretised into d bins each has $|\mathcal{S}| = d^n$ states, which grows exponentially in n. This is Bellman's *curse of dimensionality*: the state space explodes with the number of dimensions. For a modest robot with 10 joints and 100 bins per joint, $|\mathcal{S}| = 10^{20}$—hopelessly intractable.

Comparison with linear programming. The optimal value function v^* can also be found by solving a linear program with $|\mathcal{S}|$ variables and $|\mathcal{S}| \cdot |\mathcal{A}|$ constraints. LP-based methods can in principle exploit efficient LP solvers, but they scale poorly with $|\mathcal{S}|$ as well and are generally not competitive with DP for large problems.

Why sampling is essential. The limitations of DP motivate the model-free methods studied in the rest of this book. In Monte Carlo methods (Chapter 5), v^π is estimated from sampled trajectories without requiring p. In temporal-difference methods (Chapter 6), bootstrapping allows incremental updates from individual transitions. In all cases, the GPI framework is preserved but exact Bellman backups are replaced by approximate, sampled ones.

The key insight is this: *DP tells us what to compute; sampling tells us how to compute it when p is unavailable or the state space is too large.*

4.8 Worked Example: Jack's Car Rental

To illustrate policy iteration on a realistic problem, we describe the *Jack's Car Rental* example, a classic from Sutton and Barto (2018).

Problem Setup

Jack manages two car rental locations, A and B. Each day, customers arrive at each location to rent cars. Rentals and returns are approximately Poisson distributed: expected 3 rentals and 3 returns per day at A, and 4 rentals and 2 returns per day at B. Jack receives \$10 for each car rented (if available) and can move cars overnight between locations (up to 5 cars per night, at a cost of \$2 per car moved). Each location can hold at most 20 cars.

The MDP is defined as follows. The state (n_A, n_B) is the number of cars at each location at the end of the day (before overnight moves), ranging from 0 to 20 at each site. The action $a \in \{-5, \ldots, 5\}$ is the net number of cars moved from A to B (negative means moving from B to A). The reward is the expected rental income minus moving costs. The state space has $21^2 = 441$ states, and the action space has 11 actions, making this tractable for exact DP.

Policy Evolution

Figure 4.3 shows schematically how the policy evolves under policy iteration.

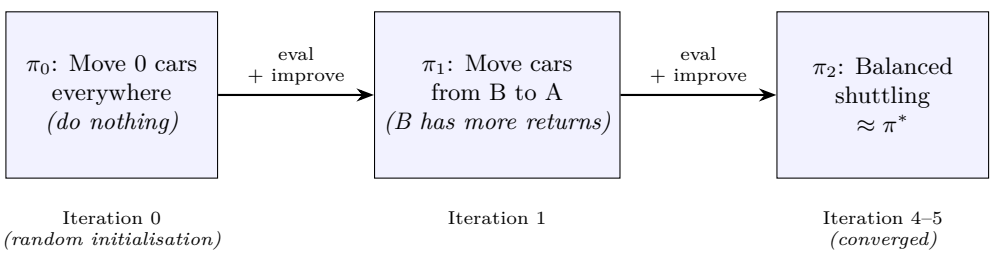


Figure 4.3: Qualitative evolution of the policy under policy iteration for the car rental problem. Early policies are crude; after 4–5 iterations the policy stabilises to the optimal shuttling strategy, which moves cars from B toward A (since B has higher expected returns, keeping cars at A captures more rental income).

Observations. The optimal policy moves cars from the location with higher expected returns (B) toward the location with higher expected rentals (A), carefully balancing the \$2 per-car transport cost against the expected gain in rental income. Policy iteration converges in 4–5 outer iterations, each requiring solving a 441×441 linear system. This demonstrates how policy iteration handles structured reward and transition dynamics efficiently, reaching the optimum far faster than pure value iteration or exhaustive search.

4.9 Summary

- Dynamic programming assumes a known model $p(s', r \mid s, a)$ and solves for the optimal value function and policy exactly.
- **Iterative policy evaluation** applies the Bellman operator T^π repeatedly; convergence is guaranteed by the contraction mapping theorem with rate γ.
- The **policy improvement theorem** guarantees that the greedy policy with respect to v^π is at least as good as π; if it equals π, then π is optimal.
- **Policy iteration** alternates evaluation and improvement and terminates in finitely many iterations because the sequence of policies is strictly improving and the policy space is finite.
- **Value iteration** applies T^* directly, combining evaluation and improvement in a single sweep, and converges geometrically to v^*.
- **Generalized Policy Iteration** is the unifying framework: all RL algorithms can be seen as interleaved evaluation and improvement, differing only in how approximate each step is.
- **Asynchronous DP** updates states in flexible order; prioritized sweeping focuses computation where Bellman errors are largest.
- DP is polynomial in $|\mathcal{S}|$ and $|\mathcal{A}|$ but impractical for large state spaces due to the curse of dimensionality, motivating the sampling-based methods of Chapters 5 and 6.

Exercises

Exercise 4.1. Let $v_0 \equiv 0$ and suppose $\|v^*\|_\infty \leq R_{\max}/(1-\gamma)$.

(a) Show that $\|v_k - v^*\|_\infty \leq \gamma^k R_{\max}/(1-\gamma)$ for the iterates $v_{k+1} = T^* v_k$.

(b) For $\gamma = 0.99$ and $R_{\max} = 1$, how many iterations of value iteration are needed to achieve $\|v_k - v^*\|_\infty \leq 10^{-3}$?

Exercise 4.2. Generalise the policy improvement theorem to *stochastic* policies π'. That is, suppose $\sum_a \pi'(a \mid s)\, q^\pi(s,a) \geq v^\pi(s)$ for all s. Prove that $v^{\pi'} \geq v^\pi$ pointwise. *Hint*: the proof is essentially the same as for deterministic policies; the operator $T^{\pi'}$ is still a contraction.

Exercise 4.3. After value iteration converges to $V \approx v^*$ with $\|V - v^*\|_\infty \leq \varepsilon$, what bound can you give on the sub-optimality of the greedy policy π extracted from V? That is, bound $v^*(s) - v^\pi(s)$ in terms of ε and γ. *Hint*: use the contraction argument twice.

Exercise 4.4. In the 4×4 gridworld with $\gamma = 1$ and the random policy ($\pi(a \mid s) = 0.25$):

(a) Write out the Bellman equations for the four corner states and the four edge-midpoint states.

(b) Verify numerically (or algebraically) that the converged values in Example 4.1 satisfy these equations.

Exercise 4.5. A finite MDP has $|\mathcal{S}| = 5$ states and $|\mathcal{A}| = 3$ actions. How many deterministic policies are there? Policy iteration visits each policy at most once. Assuming each outer iteration takes $O(|\mathcal{S}|^2|\mathcal{A}|)$ work, what is a worst-case bound on total computation? Compare with the work of exhaustive policy evaluation.

Exercise 4.6. Explain why an in-place synchronous sweep (single array, states updated left-to-right in the gridworld) can converge faster than the two-array synchronous version. Give an intuitive example in the 4×4 gridworld where an update propagates across multiple states in a single sweep.

Exercise 4.7 (Truncated policy evaluation)**.** Define *m-step policy iteration* as policy iteration in which the evaluation phase performs exactly m sweeps of T^π before each improvement step. Show that:

(a) $m = \infty$ recovers standard policy iteration.

(b) $m = 1$ recovers value iteration.

(c) For all finite m, the sequence of value functions is non-decreasing and converges to v^*.

Exercise 4.8. Show that the optimal value function v^* is the *smallest* super-solution to the Bellman optimality equation: that is, v^* is the component-wise minimum of all functions v satisfying $v(s) \geq (T^*v)(s)$ for all s. Use this to formulate finding v^* as a linear program.

Exercise 4.9 (Jack's Car Rental modification)**.** In the car rental problem of Section 4.8, Jack hires a second employee. The first employee can move one car per night for free; any additional cars still cost \$2 each. Furthermore, each location can now hold up to 25 cars, but a parking fee of \$4 is charged per night for each location that has more than 10 cars. Describe how you would modify the reward function and state space definition. Which changes affect the transition probabilities? Which affect only the reward?

Exercise 4.10. In model-based DP, we argued that the on-policy/off-policy distinction is moot. Explain precisely why. Then explain what goes wrong if we try to apply *off-policy* updates in a model-free (sampling-based) version of policy evaluation, and state the condition required to make such updates consistent.

Chapter 5

Monte Carlo Methods

To find the value of a position, simulate many games from that position, and average the results.

The dynamic-programming methods of Chapter 4 are powerful but carry a heavy prerequisite: complete knowledge of the environment's transition probabilities and reward function. In many problems of practical interest — card games, robotic locomotion, dialogue systems — such a model is simply unavailable, or is too large to make full sweeps over all states tractable. *Monte Carlo methods* eliminate this requirement entirely. Rather than computing value functions from a mathematical specification of the environment, they *learn* value functions directly from sampled episodes of interaction.

The core idea is as old as statistical simulation itself: if you want to know the expected outcome of a stochastic process, run it many times and average the results. Applied to reinforcement learning, this means generating complete episodes under a fixed policy, computing the return (the discounted sum of rewards) that followed each visit to a state or state–action pair, and averaging those returns. As the number of episodes grows, the law of large numbers guarantees convergence to the true value function.

Monte Carlo methods occupy a pivotal position in the RL landscape. They mark the transition from model-based to *model-free* learning and introduce the fundamental dichotomy between *on-policy* and *off-policy* learning that will recur throughout the remainder of this book. They require complete episodes, which makes them naturally suited to episodic tasks; the temporal-difference methods of Chapter 6 will later remove even this restriction.

5.1 Monte Carlo Prediction

Setting and Notation

We work throughout this chapter with a finite episodic MDP (see Chapter 3) with state space $\mathcal{S}$, action space $\mathcal{A}$, discount factor $\gamma \in [0, 1)$, and a fixed policy π. A single episode is a trajectory

$$S_0, A_0, R_1, S_1, A_1, R_2, \ \ldots, \ S_{\tau-1}, A_{\tau-1}, R_\tau, S_\tau = \mathrm{T},$$

where T is the terminal state and τ is the (random) episode length. The *return* from time step t onward is

$$G_t = \sum_{k=0}^{\tau-t-1} \gamma^k R_{t+1+k}. \tag{5.1}$$

The state value function is $v^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$.

We assume throughout that: (i) episodes are generated independently under π; (ii) every episode terminates almost surely, $\mathbb{P}_\pi(\tau < \infty) = 1$; (iii) rewards are bounded, $|R_t| \le R_{\max}$ a.s.; and (iv) each state s of interest is visited with positive probability per episode.

First-Visit and Every-Visit Monte Carlo

Within a single episode, a state s may be visited multiple times. Two natural averaging schemes arise from how we handle these repeated visits.

Definition 5.1 (First-visit MC estimator)**.** The **first-visit Monte Carlo** estimator of $v^\pi(s)$ averages, across episodes, the return G_t that follows the *first* time step t at which $S_t = s$ in each episode. If $G_1^{\mathrm{FV}}(s), G_2^{\mathrm{FV}}(s), \dots$ denote these returns collected from n episodes, the estimator is

$$\widehat{v}_n^{\mathrm{FV}}(s) = \frac{1}{n} \sum_{i=1}^{n} G_i^{\mathrm{FV}}(s). \tag{5.2}$$

Definition 5.2 (Every-visit MC estimator)**.** The **every-visit Monte Carlo** estimator averages the returns that follow *every* visit to s across all episodes. For episode i, let $K_i(s) = \sum_t \mathbb{1}\{S_t^{(i)} = s\}$ be the visit count and $X_i(s) = \sum_t \mathbb{1}\{S_t^{(i)} = s\} G_t^{(i)}$ the summed returns. The estimator after n episodes is

$$\widehat{v}_n^{\mathrm{EV}}(s) = \frac{\sum_{i=1}^{n} X_i(s)}{\sum_{i=1}^{n} K_i(s)}. \tag{5.3}$$

Both estimators target the same quantity $v^\pi(s)$. Their difference is statistical, not definitional.

Unbiasedness of First-Visit MC

Theorem 5.1 (First-visit MC is unbiased and consistent)**.** *Under the assumptions above, each return $G_i^{\mathrm{FV}}(s)$ is an i.i.d. random variable with mean $v^\pi(s)$ and finite variance. Consequently:*

1. $\mathbb{E}\big[\widehat{v}_n^{\mathrm{FV}}(s)\big] = v^\pi(s)$ *for every $n \ge 1$ (unbiasedness).*
2. $\widehat{v}_n^{\mathrm{FV}}(s) \xrightarrow[n\to\infty]{\text{a.s.}} v^\pi(s)$ *(strong consistency).*

Proof. **Independence.** Each $G_i^{\mathrm{FV}}(s)$ is computed from a separate, independently generated episode. Hence the sequence $G_1^{\mathrm{FV}}(s), G_2^{\mathrm{FV}}(s), \dots$ consists of independent random variables.

Identical distribution. Consider any episode and let T^* be the first time step at which $S_{T^*} = s$. By the Markov property, the distribution of the trajectory from T^* onward depends only on the current state s and not on the history $S_0, A_0, \dots, S_{T^*-1}$. Since π is fixed, the return G_{T^*} from that point has exactly the distribution of $G_t \mid S_t = s$ under policy π. Thus $G_i^{\mathrm{FV}}(s) \overset{d}{=} G_t \mid S_t = s$ for every i.

Finite variance. Because $|R_t| \le R_{\max}$ and $\tau < \infty$ a.s.,

$$|G_t| \le R_{\max} \sum_{k=0}^{\tau-1} \gamma^k \le \frac{R_{\max}}{1-\gamma} < \infty \quad \text{a.s.}$$

Hence $G_i^{\mathrm{FV}}(s)$ is bounded a.s. and has finite variance $\sigma_s^2 < \infty$.

Unbiasedness. By linearity of expectation,

$$\mathbb{E}\big[\widehat{v}_n^{\mathrm{FV}}(s)\big] = \frac{1}{n}\sum_{i=1}^{n}\mathbb{E}\big[G_i^{\mathrm{FV}}(s)\big] = v^\pi(s).$$

Strong consistency. Since $G_1^{\mathrm{FV}}(s), G_2^{\mathrm{FV}}(s), \ldots$ are i.i.d. with finite mean, the strong law of large numbers gives

$$\widehat{v}_n^{\mathrm{FV}}(s) = \frac{1}{n}\sum_{i=1}^{n} G_i^{\mathrm{FV}}(s) \xrightarrow{\text{a.s.}} v^\pi(s). \qquad \square$$

Remark 5.1. The variance of $\widehat{v}_n^{\mathrm{FV}}(s)$ equals σ_s^2/n, which decreases at the standard Monte Carlo rate of $1/\sqrt{n}$. This is in contrast to, say, bootstrap methods, which can achieve faster rates in smooth models. However, no distributional assumptions whatsoever are required here — only the existence of bounded rewards and episodic termination.

Convergence of Every-Visit MC

Lemma 5.2. *For any state s and any episode, $\mathbb{E}[X_1(s)] = v^\pi(s)\ \mathbb{E}[K_1(s)]$.*

Proof. By linearity and the tower property,

$$\begin{aligned}\mathbb{E}[X_1(s)] &= \sum_{t\geq 0}\mathbb{E}[\mathbb{1}\{S_t = s\}\, G_t] = \sum_{t\geq 0}\mathbb{E}[\mathbb{1}\{S_t = s\}\ \mathbb{E}[G_t \mid S_t]] \\ &= v^\pi(s)\sum_{t\geq 0}\mathbb{P}(S_t = s) = v^\pi(s)\ \mathbb{E}[K_1(s)]. \qquad \square\end{aligned}$$

Theorem 5.3 (Every-visit MC is consistent)**.** *If $\mathbb{E}[K_1(s)] \in (0,\infty)$ and $\mathbb{E}[|X_1(s)|] < \infty$, then $\widehat{v}_n^{\mathrm{EV}}(s) \xrightarrow{\text{a.s.}} v^\pi(s)$.*

Proof. The pairs $(X_i(s), K_i(s))$ are i.i.d. across episodes. The strong law of large numbers gives

$$\frac{1}{n}\sum_{i=1}^{n} X_i(s) \xrightarrow{\text{a.s.}} \mathbb{E}[X_1(s)], \qquad \frac{1}{n}\sum_{i=1}^{n} K_i(s) \xrightarrow{\text{a.s.}} \mathbb{E}[K_1(s)] > 0.$$

The denominator is eventually positive a.s., so by the continuous mapping theorem (ratio rule):

$$\widehat{v}_n^{\mathrm{EV}}(s) \xrightarrow{\text{a.s.}} \frac{\mathbb{E}[X_1(s)]}{\mathbb{E}[K_1(s)]} = v^\pi(s),$$

where the last equality is Lemma 5.2. $\square$

Remark 5.2. Every-visit MC is *biased* at any finite n because $\mathbb{E}[\sum X_i/\sum K_i] \neq \mathbb{E}[\sum X_i]/\mathbb{E}[\sum K_i]$ in general (ratio-estimator bias). However, the bias vanishes as $n \to \infty$, so every-visit MC is *asymptotically* unbiased. In practice, every-visit MC can have lower mean-squared error than first-visit when states are revisited frequently within an episode, because it uses more data.

Incremental Implementation

Computing averages from scratch after each episode is wasteful. Instead, we maintain a running count $N(s)$ and a running estimate $V(s)$, and update after each observed return G:

$$N(s) \leftarrow N(s) + 1, \qquad V(s) \leftarrow V(s) + \frac{1}{N(s)}\big(G - V(s)\big). \tag{5.4}$$

This is algebraically equivalent to the running sample mean and requires $O(|\mathcal{S}|)$ memory.

Algorithm 6: First-Visit Monte Carlo Prediction

```
   Input: Policy π, discount γ, number of episodes N_ep
1  Initialise V(s) ← 0, Count(s) ← 0 for all s ∈ S;
2  for episode = 1, 2, ..., N_ep do
3      Generate episode S_0, A_0, R_1, ..., S_τ using π;
4      G ← 0;
5      for t = τ − 1, τ − 2, ..., 0 (backward) do
6          G ← γG + R_{t+1};
7          if S_t does not appear in S_0, ..., S_{t−1} then
8              Count(S_t) ← Count(S_t) + 1;
9              V(S_t) ← V(S_t) + (1/Count(S_t)) (G − V(S_t));
10 return V;
```

The backward pass through the episode (lines 4–9) computes all returns G_t in $O(\tau)$ time using the recurrence $G_t = R_{t+1} + \gamma G_{t+1}$.

MC Backup Diagram

The following TikZ diagram contrasts the backup diagrams of dynamic programming and Monte Carlo. In DP, a one-step backup averages over all possible next states using the model. In MC, a single sampled trajectory is followed all the way to the terminal state.

5.2 Worked Example: Blackjack

Blackjack (also called *twenty-one*) is the canonical example used to illustrate Monte Carlo prediction in Sutton and Barto (2018). It is particularly instructive because a complete environment model (transition probabilities) is technically available but tedious to write down, whereas Monte Carlo estimation is trivial to implement by simulating card play.

The Blackjack MDP

The game is played against a dealer. The player's goal is to obtain cards summing as close to 21 as possible without exceeding 21 (going "bust"). Face cards (Jack, Queen, King) count as 10; an Ace counts as 11 unless doing so would cause the total to exceed 21, in which case it counts as 1.

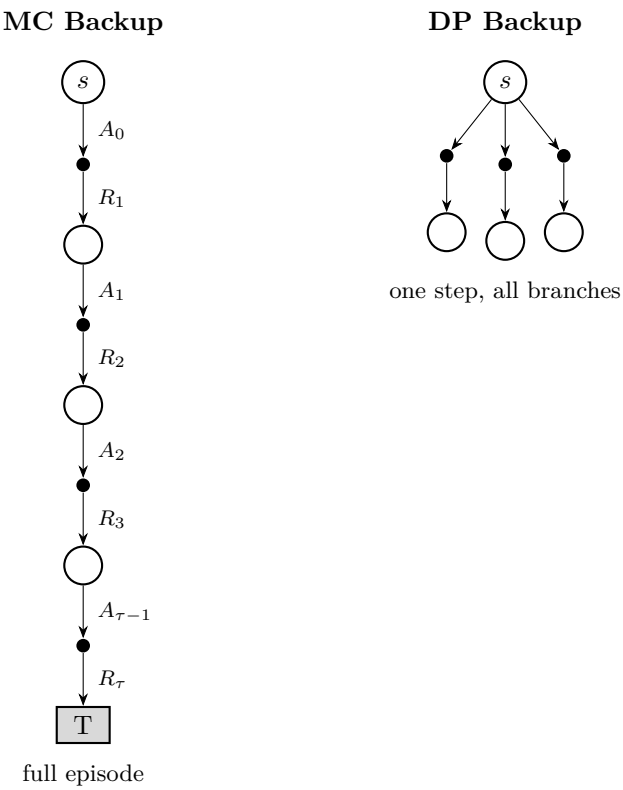


Figure 5.1: Backup diagrams for Monte Carlo (left) and dynamic programming (right). MC follows a single sampled trajectory to the terminal state; DP fans out over all possible one-step transitions using the model.

State. The state is a triple $(x, d, a) \in \{12, \ldots, 21\} \times \{1, \ldots, 10\} \times \{0, 1\}$, where:

- x = the player's current sum (we only consider sums ≥ 12; below 12, the player always hits safely);
- d = the dealer's one showing card (Ace counts as 1 here);
- $a = 1$ if the player holds a usable Ace (an Ace currently counted as 11), $a = 0$ otherwise.

This gives $|\mathcal{S}| = 10 \times 10 \times 2 = 200$ non-terminal states.

Actions. At each state the player chooses *stick* (stop drawing cards, $A = 0$) or *hit* (draw another card, $A = 1$).

Rewards. An episode ends when the player sticks or busts. After sticking, the dealer plays to completion using a fixed rule (hit until sum ≥ 17). The reward is $+1$ for a player win, -1 for a loss, and 0 for a draw. All intermediate rewards are 0. The discount factor is $\gamma = 1$.

Policy. We evaluate the *simple policy* π that sticks if $x \geq 20$ and hits otherwise.

Episode Generation and Return Computation

Example 5.1 (Sample Blackjack episode)**.** Consider the following episode generated under the simple policy:

Step t	State (x, d, a)	Action	Next card	R_{t+1}
0	(15, 6, 0)	Hit	3	0
1	(18, 6, 0)	Hit	2	0
2	(20, 6, 0)	Stick	—	—
Dealer plays: $6 + \text{hidden} = 6 + 9 = 15$, draws $4 \to 19$. Player $20 > 19$.				+1

The episode terminates with $\tau = 3$ and $R_3 = +1$. The returns (with $\gamma = 1$) are:

$$\begin{aligned} G_2 &= R_3 = +1, \\ G_1 &= R_2 + G_2 = 0 + 1 = +1, \\ G_0 &= R_1 + G_1 = 0 + 1 = +1. \end{aligned}$$

Under first-visit MC, this episode contributes:

- Return $G_0 = +1$ to the estimate of $v^\pi(15, 6, 0)$ (first and only visit).
- Return $G_1 = +1$ to $v^\pi(18, 6, 0)$.
- Return $G_2 = +1$ to $v^\pi(20, 6, 0)$.

Running 500,000 such episodes produces an accurate picture of the value function. States with $x = 20$ or $x = 21$ have values close to $+0.6$ (the player usually wins), while states with $x \in \{12, \ldots, 14\}$ facing a dealer Ace have values near -0.3. The Sutton–Barto textbook plots these values as 3-D surfaces, which reveal an intuitively clear landscape: high sums against weak dealer cards are good; low sums against strong dealer cards are bad.

The example highlights why Monte Carlo is well suited here: computing $p(s', r \mid s, a)$ exactly requires enumerating the distribution of cards drawn from an infinite deck, whereas simulating deals is trivial.

5.3 Monte Carlo Estimation of Action Values

When the environment model $p(s', r \mid s, a)$ is available, the state-value function v^π is sufficient for control: given v^π, we can compute the greedy policy by a one-step look-ahead,

$$\pi'(s) \in \operatorname*{argmax}_{a \in \mathcal{A}} \sum_{s', r} p(s', r \mid s, a) \big[r + \gamma v^\pi(s') \big].$$

Without a model, this look-ahead is impossible. We cannot compute the right-hand side because $p(s', r \mid s, a)$ is unknown. Therefore, in model-free control the natural target is the *action-value function*

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s,\ A_t = a]. \tag{5.5}$$

Given Q^π, the greedy policy is simply $\pi'(s) = \operatorname{argmax}_a Q^\pi(s, a)$, requiring no model at all.

Estimating Q^π by Monte Carlo. All the machinery of Section 5.1 applies verbatim with states s replaced by state–action pairs (s, a). We define a first-visit MC estimator for $Q^\pi(s, a)$ by collecting returns from the first visit to (s, a) in each episode, and an every-visit estimator by averaging all such returns. Theorems 5.1 and 5.3 hold with identical proofs.

The exploring starts assumption. A key difficulty arises immediately: if the policy π is deterministic, it will never choose certain actions in certain states, so the pairs (s, a) for those actions are never visited, and $Q^\pi(s, a)$ is never estimated. Without estimates for all pairs, we cannot improve the policy by choosing the greedy action.

The *exploring starts* assumption resolves this by requiring that every state–action pair has a positive probability of being the starting pair of an episode:

$$\mathbb{P}\big(S_0 = s,\ A_0 = a\big) > 0 \quad \forall\, s \in \mathcal{S},\ a \in \mathcal{A}.$$

Under exploring starts, every (s, a) pair will eventually be visited infinitely often (since episodes are generated repeatedly), ensuring consistent estimation of Q^π for all pairs. Section 5.4 develops a full control algorithm under this assumption; Section 5.5 then removes it.

5.4 Monte Carlo Control with Exploring Starts

The generalized policy iteration (GPI) framework of Chapter 4 alternates between *policy evaluation* (estimate Q^π) and *policy improvement* (choose the greedy policy with respect to the current Q). Monte Carlo control instantiates this framework by using sampled episodes in place of Bellman sweeps.

Algorithm 7: Monte Carlo Control with Exploring Starts (MC-ES)

Input: Discount γ

1 Initialise $Q(s, a)$ arbitrarily, $\pi(s) \in \mathcal{A}$ arbitrarily, $\mathit{Count}(s, a) \leftarrow 0$ for all s, a;
2 **for** *each episode* **do**
3 Choose $S_0 \sim \text{Uniform}(\mathcal{S})$, $A_0 \sim \text{Uniform}(\mathcal{A})$ (exploring starts);
4 Generate episode $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_\tau$ following π after step 0;
5 $G \leftarrow 0$;
6 **for** $t = \tau - 1, \tau - 2, \ldots, 0$ (backward) **do**
7 $G \leftarrow \gamma G + R_{t+1}$;
8 **if** (S_t, A_t) *does not appear in* $(S_0, A_0), \ldots, (S_{t-1}, A_{t-1})$ **then**
9 $\mathit{Count}(S_t, A_t) \leftarrow \mathit{Count}(S_t, A_t) + 1$;
10 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{\mathit{Count}(S_t, A_t)}\big(G - Q(S_t, A_t)\big)$;
11 $\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$;

Convergence argument. The algorithm performs a single step of policy improvement (line 11) after each return update. Formally, convergence to an optimal policy π^* follows from a fixed-point argument: if the algorithm converges, then at convergence $Q \approx Q^\pi$ (evaluation is consistent by Theorem 5.1) and π is greedy with respect to Q^π. A policy that is simultaneously greedy with respect to its own value function must be optimal (see the Policy Improvement Theorem, Chapter 4).

Why exploring starts is impractical. The assumption that each episode can start from any (s, a) pair is artificial in most real-world tasks. One cannot, for example, start a chess game with the king in an arbitrary position, or initialize a robot at an arbitrary joint angle while guaranteeing safety. The exploration requirement must instead be built into the policy itself — which is the subject of the next two sections.

5.5 On-Policy Monte Carlo Control

On-policy MC control abandons exploring starts and instead ensures that every action is tried in every state by using a *stochastic* policy that always assigns some minimum probability to each action.

Definition 5.3 (ε-soft policy)**.** A policy π is **ε-soft** if for all $s \in \mathcal{S}$ and all $a \in \mathcal{A}(s)$,

$$\pi(a \mid s) \geq \frac{\varepsilon}{|\mathcal{A}(s)|}, \qquad \varepsilon > 0. \tag{5.6}$$

Among all ε-soft policies, the natural improvement of a given ε-soft policy π is the ε-greedy policy with respect to the current Q estimate:

$$\pi'(a \mid s) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & a = \operatorname{argmax}_b Q(s,b), \\ \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & \text{otherwise.} \end{cases} \tag{5.7}$$

ε-Soft Improvement Theorem

Theorem 5.4 (ε-soft policy improvement)**.** *Let π be an ε-soft policy and let π' be the ε-greedy policy defined by (5.7) with respect to Q^π. Then $v^{\pi'}(s) \geq v^\pi(s)$ for all $s \in \mathcal{S}$.*

Proof. Fix $s \in \mathcal{S}$ and let $m = |\mathcal{A}(s)|$. Write $M = \max_{a \in \mathcal{A}(s)} Q^\pi(s,a)$.

Because π is ε-soft, we can decompose it as

$$\pi(a \mid s) = \frac{\varepsilon}{m} + \delta_a, \quad \delta_a \geq 0, \quad \sum_a \delta_a = 1 - \varepsilon.$$

Then, using the shorthand $v^\pi(s) = Q^\pi(s,\pi) := \sum_a \pi(a \mid s) Q^\pi(s,a)$,

$$\begin{aligned} v^\pi(s) &= \sum_a \pi(a \mid s)\, Q^\pi(s,a) \\ &= \frac{\varepsilon}{m} \sum_a Q^\pi(s,a) + \sum_a \delta_a\, Q^\pi(s,a) \\ &\leq \frac{\varepsilon}{m} \sum_a Q^\pi(s,a) + (1-\varepsilon)\, M. \end{aligned} \tag{5.8}$$

Now observe that the right-hand side of (5.8) is exactly

$$Q^\pi(s,\pi') = \sum_a \pi'(a \mid s)\, Q^\pi(s,a),$$

because π' places mass $1 - \varepsilon + \varepsilon/m$ on the maximising action(s) and ε/m on all others. Thus

$$Q^\pi(s,\pi') \geq v^\pi(s) = Q^\pi(s,\pi).$$

This is precisely the condition of the Policy Improvement Theorem (Chapter 4), which gives $v^{\pi'}(s) \geq v^\pi(s)$. □

Remark 5.3. The inequality becomes an equality — and hence $\pi = \pi'$ — only when $Q^\pi(s,a)$ is the same for all a in every state, i.e. when π is already optimal among all ε-soft policies. This optimum is the *ε-optimal* policy: the best achievable when forced to keep at least $\varepsilon/|\mathcal{A}(s)|$ probability on each action. It is not the globally optimal deterministic policy π^* in general. The gap shrinks as $\varepsilon \to 0$.

Algorithm 8: On-Policy First-Visit MC Control (ε-greedy)

Input: Small $\varepsilon > 0$, discount γ
Initialise $Q(s,a) \leftarrow 0$, $Count(s,a) \leftarrow 0$ for all s, a;
$\pi \leftarrow \varepsilon$-greedy w.r.t. Q;
for *each episode* **do**
  Generate episode $S_0, A_0, R_1, \ldots, S_\tau$ using π;
  $G \leftarrow 0$;
  for $t = \tau - 1, \tau - 2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$;
    if (S_t, A_t) *first visit in episode* **then**
      $Count(S_t, A_t) \leftarrow Count(S_t, A_t) + 1$;
      $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{Count(S_t, A_t)}\big(G - Q(S_t, A_t)\big)$;
  Update π to be ε-greedy w.r.t. current Q;

The ε-soft property guarantees that every (s, a) pair is visited infinitely often, so the estimates $Q(s, a)$ converge to Q^π by Theorem 5.1, and repeated application of Theorem 5.4 drives the policy toward the ε-optimal policy.

Convergence to ε-optimality. On-policy MC control converges to the best possible policy among all ε-soft policies. This is weaker than convergence to the globally optimal π^*. To recover π^*, one either uses exploring starts (Section 5.4) or, more practically, the off-policy approach of Section 5.6, which allows the target policy to be deterministic while the behavior policy handles exploration.

5.6 Off-Policy Monte Carlo via Importance Sampling

The Off-Policy Problem

In off-policy learning, data is collected by a *behavior policy* μ (which can explore freely), while we wish to estimate or improve a *target policy* π (which may be deterministic and greedy). The fundamental obstacle is that the episode distribution induced by μ differs from the distribution induced by π. Naive averaging of returns would yield estimates of Q^μ, not Q^π.

Coverage assumption. For off-policy estimation to be well posed, we require that π cannot choose any action that μ would never choose:

$$\pi(a \mid s) > 0 \implies \mu(a \mid s) > 0, \quad \forall\, s \in \mathcal{S},\ a \in \mathcal{A}. \tag{5.9}$$

Without (5.9), certain trajectories required by π would have zero probability under μ and could never appear in the data.

Importance Sampling Ratio

The key tool is the *importance sampling ratio*, which reweights the probability of observing a particular trajectory under μ to the probability of observing it under π. For a trajectory

segment $(S_t, A_t, \ldots, S_T)$ generated by μ, the ratio of the trajectory probability under π to that under μ is

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{\mu(A_k \mid S_k)}. \tag{5.10}$$

The environment transitions $p(s', r \mid s, a)$ cancel in this ratio because they are the same under both policies.

Ordinary Importance Sampling (OIS)

Definition 5.4 (Ordinary importance sampling estimator)**.** Given n episodes with returns $G_0^{(1)}, \ldots, G_0^{(n)}$ and corresponding IS weights $\rho^{(1)}, \ldots, \rho^{(n)}$, the **ordinary importance sampling** (OIS) estimator of $Q^\pi(s, a)$ is

$$\widehat{Q}_n^{\text{OIS}}(s, a) = \frac{1}{n} \sum_{i=1}^{n} \rho^{(i)} G_0^{(i)}. \tag{5.11}$$

Theorem 5.5 (OIS is unbiased)**.** *Under the coverage assumption* (5.9)*,*

$$\mathbb{E}_\mu[\rho_{0:T-1}\, G_0 \mid S_0 = s,\ A_0 = a] = Q^\pi(s, a).$$

Proof. Denote a tail trajectory by $h = (s_1, a_1, r_2, \ldots, s_\tau)$. Then

$$Q^\pi(s, a) = \sum_h \mathbb{P}_\pi(h \mid s, a)\, G(h),$$

where $G(h) = \sum_k \gamma^k r_{k+1}$. Under μ,

$$\mathbb{P}_\mu(h \mid s, a) = \prod_{k=1}^{\tau-1} \mu(a_k \mid s_k)\, p(s_{k+1}, r_{k+1} \mid s_k, a_k).$$

By definition of ρ,

$$\rho(h)\, \mathbb{P}_\mu(h \mid s, a) = \prod_{k=1}^{\tau-1} \pi(a_k \mid s_k)\, p(s_{k+1}, r_{k+1} \mid s_k, a_k) = \mathbb{P}_\pi(h \mid s, a).$$

Therefore,

$$\begin{aligned} \mathbb{E}_\mu[\rho\, G_0 \mid S_0 = s, A_0 = a] &= \sum_h \rho(h)\, G(h)\, \mathbb{P}_\mu(h \mid s, a) \\ &= \sum_h G(h)\, \mathbb{P}_\pi(h \mid s, a) = Q^\pi(s, a). \end{aligned}$$

□

Weighted Importance Sampling (WIS)

Definition 5.5 (Weighted importance sampling estimator)**.** The **weighted importance sampling** (WIS) estimator is

$$\widehat{Q}_n^{\text{WIS}}(s, a) = \frac{\sum_{i=1}^{n} \rho^{(i)}\, G_0^{(i)}}{\sum_{i=1}^{n} \rho^{(i)}}, \tag{5.12}$$

whenever the denominator is positive.

WIS is a ratio estimator. It is biased for finite n (since $\mathbb{E}[\sum \rho G / \sum \rho] \neq \mathbb{E}[\sum \rho G] / \mathbb{E}[\sum \rho]$), but asymptotically unbiased and consistent by the same ratio-limit argument as Theorem 5.3. Crucially, WIS has dramatically lower variance than OIS:

Proposition 5.6. *Let ρ be the IS weight for a single episode. For any single episode,*

$$\mathrm{Var}_\mu[\rho G] \geq \mathrm{Var}_\mu\left[\widehat{Q}_1^{\mathrm{WIS}}\right]$$

(in the sense that WIS normalises the weight so its squared expectation is bounded). In particular, if $\pi = \mu$, then $\rho \equiv 1$ and both estimators reduce to the on-policy sample mean with no increase in variance.

The intuition is that OIS can produce IS weights $\rho \gg 1$ when π strongly prefers certain actions that μ rarely takes. WIS self-normalises and clips the effective weights.

Worked Example: Off-Policy Estimation with Numbers

Example 5.2 (Off-policy MC estimation)**.** Consider a simple two-state episodic MDP. The single non-terminal state is s, with two actions $\{a_1, a_2\}$. A single step from s leads to the terminal state T with reward r.

Policies:

- Behavior: $\mu(a_1 \mid s) = 0.5$, $\mu(a_2 \mid s) = 0.5$.
- Target: $\pi(a_1 \mid s) = 0.8$, $\pi(a_2 \mid s) = 0.2$.

Rewards: action a_1 always yields $r = +2$; action a_2 always yields $r = +1$.

The true target values are:

$$Q^\pi(s, a_1) = 2, \quad Q^\pi(s, a_2) = 1.$$

Suppose we observe three episodes under μ:

Episode i	Action taken	Return $G^{(i)}$	IS weight $\rho^{(i)} = \pi(A^{(i)} \mid s)/\mu(A^{(i)} \mid s)$
1	a_1	2	$0.8/0.5 = 1.6$
2	a_2	1	$0.2/0.5 = 0.4$
3	a_1	2	$0.8/0.5 = 1.6$

Ordinary IS estimate for $v^\pi(s) = \mathbb{E}_\pi[G \mid S_0 = s] = 0.8 \times 2 + 0.2 \times 1 = 1.8$:

$$\widehat{v}_3^{\mathrm{OIS}} = \frac{1}{3}(1.6 \times 2 + 0.4 \times 1 + 1.6 \times 2) = \frac{1}{3}(3.2 + 0.4 + 3.2) = \frac{6.8}{3} \approx 2.27.$$

Weighted IS estimate:

$$\widehat{v}_3^{\mathrm{WIS}} = \frac{1.6 \times 2 + 0.4 \times 1 + 1.6 \times 2}{1.6 + 0.4 + 1.6} = \frac{6.8}{3.6} \approx 1.89.$$

With only 3 episodes, WIS is closer to the true value 1.8 in this instance. Both estimators converge to 1.8 as $n \to \infty$. The OIS deviation of $2.27 - 1.8 = 0.47$ is due to the unlucky overrepresentation of a_1; WIS compensates by normalising the weights.

Incremental Off-Policy Implementation

For WIS, the incremental update requires maintaining both the numerator sum and the denominator sum. Let $C_n = \sum_{i=1}^{n} \rho^{(i)}$ be the cumulative weight. After episode $n+1$:

$$C_{n+1} \leftarrow C_n + \rho^{(n+1)}, \tag{5.13}$$

$$Q_{n+1}(s,a) \leftarrow Q_n(s,a) + \frac{\rho^{(n+1)}}{C_{n+1}}\big(G^{(n+1)} - Q_n(s,a)\big). \tag{5.14}$$

Equations (5.13)–(5.14) process one episode at a time and require $O(|\mathcal{S}||\mathcal{A}|)$ memory.

5.7 Bias–Variance Trade-off in Monte Carlo

Bias

Monte Carlo estimators are *unbiased* (first-visit) or *asymptotically unbiased* (every-visit, WIS). This is a significant advantage over bootstrap methods such as temporal-difference learning, which introduces bias through its bootstrapping target $R + \gamma V(S')$. The MC estimate uses the actual observed return G_t, which is an unbiased sample of $v^\pi(s)$ by definition.

Variance

Despite their unbiasedness, MC estimators can have high variance, especially:

- **Long episodes.** The return $G_t = \sum_{k=0}^{\tau-t-1} \gamma^k R_{t+k+1}$ accumulates many stochastic rewards. If the episode length is L, the variance of G_t can grow as $O(L \cdot \sigma_R^2)$ for $\gamma = 1$ (where σ_R^2 is the per-step reward variance).
- **Off-policy IS weights.** As noted in Section 5.6, the OIS weight $\rho = \prod_k \pi(A_k \mid S_k)/\mu(A_k \mid S_k)$ is a product of τ terms. Even if each factor is close to 1, the product can be exponentially variable in the episode length. WIS reduces this, but at the cost of introducing bias.
- **Rare visits.** If state s is visited infrequently, the effective sample size is small and the variance of the MC estimate is large.

Comparison Table

Table 5.1: Summary of Monte Carlo estimators. "a.u. = asymptotically unbiased".

Method	Bias	Variance	Practical	Note
First-visit MC (on-policy)	None	Moderate	Yes	i.i.d. returns
Every-visit MC (on-policy)	a.u.	Lower (more data)	Yes	Ratio estimator
OIS (off-policy)	None	High	Sometimes	Product weights
WIS (off-policy)	a.u.	Lower	Yes	Self-normalised
DP policy evaluation	None	Zero (deterministic)	Model needed	Full sweep

The Fundamental Tension

The bias–variance trade-off in MC methods can be summarised as follows. *Any* method that uses only actual observed returns avoids bias but accumulates variance over the full episode length. Methods that bootstrap (e.g. TD) reduce variance by using a one-step estimate, but introduce bias from the approximate value function. This tension between MC and TD methods is a central theme of the field, and Chapter 6 will explore it in depth through the lens of the TD(λ) family, which interpolates continuously between the two extremes.

5.8 Summary

Monte Carlo methods constitute the first genuinely model-free approach to RL value estimation and control. Their key properties are:

No model required. Episodes are used directly; transition probabilities are never needed.

Unbiased estimates. First-visit MC is unbiased; every-visit and WIS are asymptotically unbiased.

Complete episodes. Unlike TD, MC must wait until episode termination before updating. This limits applicability to episodic tasks and makes MC unsuitable for very long or non-terminating episodes.

High variance. Returns aggregate many random rewards; IS weights amplify variance off-policy.

On/off-policy split. On-policy methods (with ε-soft policies) are simpler and lower-variance; off-policy methods (with IS) are more flexible but higher-variance.

GPI framework. MC control is a sample-based instantiation of the evaluation–improvement cycle of Chapter 4.

A brief historical note: Monte Carlo simulation was systematized by Stanislaw Ulam and John von Neumann in the 1940s as part of the Manhattan Project. Its application to RL was developed by Barto and Sutton in the 1990s, drawing on earlier work in adaptive control. The importance sampling connection was clarified in the foundational Sutton–Barto textbook and in the work of Precup, Sutton, and Singh (2000) on eligibility traces in the off-policy setting.

Preview of TD methods. Chapter 6 introduces temporal-difference learning, which updates value estimates after each step rather than waiting for an episode to end. TD methods bootstrap from current estimates, introducing bias but dramatically reducing variance. The TD–MC spectrum is controlled by the λ parameter in TD(λ), which continuously interpolates between single-step TD ($\lambda = 0$) and full MC ($\lambda = 1$).

Exercises

Exercise 5.1. Let $G_1^{\mathrm{FV}}(s), \ldots, G_n^{\mathrm{FV}}(s)$ be the first-visit returns for state s collected across n episodes. Assume each return has mean $v^\pi(s)$ and variance $\sigma_s^2 < \infty$.

(a) Show that $\widehat{v}_n^{\mathrm{FV}}(s)$ is unbiased.

(b) Compute $\mathrm{Var}\big[\widehat{v}_n^{\mathrm{FV}}(s)\big]$.

(c) How many episodes are needed to guarantee $\mathrm{Var}\big[\widehat{v}_n^{\mathrm{FV}}(s)\big] \le 0.01\,\sigma_s^2$?

Exercise 5.2. Show by a concrete two-state example that the every-visit MC estimator $\widehat{v}_1^{\mathrm{EV}}(s)$ is biased when computed from a single episode. *Hint*: Construct a loop: $s \to s \to \mathrm{T}$ with rewards 1 and 0, and show that the single-episode EV estimator does not equal $v^\pi(s)$.

Exercise 5.3. Derive (5.4) from the formula for the running sample mean. That is, starting from $V_n = \frac{1}{n}\sum_{i=1}^{n} G_i$, show algebraically that $V_{n+1} = V_n + \frac{1}{n+1}(G_{n+1} - V_n)$.

Exercise 5.4. In the Blackjack example of Section 5.2, the state is (x, d, a) with $x \in \{12, \dots, 21\}$.

(a) Why do we restrict to $x \ge 12$? What happens for sums below 12?

(b) Why does on-policy MC produce consistent estimates for all 200 states even though the simple policy ("stick on 20 or 21") never voluntarily takes certain actions?

(c) Estimate informally the value $v^\pi(20, \text{dealer shows } 6, \text{no usable ace})$ and justify your estimate.

Exercise 5.5. In Algorithm 7, the policy update (line 11) is applied only on the first visit to (S_t, A_t) in each episode. Suppose instead we always update Q and π for every visit (including repeated ones). Does the algorithm still converge? What changes in the convergence argument?

Exercise 5.6. In the proof of Theorem 5.4, the inequality (5.8) becomes an equality. Characterize precisely when this equality holds, and explain why at equality the policy π is ε-optimal.

Exercise 5.7. Let $X = \rho G$ be a single OIS-weighted return. Show that

$$\mathrm{Var}_\mu[\rho G] = \mathbb{E}_\mu[\rho^2 G^2] - (Q^\pi(s,a))^2.$$

Argue why $\mathbb{E}_\mu[\rho^2 G^2]$ can be very large when π and μ differ substantially. How does this motivate WIS?

Exercise 5.8. Prove that if $\pi = \mu$, then $\rho_{t:T-1} = 1$ for every episode, and both the OIS and WIS estimators reduce to the standard on-policy Monte Carlo average.

Exercise 5.9. Show that the WIS estimator $\widehat{Q}_1^{\mathrm{WIS}}(s,a)$ computed from a single episode satisfies $\widehat{Q}_1^{\mathrm{WIS}}(s,a) = G_0^{(1)}$, regardless of the IS weight $\rho^{(1)}$. Conclude that with a single episode, WIS is not unbiased in general.

Exercise 5.10. Consider a 5-step episodic MDP where the only reward is $+1$ at the final step and 0 elsewhere. The discount is $\gamma = 1$. The policy is fixed and deterministic.

(a) What is the MC return G_t for each time step $t = 0, 1, 2, 3, 4$?

(b) If we run first-visit MC for 1000 episodes, what does the estimate $V(s_0)$ converge to?

(c) Explain qualitatively why the variance of G_0 is higher than the variance of G_4. Under what condition would they be equal?

Chapter 6

Temporal-Difference Learning

If one had to identify one idea as central and novel to reinforcement learning, it would undoubtedly be temporal-difference (TD) learning.

Temporal-difference learning is the single most important idea in reinforcement learning. It synthesises two powerful strands of thought: the *sampling* approach of Monte Carlo methods, which learns directly from experience without a model of the environment, and the *bootstrapping* approach of dynamic programming, which updates estimates on the basis of other estimates rather than waiting for the final outcome. The result is a family of algorithms that can learn online, after every single transition, in both episodic and continuing tasks, without requiring a model of the environment.

This chapter develops the TD family in full generality. We begin with the simplest member, TD(0), and establish its key properties: the TD error, its zero-mean property under the true value function, and convergence. We then move to TD control algorithms — SARSA, Q-learning, Expected SARSA, and Double Q-learning — and carefully compare their on-policy and off-policy characters. We generalise to n-step returns, which interpolate between TD and Monte Carlo, and then to the full TD(λ) algorithm with eligibility traces, which combines information from all horizons simultaneously. We conclude with Generalised Advantage Estimation (GAE), a modern technique for advantage estimation in policy gradient methods that is built squarely on the TD(λ) machinery, and with a careful analysis of bias, variance, and convergence across the entire family.

6.1 TD(0) Prediction

6.1.1 From Monte Carlo to Temporal Difference

Recall from Chapter 5 that Monte Carlo (MC) prediction estimates the value function V^π by averaging complete sample returns. After each full episode, each visited state S_t receives the update

$$V(S_t) \leftarrow V(S_t) + \alpha\big(G_t - V(S_t)\big), \tag{6.1}$$

where $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ is the discounted return from time t. This estimator is unbiased—$\mathbb{E}_\pi[G_t \mid S_t = s] = V^\pi(s)$—but it has three limitations: it requires the episode to terminate before any learning can occur; it accumulates reward variance over long horizons; and it cannot be applied to continuing (non-episodic) tasks at all.

The Bellman equation for a fixed policy π offers an alternative target. Because

$$V^\pi(s) = \mathbb{E}_\pi\big[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s\big], \tag{6.2}$$

the true value of state s equals the expected immediate reward plus the discounted value of the next state. Equation (6.2) is exact when $V = V^\pi$, but even with an imperfect estimate $V \approx V^\pi$ the quantity

$$R_{t+1} + \gamma V(S_{t+1}) \tag{6.3}$$

provides a useful *one-step target* that can be computed as soon as the next transition $(S_t, A_t, R_{t+1}, S_{t+1})$ is observed. Replacing the full return G_t in (6.1) with this one-step target yields

$$\boxed{V(S_t) \leftarrow V(S_t) + \alpha\Big(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\Big).} \tag{6.4}$$

This is the **TD(0)** algorithm, also called the one-step temporal-difference method.

6.1.2 The TD Error

Definition 6.1 (TD error)**.** The **temporal-difference error** at time t is

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t). \tag{6.5}$$

The TD error measures the discrepancy between the current estimate $V(S_t)$ and the one-step target $R_{t+1} + \gamma V(S_{t+1})$. With this notation, the TD(0) update (6.4) simplifies to

$$V(S_t) \leftarrow V(S_t) + \alpha\,\delta_t.$$

The sign of δ_t has an immediate intuitive interpretation:

- $\delta_t > 0$: the observed transition was better than expected; $V(S_t)$ should be increased.
- $\delta_t < 0$: the state was overvalued; $V(S_t)$ should be decreased.
- $\delta_t = 0$: the current estimate is locally consistent with the Bellman equation.

Definition 6.2 (Bootstrapping)**.** **Bootstrapping** is the use of the current estimate of the value function as part of the target in the update rule. In TD(0), the target $R_{t+1} + \gamma V(S_{t+1})$ depends on the estimate V at the next state, so the method bootstraps. Monte Carlo does not bootstrap because G_t is the actual return, independent of V.

6.1.3 Zero-Mean Property of the TD Error

The following result is fundamental: it shows that δ_t is a valid (zero-mean) gradient signal when the value function is exact.

Theorem 6.1 (Zero mean of TD error)**.** *If $V = V^\pi$, then for every state s,*

$$\mathbb{E}_\pi[\delta_t \mid S_t = s] = 0. \tag{6.6}$$

Proof. Substituting $V = V^\pi$ into the definition (6.5),

$$\delta_t = R_{t+1} + \gamma V^\pi(S_{t+1}) - V^\pi(S_t).$$

Taking the conditional expectation given $S_t = s$:

$$\begin{aligned}\mathbb{E}_\pi[\delta_t \mid S_t = s] &= \mathbb{E}_\pi\big[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s\big] - V^\pi(s) \\ &= V^\pi(s) - V^\pi(s) = 0,\end{aligned}$$

where the second line uses the Bellman equation (6.2). □

Theorem 6.1 implies that, at the fixed point, the TD update has zero expected change. Before convergence, $\delta_t \neq 0$ on average, and its sign consistently points in the direction that reduces the Bellman error.

6.1.4 TD(0) Algorithm

Algorithm 9: TD(0) for Policy Evaluation

Input: Policy π, step size $\alpha \in (0, 1]$, discount $\gamma \in [0, 1)$
1 Initialise $V(s) \leftarrow 0$ for all $s \in \mathcal{S}$, $V(s_{\text{term}}) \leftarrow 0$;
2 **for** *each episode* **do**
3 Initialise S_0;
4 **for** $t = 0, 1, 2, \ldots$ *until* S_t *is terminal* **do**
5 Choose $A_t \sim \pi(\cdot \mid S_t)$;
6 Take action A_t; observe R_{t+1} and S_{t+1};
7 $\delta_t \leftarrow R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$;
8 $V(S_t) \leftarrow V(S_t) + \alpha\, \delta_t$;

Note that, unlike Monte Carlo, the update at line 8 occurs *immediately* after each transition — no waiting for the end of the episode.

6.1.5 Convergence of Tabular TD(0)

The convergence of TD(0) can be understood through the Bellman operator framework. For a fixed policy π, define

$$(T^\pi V)(s) = \mathbb{E}_\pi\big[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s\big].$$

The Bellman equation reads $V^\pi = T^\pi V^\pi$. Since T^π is a γ-contraction in the supremum norm, its unique fixed point is V^π. TD(0) can be interpreted as stochastic approximation of the iteration $V \leftarrow T^\pi V$: instead of computing the full expectation, it uses a single sample transition.

Theorem 6.2 (Convergence of tabular TD(0))**.** *In the tabular setting, suppose π is fixed, all states are visited infinitely often, rewards are bounded, and the step sizes $\{\alpha_t(s)\}$ satisfy the Robbins–Monro conditions*

$$\sum_{t=1}^{\infty} \alpha_t(s) = \infty, \qquad \sum_{t=1}^{\infty} \alpha_t^2(s) < \infty \quad \textit{for each } s. \tag{6.7}$$

Then $V_t(s) \to V^\pi(s)$ almost surely for all $s \in \mathcal{S}$.

The Robbins–Monro conditions (6.7) require that the step sizes are large enough for the algorithm to correct any initial error (first condition) but small enough for the noise to average out (second condition). The schedule $\alpha_t(s) = 1/N_t(s)$, where $N_t(s)$ is the visit count, satisfies both conditions. A constant step size α satisfies only the first, so it does not converge to the exact V^π but tracks it in a non-stationary environment.

6.1.6 Worked Example: Random Walk

Example 6.1 (Five-state random walk)**.** Consider a five-state Markov chain with states A, B, C, D, E arranged linearly, plus two terminal states L (left) and R (right). From any non-terminal state, the agent moves left or right with equal probability 0.5. The only nonzero reward is $+1$ on the transition into the right terminal state R; all other rewards are 0. The discount factor is $\gamma = 1$. The true state values (i.e. the probability of eventually reaching R) are

$$V^\pi(A) = \tfrac{1}{6}, \quad V^\pi(B) = \tfrac{2}{6}, \quad V^\pi(C) = \tfrac{3}{6}, \quad V^\pi(D) = \tfrac{4}{6}, \quad V^\pi(E) = \tfrac{5}{6}.$$

Suppose we initialise $V(s) = 0.5$ for all states and run TD(0) with $\alpha = 0.1$. Consider a single trajectory that goes $C \to D \to E \to R$.

Step 1: $S_t = C$, $A_t = \text{right}$, $R_{t+1} = 0$, $S_{t+1} = D$.

$$\delta_t = 0 + 1.0 \cdot 0.5 - 0.5 = 0.$$

No update occurs this step (since $V(D) = V(C) = 0.5$ initially).

Step 2: $S_t = D$, $R_{t+1} = 0$, $S_{t+1} = E$.

$$\delta_t = 0 + 0.5 - 0.5 = 0.$$

Again no change.

Step 3: $S_t = E$, $R_{t+1} = 1$, $S_{t+1} = R$ (terminal, $V(R) = 0$).

$$\delta_t = 1 + 0 - 0.5 = 0.5.$$

$$V(E) \leftarrow 0.5 + 0.1 \times 0.5 = 0.55.$$

The reward signal propagates only one step backward. Over many episodes, TD(0) gradually propagates value estimates backward through the chain. After approximately 100 episodes, the estimates are close to the true values. This example demonstrates both the speed advantage of TD (learning within each episode) and the credit-assignment challenge: each new reward signal must propagate back through the state space visit by visit.

6.2 Advantages of TD Methods

TD methods occupy a privileged position in the RL landscape because they inherit advantages from both Monte Carlo and dynamic programming while avoiding their main weaknesses. We catalogue these advantages carefully.

No complete episode required. Monte Carlo methods must wait until the end of an episode to compute G_t. This is problematic for long episodes and impossible for continuing tasks with no natural termination. TD(0) updates after every single step, making it immediately applicable to any task structure.

Works for continuing tasks. Because the TD target $R_{t+1} + \gamma V(S_{t+1})$ is always available (no terminal state needed), TD methods naturally handle continuing tasks. Monte Carlo methods cannot, in their basic form, be applied to such settings.

Lower variance than Monte Carlo. The Monte Carlo return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$ accumulates variance from every future reward. The TD target $R_{t+1} + \gamma V(S_{t+1})$ only involves one random reward and the (relatively smooth) estimate $V(S_{t+1})$. Formally, if each reward has variance σ^2 and transitions are independent, the MC variance grows roughly as $\sum_{k=0}^{T-t-1} \gamma^{2k}\sigma^2$, whereas the TD variance is approximately $\sigma^2 + \gamma^2 \operatorname{Var}[V(S_{t+1})]$.

Bias–variance trade-off. The price paid for lower variance is that the TD target is *biased*: since $V \neq V^\pi$ early in training, the target $R_{t+1} + \gamma V(S_{t+1})$ does not equal $V^\pi(S_t)$ in expectation. By Theorem 6.1, bias disappears at convergence. Monte Carlo is unbiased but high-variance. This fundamental trade-off — bootstrapping buys lower variance at the cost of bias — runs through all TD methods and is quantified in Section 6.11.

Faster credit assignment. Because TD propagates value estimates one step at a time within each episode, useful information from rewarding transitions is incorporated into earlier states more quickly than with MC. n-step TD (Section 6.8) and eligibility traces (Section 6.9) provide mechanisms to propagate credit across multiple steps efficiently.

Batch TD(0) and the maximum likelihood estimate. When a fixed batch of episodes is available, one can repeatedly apply TD(0) updates until convergence. This *batch TD(0)* converges to the *maximum likelihood* (certainty-equivalence) estimate of V^π given the data — the exact solution that would be computed by constructing the empirical MDP from the batch and solving it. In contrast, batch Monte Carlo converges to the estimate that minimises the mean-squared error on the training returns. In problems with limited data, the TD solution often generalises better because it exploits the Markov structure.

6.3 SARSA: On-Policy TD Control

6.3.1 From Prediction to Control

Policy evaluation tells us how good the current policy is. Control requires we improve it. As in Monte Carlo control (Chapter 5), the key insight is to estimate the *action-value function* $Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$ rather than $V^\pi(s)$, because greedy improvement requires comparing actions:

$$\pi'(s) \in \operatorname*{argmax}_{a \in \mathcal{A}} Q(s,a).$$

The Bellman equation for Q^π is

$$Q^\pi(s,a) = \mathbb{E}_\pi\big[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a\big]. \tag{6.8}$$

Replacing the expectation in (6.8) by a single sample and applying the resulting update gives:

6.3.2 The SARSA Update

Definition 6.3 (SARSA)**. SARSA**[1] (Rummery and Niranjan, 1994; Sutton, 1996) is the on-policy TD control algorithm with update rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big). \tag{6.9}$$

The critical feature of SARSA is that A_{t+1} is drawn from the *same* policy π that is being used to collect data. This makes SARSA **on-policy**: it evaluates and improves the policy that generates the data.

Proposition 6.3. *The target $Y_t^{\text{SARSA}} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ is an unbiased sample of the Bellman operator applied to Q:*

$$\mathbb{E}_\pi\big[Y_t^{\text{SARSA}} \mid S_t = s, A_t = a\big] = (T^\pi Q)(s, a),$$

where $(T^\pi Q)(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$.

Proof. By definition of the conditional expectation, the claim follows immediately. □

Algorithm 10: SARSA (On-Policy TD Control)

```
Input: Step size α, discount γ, exploration ε
Initialise Q(s, a) for all s, a; Q(s_term, ·) ← 0;
for each episode do
    Initialise S;
    Choose A from π(· | S) derived from Q (e.g. ε-greedy);
    for each step of episode until S is terminal do
        Take action A; observe R, S';
        Choose A' from π(· | S') (same ε-greedy);
        Q(S, A) ← Q(S, A) + α(R + γQ(S', A') − Q(S, A));
        S ← S';   A ← A';
```

6.3.3 Convergence of SARSA

Theorem 6.4 (Convergence of SARSA)**.** *In the tabular setting, SARSA converges to Q^* almost surely if: (1) all state-action pairs are visited infinitely often; (2) the policy converges to the greedy policy (e.g. $\varepsilon_t \to 0$); and (3) the step sizes satisfy the Robbins–Monro conditions* (6.7)*.*

The convergence requires the policy to become greedy in the limit, so exploration must be reduced over time. In practice, constant ε or constant α are often used, in which case SARSA converges to a neighbourhood of Q^* rather than to Q^* exactly.

[1] The name stands for the five-tuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ used in each update.

6.3.4 Worked Example: Cliff Walking

Example 6.2 (Cliff Walking)**.** Consider a 4×12 gridworld with a cliff along the bottom edge (cells $(3, 1)$ through $(3, 10)$). The agent starts at $(3, 0)$ and must reach the goal at $(3, 11)$. Stepping onto the cliff gives reward -100 and returns the agent to start. All other transitions give reward -1. Episodes terminate at the goal or after 200 steps.

SARSA with $\varepsilon = 0.1$ learns the *safe path* along the top of the grid, staying far from the cliff. Because the exploratory actions occasionally push the agent toward the cliff, SARSA evaluates the cliff cells as dangerous and avoids them. Q-learning (Section 6.4), being off-policy, evaluates each state under the *greedy* policy and therefore discovers the shortest path along the cliff edge — but this path is fragile in practice because ε-greedy exploration sometimes causes it to fall into the cliff during training. This is the canonical illustration of the on-policy/off-policy trade-off: SARSA is safer during learning; Q-learning finds the theoretically optimal policy.

6.4 Q-Learning: Off-Policy TD Control

6.4.1 The Q-Learning Update

Q-learning (Watkins, 1989) directly approximates the optimal action-value function Q^* by using the Bellman *optimality* equation as its target. Since $Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid S_t = s, A_t = a]$, a single-sample approximation gives

$$\boxed{Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\Big).} \tag{6.10}$$

The key difference from SARSA (6.9) is the **max** operator. No matter how A_{t+1} is actually chosen (whether by ε-greedy or any other behaviour policy), the target always uses the greedy action in S_{t+1}. This makes Q-learning **off-policy**: the behaviour policy collects data, but the update evaluates the greedy (target) policy.

Algorithm 11: Q-Learning (Off-Policy TD Control)

```
Input: Step size α, discount γ, exploration ε
Initialise Q(s, a) for all s, a; Q(s_term, ·) ← 0;
for each episode do
    Initialise S;
    for each step until S is terminal do
        Choose A from Q using ε-greedy;
        Take action A; observe R, S';
        Q(S, A) ← Q(S, A) + α(R + γ max_a' Q(S', a') − Q(S, A));
        S ← S';
```

6.4.2 Convergence of Q-Learning

Theorem 6.5 (Convergence of Q-learning, Watkins and Dayan 1992)**.** *In a finite MDP, if all state-action pairs are visited infinitely often, rewards are bounded, and the step sizes satisfy the Robbins–Monro conditions* (6.7)*, then Q-learning converges to Q^* almost surely.*

This result is remarkable: it holds regardless of the behaviour policy used to generate transitions, as long as every (s, a) pair is visited sufficiently often. Q-learning is thus a fully off-policy method with convergence guarantees in the tabular case.

6.4.3 Maximisation Bias

A subtle but important problem arises from using $\max_{a'} Q(S_{t+1}, a')$ as a target. When Q is an estimate with noise, the maximum over noisy values is a *positively biased* estimator of the true maximum value. This is because, by Jensen's inequality applied to the convex max function,

$$\mathbb{E}\Big[\max_{a'} Q(S_{t+1}, a')\Big] \geq \max_{a'} \mathbb{E}[Q(S_{t+1}, a')].$$

The bias can be substantial when there are many actions with similar true values but noisy estimates.

Example 6.3 (Maximisation bias)**.** Suppose state s' has three actions with true values $Q^*(s', a_1) = Q^*(s', a_2) = Q^*(s', a_3) = 0$. Estimates $\hat{Q}(s', a_i) \sim \mathcal{N}(0, \sigma^2)$ are independent Gaussian noise. Then $\mathbb{E}[\max_i \hat{Q}(s', a_i)] > 0$, a positive bias that grows with σ^2 and with the number of actions. Q-learning uses this biased maximum as a target, causing systematic overestimation.

6.5 Expected SARSA

SARSA uses the actual next action A_{t+1} in its target, introducing variance from the policy's randomness. Q-learning eliminates this variance by taking the greedy action, but incurs maximisation bias. **Expected SARSA** (van Seijen et al., 2009) takes the middle path: it replaces $Q(S_{t+1}, A_{t+1})$ with the *expectation* over the policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1})\, Q(S_{t+1}, a') - Q(S_t, A_t)\right). \tag{6.11}$$

Algorithm 12: Expected SARSA

Input: Step size α, discount γ, policy π (or ε-greedy)
Initialise $Q(s, a)$ for all s, a;
for *each episode* **do**
  Initialise S;
  for *each step until S is terminal* **do**
    Choose A from $\pi(\cdot \mid S)$;
    Take action A; observe R, S';
    Compute $\bar{Q}(S') \leftarrow \sum_{a'} \pi(a' \mid S')\, Q(S', a')$;
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big(R + \gamma\, \bar{Q}(S') - Q(S, A)\big)$;
    $S \leftarrow S'$;

Expected SARSA eliminates the variance from the random choice of A_{t+1}, so its updates are less noisy than SARSA's. It can also be used off-policy: if π is the greedy policy

(probability 1 on $\operatorname{argmax}_a Q(s,a)$, probability 0 elsewhere), Expected SARSA reduces exactly to Q-learning, since

$$\sum_{a'} \pi_{\text{greedy}}(a' \mid s')Q(s',a') = \max_{a'} Q(s',a').$$

Thus Expected SARSA generalises both SARSA (on-policy, π is the behaviour policy) and Q-learning (off-policy, π is greedy).

6.6 Double Q-Learning

Double Q-learning (van Hasselt, 2010) addresses maximisation bias by maintaining *two independent* action-value estimates, Q_1 and Q_2. The idea is to *decouple* action selection from action evaluation:

- Use Q_1 to select the greedy action: $a^* = \operatorname{argmax}_{a'} Q_1(S_{t+1}, a')$.
- Use Q_2 to evaluate that action: target $= R_{t+1} + \gamma Q_2(S_{t+1}, a^*)$.

Since Q_1 and Q_2 are independent estimates, using one to select and the other to evaluate removes the upward bias. Crucially, the roles of Q_1 and Q_2 are swapped with probability 0.5 at each step to keep both estimates up to date.

Algorithm 13: Double Q-Learning

```
Input: Step size α, discount γ, exploration ε
Initialise Q1(s,a) and Q2(s,a) for all s,a;
for each episode do
    Initialise S;
    for each step until S is terminal do
        A ← ε-greedy w.r.t. Q1(S,·) + Q2(S,·);
        Observe R, S';
        if with probability 0.5 then
            a* ← argmax_a' Q1(S',a');
            Q1(S,A) ← Q1(S,A) + α(R + γQ2(S',a*) − Q1(S,A));
        else
            a* ← argmax_a' Q2(S',a');
            Q2(S,A) ← Q2(S,A) + α(R + γQ1(S',a*) − Q2(S,A));
        S ← S';
```

Double Q-learning provably reduces maximisation bias and performs better than Q-learning in environments with many actions. The idea was later scaled to deep RL in the Double DQN algorithm (van Hasselt, Guez, and Silver, 2016), which uses the online network for action selection and the target network for evaluation.

6.7 Comparison: SARSA, Q-Learning, and Expected SARSA

Table 6.1 summarises the three main TD control algorithms.

Table 6.1: Comparison of on-policy and off-policy TD control algorithms.

Property	**SARSA**	**Q-Learning**	**Expected SARSA**
Target	$R+\gamma Q(S',A')$	$R+\gamma \max_{a'} Q(S',a')$	$R+\gamma \sum_{a'} \pi(a'\|S')Q(S',a')$
Policy	On-policy	Off-policy	Both
Goal	Q^π	Q^*	Q^π or Q^*
Variance	Medium	Low	Lower than SARSA
Bias	Unbiased	Max. bias	Unbiased
Safe?	Yes	No (cliff)	Yes

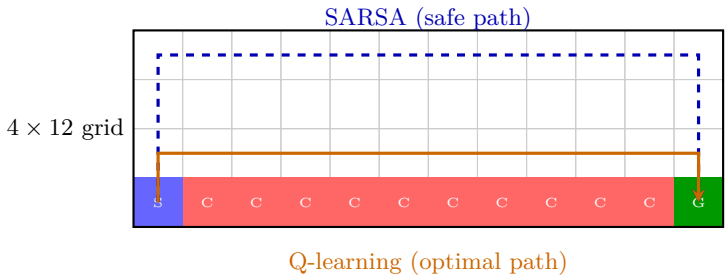


Figure 6.1: Cliff Walking gridworld. The red cells are the cliff (reward -100, reset to start). SARSA (dashed blue) learns a safe path along the top; Q-learning (solid orange) learns the theoretically shorter path along the cliff edge but is fragile under ε-greedy exploration.

The Cliff Walking example (Example 6.2 and Figure 6.1) illustrates the practical consequence of on-policy vs. off-policy evaluation. During training with ε-greedy exploration, SARSA accounts for the possibility of random actions pushing it into the cliff and therefore prefers the safer route. Q-learning evaluates states under the greedy policy — which never steps off the cliff — so it learns the cliff-edge route, which is optimal but fragile. At test time ($\varepsilon = 0$) both find equivalent paths, but SARSA achieves higher average reward during learning.

6.8 n-Step TD Methods

6.8.1 The n-Step Return

Both TD(0) and Monte Carlo are special cases of a more general family parameterised by the *lookahead horizon* $n \in \{1, 2, \dots\}$. The **n-step return** is defined as

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}), \tag{6.12}$$

and the corresponding update is

$$V(S_t) \leftarrow V(S_t) + \alpha\big(G_t^{(n)} - V(S_t)\big). \tag{6.13}$$

Proposition 6.6. *The n-step return satisfies:*

1. *At $n = 1$: $G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$, the TD(0) target.*
2. *In an episodic task with terminal time T: $G_t^{(T-t)} = G_t$, the full Monte Carlo return (using $V(S_T) = 0$).*

Proof. (1) Setting $n = 1$ in (6.12) gives $G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$. (2) With $n = T - t$ and $V(S_T) = 0$: $G_t^{(T-t)} = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} = G_t$. □

6.8.2 Bias–Variance Trade-Off in n-Step TD

As n increases, the n-step return interpolates between the biased-but-low-variance TD(0) target and the unbiased-but-high-variance MC return:

- Small n: more bootstrapping, higher bias, lower variance.
- Large n: less bootstrapping, lower bias, higher variance.

The optimal n depends on the problem: in environments with dense, low-variance rewards, small n works well; in sparse-reward environments, larger n is often beneficial.

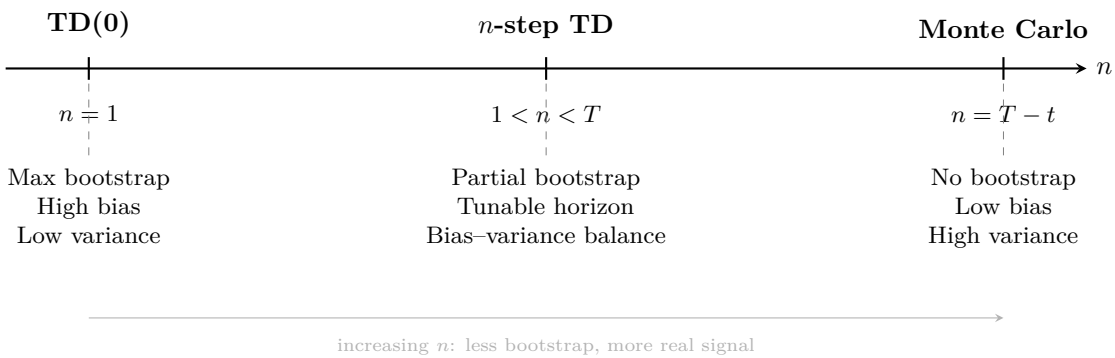


Figure 6.2: The spectrum of n-step TD methods. TD(0) ($n = 1$) and Monte Carlo ($n = T - t$) are the extreme cases. Intermediate n provides a bias–variance trade-off that can be tuned to the task.

6.8.3 n-Step SARSA

The n-step idea extends naturally to control. The n-step SARSA target is

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n}),$$

and the update is $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(G_t^{(n)} - Q(S_t, A_t))$. This algorithm shares convergence guarantees with standard SARSA under appropriate conditions.

6.9 TD(λ) and Eligibility Traces

The n-step methods of the previous section require fixing a single horizon n in advance. TD(λ) instead combines *all* n-step returns simultaneously, weighting them geometrically so that the combined target is computable online with a single extra vector, the *eligibility trace*.

6.9.1 The λ-Return

Definition 6.4 (λ-return)**.** For $\lambda \in [0, 1]$, the **λ-return** is the geometrically weighted average of all n-step returns:

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}. \tag{6.14}$$

The weight given to the n-step return is $(1-\lambda)\lambda^{n-1}$. These weights are positive, decrease geometrically with n, and sum to one:

$$\sum_{n=1}^{\infty} (1-\lambda)\lambda^{n-1} = (1-\lambda) \cdot \frac{1}{1-\lambda} = 1.$$

Thus G_t^λ is a genuine convex combination of the n-step returns.

Special cases.

- $\lambda = 0$: $G_t^0 = G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$, the TD(0) target. All weight is on the one-step return.
- $\lambda = 1$: $G_t^1 = G_t$, the complete Monte Carlo return. The geometric weights are flat (all equal to zero for finite n; the limit equals the full return).

Thus TD(λ) subsumes both TD(0) and Monte Carlo as extreme cases, just as n-step TD does — but with an elegant, differentiable interpolation.

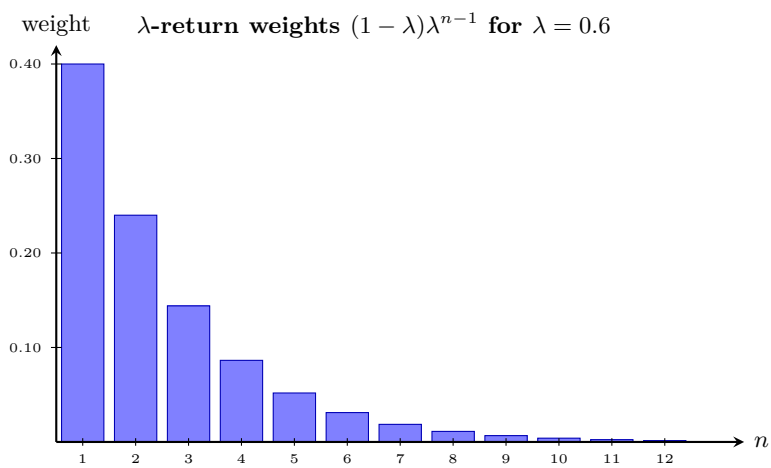


Figure 6.3: Weights $(1-\lambda)\lambda^{n-1}$ assigned to each n-step return in the λ-return ($\lambda = 0.6$). Short-horizon returns receive the most weight; the weights decay geometrically, so the influence of distant returns diminishes rapidly.

6.9.2 Forward View of TD(λ)

The **forward view** of TD(λ) defines the update target for state S_t as G_t^λ and applies

$$V(S_t) \leftarrow V(S_t) + \alpha\big(G_t^\lambda - V(S_t)\big).$$

This is conceptually clean: the target is a specific, well-defined quantity. However, computing G_t^λ requires the entire future trajectory from time t, so the forward view requires waiting until the episode terminates before any update can be applied. It is therefore not online in the operational sense, and it is equivalent to a single-pass post-hoc update over the episode.

Despite its impracticality for online learning, the forward view is the *gold standard* for understanding what TD(λ) computes. The backward view, which we develop next, is mathematically equivalent to the forward view but operates online.

6.9.3 Eligibility Traces

The key to making TD(λ) online is the **eligibility trace** $e_t(s)$, a vector over states that records how "eligible" each state is for a credit update at time t.

Definition 6.5 (Eligibility trace (accumulating))**.** The **accumulating eligibility trace** for state s is initialised to $e_0(s) = 0$ and updated each step by

$$e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbb{1}\{S_t = s\}. \tag{6.15}$$

The trace $e_t(s)$ increases by 1 each time state s is visited and decays by factor $\gamma\lambda$ at every time step. Intuitively, it measures the recency and frequency of visits to s: a state visited frequently and recently has a high trace.

An alternative is the **replacing eligibility trace**:

$$e_t(s) = \begin{cases} \gamma\lambda\, e_{t-1}(s) + 1 & \text{if } s = S_t, \\ \gamma\lambda\, e_{t-1}(s) & \text{otherwise,} \end{cases} \quad \text{capped at 1.} \tag{6.16}$$

Replacing traces prevent the trace for the current state from exceeding 1, which can stabilise learning in some settings.

6.9.4 Backward View of TD(λ): The Online Algorithm

In the **backward view**, a single TD error δ_t is computed each step, and *all* states are updated simultaneously, weighted by their eligibility traces:

$$\boxed{V(s) \leftarrow V(s) + \alpha\, \delta_t\, e_t(s), \quad \forall s \in \mathcal{S}.} \tag{6.17}$$

The TD error $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is broadcast to all states, but only states with a nonzero trace receive a meaningful update. Recently visited states — which have high traces — receive large updates; states not visited recently have near-zero traces and are barely affected. This is the "credit assignment" mechanism of eligibility traces: good or bad outcomes are credited proportionally to the recency and frequency of past visits.

In practice, only states with nonzero traces need to be updated, so the inner loop over all s is typically replaced by maintenance of an active-trace set. The algorithm requires $O(|\mathcal{S}|)$ memory per episode — one trace value per state.

6.9.5 Equivalence of Forward and Backward Views

The following theorem is the cornerstone of the eligibility trace theory.

Theorem 6.7 (Forward–backward equivalence of TD(λ))**.** *For the offline (end-of-episode) updating version of TD(λ) with constant step size α, the sum of all updates made to $V(s)$*

Algorithm 14: TD(λ) with Eligibility Traces

```
Input: Step size α, discount γ, trace parameter λ
Initialise V(s) ← 0 for all s;
for each episode do
    e(s) ← 0 for all s;
    Initialise S;
    for each step t until S is terminal do
        Choose A ~ π(· | S); observe R, S';
        δ ← R + γV(S') − V(S);
        e(S) ← e(S) + 1    (accumulating trace);
        for all s ∈ 𝒮 do
            V(s) ← V(s) + α δ e(s);
            e(s) ← γλ e(s);
        S ← S';
```

during an episode is identical under the forward view and the backward view. That is, defining $\Delta_t V(s) = \alpha(G_t^\lambda - V(S_t))\mathbb{1}\{S_t = s\}$ *(forward) and* $\Delta_t^{\mathrm{bw}} V(s) = \alpha\delta_t e_t(s)$ *(backward):*

$$\sum_{t=0}^{T-1} \Delta_t^{\mathrm{bw}} V(s) = \sum_{t=0}^{T-1} \Delta_t V(s), \quad \forall s \in \mathcal{S}. \tag{6.18}$$

Proof sketch. The key identity is

$$G_t^\lambda - V(S_t) = \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} \delta_k,$$

which expresses the λ-return error as a discounted sum of future TD errors. To verify this, expand G_t^λ using the definition (6.14), write each $G_t^{(n)}$ as a telescoping sum of TD errors (using $G_t^{(n)} = \delta_t + \gamma G_{t+1}^{(n-1)}$ recursively), then collect terms. The full algebraic derivation can be found in Sutton & Barto (2018, Chapter 12).

Once this identity is established, summing $\alpha\delta_t e_t(s)$ over t and exchanging the order of summation gives $\sum_t \alpha(G_t^\lambda - V(S_t))\mathbb{1}\{S_t = s\}$, completing the proof. □

Theorem 6.7 confirms that the online, incremental backward view computes exactly the same updates as the offline forward view. This is a profound result: it means we do not have to choose between conceptual clarity (forward) and computational efficiency (backward).

6.9.6 Practical Considerations

Remark 6.1 (Trace decay and effective horizon). The product $\gamma\lambda$ controls the effective decay rate of eligibility traces. For $\gamma\lambda = 0.9$, a state visited k steps ago has trace 0.9^k, so states visited more than ~ 20 steps ago have negligible trace. Practitioners often tune λ jointly with γ to set the desired effective horizon.

Remark 6.2 (Computational cost). The naive implementation of Algorithm 14 updates all $|\mathcal{S}|$ traces at each step, which is $O(|\mathcal{S}|)$ per step. With tabular representations this is feasible for moderate state spaces. With function approximation (Chapter 7), the trace becomes a vector of the same dimension as the parameter vector, and the per-step cost is $O(d)$ where d is the number of parameters.

Remark 6.3 (SARSA(λ) and Q(λ)). Eligibility traces extend naturally to control. **SARSA(λ)** maintains a trace $e_t(s, a)$ over state-action pairs and uses the SARSA TD error $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$ to update $Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t e_t(s, a)$. Similarly, **Q($\lambda$)** applies the Q-learning TD error with traces. SARSA(λ) is particularly popular in practice because it maintains the on-policy nature of SARSA while incorporating the multi-step credit assignment of eligibility traces.

6.9.7 Worked Example: Random Walk with TD(λ)

Example 6.4 (Effect of λ on learning speed)**.** Return to the five-state random walk of Example 6.1. Running TD(λ) with various values of λ for a fixed number of episodes and measuring the RMS error relative to the true values, we observe:

- $\lambda = 0$ (TD(0)): learns slowly; reward signal takes many episodes to propagate to distant states.
- $\lambda = 0.5$: traces ensure that a reward obtained at the right terminal propagates backward to states visited in the last several steps within the same episode, substantially accelerating learning.
- $\lambda = 0.9$: fast early learning but slightly more noisy, since traces give significant weight to distant states that may not have contributed to the final reward.
- $\lambda = 1$ (MC): slowest per-episode variance is high, but with a well-tuned step size, total learning across episodes can be competitive.

In Sutton and Barto's experiments, intermediate values $\lambda \approx 0.7$–0.9 often achieve the lowest RMS error for this task. The optimal λ depends on the step size and the task structure.

6.10 Generalised Advantage Estimation (GAE)

Generalised Advantage Estimation (Schulman et al., 2016) is a modern technique for computing low-variance advantage estimates for use in policy gradient methods (Chapter 8). It is built directly on the TD(λ) machinery and provides a compelling application of the theory developed in this chapter.

6.10.1 The Advantage Function

Definition 6.6 (Advantage function)**.** The **advantage function** of policy π is

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \tag{6.19}$$

The advantage $A^\pi(s, a)$ measures how much better action a is in state s compared to the average action under policy π. By definition:

- $A^\pi(s, a) > 0$: action a is better than average; the policy should increase the probability of a.
- $A^\pi(s, a) < 0$: action a is worse than average; decrease its probability.
- $A^\pi(s, a) = 0$: action a performs exactly at the average.

Note that $\mathbb{E}_{a \sim \pi(\cdot|s)}[A^\pi(s, a)] = 0$ for all s, so the advantage is centred.

Why advantage matters for policy gradients. The policy gradient theorem (Chapter 8) states that the gradient of the expected return with respect to the policy parameters θ is

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi\big[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \cdot \Psi_t\big],$$

where Ψ_t can be the return G_t, the action-value $Q^\pi(S_t, A_t)$, or — crucially — the advantage $A^\pi(S_t, A_t)$. Using the advantage instead of the raw return or action-value dramatically reduces variance: the baseline $V^\pi(S_t)$ is subtracted, removing all variation due to the state (which the policy cannot influence).

6.10.2 n-Step Advantage Estimates

In practice, V^π and A^π are unknown. Given a learned critic $V_\phi \approx V^\pi$, a natural estimator of the one-step advantage is the TD error:

$$\hat{A}_t^{(1)} = \delta_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t). \tag{6.20}$$

More generally, the n-step advantage estimate is

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n V_\phi(S_{t+n}) - V_\phi(S_t). \tag{6.21}$$

Note that $\hat{A}_t^{(n)} = G_t^{(n)} - V_\phi(S_t)$, where $G_t^{(n)}$ is the n-step return (6.12).

6.10.3 GAE: The λ-Weighted Advantage

Definition 6.7 (Generalised Advantage Estimation)**.** The **Generalised Advantage Estimation** $\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}$ is the exponentially weighted average of n-step advantage estimates:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}, \tag{6.22}$$

where $\delta_{t+l} = R_{t+l+1} + \gamma V_\phi(S_{t+l+1}) - V_\phi(S_{t+l})$ is the TD error at time $t + l$.

Proposition 6.8. *GAE is a λ-weighted average of n-step advantage estimates:*

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)}. \tag{6.23}$$

Proof. We use the key identity $\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} (\gamma)^l \delta_{t+l}$ (telescoping), which follows from

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n V_\phi(S_{t+n}) - V_\phi(S_t) = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}.$$

Then

$$\begin{aligned}(1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\hat{A}_t^{(n)} &= (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\sum_{l=0}^{n-1}\gamma^l\delta_{t+l}\\ &= (1-\lambda)\sum_{l=0}^{\infty}\gamma^l\delta_{t+l}\sum_{n=l+1}^{\infty}\lambda^{n-1}\\ &= (1-\lambda)\sum_{l=0}^{\infty}\gamma^l\delta_{t+l}\cdot\frac{\lambda^l}{1-\lambda}\\ &= \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}.\end{aligned}$$

□

6.10.4 Special Cases and Bias–Variance Trade-Off

- $\lambda = 0$: $\hat{A}_t^{\mathrm{GAE}(\gamma,0)} = \delta_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$. This is the one-step TD advantage — lowest variance, highest bias (relies entirely on V_ϕ).
- $\lambda = 1$: $\hat{A}_t^{\mathrm{GAE}(\gamma,1)} = \sum_{l=0}^{\infty}\gamma^l\delta_{t+l} = G_t - V_\phi(S_t)$. This is the Monte Carlo advantage estimate — lowest bias, highest variance.
- Intermediate λ: exponentially decaying weights average short-horizon (low-variance, high-bias) and long-horizon (high-variance, low-bias) estimates.

6.10.5 Connection to TD(λ)

The structure of GAE (6.22) is identical to the λ-return mechanism. In fact, if we view $V_\phi(S_t)$ as a baseline, then

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = G_t^\lambda - V_\phi(S_t),$$

where G_t^λ is the λ-return computed with the critic V_ϕ. Thus GAE is simply the TD(λ) target minus the current value estimate. The eligibility trace view provides an efficient way to compute GAE online: maintain a trace e_t and accumulate $e_t\,\delta_t$ terms.

In practice (e.g. in the PPO and TRPO algorithms), GAE is computed in a backward pass over a collected rollout of length T:

$$\begin{aligned}&A_T = 0,\\ &A_t = \delta_t + \gamma\lambda\, A_{t+1}, \quad t = T-1, T-2, \ldots, 0,\end{aligned}$$

which is a linear recursion that can be computed in $O(T)$ time.

Example 6.5 (GAE computation on a short rollout)**.** Suppose $\gamma = 0.99$, $\lambda = 0.95$, and a rollout of 4 steps yields TD errors $\delta_0 = 0.5$, $\delta_1 = -0.3$, $\delta_2 = 0.8$, $\delta_3 = 0.1$ (with $A_4 = 0$). The backward computation gives:

$$\begin{aligned}A_3 &= 0.1 + 0 = 0.1,\\ A_2 &= 0.8 + 0.99\times 0.95\times 0.1 = 0.8 + 0.094 = 0.894,\\ A_1 &= -0.3 + 0.99\times 0.95\times 0.894 = -0.3 + 0.840 = 0.540,\\ A_0 &= 0.5 + 0.99\times 0.95\times 0.540 = 0.5 + 0.508 = 1.008.\end{aligned}$$

The advantage estimate $A_0 = 1.008$ for the first step accounts not only for $\delta_0 = 0.5$ but also for the downstream positive signal $\delta_2 = 0.8$, appropriately discounted by $(\gamma\lambda)^2$.

6.11 Bias, Variance, and Convergence Analysis

6.11.1 Formal Bias Analysis

The TD(0) target $R_{t+1} + \gamma V_t(S_{t+1})$ is biased whenever $V_t \neq V^\pi$. We can decompose the bias as follows. Let V_t denote the value estimate at iteration t and let $b_t(s) = V_t(s) - V^\pi(s)$ denote the approximation error at iteration t. Then

$$\mathbb{E}_\pi[R_{t+1} + \gamma V_t(S_{t+1}) \mid S_t = s] = V^\pi(s) + \gamma\, \mathbb{E}_\pi[b_t(S_{t+1}) \mid S_t = s],$$

So the bias of the TD target equals γ times the expected approximation error at the next state. This is strictly smaller than the approximation error at s in magnitude (scaled by $\gamma < 1$), which is the mechanism by which TD updates are pulled toward V^π. As $V_t \to V^\pi$, the bias vanishes, consistent with Theorem 6.1.

Monte Carlo, on the other hand, uses the unbiased target G_t, so its bias is zero regardless of V_t.

6.11.2 Variance Analysis

The variance of the TD(0) target $R_{t+1} + \gamma V(S_{t+1})$ is bounded by

$$\mathrm{Var}[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s] \leq \sigma_R^2 + \gamma^2\, \mathrm{Var}[V(S_{t+1}) \mid S_t = s],$$

where σ_R^2 is the reward variance. When V is a smooth function (e.g. a neural network), $\mathrm{Var}[V(S_{t+1})]$ is small. In contrast, the MC return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ accumulates variance from every step. Since rewards at different time steps are generally correlated through the shared trajectory, the full expression includes cross-covariance terms:

$$\begin{aligned}\mathrm{Var}[G_t \mid S_t = s] &= \sum_{k=0}^{\infty} \gamma^{2k}\, \mathrm{Var}[R_{t+k+1} \mid S_t = s] \\ &\quad + 2 \sum_{0 \leq j < k} \gamma^{j+k}\, \mathrm{Cov}[R_{t+j+1}, R_{t+k+1} \mid S_t = s] \\ &\gg \mathrm{Var}[R_{t+1} + \gamma V(S_{t+1})].\end{aligned}$$

This variance gap grows with episode length and discount, making TD methods substantially more sample-efficient than MC in many practical settings.

6.11.3 Mean Squared Error Decomposition

For any estimator $\hat{y}$ of a target $y^* = \mathbb{E}[Y]$, the mean squared error decomposes as

$$\mathrm{MSE}(\hat{y}) = \underbrace{\big(\mathbb{E}[\hat{y}] - y^*\big)^2}_{\mathrm{Bias}^2} + \underbrace{\mathrm{Var}[\hat{y}]}_{\mathrm{Variance}}. \tag{6.24}$$

This decomposition applies to the TD target (biased, low variance) and the MC return (unbiased, high variance). Minimising MSE requires balancing both terms, which is precisely what TD(λ) does by tuning λ.

Table 6.2: Bias, variance, and convergence properties of prediction methods.

Method	Bias	Variance	Online?	Convergence
Monte Carlo	Zero	High (grows with T)	No	Yes (to V^π)
TD(0)	High early, $\to 0$	Low	Yes	Yes (Thm. 6.2)
n-step TD	Medium	Medium	Semi	Yes
TD(λ)	Tunable (λ)	Tunable (λ)	Yes	Yes

6.11.4 Comparison Table

6.11.5 The Deadly Triad

Tabular TD methods are guaranteed to converge under mild conditions. However, combining three ingredients leads to potential divergence:

1. **Function approximation** (e.g. linear or neural network representations of V).
2. **Bootstrapping** (TD updates using the current estimate as a target).
3. **Off-policy training** (learning from data generated by a different policy).

This combination — known as the *deadly triad* — can cause the value estimates to diverge to infinity even in simple linear examples (Baird's counterexample, 1995). Monte Carlo avoids the triad by not bootstrapping; on-policy methods avoid it by staying on-policy; some algorithms (e.g. gradient TD methods, emphatic TD) address it through careful algorithmic design. Chapter 7 treats these issues in depth.

6.11.6 Convergence Under the Robbins–Monro Schedule

The Robbins–Monro conditions (6.7) are necessary for almost-sure convergence of stochastic approximation algorithms. Their intuition is as follows:

- The divergence condition $\sum_t \alpha_t = \infty$ ensures the algorithm can correct any initial error, no matter how large.
- The square-summability condition $\sum_t \alpha_t^2 < \infty$ ensures accumulated noise is suppressed: the variance of the sum $\sum_t \alpha_t \epsilon_t$ (where ϵ_t is noise) is bounded by $\sigma^2 \sum_t \alpha_t^2 < \infty$.

The harmonic schedule $\alpha_t = 1/t$ satisfies both conditions. The constant schedule $\alpha_t = \alpha$ satisfies neither — leading to persistent fluctuations — but is often preferred in non-stationary problems or when rapid adaptation to distribution shifts is needed.

6.12 Summary

Temporal-difference learning unifies the Monte Carlo and dynamic programming traditions. From the single idea of bootstrapping — using current value estimates as targets — an entire family of efficient, online, model-free algorithms emerges. Table 6.3 provides a comprehensive overview of all methods introduced in this chapter.

The key insights of this chapter are:

- The TD error $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is the fundamental credit signal in RL; it has zero mean at convergence (Theorem 6.1).

Table 6.3: Summary of temporal-difference methods introduced in this chapter.

Algorithm	**Type**	**Policy**	**Target**	**Online**	**Credit**
TD(0)	Prediction	Fixed π	$R + \gamma V(S')$	Yes	1 step
n-step TD	Prediction	Fixed π	$G^{(n)}$	Semi	n steps
TD(λ)	Prediction	Fixed π	G^{λ}	Yes	Traces
SARSA	Control	On-policy	$R + \gamma Q(S', A')$	Yes	1 step
SARSA(λ)	Control	On-policy	G_Q^{λ}	Yes	Traces
Q-learning	Control	Off-policy	$R + \gamma \max_{a'} Q(S', a')$	Yes	1 step
Expected SARSA	Control	Both	$R + \gamma \bar{Q}(S')$	Yes	1 step
Double Q-learning	Control	Off-policy	Decoupled max	Yes	1 step

- TD methods trade bias for variance: bootstrapping introduces bias but reduces variance compared to Monte Carlo.
- SARSA is on-policy and safe during exploration; Q-learning is off-policy and learns the optimal policy directly.
- TD(λ) with eligibility traces elegantly bridges the gap between one-step TD and Monte Carlo, with the forward and backward views provably equivalent.
- GAE applies the λ-return machinery to advantage estimation in policy gradient methods, providing a clean bias–variance trade-off parameterised by λ.
- Convergence in the tabular setting is well understood; the combination of function approximation, bootstrapping, and off-policy data (the deadly triad) can cause divergence.

Exercises

Exercise 6.1. Suppose $V(S_t) = 3$, $R_{t+1} = 1$, $\gamma = 0.9$, and $V(S_{t+1}) = 4$.

(a) Compute the TD error δ_t.

(b) Apply one TD(0) update with step size $\alpha = 0.1$. What is the new value $V(S_t)$?

(c) Interpret the sign of δ_t: was the state overvalued or undervalued?

Exercise 6.2. Let $V = V^{\pi}$ be the exact value function. Provide a detailed proof that $\mathbb{E}_{\pi}[\delta_t \mid S_t = s] = 0$ for all s, explicitly stating at which step you use the Bellman equation for V^{π}. Then explain why this property does *not* hold when $V \neq V^{\pi}$.

Exercise 6.3. Consider the Cliff Walking environment of Example 6.2.

(a) Explain in precise terms why SARSA with $\varepsilon = 0.1$ prefers the path far from the cliff, even though the cliff-edge path is shorter.

(b) Q-learning with $\varepsilon = 0.1$ eventually learns the cliff-edge route. If we decrease ε to 0.01, how does this affect (i) the path learned by Q-learning, and (ii) the path learned by SARSA?

(c) At test time with $\varepsilon = 0$, which algorithm would you prefer, and why?

Exercise 6.4. Consider a two-action MDP where the true $Q^*(s, a_1) = Q^*(s, a_2) = 0$. Let $\hat{Q}(s, a_i) \sim \mathcal{N}(0, \sigma^2)$ be independent noisy estimates.

(a) Show that $\mathbb{E}[\max(\hat{Q}(s, a_1), \hat{Q}(s, a_2))] > 0$. (Hint: use the formula for the expected maximum of two standard normal random variables and scale by σ.)

(b) How does this bias grow as the number of actions increases?

(c) Explain how Double Q-learning (Algorithm 13) eliminates this bias.

Exercise 6.5. For the five-state random walk of Example 6.1, initialise all $V(s) = 0.5$ and suppose the trajectory $B \to C \to D \to E \to R$ is observed with rewards $(0, 0, 0, 1)$ and $\gamma = 1$.

(a) Compute the TD(0) target ($n = 1$) for state B.

(b) Compute the 2-step return $G_0^{(2)}$ for state B.

(c) Compute the full Monte Carlo return G_0 for state B.

(d) For the λ-return with $\lambda = 0.5$, compute G_0^λ for state B (include all n-step terms).

Exercise 6.6. In a tabular problem, a trajectory visits states A, B, A, C at times $t = 0, 1, 2, 3$. Using accumulating eligibility traces (6.15) with $\gamma = 0.9$ and $\lambda = 0.8$, compute the trace $e_t(s)$ for each state $s \in \{A, B, C\}$ at each time $t = 0, 1, 2, 3$. Verify that at $t = 3$, the trace $e_3(A)$ is larger than $e_3(B)$ because A was visited twice and most recently.

Exercise 6.7. State the forward–backward equivalence theorem (Theorem 6.7) in your own words. Then use the identity $G_t^\lambda - V(S_t) = \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} \delta_k$ to verify the equivalence for a specific two-step episode: states S_0, S_1, S_2 (terminal), with rewards R_1, R_2 and discount γ. Confirm that $\sum_{t=0}^{1} \alpha\, \delta_t\, e_t(S_0)$ equals $\alpha(G_0^\lambda - V(S_0))$.

Exercise 6.8. Suppose $\gamma = 0.99$, $\lambda = 0.9$, and a trajectory of length 5 yields TD errors $\delta_0 = 1, \delta_1 = 0, \delta_2 = -1, \delta_3 = 2, \delta_4 = 0$.

(a) Compute $\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}$ for $t = 0, 1, 2, 3$ using the backward recursion from Section 6.10.

(b) What would the estimates be if $\lambda = 0$? If $\lambda = 1$?

(c) Interpret the difference: for which value of λ does the advantage estimate at $t = 0$ most strongly reflect the positive signal $\delta_3 = 2$?

Exercise 6.9. The *deadly triad* refers to the combination of bootstrapping, function approximation, and off-policy training.

(a) Explain why removing any one of the three elements restores convergence guarantees.

(b) Monte Carlo with function approximation is not known to diverge. Which element of the triad is missing?

(c) On-policy TD with linear function approximation converges to a neighbourhood of V^π. Which element of the triad is missing?

(d) Propose one algorithm design choice that retains all three elements while attempting to restore convergence.

Exercise 6.10. Consider the step size schedule $\alpha_t = c/(t + t_0)$ for constants $c > 0$ and $t_0 > 0$.

(a) Show that this schedule satisfies the Robbins–Monro conditions (6.7).

(b) Show that the constant schedule $\alpha_t = \alpha$ does *not* satisfy the second condition.

(c) In a non-stationary environment where V^π changes over time, explain why a constant step size is often *preferred* despite lacking the second Robbins–Monro condition.

Exercise 6.11. Show algebraically that when the policy π in Expected SARSA (6.11) is the deterministic greedy policy with respect to Q—i.e. $\pi(a \mid s) = \mathbb{1}\{a = \operatorname{argmax}_{a'} Q(s, a')\}$—the Expected SARSA update reduces exactly to the Q-learning update (6.10).

Exercise 6.12. Fill in the following table for a simple episodic task with average episode length $T = 100$ and reward variance $\sigma^2 = 1$ per step. Assume $\gamma = 0.99$. Estimate (order of magnitude) the variance of the update target for: (a) TD(0), (b) 10-step TD, (c) Monte Carlo. Use the approximation that the n-step return variance is $\sum_{k=0}^{n-1} \gamma^{2k}\sigma^2 + \gamma^{2n} \operatorname{Var}[V(S_n)]$ and that $\operatorname{Var}[V(S_n)] \approx 0$ (perfect critic). Discuss how the bias terms change as n increases.

Part II

Function Approximation and Deep RL

Chapter 7

Value Function Approximation and Deep Reinforcement Learning

The power of neural networks is not that they memorise; it is that they generalise. In reinforcement learning, this distinction separates methods that scale from methods that do not.

The preceding six chapters have built a rigorous foundation for reinforcement learning using tabular representations. In Chapter 3 we formalised the Markov decision process framework and derived the Bellman equations; in Chapter 4 we showed how dynamic programming solves them exactly given a model; and in Chapters 5 and 6 we developed Monte Carlo and temporal-difference methods that learn from experience without a model. These algorithms possess well-understood convergence guarantees and clean mathematical structure. Yet they all share one critical assumption: that we can store and update a separate numerical entry for every state — or every state–action pair — in the problem.

This assumption breaks almost immediately in any realistic application. An Atari game presents roughly $10^{120\,000}$ distinct screen configurations; a robotic arm moving in continuous joint space has an uncountably infinite state space; a language model must assign values to every possible sequence of tokens. No computer, present or future, can store a table of that size. Even if storage were unlimited, the generalisation properties of tabular methods are poor: learning that one game state is dangerous conveys no information whatsoever about a visually similar state.

The solution is *function approximation*: instead of storing one number per state, we learn a parameterised function $V_{\mathbf{w}}(s) \approx V^{\pi}(s)$ whose parameter count d is vastly smaller than $|\mathcal{S}|$. The same set of parameters simultaneously encodes information about all states, and updating the parameters after visiting one state automatically changes the predicted values of nearby states. This is the mechanism of generalisation.

This chapter develops function approximation systematically, from first principles to the deep neural network architectures that power modern reinforcement learning. Section 7.1 quantifies the scalability problem. Section 7.2 introduces linear function approximation, the simplest setting in which convergence can be proved rigorously. Section 7.3 analyses the projected Bellman equation, which characterises the fixed point of TD methods in

function-approximation space. Section 7.4 examines semi-gradient methods and explains precisely why they are not true gradient methods. Section 7.5 confronts the *deadly triad*: the conjunction of function approximation, bootstrapping, and off-policy data that can cause divergence even with linear approximators. Section 7.6 presents Deep Q-Networks (DQN), the first algorithm to demonstrate human-level performance across many diverse tasks by stabilising deep RL training with two key engineering innovations. Section 7.7 surveys important algorithmic improvements built upon the DQN foundation. Section 7.8 surveys what convergence theory can and cannot say about nonlinear function approximation. Finally, Section 7.9 discusses practical considerations that bridge theory and engineering.

7.1 Why Tabular Methods Fail

7.1.1 The Curse of Dimensionality

To understand the necessity of function approximation, we must first quantify how dramatically the state space grows with problem complexity. In a tabular MDP with $|\mathcal{S}|$ states and $|\mathcal{A}|$ actions, representing the action-value function $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ requires storing $|\mathcal{S}| \cdot |\mathcal{A}|$ floating-point numbers. For a small gridworld with 100 cells and 4 actions, that is 400 numbers — trivial. But the size of the state space is not a linear function of the problem's input dimensionality; it is exponential.

Definition 7.1 (Curse of dimensionality)**.** Let the state be described by n features, each taking k discrete values. The total number of states is k^n. The memory and sample complexity of tabular methods therefore grows *exponentially* in the number of features n. This phenomenon is called the **curse of dimensionality**.

Three concrete domains make this vivid.

Example 7.1 (Atari video games)**.** The Atari 2600 produces 210×160 pixel frames with 128 colours per pixel. Mnih et al. (2015) preprocess frames to 84×84 greyscale and stack four consecutive frames to capture motion, giving an observation tensor with $84 \times 84 \times 4 = 28\,224$ values, each an integer in $\{0, \ldots, 255\}$. The number of distinct observations is therefore at most $256^{28\,224}$, a number with roughly 68 000 decimal digits. A tabular Q-function over this space is not merely impractical; it is conceptually absurd. Yet DQN, using a convolutional neural network with ~ 1.5 million parameters, achieved superhuman performance on many of these games. The network succeeds precisely because it generalises across visually similar frames.

Example 7.2 (Continuous robotics)**.** A simulated humanoid robot (e.g., the MuJoCo Humanoid-v4 environment) has a state vector $s \in \mathbb{R}^{376}$ comprising joint positions, joint velocities, and contact forces. This state space is not merely large — it is continuous and uncountably infinite. No discretisation scheme can reduce it to a manageable table without introducing severe approximation errors. Moreover, the action space is $\mathbb{R}^{17}$, so the argmax operation central to Q-learning becomes a continuous optimisation problem at every step.

Example 7.3 (Language and text)**.** In applications of RL to language model fine-tuning (discussed further in Chapter 14), the state is the current token prefix — the entire conversation history up to the current step. With a vocabulary of size 50 000 and a context window of 2 048 tokens, the number of possible prefixes is astronomically large. Furthermore, the semantics of two prefixes differing by a single word can be nearly identical, so any useful representation must achieve semantic generalisation, not just syntactic.

7.1.2 The Generalisation Argument

Beyond the pure storage problem, tabular methods have a fundamental statistical inefficiency: every state is treated as a completely separate object. Learning that the value of state s is high conveys no information about state s', even if s and s' are visually identical up to the position of one pixel. In the language of statistics, tabular methods have zero *generalisation*: each entry must be estimated independently, from transitions that visit that specific state.

In large state spaces, most states are visited at most once, or never at all, during any reasonable training run. Tabular methods assign such states their initial values (often zero) indefinitely. Function approximation addresses this by sharing statistical strength across states: a visit to state s implicitly updates the estimated values of all states s' for which $\varphi(s')$ is similar to $\varphi(s)$, where φ denotes the feature representation. This *implicit regularisation* is the core benefit of parameterised value functions.

7.2 Linear Value Function Approximation

7.2.1 Feature Vectors and the Linear Ansatz

The simplest parameterised approximation represents the value function as a linear combination of fixed features.

Definition 7.2 (Linear value function approximation)**.** Let $\varphi : \mathcal{S} \to \mathbb{R}^d$ be a **feature map** assigning each state a feature vector. A **linear value function approximation** takes the form

$$V_{\mathbf{w}}(s) = \mathbf{w}^\top \varphi(s) = \sum_{i=1}^{d} w_i \varphi_i(s), \tag{7.1}$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector. For action-value functions one uses features $\varphi(s, a) \in \mathbb{R}^d$ and writes $Q_{\mathbf{w}}(s, a) = \mathbf{w}^\top \varphi(s, a)$.

The linear ansatz is attractive for three reasons. First, it is the simplest possible parameterisation: the approximation is uniquely determined by d real numbers regardless of how large $|\mathcal{S}|$ is. Second, many interesting feature maps can be designed by hand — tile codings, radial basis functions, polynomial bases, Fourier features — each encoding specific prior beliefs about the structure of the value function. Third, and most importantly, linear function approximation is the setting where TD convergence can be established with mathematical rigour, as we will see in Section 7.3.

Example 7.4 (Polynomial features for a one-dimensional corridor)**.** Consider a corridor of $N = 1\,000$ discrete positions. In the tabular case, V^π requires $N = 1\,000$ parameters. If, however, the true value function is smooth — which is likely if the reward is concentrated at one end and the discount factor is moderate — we can approximate it with the polynomial basis

$$\varphi(s) = \big(1,\ s/N,\ (s/N)^2,\ (s/N)^3\big)^\top \in \mathbb{R}^4.$$

This reduces the parameter count from 1 000 to 4, at the cost of being unable to represent rapidly varying value functions exactly. The approximation error incurred by restricting to this function class is called the **representation error**, and it is unavoidable whenever the true value function lies outside the approximation family.

7.2.2 Tile Coding

A particularly useful feature map for continuous state spaces is *tile coding*, developed by Sutton (1996). The state space $\mathbb{R}^n$ is covered by multiple overlapping grids, called *tilings*. For a given state s, exactly one tile in each tiling is active, giving a binary feature vector whose non-zero entries indicate which tile is active in each tiling. If there are m tilings each partitioning the space into k tiles, the feature vector has dimension mk and contains exactly m ones. The resulting linear approximation is equivalent to a lookup table of tile values, but the overlapping structure of the tilings provides smooth interpolation between nearby states.

7.2.3 Linear TD(0)

Recall from Chapter 6 that TD(0) for policy evaluation maintains a value estimate and updates it after each transition $(S_t, A_t, R_{t+1}, S_{t+1})$ using the TD error

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

When V is parameterised as $V_{\mathbf{w}}(s) = \mathbf{w}^\top \varphi(s)$, the natural extension is to update the weight vector in the direction that reduces the squared TD error. This yields the **semi-gradient TD(0)** update (analysed in detail in Section 7.4):

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\, \delta_t\, \varphi(S_t), \tag{7.2}$$

where $\delta_t = R_{t+1} + \gamma \mathbf{w}^\top \varphi(S_{t+1}) - \mathbf{w}^\top \varphi(S_t)$. Observe that in the linear case $\nabla_{\mathbf{w}} V_{\mathbf{w}}(S_t) = \varphi(S_t)$, so (7.2) is the semi-gradient step applied to $V_{\mathbf{w}}$.

A critical property of this update is that changing $\mathbf{w}$ affects *all states simultaneously*: the value prediction at state s' changes by $\alpha\delta_t\, \varphi(s') \cdot \varphi(S_t)$, proportional to how similar s' is to S_t under the feature inner product. This is the mechanism of generalisation in linear function approximation.

Theorem 7.1 (Convergence of linear TD(0) with on-policy sampling)**.** *Let π be a fixed policy and let μ be its stationary distribution over $\mathcal{S}$ (assumed unique and positive everywhere). Assume the feature matrix $\Phi \in \mathbb{R}^{|\mathcal{S}| \times d}$ has full column rank. If samples (S_t, R_{t+1}, S_{t+1}) are drawn on-policy, i.e., $S_t \sim \mu$ in steady state, and the step sizes satisfy $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$, then the linear TD(0) iterates $\mathbf{w}_t$ converge almost surely to a unique weight vector $\mathbf{w}_{\mathrm{TD}}$ satisfying the projected Bellman equation $\Phi \mathbf{w}_{\mathrm{TD}} = \Pi_\mu T^\pi(\Phi \mathbf{w}_{\mathrm{TD}})$ (defined in Section 7.3).*

The proof of Theorem 7.1 relies on the theory of stochastic approximation and the contraction properties of the projected Bellman operator, which we now develop.

7.3 The Projected Bellman Equation

7.3.1 Geometry of Function Approximation

To understand what linear TD(0) actually converges to, we need to think geometrically. The linear function class $\mathcal{F} = \{\Phi \mathbf{w} : \mathbf{w} \in \mathbb{R}^d\}$ is a d-dimensional subspace of $\mathbb{R}^{|\mathcal{S}|}$ (identifying value functions with vectors indexed by states). The Bellman operator T^π maps any value function $v \in \mathbb{R}^{|\mathcal{S}|}$ to a new function $T^\pi v$, but in general $T^\pi v \notin \mathcal{F}$ even when $v \in \mathcal{F}$. Consequently, TD methods operating within $\mathcal{F}$ cannot follow T^π directly.

The resolution is to *project* $T^\pi v$ back onto $\mathcal{F}$ after each Bellman step.

Definition 7.3 (μ-weighted inner product and projection)**.** Let $\mu : \mathcal{S} \to (0, \infty)$ be a positive weighting (typically the on-policy stationary distribution of π). Define the **μ-weighted norm**

$$\|v\|_\mu^2 = \sum_{s \in \mathcal{S}} \mu(s)\, v(s)^2 = \mathbf{v}^\top D_\mu \mathbf{v}, \tag{7.3}$$

where $D_\mu = \mathrm{diag}(\mu)$. The **orthogonal projection** of v onto $\mathcal{F}$ in this norm is

$$\Pi_\mu v = \Phi\big(\Phi^\top D_\mu \Phi\big)^{-1} \Phi^\top D_\mu v = \operatorname*{argmin}_{f \in \mathcal{F}} \|f - v\|_\mu. \tag{7.4}$$

The matrix $\Pi_\mu = \Phi(\Phi^\top D_\mu \Phi)^{-1}\Phi^\top D_\mu$ is the orthogonal projector in the μ-weighted inner product. It is idempotent ($\Pi_\mu^2 = \Pi_\mu$) and satisfies $\|I - \Pi_\mu\|_\mu \le 1$, meaning that projection can only bring a vector *closer* to $\mathcal{F}$, never push it further away.

Definition 7.4 (Projected Bellman equation)**.** The **projected Bellman equation** for policy π is

$$\hat{V} = \Pi_\mu T^\pi \hat{V}, \qquad \hat{V} \in \mathcal{F}. \tag{7.5}$$

A solution $\hat{V} = \Phi \mathbf{w}_{\mathrm{TD}}$ is called the **TD fixed point**.

The projected Bellman equation has a clear geometric interpretation: find a function in the approximation class $\mathcal{F}$ that, after one Bellman backup and re-projection, returns to itself. It is the best we can hope for when exact Bellman consistency is unachievable within $\mathcal{F}$.

Example 7.5 (Projected Bellman equation with three states)**.** Suppose $|\mathcal{S}| = 3$ with uniform weighting $\mu = (1/3, 1/3, 1/3)$, and our feature matrix is

$$\Phi = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \in \mathbb{R}^{3\times 2}.$$

The approximation family $\mathcal{F}$ has dimension 2, so values at all three states cannot be set independently: $V(s_3) = V(s_1) + V(s_2)$ always. The projection $\Pi_\mu v$ of any $v \in \mathbb{R}^3$ finds the weights $\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} \|v - \Phi\mathbf{w}\|_\mu^2$ by solving $\Phi^\top D_\mu \Phi \mathbf{w}^* = \Phi^\top D_\mu v$, yielding

$$\mathbf{w}^* = \tfrac{1}{2}\begin{pmatrix} v_1 + v_3 - v_2 \\ v_2 + v_3 - v_1 \end{pmatrix},$$

so the projection "splits the contribution of s_3" across the two basis directions.

7.3.2 Existence of the TD Fixed Point

Theorem 7.2 (TD fixed point existence and uniqueness)**.** *Under the assumptions of Theorem 7.1, the projected Bellman operator $\Pi_\mu T^\pi : \mathcal{F} \to \mathcal{F}$ is a contraction in the μ-norm with contraction factor γ:*

$$\|\Pi_\mu T^\pi u - \Pi_\mu T^\pi v\|_\mu \le \gamma \|u - v\|_\mu \qquad \forall\, u, v \in \mathcal{F}. \tag{7.6}$$

Consequently, the projected Bellman equation (7.5) has a unique solution $\hat{V} = \Phi \mathbf{w}_{\mathrm{TD}} \in \mathcal{F}$.

Proof. The Bellman operator T^π is itself a contraction in the μ-norm with factor γ (this follows from the contraction property of T^π in the sup-norm and the equivalence of these norms for positive μ). Formally, for any u, v:

$$\|T^\pi u - T^\pi v\|_\mu \le \gamma \|u - v\|_\mu.$$

Since Π_μ is a non-expansive projection ($\|\Pi_\mu x\|_\mu \le \|x\|_\mu$ for all x), we have

$$\|\Pi_\mu T^\pi u - \Pi_\mu T^\pi v\|_\mu \le \|T^\pi u - T^\pi v\|_\mu \le \gamma \|u - v\|_\mu.$$

Because $\gamma < 1$ and $\mathcal{F}$ is a complete metric space (a closed subspace of $\mathbb{R}^{|\mathcal{S}|}$), the Banach fixed-point theorem guarantees a unique fixed point. □

7.3.3 Approximation Error at the TD Fixed Point

The TD fixed point $\hat{V}$ minimises a particular measure of error, but it does not minimise the mean-squared error $\|\hat{V} - V^\pi\|_\mu^2$ directly. The following classical bound (Tsitsiklis and Van Roy, 1997) quantifies how far the TD fixed point deviates from the best possible approximation.

Theorem 7.3 (TD fixed-point approximation bound)**.** *Let $\hat{V}^* = \Pi_\mu V^\pi$ be the best approximation of V^π in $\mathcal{F}$. Then*

$$\|\hat{V} - V^\pi\|_\mu \le \frac{1}{\sqrt{1-\gamma^2}} \|\hat{V}^* - V^\pi\|_\mu. \tag{7.7}$$

The factor $1/\sqrt{1-\gamma^2}$ is a "price" paid for bootstrapping: the TD fixed point may be up to $1/\sqrt{1-\gamma^2}$ times worse than the best-in-class approximation. For $\gamma = 0.99$ this factor is approximately 7.1, illustrating that bootstrapping can significantly amplify approximation errors in problems with long time horizons.

7.4 Semi-Gradient Methods

7.4.1 True Gradient Descent for Prediction

Before examining semi-gradient methods, it is instructive to understand what true gradient descent would look like for the prediction problem. Define the **mean-squared value error**

$$\overline{\mathrm{VE}}(\mathbf{w}) = \|V^\pi - V_\mathbf{w}\|_\mu^2 = \sum_{s \in \mathcal{S}} \mu(s) \big[V^\pi(s) - V_\mathbf{w}(s)\big]^2, \tag{7.8}$$

where μ is the on-policy distribution. The true gradient of $\overline{\mathrm{VE}}$ with respect to $\mathbf{w}$ is

$$-\frac{1}{2}\nabla_\mathbf{w} \overline{\mathrm{VE}}(\mathbf{w}) = \sum_s \mu(s) \big[V^\pi(s) - V_\mathbf{w}(s)\big] \nabla_\mathbf{w} V_\mathbf{w}(s). \tag{7.9}$$

Unfortunately, $V^\pi(s)$ is unknown, so we cannot evaluate this gradient directly. Monte Carlo methods substitute the sample return $G_t \approx V^\pi(S_t)$, giving the update

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \big[G_t - V_\mathbf{w}(S_t)\big] \nabla_\mathbf{w} V_\mathbf{w}(S_t).$$

This is a true stochastic gradient step because $\mathbb{E}[G_t \mid S_t] = V^\pi(S_t)$ and the gradient $\nabla_\mathbf{w} V_\mathbf{w}(S_t)$ does not appear in the target G_t.

7.4.2 Why TD Breaks True Gradient Descent

TD methods replace G_t with the one-step bootstrap target $R_{t+1} + \gamma V_\mathbf{w}(S_{t+1})$. The resulting update is

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \underbrace{\big[R_{t+1} + \gamma V_\mathbf{w}(S_{t+1}) - V_\mathbf{w}(S_t)\big]}_{\delta_t} \nabla_\mathbf{w} V_\mathbf{w}(S_t). \tag{7.10}$$

This *looks* like a gradient step, but it is not.

Definition 7.5 (Semi-gradient method)**.** A **semi-gradient** method is one in which the update direction is computed as if δ_t were an error signal independent of $\mathbf{w}$, i.e., the gradient is taken only through $V_{\mathbf{w}}(S_t)$ and not through the target $V_{\mathbf{w}}(S_{t+1})$. The update (7.10) is a semi-gradient step because the gradient of δ_t with respect to $\mathbf{w}$ via its dependence on $V_{\mathbf{w}}(S_{t+1})$ is ignored.

To see why the gradient through $V_{\mathbf{w}}(S_{t+1})$ is omitted, note that if we computed the true gradient of $\frac{1}{2}\delta_t^2$ we would obtain

$$\nabla_{\mathbf{w}} \tfrac{1}{2}\delta_t^2 = \delta_t\big(\gamma \nabla_{\mathbf{w}} V_{\mathbf{w}}(S_{t+1}) - \nabla_{\mathbf{w}} V_{\mathbf{w}}(S_t)\big).$$

The TD semi-gradient drops the $\gamma \nabla_{\mathbf{w}} V_{\mathbf{w}}(S_{t+1})$ term. This omission means the update direction is *not* the negative gradient of any fixed objective function. As a consequence, standard convergence theorems for stochastic gradient descent do not apply directly.

Remark 7.1. The omission of the bootstrap gradient is deliberate, not an error. If one were to include it, the resulting *residual gradient* algorithm converges in the mean-squared Bellman error but suffers from a "double-sampling" problem: computing the full gradient requires two independent samples of S_{t+1} for the same (S_t, A_t), which is typically unavailable in a single interaction stream. Semi-gradient methods avoid this by treating the target as fixed; their practical convergence properties are generally superior despite the lack of a gradient guarantee.

7.4.3 Semi-Gradient TD Algorithm

Algorithm 15: Semi-Gradient TD(0) for Policy Evaluation

Input: Policy π, feature map φ, step size $\alpha > 0$, discount $\gamma \in [0, 1)$
Initialise $\mathbf{w} \in \mathbb{R}^d$ (e.g., $\mathbf{w} \leftarrow \mathbf{0}$);
for *each episode* **do**
  Initialise state S_0;
  for $t = 0, 1, 2, \ldots$ *until* S_t *is terminal* **do**
    Choose $A_t \sim \pi(\cdot \mid S_t)$;
    Observe R_{t+1} and S_{t+1};
    $\delta_t \leftarrow R_{t+1} + \gamma \mathbf{w}^\top \varphi(S_{t+1}) - \mathbf{w}^\top \varphi(S_t)$;
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\, \delta_t\, \varphi(S_t)$;

7.4.4 Convergence of Semi-Gradient TD

For the linear case, semi-gradient TD(0) under on-policy sampling converges to the TD fixed point $\mathbf{w}_{\mathrm{TD}}$ characterised by Theorem 7.2. The argument relies on the *ordinary differential equation (ODE) method*: one shows that the mean update field

$$\bar{f}(\mathbf{w}) = \mathbb{E}_\mu[\delta_t \varphi(S_t) \mid \mathbf{w}] = \Phi^\top D_\mu (T^\pi(\Phi \mathbf{w}) - \Phi \mathbf{w})$$

is a contraction toward $\mathbf{w}_{\mathrm{TD}}$, and then appeals to a standard theorem (Kushner and Yin, 2003) that says stochastic approximation converges to the equilibrium of the associated ODE.

For *nonlinear* function approximators, even on-policy semi-gradient TD is not guaranteed to converge. In practice, neural-network-based TD methods often converge to useful solutions, but counter-examples exist. Convergence in the nonlinear case remains an active research area.

7.5 The Deadly Triad

7.5.1 Three Ingredients for Instability

The convergence guarantee of Theorem 7.1 hinges critically on the words "on-policy sampling". When any one of three conditions is violated, the well-behaved landscape of linear TD breaks down. When all three violations occur simultaneously, even linear function approximation can diverge. Sutton and Barto (2018) term this dangerous combination the **deadly triad**.

Definition 7.6 (The deadly triad)**.** The **deadly triad** refers to the simultaneous presence of three elements:

1. **Function approximation:** using a parameterised function $V_{\mathbf{w}}$ (or $Q_{\mathbf{w}}$) with $d \ll |\mathcal{S}|$ instead of a separate table entry per state.
2. **Bootstrapping:** forming targets from the current estimate, as in TD methods, rather than waiting for the full return, as in Monte Carlo.
3. **Off-policy training:** updating parameters using data generated by a *behaviour policy* b that differs from the *target policy* π being evaluated or improved.

The combination of all three can cause parameter divergence even with linear approximators and even in the limit of infinite data.

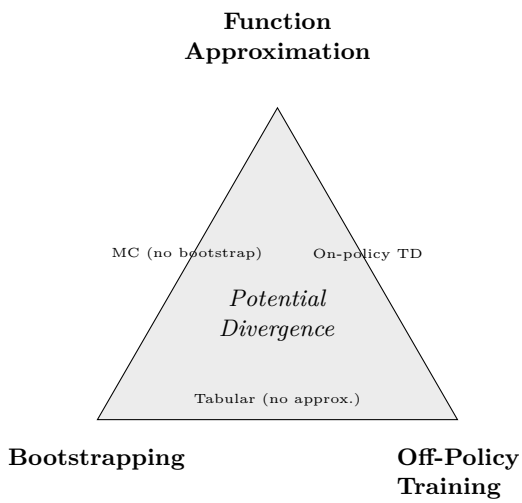


Figure 7.1: The deadly triad. Each edge of the triangle is "safe": two of the three ingredients together are manageable. Only when all three vertices are active simultaneously does instability become unavoidable in general. The edge labels indicate which element is absent along each side.

It is important to understand that each pair of ingredients is manageable:

- *Function approximation + bootstrapping, on-policy:* Linear TD(0) converges to the TD fixed point (Theorem 7.1).
- *Function approximation + off-policy, no bootstrap:* Off-policy Monte Carlo with linear function approximation converges because the gradient is a true gradient of the MSVE.
- *Bootstrapping + off-policy, tabular:* Q-learning converges to Q^* in the tabular setting (Chapter 6, Theorem on Q-learning convergence).

The problem emerges only when all three are combined.

7.5.2 Baird's Counterexample

The clearest illustration of the deadly triad is *Baird's counterexample* (Baird, 1995). Consider a simple MDP with 7 states and 2 actions. Six states are arranged in a star pattern; the seventh is a central hub. The behaviour policy always chooses the first action (which transitions to one of the six star states, uniformly at random) with probability $\frac{6}{7}$ and the second action (which transitions to the hub) with probability $\frac{1}{7}$. The target policy always takes the second action. All rewards are zero, and $\gamma = 0.99$.

The feature vectors are chosen so that the approximation family is linear. Specifically, the feature vector for hub state s_7 is $(0, 0, 0, 0, 0, 0, 1, 2)^\top$ and for star state s_i $(i = 1, \ldots, 6)$ is the vector with a 2 in position i and a 1 in position 8. The weight vector $\mathbf{w} \in \mathbb{R}^8$ is initialised to $(1, 1, 1, 1, 1, 1, 10, 1)^\top$.

Theorem 7.4 (Baird's counterexample)**.** *Under semi-gradient off-policy TD(0) with the feature vectors described above, the weight vector $\mathbf{w}_t$ diverges to infinity almost surely, even though the feature matrix has full column rank and the step size satisfies the standard Robbins–Monro conditions.*

The mechanism of divergence is subtle: the off-policy distribution μ concentrates too much weight on the star states (which are frequently visited under the behaviour policy) while the Bellman updates primarily "push" the values of those states up, since the target policy prefers to go to the hub. The projection back onto the function class then amplifies this push rather than damping it, creating a positive feedback loop.

Remark 7.2 (What Baird's example teaches us)*.* Baird's example is not a pathological curiosity; it is a fundamental result about the limits of TD methods. It tells us that the combination of semi-gradient bootstrapping and off-policy data is inherently dangerous, regardless of how well-behaved the feature representation is. The solutions explored in practice — importance sampling corrections, gradient-TD methods (TDC, GTD2), and the engineering stabilisations of DQN — all represent different strategies for escaping the triad's dangers without sacrificing the benefits of off-policy data.

7.6 Deep Q-Networks

7.6.1 From Linear to Nonlinear Approximation

The successes of linear function approximation are real but limited: the method requires hand-designed features, and in high-dimensional perceptual domains such as raw image pixels, no human designer can specify the right features in advance. The natural next step is to use a deep neural network as the approximator, letting the network learn its own internal representation from data.

The challenge is that deep networks bring all the difficulties of the deadly triad into sharp focus. In Q-learning with a neural network Q_θ and greedy action selection, the target for state–action pair (S_t, A_t) is

$$y_t = R_{t+1} + \gamma \max_{a'} Q_\theta(S_{t+1}, a'), \tag{7.11}$$

and the loss minimised is $(y_t - Q_\theta(S_t, A_t))^2$. But both the prediction $Q_\theta(S_t, A_t)$ and the target y_t depend on the same parameters θ, so each gradient step simultaneously changes the prediction *and* the target. This creates a moving-target problem: the network chases a quantity that shifts with every update, leading to oscillations and divergence in practice.

Two further pathologies compound the problem. The transitions $(S_t, A_t, R_{t+1}, S_{t+1})$ collected by a sequential agent are *highly correlated*: consecutive transitions share the same state sequence. Training a neural network on correlated mini-batches violates the i.i.d. assumption of stochastic gradient descent and causes the network to overfit to recent experience, "forgetting" earlier learning. Additionally, the distribution of training data changes as the policy improves, making the optimisation landscape non-stationary.

7.6.2 Experience Replay

The first stabilising innovation introduced by Mnih et al. (2013, 2015) is **experience replay**. As the agent interacts with the environment, each transition $(S_t, A_t, R_{t+1}, S_{t+1})$ is stored in a *replay buffer* $\mathcal{D}$ of capacity N. During training, mini-batches of transitions are sampled *uniformly at random* from $\mathcal{D}$:

$$\{(s_i, a_i, r_i, s_i')\}_{i=1}^{B} \sim \text{Uniform}(\mathcal{D}). \tag{7.12}$$

This simple change has two important effects. First, it *decorrelates* training data: successive mini-batches are drawn from many different time steps, breaking the temporal correlation that destabilises on-line updates. Second, each transition is used for potentially many gradient steps, improving data efficiency. The cost is that replay requires storing a large buffer in memory and introduces an implicit off-policy component: transitions stored long ago were generated by an older, different policy.

7.6.3 Target Networks

The second stabilising innovation is the **target network**. DQN maintains two networks with the same architecture: the *online network* Q_θ (also called the *behaviour network*) and the *target network* Q_{θ^-}. The online network is updated at every gradient step. The target network's parameters θ^- are held fixed for C steps (typically $C = 10\,000$) and then synchronised with the online network: $\theta^- \leftarrow \theta$.

With the target network, the loss function becomes

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\right)^2\right]. \tag{7.13}$$

The gradient is taken only with respect to θ, treating θ^- as a constant. By freezing the target for C steps, the method reduces the correlation between the parameters being updated and the target they are chasing. The target is not perfectly stationary — it still moves every C steps — but in practice this significantly stabilises training.

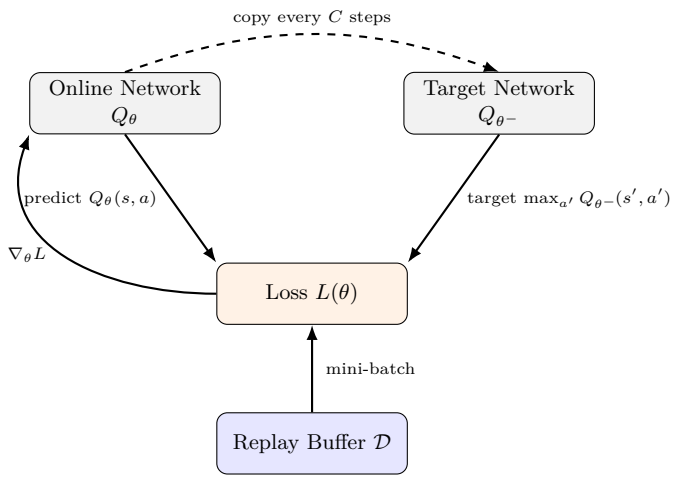


Figure 7.2: DQN architecture. The online network Q_θ selects actions and is updated every step. The target network Q_{θ^-} computes the TD target and is synchronised every C steps. Transitions are stored in the replay buffer $\mathcal{D}$ and sampled in random mini-batches for training. Dashed arrow: periodic hard copy $\theta^- \leftarrow \theta$.

7.6.4 DQN Algorithm

7.6.5 Architecture and Atari Results

For Atari games, the DQN network is a convolutional architecture: three convolutional layers (with 32, 64, and 64 filters of sizes 8×8, 4×4, 3×3 and strides 4, 2, 1 respectively) followed by a fully connected layer of 512 units and a linear output layer with one unit per action. The input is four stacked 84×84 greyscale frames, preprocessed from the raw 210×160 RGB output.

Mnih et al. (2015) evaluated DQN on 49 Atari games using a *single fixed architecture and hyperparameters* across all games. DQN achieved above-human-level performance on 29 of the 49 games and set a new state of the art on almost all of them. Crucially, DQN receives only raw pixels and the game score as input, with no game-specific feature engineering. This demonstrated for the first time that a single deep RL algorithm could achieve broad competence across diverse tasks purely from high-dimensional sensory input.

7.7 Extensions of DQN

The DQN paper sparked a wave of algorithmic improvements. We survey three that have become standard components of modern deep RL systems.

7.7.1 Double DQN

A subtle but important pathology of Q-learning is **maximisation bias**: the $\max_{a'} Q(s', a')$ operator is an overestimate of $\max_{a'} Q^*(s', a')$ when Q has any noise, because the maximum of noisy estimates is biased upward. Van Hasselt et al. (2016) showed that this bias is present and harmful in DQN, causing the agent to overestimate values and take suboptimal actions.

Algorithm 16: Deep Q-Network (DQN)

Input: Env., network Q_θ, replay capacity N, batch size B, update frequency C, ε-schedule, γ, α

Initialise θ randomly; $\theta^- \leftarrow \theta$; $\mathcal{D} \leftarrow \emptyset$;
for *each episode* **do**
  Reset environment; observe s_0;
  for $t = 0, 1, 2, \ldots$ **do**
    With probability ε: $a_t \leftarrow$ random action; else $a_t \leftarrow \operatorname{argmax}_a Q_\theta(s_t, a)$;
    Execute a_t; observe r_{t+1}, s_{t+1}, done flag;
    Store $(s_t, a_t, r_{t+1}, s_{t+1})$ in $\mathcal{D}$ (discard oldest if full);
    if $|\mathcal{D}| \geq B$ **then**
      Sample mini-batch $\{(s_i, a_i, r_i, s_i')\}_{i=1}^B \sim \mathcal{D}$;
      for *each transition* i **do**
        if s_i' *is terminal* **then** $y_i \leftarrow r_i$;
        else $y_i \leftarrow r_i + \gamma \max_{a'} Q_{\theta^-}(s_i', a')$;
      Perform gradient step: $\theta \leftarrow \theta - \alpha \nabla_\theta \frac{1}{B} \sum_i (y_i - Q_\theta(s_i, a_i))^2$;
      if $t \bmod C = 0$ **then**
        $\theta^- \leftarrow \theta$
    if *done* **then break**;

The fix is **Double DQN** (DDQN): decouple action selection from value estimation by using the online network to choose the greedy action and the target network to evaluate it:

$$y_t^{\text{DDQN}} = R_{t+1} + \gamma Q_{\theta^-}\left(S_{t+1}, \operatorname*{argmax}_{a'} Q_\theta(S_{t+1}, a')\right). \tag{7.14}$$

Compared to DQN's target in (7.13), the only change is that the argmax and the Q-evaluation are performed by different networks. This simple modification substantially reduces overestimation and improves performance on many Atari games with negligible computational overhead.

7.7.2 Dueling DQN

Wang et al. (2016) observed that in many states, the value of the state $V^\pi(s)$ is more informative than the individual action values $Q^\pi(s, a)$. When the agent is in a safe state, the choice of action matters little; what matters is the state itself. They proposed the **dueling architecture**, which splits the network's final layers into two streams: one estimating the state-value function $V_\theta(s)$ and one estimating the **advantage function**

$$A_\theta(s, a) = Q_\theta(s, a) - V_\theta(s).$$

The streams are combined as

$$Q_\theta(s, a) = V_\theta(s) + \left(A_\theta(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A_\theta(s, a')\right), \tag{7.15}$$

where the mean subtraction ensures identifiability (since V and A are not separately identifiable without it). The dueling architecture allows the network to learn which states are

valuable without having to learn the effect of each action, which is beneficial when actions have similar effects in most states.

7.7.3 Prioritised Experience Replay

In standard replay, transitions are sampled uniformly from $\mathcal{D}$. Schaul et al. (2016) argued that transitions with large TD errors contain more "surprising" information and should be replayed more often. **Prioritised experience replay** (PER) assigns each transition i a priority

$$p_i = |\delta_i|^\alpha + \epsilon,$$

where δ_i is the most recent TD error for transition i, $\alpha > 0$ controls the degree of prioritisation, and $\epsilon > 0$ prevents zero priority. The sampling probability is $P(i) = p_i / \sum_j p_j$. Importance-sampling weights $w_i = (N \cdot P(i))^{-\beta}$ (with $\beta \to 1$ as training progresses) correct for the non-uniform sampling to maintain unbiased gradient estimates. PER typically leads to faster learning and better final performance, particularly in sparse-reward environments where informative transitions are rare.

7.8 Convergence Analysis

7.8.1 What We Can Prove

The convergence landscape for value function approximation is considerably more nuanced than for tabular RL. Table 7.1 summarises the main results.

Table 7.1: Convergence summary for value function approximation methods. "Converges" means convergence to a useful (possibly approximate) solution almost surely or in mean square. "May diverge" means counter-examples to convergence are known.

Method	**Approximation**	**Sampling**	**Guarantee**
MC gradient	Linear / Nonlinear	On / Off	Converges (local min.)
Semi-gradient TD	Linear	On-policy	Converges (TD f.p.)
Semi-gradient TD	Linear	Off-policy	May diverge
Semi-gradient TD	Nonlinear	On-policy	May diverge
Gradient-TD (GTD2)	Linear	Off-policy	Converges (TD f.p.)
Semi-gradient Q-learning	Linear	Off-policy	May diverge
DQN	Nonlinear (deep)	Off-policy	No guarantee

7.8.2 Gradient-TD Methods

To escape the deadly triad while preserving off-policy learning, several authors developed **gradient-TD methods** — algorithms that minimise a surrogate objective whose true gradient is tractable. The key insight (Sutton et al., 2009) is to define the *projected Bellman error*

$$\overline{\mathrm{PBE}}(\mathbf{w}) = \|\Pi_\mu (T^\pi V_\mathbf{w} - V_\mathbf{w})\|_\mu^2,$$

which is zero at the TD fixed point. Unlike the MSVE, the gradient of $\overline{\text{PBE}}$ can be estimated with a single sample (with an auxiliary parameter vector), avoiding the double-sampling problem. The resulting algorithms (TDC and GTD2) converge to $\mathbf{w}_{\text{TD}}$ under off-policy sampling, breaking one leg of the deadly triad. Their practical adoption has been limited, however, because they require careful tuning of two step sizes and can converge slowly.

7.8.3 The Nonlinear Case

For deep neural networks, no convergence guarantee analogous to Theorem 7.1 is known, and genuine counter-examples to convergence with nonlinear approximators exist (Tsitsiklis and Van Roy, 1997). The engineering innovations of DQN — experience replay and target networks — are heuristic stabilisers that work well in practice without providing theoretical guarantees.

One partial result is available: if the DQN loss (7.13) is viewed as a supervised regression problem with fixed targets (i.e., θ^- is held constant), then standard neural-network training converges to a local minimum of $L(\theta)$ for that fixed target. The difficulty is that the target changes periodically. If the target network is updated infinitely slowly (formally, $C \to \infty$ with the online network fully converged at each step), one recovers an iterative sequence of supervised problems whose fixed points are related to the Q-learning fixed point. This *fitted value iteration* perspective (Gordon, 1999; Riedmiller, 2005) provides some theoretical grounding for DQN's behaviour.

Remark 7.3 (Mean-field theory perspective). Recent work in the *mean-field* limit (infinite-width networks) and the *neural tangent kernel* (NTK) regime has provided new tools for analysing neural-network-based RL. In the NTK limit, the network behaves approximately linearly in its parameters throughout training, recovering the convergence guarantees of linear TD. While this regime is not fully realistic (real networks are far narrower than required), it suggests that "sufficiently over-parameterised" networks may inherit some linear-case stability, and supports empirical observations that wide networks train more stably in deep RL.

7.9 Applications and Practical Considerations

7.9.1 Feature Engineering vs. End-to-End Learning

A central design decision in any value-function-approximation system is whether to hand-engineer features or learn them end-to-end.

Hand-designed features (tile codings, radial basis functions, Fourier bases) are interpretable, require less data, and admit convergence guarantees under linear approximation. They are the right choice when domain knowledge is rich and the state space is moderate in complexity. For example, a robot navigation task on a known map might use as features the robot's (x, y) coordinates, its heading, and the distance to the nearest obstacle — a four-dimensional representation that collapses thousands of LIDAR readings to the quantities that actually matter.

End-to-end deep learning is necessary when the raw observation is high-dimensional (images, audio, text) and the relevant features are not known in advance. The cost is a much larger parameter count, longer training time, reduced interpretability, and weaker theoretical guarantees. In practice, end-to-end deep RL benefits from: (i) sufficient network capacity to represent the value function; (ii) feature normalisation to prevent gradient explosion; (iii) a

well-tuned ε-greedy schedule to balance exploration and exploitation; and (iv) a large enough replay buffer to break temporal correlations.

7.9.2 Hyperparameter Sensitivity

Deep RL algorithms are notoriously sensitive to hyperparameters. Henderson et al. (2018) demonstrated that the reported performance of deep RL algorithms can vary by more than an order of magnitude depending on random seeds, implementation details, and hyperparameter choices. The following hyperparameters most critically affect DQN performance:

Replay buffer size N: Too small and the buffer fills with correlated recent experience; too large and old, stale transitions from an early policy dominate. Typical values: 10^5–10^6.

Target update frequency C: Too small causes the moving-target problem; too large means the target lags far behind, slowing learning. Typical: 10^3–10^4 steps.

Batch size B: Larger batches give more stable gradient estimates but increase memory and compute per update. Typical: 32–256.

ε-schedule: The decay rate from 1.0 to a small final ε must be matched to the task; too fast leads to premature exploitation; too slow wastes sample budget.

7.9.3 Practical Recommendations

For practitioners implementing deep RL for the first time:

- Start with the simplest baseline: tabular Q-learning or linear function approximation, if the state space is manageable. Verify that the environment and reward signal are correct before introducing neural networks.
- When moving to deep networks, use an established implementation (e.g., Stable Baselines3, CleanRL) rather than implementing from scratch, to avoid subtle bugs in replay buffers and target networks.
- Plot the TD error, average episode return, and Q-value estimates throughout training. Exploding Q-values are a reliable signal of the moving-target pathology.
- If training is unstable, reduce the learning rate, increase the target network update frequency C, or switch from hard target updates ($\theta^- \leftarrow \theta$) to soft updates ($\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$, $\tau \ll 1$), as used in DDPG and SAC.
- Normalise rewards to $[-1, 1]$ or clip gradients to prevent early divergence.

7.9.4 Historical Notes

Function approximation in RL has a long history. Bellman himself recognised the curse of dimensionality in the 1950s. The convergence of linear TD(0) was established by Tsitsiklis and Van Roy (1997). The deadly triad was identified and named by Sutton (1995). Neural network function approximation in RL dates at least to Tesauro's TD-Gammon (1992), which learned to play backgammon at near-grandmaster level using a hand-crafted two-layer

network trained with TD methods. DQN (Mnih et al., 2013, published in full in 2015) was the first demonstration of end-to-end deep RL achieving superhuman performance on diverse tasks. Double DQN (Van Hasselt et al., 2016), Dueling DQN (Wang et al., 2016), and Prioritised Experience Replay (Schaul et al., 2016) were all published in the same year, collectively forming the "DQN era" of deep RL. The Rainbow algorithm (Hessel et al., 2018) combined all six extensions into a single agent, achieving significantly improved sample efficiency.

Exercises

Exercise 7.1. Consider a robot navigating a grid world described by an n-dimensional binary state vector (each dimension is 0 or 1).

1. How many states does a tabular Q-function with $|\mathcal{A}| = 4$ require to store?
2. Suppose instead we use a linear function approximation with $d = 10n$ features, where $d \ll 2^n$. Express the ratio of storage required by the linear approximation to the tabular representation as a function of n.
3. At what value of n does the linear approximation first require less storage than the tabular representation (assuming $d = 10n$)?
4. Discuss one approximation error that the linear representation introduces compared to the tabular one.

Exercise 7.2. Let $|\mathcal{S}| = 4$, $d = 2$, $\mu = (1/4, 1/4, 1/4, 1/4)$, and

$$\Phi = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}.$$

1. Verify that Φ has full column rank.
2. Compute the projection matrix $\Pi_\mu = \Phi(\Phi^\top D_\mu \Phi)^{-1}\Phi^\top D_\mu$.
3. Apply the projection to the vector $v = (3, 1, 2, 4)^\top$. What is $\Pi_\mu v$?
4. Verify that $\|v - \Pi_\mu v\|_\mu \le \|v - f\|_\mu$ for $f = (2, 2, 2, 2)^\top$.

Exercise 7.3. Consider the mean-squared value error $\overline{\mathrm{VE}}(\mathbf{w}) = \sum_s \mu(s)[V^\pi(s) - V_\mathbf{w}(s)]^2$. The TD target for a transition (s, r, s') is $y = r + \gamma V_\mathbf{w}(s')$.

1. Write out $-\frac{1}{2}\nabla_\mathbf{w}[y - V_\mathbf{w}(s)]^2$ treating y as a constant (the semi-gradient).
2. Write out $-\frac{1}{2}\nabla_\mathbf{w}[y - V_\mathbf{w}(s)]^2$ treating y as a function of $\mathbf{w}$ (the full gradient).
3. Identify the term that the semi-gradient omits. Under what condition is this term zero?

Exercise 7.4. The approximation error bound at the TD fixed point (Theorem 7.3) is $\|\hat{V} - V^\pi\|_\mu \le \frac{1}{\sqrt{1-\gamma^2}}\|\hat{V}^* - V^\pi\|_\mu$.

1. Compute the bound factor for $\gamma \in \{0.5, 0.9, 0.99, 0.999\}$.

2. As $\gamma \to 1$, the factor diverges. Give an intuitive explanation of why bootstrapping with near-unit discount amplifies approximation errors.

3. Is the bound tight in general? What would need to be true of the MDP and feature set for the bound to be achieved with equality?

Exercise 7.5. For each of the following algorithms, identify which elements of the deadly triad are present and state whether convergence is guaranteed:

1. Monte Carlo with linear function approximation, on-policy.

2. Q-learning in a tabular MDP with $|\mathcal{S}| = 10^6$.

3. Off-policy TD(0) with a neural network.

4. SARSA with linear function approximation, on-policy.

5. DQN on Atari.

Exercise 7.6. Show that the DQN loss (7.13) reduces to the standard Q-learning update (7.11) in the special case of a tabular (one-hot feature) representation with a step size $\alpha = 1$ and no target network ($\theta^- = \theta$). What does this imply about DQN's relationship to Q-learning?

Exercise 7.7. Let $Q(s, a) = Q^*(s, a) + \varepsilon_{s,a}$ where $\varepsilon_{s,a}$ are i.i.d. zero-mean noise terms with variance σ^2.

1. Show that $\mathbb{E}[\max_a Q(s, a)] \geq \max_a Q^*(s, a)$ (maximisation bias).

2. Suppose actions are evaluated with an independent copy $\tilde{Q}(s, a) = Q^*(s, a) + \tilde{\varepsilon}_{s,a}$, where $\tilde{\varepsilon}$ is independent of ε. Is $\mathbb{E}[\tilde{Q}(s, \operatorname{argmax}_a Q(s, a))]$ still biased upward? Justify your answer.

3. Relate your findings to the Double DQN target (7.14).

Exercise 7.8. In prioritised experience replay, transitions are sampled with probability $P(i) = p_i / \sum_j p_j$ and corrected with importance-sampling weights $w_i = (N \cdot P(i))^{-\beta}$.

1. Show that if all transitions have equal priority $p_i = p$, then $w_i = 1$ for all i (uniform sampling, no correction needed).

2. Explain why importance-sampling corrections are necessary for convergence in the limit $\beta = 1$. What goes wrong when $\beta = 0$?

3. Describe a situation where PER might *hurt* performance compared to uniform replay.

Exercise 7.9. Fitted Value Iteration. Consider the following algorithm: given a dataset $\mathcal{D}$ of transitions, repeatedly solve the supervised regression problem

$$\mathbf{w}_{k+1} = \operatorname*{argmin}_{\mathbf{w}} \sum_{(s,a,r,s') \in \mathcal{D}} \big(r + \gamma \max_{a'} Q_{\mathbf{w}_k}(s', a') - Q_{\mathbf{w}}(s, a)\big)^2.$$

1. Show that for linear function approximation, this is equivalent to solving a sequence of linear least-squares problems.

2. Prove that if $Q_{\mathbf{w}}$ is a tabular representation (one parameter per (s, a) pair) and $\mathcal{D}$ covers all state–action pairs, the algorithm converges to Q^*.

3. Explain how DQN with periodic target-network copies approximates fitted value iteration. What is the key approximation made?

Chapter 8

Policy Gradient Methods

Instead of looking for the policy indirectly through a value function, we look for it directly. We ask: given the current policy, in which direction should we nudge the parameters so that expected reward increases? That question has a beautiful answer, and it goes by the name of the policy gradient theorem.

All of the methods studied in Chapters 4–7 share a common architecture: first estimate a value function, then derive a policy from it by acting greedily or ε-greedily. Value functions are useful intermediate objects, but they are not strictly necessary. One can instead parameterise the policy directly — as a differentiable function $\pi_\theta(a \mid s)$ of parameters θ — and optimise those parameters to maximise expected return. This is the **policy gradient** idea, and it is the subject of the present chapter.

Policy gradient methods have three compelling advantages over value-based methods. First, they handle continuous and high-dimensional action spaces naturally: sampling from $\pi_\theta(\cdot \mid s)$ requires no inner maximisation. Second, the learned policy can be stochastic, which is sometimes provably optimal (e.g. in partially observed environments or adversarial games) and is always a well-defined probability distribution. Third, and perhaps most important for modern applications, the gradient of expected return with respect to policy parameters can be estimated purely from sampled trajectories, without ever needing to differentiate through the dynamics of the environment.

The chapter is organised as follows. Section 8.1 explains the motivation for direct policy optimisation. Section 8.2 derives the policy gradient theorem — the mathematical foundation of the entire family — including full proofs of the likelihood-ratio trick, the causality reduction to reward-to-go, and the compact form involving Q^{π_θ}. Section 8.3 presents the REINFORCE algorithm with a worked numerical example. Section 8.4 proves that subtracting a state-dependent baseline leaves the gradient unbiased and derives the optimal baseline. Section 8.5 combines value-function estimation with the policy gradient, giving REINFORCE with a learned baseline. Section 8.6 develops REINFORCE Leave-One-Out (RLOO), a modern estimator central to large-language-model alignment. Section 8.7 introduces the natural policy gradient, which corrects the ordinary gradient for the curvature of the policy manifold. Section 8.8 analyses convergence and the bias-variance tradeoff. Section 8.9 compares estimators in a summary table. The chapter closes with exercises.

8.1 Why Optimise the Policy Directly?

8.1.1 Limitations of the Value-Based Approach

The value-based pipeline is: learn $\hat{Q} \approx Q^*$, then set $\pi(s) = \operatorname{argmax}_a \hat{Q}(s, a)$. This pipeline breaks in two important ways when the action space is large or continuous.

1. *Continuous actions.* If $\mathcal{A} \subseteq \mathbb{R}^m$, computing $\operatorname{argmax}_{a \in \mathbb{R}^m} \hat{Q}(s, a)$ is itself a continuous optimisation problem that must be solved at every time step during execution. Methods such as DDPG circumvent this by learning a separate maximiser network, but the result is an actor-critic architecture — which already moves beyond pure value-based learning.

2. *Very large discrete vocabularies.* In token-level language generation, $|\mathcal{A}|$ is the vocabulary size (often 50,000 or more), and the episode length can be hundreds of steps. Storing a separate value for every (s, a) pair is infeasible; and even with function approximation, enumerating all actions to find the greedy one is expensive.

Both difficulties vanish when the policy is a parameterised distribution $\pi_\theta(a \mid s)$: at execution time one simply draws $A_t \sim \pi_\theta(\cdot \mid S_t)$, regardless of the size of $\mathcal{A}$.

8.1.2 Stochastic Policies Are Sometimes Necessary

Beyond scalability, there are principled reasons to want stochastic policies. In partially observed environments the optimal policy is generally stochastic: the agent may need to randomise to hedge against hidden state. In adversarial games a deterministic policy is exploitable, whereas a mixed-strategy equilibrium requires randomisation. In exploration, a stochastic policy samples actions without requiring a separate ε-greedy mechanism, and the degree of exploration is automatically controlled by the entropy of π_θ.

Remark 8.1. A deterministic policy $\mu_\theta(s)$ is a special case: it corresponds to a distribution concentrated at a single action. The *deterministic policy gradient* (DPG) theorem of Silver et al. (2014) extends the framework developed here to that degenerate case, and leads to algorithms such as DDPG and TD3. We will not pursue this direction further; it is taken up in Chapter 9.

8.1.3 The Objective Function

In an episodic task of horizon T we optimise the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}\right], \tag{8.1}$$

where the trajectory $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T)$ is distributed according to

$$p_\theta(\tau) = \rho_0(S_0) \prod_{t=0}^{T-1} \pi_\theta(A_t \mid S_t)\, p(S_{t+1}, R_{t+1} \mid S_t, A_t), \tag{8.2}$$

ρ_0 is the initial-state distribution, and p is the (unknown) transition kernel. In a continuing task the objective is the average reward $J(\theta) = \lim_{T\to\infty} \frac{1}{T}\, \mathbb{E}[\sum_{t=0}^{T-1} R_{t+1}]$; we focus on the episodic case for clarity, noting that the policy gradient theorem holds in both settings.

The central question is: how does $J(\theta)$ change as we vary θ? Naively differentiating (8.1) seems to require differentiating through the unknown environment dynamics p. The policy gradient theorem shows that this is unnecessary.

8.2 The Policy Gradient Theorem

8.2.1 The Likelihood-Ratio Trick

We begin by computing $\nabla_\theta J(\theta)$ directly from the definition. Because the horizon is finite, we may differentiate under the expectation:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau p_\theta(\tau)\, R(\tau) = \sum_\tau \nabla_\theta p_\theta(\tau)\, R(\tau), \tag{8.3}$$

where $R(\tau) = \sum_{t=0}^{T-1} \gamma^t R_{t+1}$ is the total discounted return of trajectory τ. This sum is not an expectation, so it cannot be estimated by sampling — yet. The **likelihood-ratio trick** (also called the *score function trick* or *REINFORCE trick*) converts it into one. For any $p_\theta(\tau) > 0$:

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau). \tag{8.4}$$

Substituting into (8.3) gives

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\big[R(\tau)\, \nabla_\theta \log p_\theta(\tau)\big]. \tag{8.5}$$

Now we exploit the factored form (8.2). Taking logarithms:

$$\log p_\theta(\tau) = \log \rho_0(S_0) + \sum_{t=0}^{T-1} \log \pi_\theta(A_t \mid S_t) + \sum_{t=0}^{T-1} \log p(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

The first and third terms do not depend on θ, so their gradients vanish:

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{8.6}$$

Crucially, this formula involves only the *policy*, not the transition kernel. Substituting into (8.5) yields the basic policy gradient estimator:

Theorem 8.1 (Basic policy gradient estimator)**.** *Under the assumptions above,*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\left(\sum_{t=0}^{T-1} \gamma^t R_{t+1}\right) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t)\right]. \tag{8.7}$$

Equation (8.7) is already a valid gradient estimator: given a single trajectory τ, the quantity inside the expectation is an unbiased sample of $\nabla_\theta J(\theta)$. But it has a serious variance problem: the full return $R(\tau)$ multiplies the log-gradient of every action in the trajectory, even actions that occurred *before* the rewards. Intuitively, what happened at $t = 3$ cannot retroactively change the reward received at $t = 1$. The next lemma formalises this.

8.2.2 The Causality Lemma and Reward-to-Go

Lemma 8.2 (Causality)**.** *For every $t \in \{0, \dots, T-1\}$,*

$$\mathbb{E}_{\tau\sim p_\theta}\left[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \sum_{k=0}^{t-1} \gamma^k R_{k+1}\right] = 0. \tag{8.8}$$

Proof. Let $C_t = \sum_{k=0}^{t-1} \gamma^k R_{k+1}$ denote the cumulative discounted reward *before* time t. This quantity is measurable with respect to the history $(S_0, A_0, R_1, \ldots, S_t)$ and in particular is independent of the choice of A_t given S_t. By the tower property of conditional expectation,

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, C_t] = \mathbb{E}[C_t\, \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t, \text{history}_{<t}]]\,.$$

For the inner expectation, fix $S_t = s$. Then $A_t \sim \pi_\theta(\cdot \mid s)$, so

$$\begin{aligned}\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t = s] &= \sum_a \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s) \\ &= \sum_a \nabla_\theta \pi_\theta(a \mid s) = \nabla_\theta \sum_a \pi_\theta(a \mid s) = \nabla_\theta 1 = 0.\end{aligned}$$

Because the inner conditional expectation is identically zero, the outer expectation is also zero. □

Remark 8.2. The key computation $\sum_a \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s) = \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$ is called the **score function identity**. It is the algebraic engine that drives both the baseline theorem and the causality lemma.

Define the **reward-to-go** from step t as

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}, \qquad \bar{G}_t = \gamma^t G_t = \sum_{k=t}^{T-1} \gamma^k R_{k+1}. \tag{8.9}$$

The full return decomposes as $R(\tau) = C_t + \bar{G}_t$. Applying Lemma 8.2 to (8.7) we obtain the reduced estimator:

Theorem 8.3 (Reward-to-go policy gradient)**.**

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, \bar{G}_t\right] = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, G_t\right]. \tag{8.10}$$

Proof. Starting from Theorem 8.1, swap the order of summation over t and the reward sum. For each fixed t, decompose $R(\tau) = C_t + \bar{G}_t$ and apply the linearity of expectation:

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, R(\tau)] = \underbrace{\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, C_t]}_{=0 \text{ by Lemma 8.2}} + \mathbb{E}\big[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, \bar{G}_t\big]\,.$$

Summing over t and using $\bar{G}_t = \gamma^t G_t$ gives (8.10). □

8.2.3 The Policy Gradient Theorem: Compact Form

The reward-to-go form (8.10) is the most practical estimator. It can also be written in a compact, state-action form by averaging over the discounted state-visitation measure.

Definition 8.1 (Discounted visitation measure)**.**

$$d^\gamma_{\pi_\theta}(s) = \sum_{t=0}^{T-1} \gamma^t \mathbb{P}_{\pi_\theta}(S_t = s).$$

Theorem 8.4 (Policy Gradient Theorem)**.**

$$\nabla_\theta J(\theta) = \sum_{s\in\mathcal{S}} d^\gamma_{\pi_\theta}(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s,a) = \mathbb{E}_{S\sim d^\gamma_{\pi_\theta},\, A\sim\pi_\theta(\cdot\mid S)}\left[\nabla_\theta \log \pi_\theta(A \mid S)\, Q^{\pi_\theta}(S,A)\right]. \tag{8.11}$$

Proof. Begin from (8.10). By linearity of expectation,

$$\nabla_\theta J(\theta) = \sum_{t=0}^{T-1} \gamma^t\, \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, G_t].$$

Apply the tower property conditioning on (S_t, A_t):

$$\begin{aligned}
&\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, G_t] \\
&\quad = \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, \mathbb{E}[G_t \mid S_t, A_t]] \\
&\quad = \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, Q^{\pi_\theta}(S_t, A_t)].
\end{aligned}$$

Expanding the outer expectation as a sum over (s,a) and using the identity $\pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s) = \nabla_\theta \pi_\theta(a \mid s)$:

$$\nabla_\theta J(\theta) = \sum_{t=0}^{T-1} \gamma^t \sum_s \mathbb{P}(S_t = s) \sum_a \nabla_\theta \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s,a).$$

Collecting the coefficient of each state s gives $\sum_t \gamma^t \mathbb{P}(S_t = s) = d^\gamma_{\pi_\theta}(s)$, which is Definition 8.1, and the result follows. □

Remark 8.3. Theorem 8.4 is remarkable: the gradient of $J(\theta)$ with respect to the policy parameters depends on the transition dynamics *only through* the action-value function Q^{π_θ} — and even that can be estimated from samples, without knowing p. This is what makes policy gradient methods truly model-free.

8.3 REINFORCE

8.3.1 The Algorithm

REINFORCE (Williams, 1992) is the direct Monte Carlo implementation of Theorem 8.3. Since $\mathbb{E}_{\pi_\theta}[G_t \mid S_t, A_t] = Q^{\pi_\theta}(S_t, A_t)$, we can substitute the sampled return G_t for the true Q^{π_θ}:

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \gamma^t G_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{8.12}$$

This update is an unbiased stochastic gradient ascent step on $J(\theta)$.

Several implementation notes are worth making. First, the factor γ^t in front of the update shrinks the step size for later time steps, reflecting the lower discounted weight of the corresponding gradient term. Some implementations omit this factor; doing so corresponds to optimising a slightly different (undiscounted) objective but is common in practice. Second, since all updates in an episode can be computed at the episode's end, the loop over t can be vectorised: compute all G_t values by a single backward pass, then sum the weighted gradients. Third, in the case of a softmax policy over actions, $\nabla_\theta \log \pi_\theta(a \mid s)$ is the score function, which equals $\phi(s,a) - \sum_{a'} \pi_\theta(a' \mid s)\, \phi(s,a')$ for feature vectors $\phi(s,a)$.

Algorithm 17: REINFORCE (Williams, 1992)

Input: Policy π_θ, step size α, discount γ
Initialise θ arbitrarily;
for *each episode* **do**
  Generate $\tau = (S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T)$ following π_θ;
  for $t = 0, 1, \ldots, T-1$ **do**
    Compute $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$;
    $\theta \leftarrow \theta + \alpha\, \gamma^t G_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)$;

8.3.2 A Worked Example

Example 8.1 (REINFORCE on a two-state MDP)**.** Consider the following simple episodic MDP:

- Two states: $\mathcal{S} = \{s_1, s_2\}$; two actions: $\mathcal{A} = \{a_L, a_R\}$.
- Initial state always s_1. Episode length $T = 2$.
- Transitions: from s_1, action a_L leads to s_2 with reward $R = 0$; action a_R stays in s_1 with reward $R = 0$. From s_2 (or s_1 at $t = 1$), a fixed terminal reward of $+1$ is received for a_R and 0 for a_L.
- Policy: $\pi_\theta(a_R \mid s_1) = \sigma(\theta_1)$, $\pi_\theta(a_R \mid s_2) = \sigma(\theta_2)$, where σ is the sigmoid function.

Set $\gamma = 1$, $\alpha = 0.1$, and initialise $\theta_1 = \theta_2 = 0$ (uniform policy, each action probability 0.5).

Episode: Suppose the sampled trajectory is $(s_1, a_L, 0, s_2, a_R, 1)$.

Compute the reward-to-go:

$$G_0 = 0 + 1 = 1, \qquad G_1 = 1.$$

Score function gradients (with $\theta_1 = \theta_2 = 0$, so $\pi = 0.5$ for both actions):

$$\nabla_{\theta_1} \log \pi_\theta(a_L \mid s_1) = \frac{\partial}{\partial \theta_1} \log(1 - \sigma(\theta_1)) = -\sigma(\theta_1) = -0.5,$$

$$\nabla_{\theta_2} \log \pi_\theta(a_R \mid s_2) = \sigma(\theta_2)(1 - \sigma(\theta_2)) \cdot \frac{1}{\sigma(\theta_2)} = 1 - 0.5 = 0.5.$$

Gradient updates ($\gamma = 1$):

$$\Delta\theta_1 = \alpha \cdot \gamma^0 \cdot G_0 \cdot (-0.5) = 0.1 \cdot 1 \cdot 1 \cdot (-0.5) = -0.05,$$

$$\Delta\theta_2 = \alpha \cdot \gamma^1 \cdot G_1 \cdot 0.5 = 0.1 \cdot 1 \cdot 1 \cdot 0.5 = +0.05.$$

After the update: $\theta_1 = -0.05$ (making a_L slightly less likely from s_1) and $\theta_2 = +0.05$ (making a_R more likely from s_2). Intuitively the algorithm is correct: the trajectory that yielded positive return pushes the policy toward actions that generated it.

Remark 8.4. REINFORCE converges to a local maximum of $J(\theta)$ under standard stochastic-approximation conditions (diminishing step sizes, $\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$). However, its variance can be very large, because G_t can vary enormously across episodes. The next two sections address variance reduction.

8.4 Variance Reduction: Baselines

8.4.1 The Baseline Theorem

A **baseline** $b(s)$ is any function of the state (but not of the action) that is subtracted from the return in the policy gradient estimator. The key result is that doing so does not introduce bias.

Theorem 8.5 (Baseline theorem)**.** *For any function $b : \mathcal{S} \to \mathbb{R}$,*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\,\big(G_t - b(S_t)\big)\right]. \tag{8.13}$$

Proof. It suffices to show that the additional term introduced by b has zero expectation. For each fixed t,

$$\begin{aligned}\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, b(S_t)] &= \mathbb{E}[b(S_t)\ \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t]] \\ &= \mathbb{E}[b(S_t) \cdot 0] = 0,\end{aligned}$$

where the inner expectation vanishes by the score function identity (see the proof of Lemma 8.2). Summing over t and applying linearity of expectation gives (8.13). □

8.4.2 The Optimal Baseline

Since the baseline does not change the mean of the estimator, we are free to choose it to *minimise variance*. Consider the variance of a single-step term $g_t(\theta) = \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\,(G_t - b)$ (suppressing state dependence for clarity). The variance of a scalar random variable X satisfies $\mathrm{Var}[X - c] = \mathrm{Var}[X] - 2c\ \mathrm{Cov}[X, 1] + c^2\ \mathrm{Var}[1]$; for a vector $\mathbf{v} = \mathbf{u}\,(G_t - b)$ the analogous computation applied per component gives an optimal baseline for the k-th component of the gradient:

$$b_k^*(s) = \frac{\mathbb{E}\big[(\nabla_\theta \log \pi_\theta(A_t \mid S_t))_k^2\, G_t \;\big|\; S_t = s\big]}{\mathbb{E}[(\nabla_\theta \log \pi_\theta(A_t \mid S_t))_k^2 \mid S_t = s]}. \tag{8.14}$$

This is a ratio of expectations and depends on the component index k. The optimal baseline is therefore a vector, and it is generally intractable. In practice, the state-value function $V^{\pi_\theta}(s)$ is used instead.

8.4.3 The State-Value Baseline

The most natural and widely used baseline is $b(S_t) = V^{\pi_\theta}(S_t)$. Subtracting the value function from the return yields the **advantage function**:

$$A^{\pi_\theta}(s, a) = Q^{\pi_\theta}(s, a) - V^{\pi_\theta}(s). \tag{8.15}$$

The policy gradient theorem with the value baseline becomes

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, A^{\pi_\theta}(S_t, A_t)\right]. \tag{8.16}$$

Since $\mathbb{E}_\pi[A^\pi(s, a) \mid s] = 0$ by definition, the advantage is mean-zero under the policy, which often drastically reduces estimator variance. In practice V^{π_θ} is unknown and must itself be estimated; the next section addresses this.

Example 8.2 (Effect of a baseline on variance)**.** Consider a bandit (single-state) environment with two actions. Action a_1 gives reward 10 always; action a_2 gives reward 10.1 always. Without a baseline, the gradient signal is proportional to 10 or 10.1 — both large and nearly equal. The estimator struggles to distinguish the slightly better action a_2 amid the noise. With the baseline $b = 10.05$ (the mean reward), the gradient signal is proportional to -0.05 or $+0.05$, which is centred at zero and has far lower absolute magnitude, reducing variance substantially.

8.5 REINFORCE with Learned Baseline

In the previous section, the baseline $b(s) = V^{\pi_\theta}(s)$ was taken as known. In practice it must be estimated. The simplest approach is to maintain a separate parameterised value function $\hat{V}_w(s)$ (the *critic*) alongside the policy (the *actor*), and train both simultaneously.

Algorithm 18: REINFORCE with Learned Baseline

Input: Policy π_θ, value function $\hat{V}_w$, step sizes α_θ, α_w, discount γ
Initialise θ, w arbitrarily;
for *each episode* **do**
  Generate $\tau = (S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T)$ following π_θ;
  for $t = 0, 1, \ldots, T-1$ **do**
    $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$;
    $\delta_t \leftarrow G_t - \hat{V}_w(S_t)$; `// Advantage estimate`
    $w \leftarrow w + \alpha_w \, \delta_t \, \nabla_w \hat{V}_w(S_t)$; `// Value function update`
    $\theta \leftarrow \theta + \alpha_\theta \, \gamma^t \, \delta_t \, \nabla_\theta \log \pi_\theta(A_t \mid S_t)$; `// Policy update`

The value function update minimises the mean-squared error $\mathbb{E}[(G_t - \hat{V}_w(S_t))^2]$, which is a supervised regression problem. Note that the gradient used for w is $\nabla_w \hat{V}_w(S_t)$, and the target G_t is a sample (not bootstrapped), so the value update is a standard gradient step — not a semi-gradient step as in TD(0) (see Chapter 6).

Remark 8.5. Introducing a separate critic opens the door to the full actor-critic family, where the critic uses TD bootstrapping rather than Monte Carlo returns. This trades the unbiasedness of REINFORCE for reduced variance, at the cost of introducing bias from function approximation. Actor-critic methods are the subject of Chapter 9.

A common concern is whether the learned baseline itself introduces bias. It does not: the key is that $\hat{V}_w(S_t)$ depends only on the state S_t, not on the action A_t chosen at that step. Therefore Theorem 8.5 applies regardless of how $\hat{V}_w$ was obtained. The policy gradient estimate $\delta_t \, \nabla_\theta \log \pi_\theta(A_t \mid S_t)$ remains unbiased for $\nabla_\theta J(\theta)$ even when $\hat{V}_w \neq V^{\pi_\theta}$, though suboptimal $\hat{V}_w$ will not reduce variance as effectively as the true value function.

8.6 RLOO: REINFORCE Leave-One-Out

8.6.1 Motivation: Policy Gradient in the Language Model Setting

Modern large-language-model (LLM) alignment pipelines (see Chapter 14) fine-tune a language model π_θ using a reward signal $r(\mathbf{y} \mid \mathbf{x})$ that scores a generated completion $\mathbf{y}$ given a prompt $\mathbf{x}$. In this setting the "action" is an entire completion (or, equivalently, a sequence

of token-level actions), and the "state" is the prompt. One therefore faces the policy gradient problem with:

- Very high variance per sample (completions can vary enormously).
- No access to a separately trained critic (training a value head adds memory and instability).
- A natural batch structure: for each prompt $\mathbf{x}$ in a mini-batch, the model generates K independent completions $\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(K)} \sim \pi_\theta(\cdot \mid \mathbf{x})$.

The batch structure is the key. RLOO exploits it to construct a low-variance, unbiased, critic-free baseline.

8.6.2 The RLOO Estimator

Fix a prompt $\mathbf{x}$ and draw K i.i.d. completions $\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(K)} \sim \pi_\theta(\cdot \mid \mathbf{x})$ with corresponding rewards $r_i = r(\mathbf{y}^{(i)} \mid \mathbf{x})$. The **RLOO baseline** for sample i is the average reward of the remaining $K-1$ samples:

$$b_i = \frac{1}{K-1} \sum_{j \neq i} r_j = \frac{1}{K-1} \left(\sum_{j=1}^{K} r_j - r_i \right). \tag{8.17}$$

The **RLOO policy gradient estimator** is then

$$\hat{g}_{\text{RLOO}} = \frac{1}{K} \sum_{i=1}^{K} (r_i - b_i) \nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}), \tag{8.18}$$

where we write $\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}) = \sum_t \nabla_\theta \log \pi_\theta(y_t^{(i)} \mid \mathbf{x}, y_{<t}^{(i)})$ for the token-sum score.

Substituting (8.17) into (8.18) we can also write

$$\hat{g}_{\text{RLOO}} = \frac{1}{K} \sum_{i=1}^{K} \left(r_i - \frac{\bar{r} - r_i/K}{(K-1)/K} \right) \nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}), \tag{8.19}$$

where $\bar{r} = \frac{1}{K} \sum_j r_j$. A cleaner expression is

$$r_i - b_i = r_i - \frac{\sum_{j \neq i} r_j}{K-1} = \frac{K\, r_i - \sum_{j=1}^{K} r_j}{K-1} = \frac{K}{K-1} (r_i - \bar{r}). \tag{8.20}$$

Thus the advantage used for sample i is a rescaled deviation of r_i from the batch mean, with a correction factor $K/(K-1)$ that accounts for the leave-one-out structure.

8.6.3 Unbiasedness of RLOO

Theorem 8.6 (RLOO is unbiased)**.** *The RLOO estimator $\hat{g}_{RLOO}$ defined in (8.18) is an unbiased estimator of $\nabla_\theta J(\theta)$:*

$$\mathbb{E}[\hat{g}_{RLOO}] = \nabla_\theta J(\theta).$$

Proof. It suffices to show that b_i satisfies the conditions of the baseline theorem (Theorem 8.5), i.e. that it is independent of the action $\mathbf{y}^{(i)}$ given the state (prompt $\mathbf{x}$). The baseline $b_i = \frac{1}{K-1} \sum_{j \neq i} r_j$ is a function of the $K-1$ other completions $\mathbf{y}^{(j)}$, $j \neq i$. Because all K

Algorithm 19: RLOO Policy Gradient Update

Input: Policy π_θ, reward function r, batch size K, step size α

for *each prompt* $\mathbf{x}$ *in the mini-batch* **do**
  Sample K completions $\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(K)} \sim \pi_\theta(\cdot \mid \mathbf{x})$;
  Compute rewards $r_i \leftarrow r(\mathbf{y}^{(i)} \mid \mathbf{x})$ for $i = 1, \ldots, K$;
  Compute batch mean $\bar{r} \leftarrow \frac{1}{K}\sum_{i=1}^{K} r_i$;
  for $i = 1, \ldots, K$ **do**
    $b_i \leftarrow \frac{K}{K-1}(\bar{r} - r_i/K) \cdot \frac{1}{1} = \frac{1}{K-1}\big(\sum_j r_j - r_i\big)$; `// LOO baseline`
    $\hat{A}_i \leftarrow r_i - b_i = \frac{K}{K-1}(r_i - \bar{r})$;
  $\theta \leftarrow \theta + \alpha \cdot \frac{1}{K}\sum_{i=1}^{K} \hat{A}_i \nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})$;

completions are drawn independently from $\pi_\theta(\cdot \mid \mathbf{x})$, the baseline b_i is independent of $\mathbf{y}^{(i)}$ conditionally on $\mathbf{x}$.

Formally, taking the expectation of the i-th term:

$$\mathbb{E}\Big[(r_i - b_i)\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \mathbb{E}\Big[r_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] - \mathbb{E}\Big[b_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big].$$

For the second term, condition on all $\mathbf{y}^{(j)}$ with $j \neq i$ (which determines b_i):

$$\mathbb{E}\Big[b_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \mathbb{E}\Big[b_i\;\mathbb{E}\Big[\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}) \;\Big|\; \mathbf{x},\, \{r_j\}_{j\neq i}\Big]\Big].$$

The inner expectation is exactly the score function identity applied to the distribution $\pi_\theta(\cdot \mid \mathbf{x})$:

$$\mathbb{E}_{\mathbf{y}^{(i)}\sim\pi_\theta(\cdot\mid\mathbf{x})}\Big[\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \nabla_\theta \sum_{\mathbf{y}} \pi_\theta(\mathbf{y} \mid \mathbf{x}) = \nabla_\theta 1 = 0.$$

Therefore the subtracted baseline term is zero, and

$$\mathbb{E}\Big[(r_i - b_i)\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \mathbb{E}\Big[r_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \nabla_\theta J(\theta)\big|_{\text{single sample}}.$$

Averaging over $i = 1, \ldots, K$ gives $\hat{g}_{\text{RLOO}}$ whose expectation equals $\nabla_\theta J(\theta)$. □

8.6.4 Variance Reduction of RLOO

To understand why RLOO reduces variance relative to plain REINFORCE, compare the two estimators. Plain REINFORCE uses the full return r_i as the gradient weight (no baseline):

$$\hat{g}_{\text{RF}} = \frac{1}{K}\sum_{i=1}^{K} r_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}).$$

Let $g_i = \nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})$ and $\mu = \mathbb{E}[r_i]$. Then $\mathrm{Var}[r_i\,g_i] \approx \mathrm{Var}[r_i]\,\mathrm{Var}[g_i] + \mu^2\,\mathrm{Var}[g_i]$ (ignoring cross-terms), which grows with the magnitude of μ. RLOO replaces r_i with $r_i - b_i \approx r_i - \mu$ (up to the $K/(K-1)$ correction), whose mean is zero. The dominant μ^2 term is therefore eliminated, and the variance scales with $\mathrm{Var}[r_i]$ rather than $\mathbb{E}[r_i^2]$.

More precisely, for large K the RLOO baseline $b_i \to \mu = \mathbb{E}_{\pi_\theta}[r]$ (by the law of large numbers), and RLOO approaches the oracle baseline $b = \mu$. The residual variance of $r_i - b_i$ is $\mathrm{Var}[r_i]/(K-1)$, compared to $\mathrm{Var}[r_i] + \mu^2$ for no baseline. When rewards have a large positive mean ($\mu \gg 0$), the variance reduction can be dramatic.

8.6.5 Connection to RLHF

In RLHF pipelines (Chapter 14), a reward model $r_\phi(\mathbf{y} \mid \mathbf{x})$ provides scalar feedback on model completions. RLOO fits naturally: for each prompt $\mathbf{x}$ one samples K completions from the current policy π_θ, scores them with the reward model, and computes the RLOO gradient. No critic network is needed, which saves GPU memory and removes the instability that can arise when a critic is jointly trained with the policy.

RLOO was introduced by Ahmadian et al. (2024) and shown to outperform PPO-based RLHF baselines with the same number of reward-model queries, largely due to the variance reduction from the leave-one-out baseline and the elimination of the critic. It can also be combined with a KL-divergence penalty against a reference policy π_{ref}, replacing r_i with $r_i - \beta \log(\pi_\theta(\mathbf{y}^{(i)})/\pi_{\text{ref}}(\mathbf{y}^{(i)}))$, which controls how far the fine-tuned model drifts from the pre-trained model.

Remark 8.6. RLOO is closely related to Group Relative Policy Optimisation (GRPO), which also uses a batch of samples per prompt and normalises advantages within the group. The key difference is that GRPO divides by the within-group standard deviation (standardising the advantage), while RLOO uses the unscaled leave-one-out deviation. Both belong to a family of "self-baseline" estimators that extract a variance-reducing signal from the batch itself, without a separately trained critic.

8.7 Natural Policy Gradient

8.7.1 The Problem with the Ordinary Gradient

The ordinary gradient $\nabla_\theta J(\theta)$ is defined with respect to the Euclidean metric on parameter space. A gradient step $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$ moves by a fixed Euclidean distance in θ-space, but the corresponding change in the policy distribution π_θ depends on the local geometry of the policy manifold. Near a deterministic policy (where some action probability is close to 1), a small step in θ causes a large change in π_θ; near a uniform policy, the same step causes a smaller distributional change. The ordinary gradient is therefore poorly calibrated: its magnitude reflects Euclidean distance in parameter space, not distributional change.

8.7.2 The Fisher Information Matrix

The natural policy gradient corrects for this by using the **Fisher information matrix** of the policy as a Riemannian metric:

$$F(\theta) = \mathbb{E}_{S \sim d^{\gamma}_{\pi_\theta},\, A \sim \pi_\theta(\cdot \mid S)} \left[\nabla_\theta \log \pi_\theta(A \mid S)\, \nabla_\theta \log \pi_\theta(A \mid S)^\top \right]. \tag{8.21}$$

This matrix measures the expected outer product of score vectors; it captures how sensitively the distribution π_θ responds to small changes in θ. Equivalently, $F(\theta)$ is the Hessian of the KL divergence $D_{\text{KL}}(\pi_\theta \parallel \pi_{\theta'})$ with respect to θ' evaluated at $\theta' = \theta$.

The **natural gradient** is the steepest-ascent direction in the space of distributions, measured by the KL divergence:

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1} \nabla_\theta J(\theta). \tag{8.22}$$

The natural policy gradient update is

$$\theta \leftarrow \theta + \alpha\, F(\theta)^{-1} \nabla_\theta J(\theta). \tag{8.23}$$

Theorem 8.7 (Amari, 1998; Kakade, 2002)**.** *The natural gradient* $\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1}\nabla_\theta J(\theta)$ *is the steepest-ascent direction for* J *subject to the constraint that the KL divergence between old and new policy satisfies* $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\theta+\Delta\theta}) \le \epsilon$*, for small* ϵ*.*

Proof sketch. Constrained optimisation of the first-order Taylor expansion $J(\theta + \Delta\theta) \approx J(\theta) + \nabla_\theta J(\theta)^\top \Delta\theta$ subject to $\Delta\theta^\top F(\theta)\Delta\theta \le \epsilon$ (the KL ball constraint in parameter space) gives, by the Cauchy-Schwarz inequality in the F-weighted inner product, the optimal direction $\Delta\theta^* \propto F(\theta)^{-1}\nabla_\theta J(\theta)$. □

8.7.3 Practical Considerations

Computing $F(\theta)^{-1}$ directly requires $O(d^2)$ memory and $O(d^3)$ time, which is infeasible for large neural networks. Several approximations are used in practice:

- **Conjugate gradient.** Solve $F(\theta)\,\tilde{g} = \nabla_\theta J(\theta)$ iteratively. Each step requires a Fisher-vector product, which can be computed in $O(d)$ using back-propagation through the score function. This is the approach used in TRPO (Schulman et al., 2015).
- **Kronecker-factored approximation (K-FAC).** Approximate $F(\theta)$ as a block-diagonal matrix with Kronecker structure, exploiting the layer-wise factorisation of neural networks (Martens & Grosse, 2015).
- **Proximal policy optimisation (PPO).** Rather than explicitly inverting the Fisher, PPO uses a clipped surrogate objective to implicitly constrain the KL divergence, achieving similar behaviour at much lower cost (Schulman et al., 2017).

The natural policy gradient is conceptually important because it provides the theoretical foundation for trust-region and proximal methods; its practical implementations are the workhorses of modern deep RL and LLM fine-tuning.

8.8 Convergence Analysis

8.8.1 Convergence of REINFORCE

REINFORCE is a stochastic gradient ascent algorithm applied to $J(\theta)$. Its convergence properties follow from the general theory of stochastic approximation (Robbins & Monro, 1951), provided the gradient estimator is unbiased (established in Theorem 8.3) and the step sizes satisfy the Robbins-Monro conditions:

$$\sum_{k=1}^{\infty} \alpha_k = \infty, \qquad \sum_{k=1}^{\infty} \alpha_k^2 < \infty. \tag{8.24}$$

Theorem 8.8 (Convergence of REINFORCE)**.** *Assume* $\pi_\theta(a \mid s)$ *is twice continuously differentiable in* θ*, and that the gradient* $\nabla_\theta \log \pi_\theta$ *is bounded. If the step sizes satisfy* (8.24) *and* $J(\theta)$ *is bounded above, then the iterates of REINFORCE satisfy*

$$\liminf_{k\to\infty} \|\nabla_\theta J(\theta_k)\|^2 = 0 \quad \textit{almost surely.}$$

That is, REINFORCE converges to a stationary point of J*.*

The theorem guarantees convergence to a *stationary point*, which may be a local maximum, a saddle point, or (in degenerate cases) a local minimum. Global optimality is not guaranteed for non-convex J. However, for *tabular* policies (no function approximation, finite state and action spaces), the policy gradient objective has no spurious local maxima under mild conditions (Agarwal et al., 2021), so convergence to the global optimum can be guaranteed.

8.8.2 Convergence Rate

For the non-convex setting, the state of the art is a $\tilde{O}(1/\sqrt{K})$ convergence rate to a stationary point in terms of the number of trajectories K:

Theorem 8.9 (Convergence rate, informal)**.** *Under standard regularity conditions, REINFORCE with a constant step size α finds a point θ with $\|\nabla_\theta J(\theta)\|^2 \leq \epsilon$ using at most $O(\sigma^2/(\alpha\epsilon))$ episodes, where σ^2 is an upper bound on the variance of the gradient estimator.*

This rate reveals the central role of variance: reducing σ^2 (via baselines or RLOO) directly reduces the sample complexity. With a baseline that reduces variance by a factor of c, one needs c times fewer episodes to reach the same accuracy.

8.8.3 The Bias-Variance Tradeoff

All policy gradient estimators involve a bias-variance tradeoff in how they estimate $Q^{\pi_\theta}(s, a)$:

- **Monte Carlo return G_t:** unbiased but high variance (accumulates reward noise over the full episode).
- **TD(0) target $R_{t+1} + \gamma\hat{V}(S_{t+1})$:** biased (due to the approximation error in $\hat{V}$) but low variance (only one step of noise).
- **n-step return:** interpolates between the two extremes; increasing n reduces bias and increases variance.
- **GAE (Generalised Advantage Estimation, Section 6.10) with $\lambda \in [0, 1]$:** exponentially weighted average of n-step returns, combining low bias (λ near 1) with low variance (λ near 0).

The REINFORCE estimator corresponds to the $n = T$ (full Monte Carlo) extreme. RLOO keeps the Monte Carlo return but reduces variance through the leave-one-out baseline, while actor-critic methods (Chapter 9) reduce variance through bootstrapping at the cost of some bias.

Remark 8.7. In practice, variance is typically the dominant problem for policy gradient methods, especially in long-horizon tasks or tasks with large reward scales. The baseline and reward-to-go reductions together often reduce variance by one to two orders of magnitude compared to the naive estimator (8.7), making the difference between convergence and divergence.

8.9 Comparison of Policy Gradient Estimators

Table 8.1 summarises the properties of the estimators developed in this chapter. We use the following notation: $\hat{g}$ denotes the gradient estimator, b denotes the baseline, and K denotes the number of samples per prompt/state.

Several observations are worth drawing from Table 8.1.

First, all estimators in the "Bias" column marked "Unbiased" are unbiased as estimators of $\nabla_\theta J(\theta)$: the policy gradient is an exact gradient of the true objective. Bias enters only when one replaces Monte Carlo returns with bootstrapped targets (as in actor-critic methods), or when value function approximation error propagates into the advantage estimate.

Table 8.1: Comparison of policy gradient estimators.

Estimator	Baseline	Bias/Var	Critic?	Notes
REINFORCE	None	Unbias./V.High	No	Simplest; rarely used alone
REINFORCE+b	Fixed b	Unbias./High	No	Reduces mean; suboptimal
REINFORCE+$\hat{V}$	$\hat{V}_w(s)$	Unbias./Med	Yes	Most common baseline
RLOO	LOO mean	Unbias./Low	No	$K \geq 2$ samples; no critic
Actor-Critic	$\hat{V}_w(s)$	Biased/Low	Yes	Bootstrap bias
Natural PG	None	Unbias./Med	No	Better geometry
TRPO/PPO	$\hat{V}_w(s)$	Biased/Low	Yes	KL/clip; state of art

Second, the "Critic?" column reveals a fundamental design choice. Critic-free methods (REINFORCE, RLOO) are simpler but rely on high-variance Monte Carlo returns; critic-based methods (actor-critic, PPO) achieve lower variance at the cost of a separate network, additional hyperparameters (critic learning rate, number of update epochs), and potential instability when the critic is inaccurate.

Third, RLOO achieves the best of both worlds — no critic, yet substantially lower variance than plain REINFORCE — by exploiting the natural batch structure of LLM training. For single-sample settings ($K = 1$), RLOO reduces to plain REINFORCE (the LOO baseline is zero with a single sample), so the improvement is contingent on having $K \geq 2$.

Fourth, the natural policy gradient and its practical descendants (TRPO, PPO) are preferable when large parameter updates are needed, since they are invariant to reparametrisation of the policy: the update direction is the same regardless of how π_θ is represented mathematically.

Exercises

Exercise 8.1. Let $\pi_\theta(a \mid s) = \mathrm{softmax}(\theta_a)$ be a tabular softmax policy over n actions.

1. Show that $\nabla_{\theta_a} \log \pi_\theta(a' \mid s) = \mathbb{1}[a = a'] - \pi_\theta(a \mid s)$.
2. Verify the score function identity: $\sum_a \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) = 0$.
3. Suppose $n = 3$, $\theta = (1, 0, -1)$, and an action $a = 2$ is observed with return $G = 5$. Compute the REINFORCE update $\Delta\theta$ with step size $\alpha = 0.01$.

Exercise 8.2. Prove the causality lemma (Lemma 8.2) directly from the definition of $p_\theta(\tau)$, without using the tower property. Hint: write out the sum over trajectories explicitly and factor it over time steps.

Exercise 8.3. Consider a one-state, two-action bandit. Let $r(a_1) = 1$ and $r(a_2) = 3$ with probability 1. Let $\pi_\theta(a_1) = \sigma(-\theta)$, $\pi_\theta(a_2) = \sigma(\theta)$.

1. Compute $J(\theta) = \mathbb{E}[r]$ as a function of θ.
2. Compute $\nabla_\theta J(\theta)$ analytically.

3. Compute the variance of the REINFORCE gradient estimator (no baseline) as a function of θ.

4. Show that the baseline $b = \mathbb{E}[r] = J(\theta)$ reduces the variance. Compute the variance with this baseline.

5. Which baseline is optimal according to (8.14)?

Exercise 8.4. Prove the unbiasedness of the RLOO estimator directly, without invoking Theorem 8.5. Specifically, for $K = 2$ completions $y^{(1)}, y^{(2)} \sim \pi_\theta(\cdot \mid x)$ with rewards r_1, r_2, show that

$$\mathbb{E}\Big[(r_1 - r_2)\,\nabla_\theta \log \pi_\theta(y^{(1)} \mid x) + (r_2 - r_1)\,\nabla_\theta \log \pi_\theta(y^{(2)} \mid x)\Big] = 2\,\nabla_\theta J(\theta).$$

Exercise 8.5. In the notation of Section 8.6, let $r_1, \ldots, r_K$ be i.i.d. with mean μ and variance σ^2, and let $g_1, \ldots, g_K$ be the score functions (independent of the rewards by the score function identity, so treat them as fixed for this exercise). Show that

$$\mathrm{Var}\left[\frac{1}{K}\sum_{i=1}^{K}(r_i - b_i)\,g_i\right] \approx \frac{\sigma^2}{K-1}\cdot\frac{1}{K}\sum_{i=1}^{K} g_i^2$$

for large K, and compare this with the variance of the plain REINFORCE estimator $\frac{1}{K}\sum_i r_i g_i$. Under what conditions is the variance reduction greatest?

Exercise 8.6.

1. Let π_θ be a Gaussian policy: $\pi_\theta(a \mid s) = \mathcal{N}(a;\, \mu_\theta(s),\, \sigma^2)$ with fixed variance σ^2. Compute the Fisher information matrix $F(\theta)$ in terms of $\nabla_\theta \mu_\theta(s)$.

2. Show that when $\mu_\theta(s) = \theta^\top \phi(s)$ is linear, the natural gradient reduces to $\tilde{\nabla}_\theta J = \sigma^2\, F_\phi^{-1} \nabla_\theta J$ where F_ϕ is the feature covariance matrix.

3. Explain intuitively why the natural gradient update is invariant to reparametrisation of the policy.

Exercise 8.7. Consider a continuous-action MDP with $\mathcal{A} = \mathbb{R}$ and a Gaussian policy $\pi_\theta(a \mid s) = \mathcal{N}(a;\, \mu_\theta(s),\, e^{2\rho})$, where $\theta = (\theta_\mu, \rho)$ parameterises the mean network and log-standard-deviation.

1. Derive $\nabla_{\theta_\mu} \log \pi_\theta(a \mid s)$ and $\nabla_\rho \log \pi_\theta(a \mid s)$.

2. Interpret the gradient with respect to ρ: when does the update increase entropy, and when does it decrease?

3. Argue that the REINFORCE update for ρ implements automatic entropy regulation through the reward signal alone.

Exercise 8.8.

1. State the Robbins-Monro conditions (8.24) and give one example of a step-size sequence that satisfies them and one that does not.

2. Consider REINFORCE with a constant step size $\alpha > 0$. Why does this *not* satisfy the Robbins-Monro conditions, and what does the algorithm converge to instead?

3. Explain the bias-variance tradeoff between REINFORCE (Monte Carlo return) and an actor-critic method (TD target) in the context of Theorem 8.9. How does reducing variance via a baseline affect the sample complexity bound?

Exercise 8.9. In a language model setting, the policy π_θ generates token sequences autoregressively. Suppose we use RLOO with $K = 4$ completions per prompt.

1. Write out the RLOO advantage $\hat{A}_i = r_i - b_i$ for $i = 1, 2, 3, 4$ using the formula in (8.20).
2. Suppose the four rewards are $r = (0.2, 0.8, 0.5, 0.3)$. Compute all four advantages.
3. Show that $\sum_{i=1}^{4} \hat{A}_i = 0$. Is this always true? Prove it.
4. Discuss why the zero-sum property of RLOO advantages (when summed over the batch) can be advantageous or disadvantageous in practice.

Exercise 8.10. You are designing an RL system to fine-tune a pre-trained language model on a text-generation task. You have access to a black-box reward function and can generate up to $K = 8$ completions per prompt in parallel.

1. Compare REINFORCE with a learned value baseline, RLOO, and PPO on the following dimensions: (a) memory requirements, (b) number of hyperparameters, (c) potential for bias in the gradient estimate, and (d) ease of implementation.
2. Under what circumstances would you choose RLOO over PPO? Under what circumstances would you prefer PPO?
3. Propose a simple experiment to empirically determine the optimal K for RLOO in your setting.

Chapter 9

Actor-Critic Methods and Proximal Policy Optimization

The idea of having two interacting components, one for behavior and one for criticism, seems to be as old as the idea of reinforcement learning itself. —Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction*

Chapter 8 developed the policy gradient theorem and the REINFORCE algorithm, establishing that one can directly optimize a parameterized policy by following the gradient of the expected return. REINFORCE is conceptually elegant but suffers from a fundamental statistical weakness: its gradient estimator uses the full discounted return G_t as a signal for every action taken in an episode. Because G_t is a sum of many random rewards stretching to the end of the episode, its variance grows with the horizon, leading to slow and noisy learning.

The remedy introduced in this chapter is to replace G_t with a lower-variance estimate of how much better than average a particular action was — the *advantage function* — and to learn this estimate online using a second neural network called the *critic*. The resulting class of algorithms is known as **actor-critic** methods. The actor (the policy) and the critic (the value function) interact on every time step: the critic evaluates the actor's choices, and the actor uses that evaluation to update its parameters.

From this foundation we build progressively more powerful algorithms. Generalized Advantage Estimation (GAE) provides a flexible bias-variance tradeoff for the advantage signal. Asynchronous Advantage Actor-Critic (A3C) exploits parallel workers to decorrelate experience and accelerate training. Trust Region Policy Optimization (TRPO) introduces the key insight that policy updates must be constrained to remain close to the current policy. Proximal Policy Optimization (PPO) achieves the same effect with a simple clipping mechanism that is fast, easy to implement, and remarkably robust in practice. Finally, we study how PPO is deployed for fine-tuning large language models with human feedback, the paradigm known as RLHF.

9.1 The Advantage Function

9.1.1 Definition and Basic Properties

Recall from Chapter 8 that the policy gradient theorem expresses the gradient of the objective in terms of the action-value function:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim d^{\gamma}_{\pi_\theta},\, A\sim\pi_\theta(\cdot|S)}\big[\nabla_\theta \log \pi_\theta(A \mid S)\; Q^{\pi_\theta}(S,A)\big]. \tag{9.1}$$

Chapter 8 also showed that any *state-dependent* baseline $b(S_t)$ can be subtracted from $Q^{\pi_\theta}(S_t, A_t)$ without changing the expected gradient (Theorem 8.5 in Chapter 8). The best possible baseline is the one that most reduces variance, and under mild conditions the optimal choice is precisely the state-value function $V^{\pi_\theta}(s)$. The difference that remains after subtracting this baseline is so important that it deserves its own name.

Definition 9.1 (Advantage function)**.** For a policy π and the induced action-value and state-value functions Q^π and V^π, the **advantage function** is

$$A^\pi(s,a) \;=\; Q^\pi(s,a) - V^\pi(s). \tag{9.2}$$

Intuitively, $A^\pi(s,a)$ measures the relative quality of action a compared with the average action the policy would take in state s. A positive advantage means that a is better than average; a negative advantage means it is worse. This centring property is formalized in the following proposition.

Proposition 9.1 (Zero-mean advantage)**.** *For any policy π and any state $s \in \mathcal{S}$,*

$$\mathbb{E}_{A\sim\pi(\cdot|s)}\big[A^\pi(s,A)\big] = 0. \tag{9.3}$$

Proof. By Definition 9.1 and linearity of expectation,

$$\mathbb{E}_{A\sim\pi(\cdot|s)}\big[A^\pi(s,A)\big] = \mathbb{E}_{A\sim\pi(\cdot|s)}\big[Q^\pi(s,A)\big] - V^\pi(s).$$

The Bellman equation (Chapter 3, Equation (3.12)) gives $V^\pi(s) = \sum_a \pi(a|s)\, Q^\pi(s,a) = \mathbb{E}_{A\sim\pi(\cdot|s)}[Q^\pi(s,A)]$, so the right-hand side is zero. □

Substituting $V^{\pi_\theta}(S_t)$ for the baseline in the policy gradient formula (9.1) yields the **advantage form** of the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim d^{\gamma}_{\pi_\theta},\, A\sim\pi_\theta(\cdot|S)}\big[\nabla_\theta \log \pi_\theta(A \mid S)\; A^{\pi_\theta}(S,A)\big]. \tag{9.4}$$

This is the version of the policy gradient theorem that underlies all actor-critic algorithms.

9.1.2 Why the Advantage Reduces Variance

To see concretely why using $A^\pi(s,a)$ instead of $Q^\pi(s,a)$ helps, consider a simple example.

Example 9.1 (Advantage versus raw return)**.** Suppose an agent is in state s where every action yields a high return because s is already a very good state. Concretely, assume $Q^\pi(s,a) \in \{8, 10, 12\}$ with equal probability for three actions, so $V^\pi(s) = 10$. The raw action-value scores have mean 10 and variance $\mathrm{Var}[Q^\pi(s,\cdot)] = \frac{4+0+4}{3} \approx 2.67$.

The advantage scores are $A^\pi(s,a) \in \{-2, 0, 2\}$, which have mean 0 (consistent with Proposition 9.1) and the same variance 2.67. At first glance the variance is unchanged, but notice the crucial difference: because the policy gradient update is $\nabla_\theta \log \pi_\theta(a|s) \cdot A^\pi(s,a)$,

updates for the mediocre action ($A = 0$) are zero regardless of the policy parameters. Using the raw Q would produce a nonzero gradient signal of magnitude 10 even for the neutral action, polluting the update with a large global offset that must be cancelled across many samples. The advantage removes this offset perfectly and in expectation. □

More formally, the variance of the policy gradient estimator with the optimal baseline $b^*(s)$ can be shown to satisfy

$$\mathrm{Var}\big[\nabla_\theta \log \pi_\theta(A|S)\,(Q^\pi(S,A) - b^*(S))\big] \;\le\; \mathrm{Var}\big[\nabla_\theta \log \pi_\theta(A|S)\,Q^\pi(S,A)\big], \tag{9.5}$$

and the reduction can be dramatic in environments where the value function varies substantially across states but the advantage variation within each state is modest.

9.2 The Actor-Critic Architecture

9.2.1 Two-Component Design

An actor-critic algorithm maintains two interacting components.

Definition 9.2 (Actor and critic)**.** An **actor-critic** algorithm maintains:

- an **actor**: a parameterized policy $\pi_\theta(a \mid s)$ whose parameters θ are updated to maximize $J(\theta)$;
- a **critic**: a parameterized value function $V_\phi(s)$ whose parameters ϕ are updated to approximate $V^{\pi_\theta}(s)$.

The name reflects the metaphor of a performer (the actor) and a reviewer (the critic): after every action, the critic signals whether that action was above or below average, and the actor adjusts accordingly.

Figure 9.1 illustrates the information flow. The environment emits a state S_t; the actor selects action A_t; the environment transitions to S_{t+1} and emits reward R_{t+1}. The critic uses (S_t, R_{t+1}, S_{t+1}) to compute a TD error δ_t, which serves simultaneously as the critic's learning signal (it measures how wrong the critic's own prediction was) and as an estimate of the advantage to guide the actor's update.

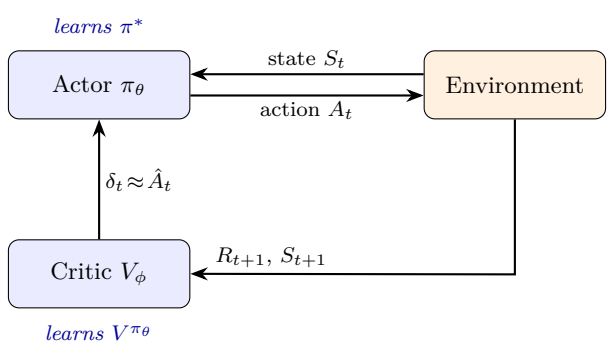


Figure 9.1: Information flow in an actor-critic algorithm. The actor selects actions; the environment returns rewards and next states; the critic produces a TD error δ_t that guides both critic and actor updates.

9.2.2 Shared versus Separate Networks

In practice, the actor and critic can be implemented in two ways. When they are **separate networks**, the actor π_θ and critic V_ϕ have completely disjoint parameters, which makes the two learning objectives independent and often easier to tune. When they **share a trunk** (a common set of hidden layers), the two heads share a representation, reducing the total number of parameters and encouraging the hidden layers to extract features useful for both policy and value estimation. Shared trunks are the default in modern implementations, especially for complex observation spaces such as images or natural-language tokens, where learning good representations is itself a significant challenge.

9.3 TD Error as Advantage Estimate

9.3.1 The One-Step Estimate

Although the advantage form (9.4) of the policy gradient is theoretically clean, it requires knowledge of both Q^{π_θ} and V^{π_θ}. Computing both is redundant: because $Q^\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s, A_t = a]$, we can recover the advantage from the value function alone by taking the observed transition as a sample. The resulting quantity is exactly the one-step TD error introduced in Chapter 6.

Theorem 9.2 (TD error is an unbiased advantage estimate)**.** *Let $V_\phi = V^{\pi_\theta}$ (the critic is perfect). Then for any transition $(S_t, A_t, R_{t+1}, S_{t+1})$ sampled under π_θ,*

$$\mathbb{E}\big[\delta_t \;\big|\; S_t = s,\ A_t = a\big] \;=\; A^{\pi_\theta}(s,a), \tag{9.6}$$

where $\delta_t = R_{t+1} + \gamma V^{\pi_\theta}(S_{t+1}) - V^{\pi_\theta}(S_t)$.

Proof. Taking the conditional expectation of δ_t given $S_t = s$ and $A_t = a$:

$$\mathbb{E}[\delta_t \mid S_t = s, A_t = a] = \mathbb{E}\big[R_{t+1} + \gamma V^{\pi_\theta}(S_{t+1}) \mid S_t = s, A_t = a\big] - V^{\pi_\theta}(s).$$

By the Bellman equation for Q^{π_θ} (Chapter 3),

$$\mathbb{E}\big[R_{t+1} + \gamma V^{\pi_\theta}(S_{t+1}) \mid S_t = s, A_t = a\big] = Q^{\pi_\theta}(s,a),$$

which follows because the Markov property allows us to factor $\mathbb{E}[G_{t+1} \mid S_t, A_t] = \mathbb{E}[\mathbb{E}[G_{t+1} \mid S_{t+1}] \mid S_t, A_t] = \mathbb{E}[V^{\pi_\theta}(S_{t+1}) \mid S_t, A_t]$. Therefore,

$$\mathbb{E}[\delta_t \mid S_t = s, A_t = a] = Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s) = A^{\pi_\theta}(s,a). \qquad \square$$

Theorem 9.2 is the key result that makes actor-critic algorithms practical. Instead of maintaining separate function approximators for Q^π and V^π, we need only learn V_ϕ; the TD error δ_t is then a single-sample, unbiased estimate of the advantage.

Remark 9.1. The unbiasedness holds only when $V_\phi = V^{\pi_\theta}$. In practice the critic is imperfect, so the TD error introduces *bias* into the advantage estimate. The bias-variance tradeoff is the central tension in actor-critic methods: a well-trained critic reduces variance substantially but introduces systematic errors. This is the same tension that distinguishes TD(0) from Monte Carlo in Chapter 6.

Algorithm 20: One-Step On-Policy Actor-Critic (TD(0) Critic)

Input: Policy network π_θ, value network V_ϕ, step sizes $\alpha_\theta, \alpha_\phi > 0$, discount γ

Initialize θ and ϕ randomly
for *each episode* **do**
  Observe initial state S_0
  for $t = 0, 1, 2, \ldots$ *until terminal* **do**
    Sample action $A_t \sim \pi_\theta(\cdot \mid S_t)$
    Observe R_{t+1} and S_{t+1}
    Compute TD error: $\delta_t \leftarrow R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$
    `// Critic update: semi-gradient TD(0)`
    $\phi \leftarrow \phi + \alpha_\phi \, \delta_t \, \nabla_\phi V_\phi(S_t)$
    `// Actor update: policy gradient with TD-error advantage`
    $\theta \leftarrow \theta + \alpha_\theta \, \gamma^t \, \delta_t \, \nabla_\theta \log \pi_\theta(A_t \mid S_t)$

9.3.2 One-Step Actor-Critic Algorithm

Plugging δ_t into the advantage-form policy gradient gives the simplest on-policy actor-critic update.

The critic update in Algorithm 20 is exactly the semi-gradient TD(0) update derived in Chapter 6 (Equation (6.18)). The actor update replaces the full return G_t of REINFORCE with the TD error δ_t, which is available immediately after each step and has much lower variance. Crucially, neither update requires waiting for the episode to end, making actor-critic methods applicable to continuing (non-episodic) tasks.

Example 9.2 (CartPole with one-step actor-critic)**.** Consider the CartPole environment, where the state is a four-dimensional vector (cart position, cart velocity, pole angle, pole angular velocity), the action is binary (push left or right), and the episode terminates when the pole falls or the cart leaves the track.

The actor is a two-layer network with softmax output for the two actions; the critic is a two-layer network producing a scalar $V_\phi(s)$. After a step where the pole nearly falls and then recovers,

$$R_{t+1} = 1, \quad V_\phi(S_t) = 4.2, \quad V_\phi(S_{t+1}) = 3.1,$$

so $\delta_t = 1 + 0.99 \times 3.1 - 4.2 = -0.131$. The negative TD error signals that the state S_t was overvalued — the agent is not doing as well as expected — and the actor update slightly decreases the probability of the action taken at S_t. Conversely, on a step where the pole is stable and the critic underestimated the situation, $\delta_t > 0$ increases the probability of the stabilizing action. This immediate, per-step feedback is what distinguishes actor-critic from REINFORCE. □

9.4 Generalized Advantage Estimation

9.4.1 Multi-Step Returns and the n-Step Advantage

The one-step advantage estimate δ_t has low variance but is biased whenever $V_\phi \neq V^{\pi_\theta}$. At the other extreme, the Monte Carlo return $G_t - V_\phi(S_t)$ is an unbiased estimate of the advantage but has high variance because it sums all future rewards. Chapter 6 showed that

n-step returns interpolate between these poles: the n-step TD target is

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_\phi(S_{t+n}).$$

The corresponding n-step advantage estimate is

$$\hat{A}_t^{(n)} = G_t^{(n)} - V_\phi(S_t) = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n V_\phi(S_{t+n}) - V_\phi(S_t). \tag{9.7}$$

As $n \to \infty$, $\hat{A}_t^{(n)} \to G_t - V_\phi(S_t)$, the Monte Carlo advantage estimate. As $n = 1$, we recover δ_t.

Rather than committing to a fixed n, Generalized Advantage Estimation (GAE), introduced by Schulman et al. (2016), takes an exponentially weighted average over all n-step estimates.

9.4.2 GAE Definition and Derivation

Definition 9.3 (Generalized Advantage Estimation)**.** For parameters $\gamma \in (0, 1)$ and $\lambda \in [0, 1]$, define the TD errors

$$\delta_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t),$$

and the **GAE advantage estimate** as

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \, \delta_{t+l}. \tag{9.8}$$

The formula (9.8) looks like an exponentially-decaying sum over one-step TD errors starting at time t. It is equivalent to TD(λ) for value estimation, applied here to advantage estimation. The following proposition makes the connection with n-step advantages explicit.

Proposition 9.3 (GAE as weighted average of n-step advantages)**.** *The GAE estimate equals a $(1 - \lambda)$-normalised mixture of all n-step advantage estimates:*

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1 - \lambda) \sum_{n=1}^{T-t} \lambda^{n-1} \, \hat{A}_t^{(n)}. \tag{9.9}$$

Proof. We expand $\hat{A}_t^{(n)}$ from (9.7) in terms of TD errors. Note that

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n V_\phi(S_{t+n}) - V_\phi(S_t).$$

Adding and subtracting $\gamma^l V_\phi(S_{t+l})$ for each intermediate step,

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \big(R_{t+l+1} + \gamma V_\phi(S_{t+l+1}) - V_\phi(S_{t+l})\big) = \sum_{l=0}^{n-1} \gamma^l \, \delta_{t+l}.$$

Now form the weighted sum:

$$
\begin{aligned}
(1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\hat{A}_t^{(n)} &= (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\sum_{l=0}^{n-1}\gamma^l\,\delta_{t+l} \\
&= (1-\lambda)\sum_{l=0}^{\infty}(\gamma\lambda)^l\,\delta_{t+l}\underbrace{\sum_{n=l+1}^{\infty}(1-\lambda)\lambda^{n-1-l}}_{=1} \\
&= \sum_{l=0}^{\infty}(\gamma\lambda)^l\,\delta_{t+l},
\end{aligned}
$$

where the interchange of summation is valid for finite-horizon episodes (the inner sum is finite), and the bracketed geometric series sums to 1 because $(1-\lambda)\sum_{k=0}^{\infty}\lambda^k = 1$. Truncating at $l = T-t-1$ gives (9.9). □

9.4.3 The λ Parameter: Bias-Variance Tradeoff

The role of λ is to interpolate between two extremes.

- $\lambda = 0$: Only the first term survives in (9.8), so $\hat{A}_t^{\mathrm{GAE}(\gamma,0)} = \delta_t$. This is the one-step TD error — minimum variance, but biased whenever $V_\phi \neq V^{\pi_\theta}$.
- $\lambda = 1$: The sum extends to the end of the episode and $\hat{A}_t^{\mathrm{GAE}(\gamma,1)} = \sum_{l=0}^{T-t-1}\gamma^l\delta_{t+l} = G_t - V_\phi(S_t)$, the Monte Carlo advantage estimate. This is unbiased (for any V_ϕ) but has maximum variance.

For intermediate λ, the effective look-ahead decays exponentially: terms δ_{t+l} are weighted by $(\gamma\lambda)^l$, so only roughly the first $1/(1-\gamma\lambda)$ steps contribute significantly. Values $\lambda \in [0.90, 0.97]$ with $\gamma \approx 0.99$ are typical in modern implementations, offering a good balance for episodes of moderate length.

Figure 9.2 visualizes how the effective weight on step-l TD errors changes with λ.

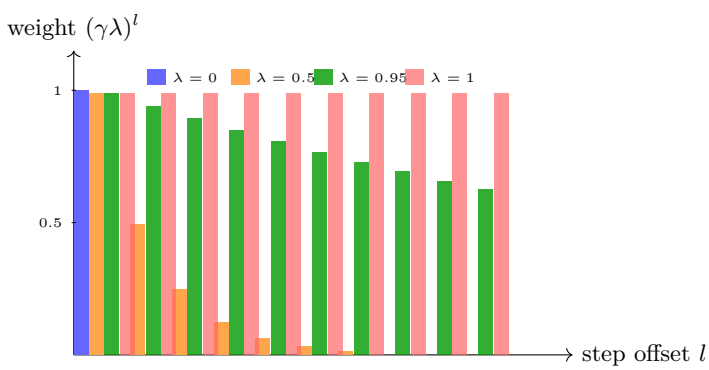


Figure 9.2: Effective weights $(\gamma\lambda)^l$ on step-offset-l TD errors in GAE, for $\gamma = 0.99$ and four values of λ. At $\lambda = 0$ only the immediate TD error contributes; at $\lambda = 1$ all steps receive nearly uniform weight (decaying only by $\gamma^l \approx 1$). Intermediate λ gives a soft exponential cutoff.

9.4.4 The GAE Target for the Critic

GAE also provides a natural target for training the critic. Because $\hat{A}_t^{\text{GAE}} = \hat{V}_t - V_\phi(S_t)$, where

$$\hat{V}_t = \hat{A}_t^{\text{GAE}} + V_\phi(S_t), \tag{9.10}$$

the critic can be trained by minimizing the mean-squared error $\mathbb{E}[(V_\phi(S_t) - \hat{V}_t)^2]$ with $\hat{V}_t$ treated as a fixed target. This creates a consistent pair: the actor uses $\hat{A}_t^{\text{GAE}}$ for the policy gradient, and the critic uses $\hat{V}_t$ as a regression target.

9.5 A2C and A3C

9.5.1 Parallelism as a Variance Reduction Tool

One practical challenge with on-policy actor-critic algorithms is that consecutive transitions $\{(S_t, A_t, R_{t+1}, S_{t+1})\}$ collected by a single agent are highly *correlated*: the agent follows a single Markov chain, so nearby samples share similar states. High correlation inflates the effective variance of the gradient estimate even when the per-sample variance is low.

Mnih et al. (2016) proposed the **Asynchronous Advantage Actor-Critic** (A3C) as a solution. The idea is to run multiple independent *worker* processes, each with its own copy of the environment and its own local copy of the network parameters. Workers generate experience independently and asynchronously push gradient updates to a central parameter server, which aggregates the updates.

Definition 9.4 (A3C architecture)**.** In A3C, K worker processes run in parallel. Each worker k:

1. Copies global parameters (θ, ϕ) to local copies (θ_k, ϕ_k).
2. Collects a short trajectory of $t_{\max}$ transitions using π_{θ_k}.
3. Computes actor and critic gradients using the advantage estimate $\hat{A}_t$ from (9.7).
4. Asynchronously applies gradients to the global (θ, ϕ).

Because different workers typically find themselves in different states and explore different parts of the state space simultaneously, their gradients are nearly uncorrelated even without experience replay. This decorrelation is the key benefit of asynchronous parallelism.

9.5.2 A2C: the Synchronous Variant

Advantage Actor-Critic (A2C) is the synchronous version of A3C. All K workers collect a batch of transitions simultaneously, and the parameter update is performed only after all workers have reported. Because updates are synchronous, there is no inconsistency between the parameters used to generate data and those being updated.

In practice A2C achieves performance comparable to A3C and is easier to implement reproducibly because the synchronous design eliminates race conditions. Modern deep RL pipelines almost universally use A2C-style synchronous batching as the backbone, later enhanced with the trust-region constraints developed in the next section.

Algorithm 21: Advantage Actor-Critic (A2C)

Input: K parallel workers, rollout length T_{roll}, discount γ, GAE parameter λ
Initialize global θ, ϕ
repeat
  for *each worker* $k = 1, \ldots, K$ in parallel **do**
    Run π_θ for T_{roll} steps; collect $\{(S_t^k, A_t^k, R_{t+1}^k)\}_{t=0}^{T_{\text{roll}}-1}$
    Compute $\hat{A}_t^k$ via GAE (9.8)
    Compute $\hat{V}_t^k$ via (9.10)
  Aggregate gradients across all workers
  Update θ using actor gradient $\frac{1}{KT_{\text{roll}}} \sum_k \sum_t \hat{A}_t^k \nabla_\theta \log \pi_\theta(A_t^k | S_t^k)$
  Update ϕ by minimizing $\frac{1}{KT_{\text{roll}}} \sum_k \sum_t (V_\phi(S_t^k) - \hat{V}_t^k)^2$
until *convergence*

9.6 Trust Region Methods

9.6.1 Off-Policy Reuse via Importance Sampling

On-policy methods like A2C collect a batch of transitions and then discard them immediately after a single gradient update. This is wasteful: each environment interaction is expensive (in simulation time, computation, or real-world cost), but the on-policy constraint forces every sample to be thrown away once the parameters change.

A natural remedy is importance sampling (introduced in Chapter 5 in the context of off-policy Monte Carlo). Suppose data was collected using an *old* policy $\pi_{\theta_{\text{old}}}$ and we wish to evaluate or optimize the *new* policy π_θ. Define the per-step **importance ratio**:

$$r_t(\theta) = \frac{\pi_\theta(A_t \mid S_t)}{\pi_{\theta_{\text{old}}}(A_t \mid S_t)}. \tag{9.11}$$

By standard importance sampling, the policy gradient $\mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(A|S)\, A^\pi(S, A)]$ can be rewritten, under the approximation that the state distribution induced by π_θ is close to that of $\pi_{\theta_{\text{old}}}$, as

$$L(\theta) = \mathbb{E}_{\pi_{\theta_{\text{old}}}}\big[r_t(\theta)\, \hat{A}_t\big]. \tag{9.12}$$

The function $L(\theta)$ is called the **surrogate objective**. It can be optimized using data from $\pi_{\theta_{\text{old}}}$, which enables multiple gradient steps on the same batch.

Proposition 9.4 (Surrogate gradient equals policy gradient at θ_{old})**.**

$$\nabla_\theta L(\theta)\big|_{\theta=\theta_{\text{old}}} = \nabla_\theta J(\theta)\big|_{\theta=\theta_{\text{old}}}. \tag{9.13}$$

Proof. At $\theta = \theta_{\text{old}}$, $r_t(\theta) = 1$ for all transitions. Differentiating $L(\theta) = \mathbb{E}[r_t(\theta)\, \hat{A}_t]$ gives

$$\nabla_\theta L(\theta)\big|_{\theta_{\text{old}}} = \mathbb{E}\big[\nabla_\theta r_t(\theta)\big|_{\theta_{\text{old}}} \cdot \hat{A}_t\big].$$

Since $\nabla_\theta r_t(\theta) = \frac{\nabla_\theta \pi_\theta(A_t|S_t)}{\pi_{\theta_{\text{old}}}(A_t|S_t)}$, at $\theta = \theta_{\text{old}}$ this equals $\nabla_\theta \log \pi_\theta(A_t|S_t)|_{\theta_{\text{old}}}$, giving the policy gradient. □

9.6.2 The Danger of Large Updates

Proposition 9.4 shows that the surrogate objective is locally correct, but what happens when θ moves far from θ_{old}?

Proposition 9.5 (Monotone improvement bound)**.** *There exists a constant $C > 0$ depending on γ and $\max_{s,a} |A^{\pi_\theta}(s,a)|$ such that*

$$J(\theta) \geq L(\theta) - C \cdot \max_s D_{\text{KL}}(\pi_\theta(\cdot|s) \parallel \pi_{\theta_{\text{old}}}(\cdot|s)) \,. \tag{9.14}$$

The bound (9.14), due to Schulman et al. (2015), says that $J(\theta)$ is lower-bounded by $L(\theta)$ minus a term proportional to the KL divergence of the new policy from the old one. If the KL is small, the surrogate $L(\theta)$ is a reliable approximation to $J(\theta)$, so maximizing L guarantees improvement in J. If the KL is large, the bound is uninformative: the surrogate could rise while the true objective falls.

This is the fundamental motivation for the trust region approach: we should only move θ within a region where the KL divergence from θ_{old} is small enough that the surrogate is trustworthy.

9.6.3 Trust Region Policy Optimization (TRPO)

Trust Region Policy Optimization (TRPO), introduced by Schulman et al. (2015), solves the constrained optimization problem:

$$\max_\theta \; L(\theta) \quad \text{subject to} \quad \max_s \; D_{\text{KL}}(\pi_\theta(\cdot|s) \parallel \pi_{\theta_{\text{old}}}(\cdot|s)) \leq \delta, \tag{9.15}$$

where $\delta > 0$ is a hyperparameter controlling the size of the trust region. In practice the maximum over states is replaced by an empirical average.

Solving (9.15) requires computing the Fisher information matrix (the Hessian of the KL constraint) and solving a constrained quadratic programme. This is done approximately via conjugate gradient with a line search, but the procedure is computationally heavy: it requires multiple Hessian-vector products per update, scaling poorly to networks with millions of parameters. TRPO is theoretically well-motivated but rarely used in practice for large models.

PPO, developed next, provides a much simpler approximation to the same idea.

9.7 Proximal Policy Optimization

9.7.1 From Trust Regions to Clipping

Proximal Policy Optimization (PPO), introduced by Schulman et al. (2017), replaces the explicit KL constraint of TRPO with a simple clipping operation on the importance ratio. The intuition is that $r_t(\theta) = 1$ when $\theta = \theta_{\text{old}}$, and that allowing $r_t(\theta)$ to deviate far from 1 corresponds to a large policy change. PPO directly prevents large deviations by clipping r_t to the interval $[1-\varepsilon, 1+\varepsilon]$ for a hyperparameter $\varepsilon > 0$ (typically $\varepsilon \in [0.1, 0.3]$).

Definition 9.5 (PPO clipped objective)**.** The **PPO clipped surrogate objective** is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\; \text{clip}\big(r_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon\big)\,\hat{A}_t\Big)\Big]\,, \tag{9.16}$$

where $\text{clip}(x, a, b) = \max(a, \min(b, x))$.

9.7.2 Case Analysis of the Clipping Mechanism

Theorem 9.6 (Correctness of PPO clipping)**.** *For each transition, the clipped objective L^{CLIP} has the following properties:*

(i) If $\hat{A}_t > 0$: L^{CLIP} is non-decreasing in $r_t(\theta)$ for $r_t \leq 1+\varepsilon$, and constant for $r_t > 1+\varepsilon$. The gradient is zero when $r_t > 1+\varepsilon$.

(ii) If $\hat{A}_t < 0$: L^{CLIP} is non-increasing in $r_t(\theta)$ for $r_t \geq 1-\varepsilon$, and constant for $r_t < 1-\varepsilon$. The gradient is zero when $r_t < 1-\varepsilon$.

(iii) At $\theta = \theta_{\text{old}}$ (so $r_t = 1 \in [1-\varepsilon, 1+\varepsilon]$), the gradient of L^{CLIP} equals the policy gradient.

Proof. Fix a transition and write $r = r_t(\theta)$, $A = \hat{A}_t$. Let $f_1(r) = rA$ and $f_2(r) = \text{clip}(r, 1-\varepsilon, 1+\varepsilon) \cdot A$.

Case (i): $A > 0$. Then f_1 is strictly increasing in r. For $r \leq 1+\varepsilon$, the clip is inactive: $f_2(r) = rA = f_1(r)$. For $r > 1+\varepsilon$, $f_2(r) = (1+\varepsilon)A$, a constant. So

$$\min(f_1, f_2) = \begin{cases} rA & r \leq 1+\varepsilon, \\ (1+\varepsilon)A & r > 1+\varepsilon. \end{cases}$$

This is non-decreasing and its derivative with respect to r is zero for $r > 1+\varepsilon$. No gradient incentive exists to increase the probability of a good action beyond the $(1+\varepsilon)$ cap.

Case (ii): $A < 0$. Then f_1 is strictly decreasing. For $r \geq 1-\varepsilon$, $f_2(r) = rA = f_1(r)$. For $r < 1-\varepsilon$, $f_2(r) = (1-\varepsilon)A$, a constant, and $f_1(r) < f_2(r)$ since $A < 0$. So $\min(f_1, f_2) = f_1(r) = rA$ for $r < 1-\varepsilon$ (where $f_1 < f_2$) and $\min(f_1, f_2) = rA = f_2(r)$ for $r \in [1-\varepsilon, \infty)$. Concretely,

$$\min(f_1, f_2) = \begin{cases} rA & r < 1-\varepsilon, \\ rA & r \geq 1-\varepsilon. \end{cases}$$

We now proceed more carefully. For $A < 0$ and $r < 1-\varepsilon$: $f_2(r) = (1-\varepsilon)A$ (constant, clipped from below), and $f_1(r) = rA > (1-\varepsilon)A$ since $r < 1-\varepsilon$ and $A < 0$. So $f_1 > f_2$, thus $\min(f_1, f_2) = f_2 = (1-\varepsilon)A$, constant. For $r \geq 1-\varepsilon$: clip is inactive, $f_2 = rA = f_1$, so $\min = rA$. Hence

$$\min(f_1, f_2) = \begin{cases} (1-\varepsilon)A & r < 1-\varepsilon, \\ rA & r \geq 1-\varepsilon. \end{cases}$$

The gradient with respect to r is zero for $r < 1-\varepsilon$: there is no incentive to aggressively reduce the probability of a bad action beyond the $(1-\varepsilon)$ floor.

Case (iii): At $r = 1 \in (1-\varepsilon, 1+\varepsilon)$ the clip is inactive in both cases, so $L^{\text{CLIP}} = \mathbb{E}[r_t(\theta)\hat{A}_t] = L(\theta)$ in a neighbourhood of θ_{old}. By Proposition 9.4, $\nabla_\theta L^{\text{CLIP}}|_{\theta_{\text{old}}} = \nabla_\theta J(\theta)|_{\theta_{\text{old}}}$. □

The min operator in Definition 9.5 is crucial: it makes L^{CLIP} a *pessimistic lower bound* on the surrogate $r_t(\theta)\hat{A}_t$. Figure 9.3 illustrates the two cases.

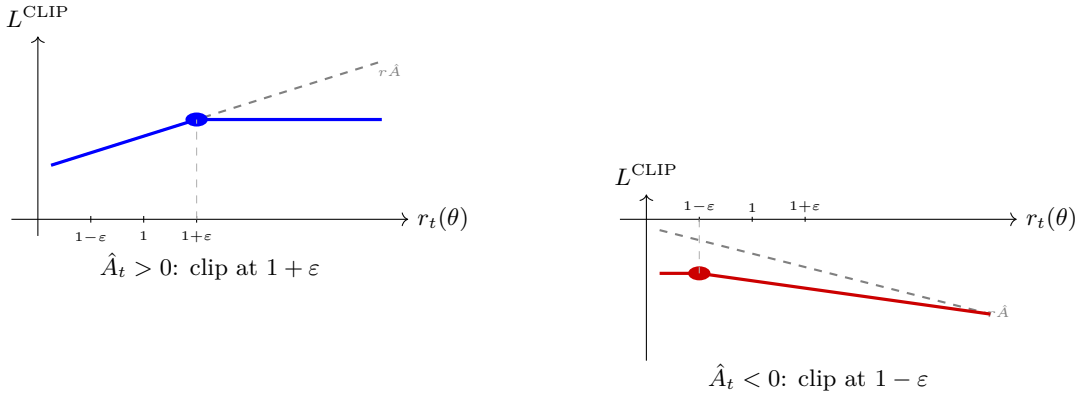


Figure 9.3: The PPO clipping mechanism. *Left*: when $\hat{A}_t > 0$, the objective is capped at $r_t = 1 + \varepsilon$ (blue), preventing excessive increases in action probability. *Right*: when $\hat{A}_t < 0$, the objective is floored at $r_t = 1 - \varepsilon$ (red), preventing excessive decreases. In both cases the min operator makes L^{CLIP} a pessimistic lower bound on the unconstrained surrogate (dashed grey).

9.7.3 Full PPO Objective

In practice PPO optimizes a composite objective that combines the clipped actor loss with a critic regression loss and an entropy bonus:

$$\mathcal{L}(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_v \, \mathbb{E}\big[(V_\phi(S_t) - \hat{V}_t)^2\big] - c_H \, \mathbb{E}\big[-\log \pi_\theta(A_t \mid S_t)\big], \tag{9.17}$$

where $\hat{V}_t$ is the GAE target (9.10), $c_v > 0$ weights the value loss, and $c_H > 0$ weights the entropy bonus $H(\pi_\theta(\cdot|S_t))$. The three terms serve distinct purposes:

$-L^{\text{CLIP}}$	Maximizes the policy objective with a conservative trust-region constraint.
$c_v \, L^{\text{VF}}$	Trains the critic to predict V^{π_θ}, which is needed for accurate advantage estimation.
$-c_H \, H$	Adds an entropy bonus to discourage the policy from collapsing prematurely to a near-deterministic distribution, maintaining exploration throughout training.

When actor and critic share network layers, the entire objective (9.17) is differentiated jointly.

9.7.4 The PPO Algorithm

Several design choices in Algorithm 22 deserve comment.

Remark 9.2 (Advantage normalization). Subtracting the batch mean and dividing by the batch standard deviation (step "Normalize advantages") is a standard engineering practice. It keeps the magnitude of the gradient roughly constant across episodes regardless of the reward scale, which reduces sensitivity to the learning rate and stabilizes training. It introduces a mild bias because the normalization changes the relative weighting of transitions, but this is empirically beneficial.

Algorithm 22: Proximal Policy Optimization (PPO)

Input: Rollout length T, workers K, epochs K_e, minibatch size M, clip ε, λ for GAE

Initialize θ, ϕ
repeat
  `// Data collection`
  for *each of K workers in parallel* **do**
    Run π_θ for T steps; store $\{S_t, A_t, R_{t+1}, S_{t+1}\}$
    Compute $\hat{A}_t^{\text{GAE}}$ via (9.8) and $\hat{V}_t$ via (9.10)
  Normalize advantages: $\hat{A}_t \leftarrow (\hat{A}_t - \bar{A})/\hat{\sigma}_A$ across all workers
  Fix $\theta_{\text{old}} \leftarrow \theta$
  `// Multiple optimization epochs on collected data`
  for *epoch* $= 1, \ldots, K_e$ **do**
    for *each minibatch $\mathcal{B}$ of size M* **do**
      Compute $r_t(\theta) = \pi_\theta(A_t|S_t)/\pi_{\theta_{\text{old}}}(A_t|S_t)$ for each $(S_t, A_t) \in \mathcal{B}$
      Compute $\mathcal{L}(\theta, \phi)$ via (9.17)
      Update θ, ϕ by one step of Adam
until *convergence*

Remark 9.3 (Multiple epochs). PPO reuses each batch for K_e gradient steps. This substantially improves sample efficiency compared with pure on-policy methods (where data is discarded after one gradient step), while the clipping ensures that the policy does not drift too far from the data-collection policy within a single batch. Typical values are $K_e \in [3, 10]$.

Example 9.3 (PPO on a simple MDP)**.** Consider a three-state chain MDP with states $\{s_1, s_2, s_3\}$ where s_3 is terminal. The agent has two actions (left/right). Suppose at the current iteration:

- $\hat{A}_t = 0.8$ (the taken action was significantly above average).
- $r_t(\theta) = 1.35$ after a gradient step (the new policy assigns 35% more probability to this action than the old policy did).
- $\varepsilon = 0.2$.

Without clipping, the contribution to L would be $1.35 \times 0.8 = 1.08$. With clipping, r_t is capped at $1 + \varepsilon = 1.2$, giving $1.2 \times 0.8 = 0.96$. Since $0.96 < 1.08$, the min selects 0.96. The gradient of L^{CLIP} with respect to θ is zero for this sample because $r_t > 1 + \varepsilon$: the policy has already moved far enough in the direction of increasing this action's probability, and PPO provides no additional gradient to push it further. □

9.8 PPO for Language Models

9.8.1 Language Models as RL Policies

The application of PPO to the fine-tuning of large language models (LLMs) via human feedback — commonly called RLHF—has become one of the most practically important uses

of reinforcement learning. Chapter 11 studies language models as RL agents in detail; here we focus specifically on the PPO component of the RLHF pipeline.

A language model π_θ defines an autoregressive policy over tokens. Given a prompt $x = (x_1, \ldots, x_m)$, the model generates a response $y = (y_1, \ldots, y_T)$ one token at a time:

$$\pi_\theta(y \mid x) = \prod_{t=1}^{T} \pi_\theta(y_t \mid x,\, y_1, \ldots, y_{t-1}). \tag{9.18}$$

This maps to an MDP where the state at step t is $S_t = (x, y_1, \ldots, y_{t-1})$, the action is the next token $A_t = y_t \in \mathcal{V}$ (the vocabulary), and the episode terminates when an end-of-sequence token is generated. The action space has size $|\mathcal{V}| \in [32{,}000,\ 128{,}000]$ — far larger than classical RL benchmarks — making value-based methods impractical and policy-based methods like PPO natural.

9.8.2 Reward Modeling and the KL Penalty

In RLHF, the reward signal comes from a separately trained **reward model** $r_\psi(x, y)$ that was fitted to human preference comparisons. In the simplest setting, a scalar reward is emitted only at the end of the episode: $R_t = 0$ for $t < T$ and $R_T = r_\psi(x, y)$.

Without further regularization, however, the policy rapidly learns to exploit the reward model — a phenomenon called **reward hacking**. The policy may produce outputs that score highly under r_ψ but diverge from coherent, safe, and factually accurate language. The standard remedy is a per-token KL-divergence penalty relative to the *reference policy* π_{ref} (typically the supervised fine-tuned model before RL training):

$$\tilde{R}_t(x, y) = \begin{cases} r_\psi(x, y) - \beta \log \dfrac{\pi_\theta(y_T \mid x, y_{<T})}{\pi_{\text{ref}}(y_T \mid x, y_{<T})}, & t = T, \\ -\beta \log \dfrac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}, & t < T, \end{cases} \tag{9.19}$$

where $\beta > 0$ controls the strength of the KL penalty.

Proposition 9.7 (RLHF total reward)**.** *The sum of adjusted per-step rewards satisfies*

$$\sum_{t=1}^{T} \tilde{R}_t = r_\psi(x, y) - \beta\, D_{\text{KL}}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))\,, \tag{9.20}$$

where the KL divergence is computed over the joint sequence distribution.

Proof. Summing (9.19) over $t = 1, \ldots, T$,

$$\sum_{t=1}^{T} \tilde{R}_t = r_\psi(x, y) - \beta \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}.$$

The KL divergence between two autoregressive distributions $\pi_\theta(\cdot|x)$ and $\pi_{\text{ref}}(\cdot|x)$ factors as

$$D_{\text{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)) = \mathbb{E}_{y \sim \pi_\theta}\left[\sum_{t=1}^{T} \log \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\text{ref}}(y_t|x, y_{<t})}\right],$$

so the per-sample sum $\sum_t \log[\pi_\theta/\pi_{\text{ref}}]$ is an unbiased Monte Carlo estimate of the KL. Substituting gives (9.20). □

Proposition 9.7 reveals the objective that RLHF implicitly optimizes: maximize reward model score minus a KL-weighted penalty for diverging from the reference policy. As $\beta \to 0$, the penalty vanishes and the policy can drift arbitrarily far from π_{ref}, risking reward hacking. As $\beta \to \infty$, the optimal policy is π_{ref} itself and no reinforcement learning occurs.

9.8.3 The Four-Model RLHF Architecture

A standard PPO-RLHF training run maintains four neural network models simultaneously:

Actor π_θ — The policy being trained. Its parameters are updated at every PPO iteration.

Critic V_ϕ — A value head — typically a single linear layer applied to the final hidden state of the last token — that estimates the expected cumulative adjusted reward from state S_t.

Reference model π_{ref} — A frozen copy of the supervised fine-tuned model, used only to compute the KL penalty in (9.19).

Reward model r_ψ — A frozen model (usually the same architecture as the actor, with an added scalar head) trained on human preference data to output a scalar quality score.

The advantages $\hat{A}_t$ are computed via GAE applied to the adjusted rewards $\tilde{R}_t$ from (9.19), and the PPO clipped objective (9.16) is applied with the vocabulary-level importance ratios $r_t(\theta) = \pi_\theta(y_t|x, y_{<t})/\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})$.

Example 9.4 (PPO-RLHF on a question-answering task)**.** Suppose the prompt is "Explain Fermat's Last Theorem." and the policy generates a 120-token response y. The reward model scores the full response: $r_\psi(x, y) = 0.78$. At each token step $t < 120$, the per-token log-ratio $\log[\pi_\theta(y_t|x, y_{<t})/\pi_{\text{ref}}(y_t|x, y_{<t})]$ is computed; if the policy has drifted modestly from the reference, each term might be around 0.01, contributing a cumulative KL penalty of $\beta \times 0.01 \times 120 = 1.2\beta$. With $\beta = 0.05$, the total adjusted reward is $0.78 - 0.06 = 0.72$.

The critic $V_\phi(S_t)$ predicts the expected total adjusted reward from the partial sequence seen so far. After each token is generated, $\delta_t = \tilde{R}_t + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$ is computed, and GAE aggregates these errors into per-token advantage estimates. A token such as "Wiles" (as in Andrew Wiles) that is both relevant and specific may push the critic's prediction upward, yielding a positive advantage that increases the probability of this token in future PPO updates. □

9.8.4 Practical Considerations

The following guidelines distill empirical best practices from the RLHF literature (Ouyang et al. 2022, Bai et al. 2022, Ziegler et al. 2019).

KL monitoring and control. Track the running average of $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ across prompt batches. A rapid increase signals that the policy is diverging from the reference model. If the KL exceeds a preset threshold (e.g. 10 nats), either increase β, reduce the learning rate, or halt training and diagnose the reward model.

Advantage normalization. Normalize $\hat{A}_t$ to zero mean and unit variance across the minibatch before computing the clipped objective. For language generation, episodes (responses) have varying lengths; normalization prevents long sequences from dominating gradient updates purely because they accumulate more per-step terms.

Critic initialization. Initialize the value head on top of the same pre-trained backbone as the actor. Starting from a randomly initialized critic produces wildly inaccurate advantage estimates in early training, destabilizing the actor before it has learned anything useful.

Epoch count per batch. Use $K_e \in [1, 4]$ optimization epochs per data batch for RLHF, versus $K_e \in [4, 10]$ for classical RL environments. Because the reward model can be exploited (unlike a true environment), too many epochs on the same batch encourage the actor to overfit to the reward model signal, accelerating reward hacking.

Reward hacking detection. Periodically evaluate the trained policy on held-out prompts judged by humans (or by a separate, independent evaluator). If the reward model score continues rising while human judgments plateau or decline, reward hacking is occurring. Mitigations include refreshing the reward model with additional preference data, increasing β, or incorporating a supervised fine-tuning loss alongside the PPO objective.

9.9 Convergence and Practical Considerations

9.9.1 Convergence Properties

The convergence theory for actor-critic methods is substantially more complex than for pure TD or pure policy gradient methods, because both components are being updated simultaneously and their objectives interact.

For the *linear* function approximation case with a fixed policy, the TD(0) critic converges to the solution of the projected Bellman equation, as established in Chapter 6. When the actor is also updated, convergence requires **two-timescale stochastic approximation**: the critic's step size α_ϕ must decrease faster than the actor's α_θ, ensuring the critic tracks V^{π_θ} faster than π_θ changes.

Theorem 9.8 (Two-timescale convergence, sketch)**.** *Under the following conditions:*

(i) linear function approximation for the critic,

(ii) Robbins-Monro step sizes satisfying $\sum_t \alpha_\phi(t) = \infty$, $\sum_t \alpha_\phi(t)^2 < \infty$, *and* $\alpha_\theta(t)/\alpha_\phi(t) \to 0$,

(iii) the policy gradient ODE has a unique globally attracting equilibrium (or, more generally, the iterates remain bounded),

the actor-critic algorithm converges almost surely to a stationary point of the objective $J(\theta)$.

For *nonlinear* function approximation (neural networks), no comparably clean theory exists. In practice, the actor and critic are updated with the same optimizer (e.g. Adam) and the same or similar learning rates, which violates the two-timescale requirement. Yet PPO is empirically stable across a wide range of tasks, suggesting that the neural networks act as their own effective regularizers.

9.9.2 Comparison of Policy Gradient Algorithms

Table 9.1 summarizes the key properties of the policy gradient family discussed in Chapters 8 and 9.

Table 9.1: Comparison of policy gradient algorithms. "On-policy" means each data sample is used for at most one gradient update.

Algorithm	Baseline	Online?	Multiple epochs?	Trust region?
REINFORCE	None / constant	No	No	No
REINFORCE + baseline	$V_\phi(s)$	No	No	No
One-step AC	TD(0) critic	Yes	No	No
A2C / A3C	GAE critic	Yes	No	No
TRPO	GAE critic	Yes	No	KL constraint
PPO	GAE critic	Yes	Yes	Clip ε

9.9.3 Hyperparameter Sensitivity and Implementation Tips

PPO is notably robust to hyperparameter choices relative to TRPO, which requires careful tuning of the conjugate gradient step count and line-search parameters. Nevertheless, the following guidelines are helpful in practice.

Clip parameter ε Typical values are 0.1 to 0.3. Smaller ε gives more conservative updates and is safer for fine-tuning large pretrained models. $\varepsilon = 0.2$ is the most common default.

GAE λ Values 0.90–0.97 work well across most tasks. Longer episodes warrant larger λ (to capture long-range credit).

Value coefficient c_v Typically 0.5; can be reduced if the value network is separately optimized or if gradient magnitudes are already well-matched.

Entropy coefficient c_H Start with 0.01; increase if the policy collapses prematurely to a near-deterministic distribution.

Epochs per batch K_e Values 4–8 for continuous control; 1–4 for RLHF. Monitor the KL between π_θ and $\pi_{\theta_{\text{old}}}$ and stop epochs early if KL exceeds $\sim 1.5\varepsilon$.

Batch and minibatch size Larger batches reduce gradient noise but require more memory. A rollout of 2048 steps with minibatch size 64 is a common starting point for continuous control.

Learning rate 3×10^{-4} with Adam is a robust default. Linear or cosine decay schedules often help for fine-tuning.

Observation normalization Normalize observations to zero mean and unit variance (with running statistics) and clip rewards to a fixed range (e.g. $[-5, 5]$) to prevent the critic from being overwhelmed by outlier rewards.

Exercises

Exercise 9.1. Consider an MDP with three actions available in state s. Suppose $Q^\pi(s, a_1) = 5$, $Q^\pi(s, a_2) = 8$, $Q^\pi(s, a_3) = 2$, and the policy assigns probabilities $\pi(a_1|s) = 0.5$, $\pi(a_2|s) = 0.3$, $\pi(a_3|s) = 0.2$.

(a) Compute $V^\pi(s)$ and the advantage $A^\pi(s, a_i)$ for each action.

(b) Verify that $\mathbb{E}_{A\sim\pi(\cdot|s)}[A^\pi(s, A)] = 0$.

(c) A gradient step increases $\pi(a_2|s)$. Explain why using $A^\pi(s, a_2) = 2$ (rather than $Q^\pi(s, a_2) = 8$) as the gradient weight leads to a smaller but more informative update.

Exercise 9.2. A critic with $V_\phi = V^{\pi_\theta}$ observes a transition $(S_t, A_t, R_{t+1}, S_{t+1})$ with $R_{t+1} = 2$, $\gamma = 0.9$, $V^\pi(S_t) = 3$, $V^\pi(S_{t+1}) = 4$.

(a) Compute the TD error δ_t.

(b) Is the action A_t above or below average in state S_t? Justify using the sign of δ_t.

(c) After one actor update with step size $\alpha_\theta = 0.01$ and $\gamma^t = 1$, by how much does $\log \pi_\theta(A_t|S_t)$ change?

Exercise 9.3. Let $T = 4$ (steps $t = 0, 1, 2, 3$, terminal at $t = 4$) with TD errors $\delta_0 = 0.4$, $\delta_1 = -0.2$, $\delta_2 = 0.6$, $\delta_3 = 0.1$. Take $\gamma = 1$ and $\lambda = 0.8$.

(a) Compute $\hat{A}_0^{\text{GAE}}$ using Definition 9.3.

(b) Verify that your answer matches the formula from Proposition 9.3 by computing the weighted mixture of n-step advantages $\hat{A}_0^{(n)}$ for $n = 1, 2, 3, 4$.

(c) Compute $\hat{A}_0^{\text{GAE}}$ for $\lambda = 0$ and $\lambda = 1$ and interpret the results.

Exercise 9.4. Consider the PPO clipped objective with $\varepsilon = 0.2$.

(a) Compute L^{CLIP} contributions for the following three transitions and indicate whether the clip is active:

  (i) $\hat{A}_t = 1.5$, $r_t(\theta) = 0.8$.

  (ii) $\hat{A}_t = 1.5$, $r_t(\theta) = 1.3$.

  (iii) $\hat{A}_t = -0.5$, $r_t(\theta) = 0.75$.

(b) For case (ii) above, explain why the gradient of L^{CLIP} with respect to θ is zero even though $\hat{A}_t > 0$ and the action should be encouraged. Why is this conservative behaviour desirable?

Exercise 9.5. This exercise works through the surrogate gradient (Proposition 9.4) in detail.

(a) Show that $r_t(\theta)|_{\theta=\theta_{\text{old}}} = 1$ for all transitions.

(b) Write $\nabla_\theta r_t(\theta)$ in terms of the policy and show it equals $r_t(\theta)\, \nabla_\theta \log \pi_\theta(A_t|S_t)$.

(c) Conclude that $\nabla_\theta L(\theta)|_{\theta_{\text{old}}} = \mathbb{E}[\nabla_\theta \log \pi_{\theta_{\text{old}}}(A_t|S_t)\, \hat{A}_t]$ and explain why this is exactly the policy gradient.

Exercise 9.6. In the RLHF adjusted reward (9.19), set $\gamma = 1$.

(a) Prove Proposition 9.7 from first principles, using only the chain rule of KL divergence for autoregressive models.

(b) Show that the optimal policy for the objective $\max_\pi \mathbb{E}[r_\psi(x,y)] - \beta\, D_{\mathrm{KL}}(\pi(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x))$ is $\pi^*(y|x) \propto \pi_{\mathrm{ref}}(y|x)\, e^{r_\psi(x,y)/\beta}$.

(c) Interpret the optimal policy: what happens as $\beta \to 0$? As $\beta \to \infty$? What does this imply for the choice of β in practice?

Exercise 9.7. Fill in the following comparison table for three algorithms: REINFORCE with baseline, one-step actor-critic, and PPO. For each row, answer "yes" / "no" or give a brief phrase.

Property	REINFORCE+BL	One-step AC	PPO
Requires full episode?			
Data reuse across updates?			
Advantage estimator			
Online (step-by-step) updates?			
Trust-region constraint?			

Exercise 9.8. Explain, using the decomposition in Proposition 9.3, why GAE with $\lambda < 1$ has lower variance than the Monte Carlo advantage estimate $G_t - V_\phi(S_t)$, but introduces bias.

(a) Identify the source of variance in the Monte Carlo estimate (Hint: count the number of independent random variables in G_t).

(b) Identify the source of bias in the GAE estimate for $\lambda < 1$ (Hint: consider what happens when $V_\phi \neq V^\pi$).

(c) State informally why $\lambda \approx 0.95$ is a sensible default for episodes of length ~ 1000.

Exercise 9.9. The two-timescale convergence result (Theorem 9.8) requires $\alpha_\theta(t)/\alpha_\phi(t) \to 0$.

(a) Explain intuitively why the critic must learn faster than the actor. What goes wrong if the actor updates are as fast as the critic's?

(b) Propose a concrete pair of step-size sequences $\alpha_\phi(t) = 1/(t+1)$ and $\alpha_\theta(t)$ satisfying the two-timescale condition.

(c) In modern PPO implementations with Adam, both actor and critic use the same optimizer and learning rate. What practical mechanism replaces the two-timescale requirement?

Exercise 9.10 (Challenge)**.** This exercise establishes a precise connection between TRPO and PPO.

(a) Show that the TRPO constraint $D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\theta_{\mathrm{old}}}) \le \delta$ implies $|r_t(\theta) - 1| \le \sqrt{2\delta}$ for any single action-state pair, using Pinsker's inequality $\|\pi - q\|_1 \le \sqrt{2\, D_{\mathrm{KL}}(\pi \,\|\, q)}$.

(b) Using part (a), argue that the PPO clip at ε is a first-order approximation to the TRPO KL constraint with $\delta \approx \varepsilon^2/2$.

(c) Discuss one advantage and one disadvantage of PPO's approximation relative to exact TRPO.

Part III

RL for Language Models

Chapter 10

Reward Shaping and Reward Models

Garbage in, garbage out. The quality of a learned policy can never exceed the quality of the reward signal it optimises.

Every reinforcement learning algorithm reduces, at bottom, to the same imperative: maximise the cumulative reward. This makes the reward signal the single most consequential design choice in any RL system. A reward that is misspecified, — even subtly, — will produce a policy that is, at best, suboptimal and, at worst, actively harmful. The history of applied RL is littered with cautionary tales: a simulated robot that learned to move by vibrating instead of walking, a game-playing agent that discovered a high-scoring exploit the designers never intended, and — in the modern era of language model alignment — chatbots that learned to produce verbose, sycophantic prose because a proxy reward model happened to prefer it.

This chapter attacks the reward design problem from two complementary angles. The first half develops the classical theory of *reward shaping*, which asks: given an environment with a well-defined but sparse or otherwise inconvenient true reward, how can we augment that reward with additional terms that speed up learning without altering the optimal policy? The cornerstone result is the potential-based shaping theorem of Ng, Harada, and Russell (1999), which provides a precise characterisation of all reward augmentations that are guaranteed to be policy-preserving. The second half turns to *learned reward models*: what to do when the true reward is not even known in closed form, and must instead be inferred from human judgements, outcome labels, or step-level annotations. We derive the Bradley–Terry preference model, the cross-entropy loss for training preference reward models, and then treat the distinction between outcome reward models (ORM) and process reward models (PRM) in depth. We close with reward overoptimisation, — Goodhart's Law in RL, — and the KL-penalty mechanism that guards against it.

This chapter occupies a pivotal position in the book. It bridges Part II (classical RL methods, Chapters 4–9) and Part III (RL for language models). The reward models trained here feed directly into the PPO fine-tuning pipeline (Chapter 14), the DPO objective, and the GRPO algorithm described in later chapters. Readers who have mastered the TD and policy-gradient machinery of Part II will find the reward-modelling material in the second half of this chapter to be a natural extension, cast in the language of large-scale language models.

10.1 Potential-Based Reward Shaping

10.1.1 The Sparse-Reward Problem

Many environments present the agent with a reward signal that is sparse: positive feedback arrives only when a specific goal is reached, and zero reward is received for every other transition. Consider a robot navigating a large maze: the true reward is $+1$ at the exit and 0 everywhere else. An agent exploring at random will find the exit only after an astronomically large number of steps, and until it does, it receives no gradient information at all. Similarly, a language model receiving reward only at the end of a long response must credit each of the hundreds of preceding tokens with a share of that terminal signal — the credit-assignment problem magnified to an extreme degree.

The intuitive fix is to *shape* the reward: add auxiliary terms that guide the agent toward the goal. A maze-navigating robot might receive a small bonus proportional to how much it reduced its Euclidean distance to the exit. A language model might receive intermediate rewards for producing grammatically correct sentences. However, reward shaping is dangerous. Naively added bonus terms can change the optimal policy: a robot that receives a large bonus for approaching a wall (which happens to be near the exit) may learn to hover near that wall rather than reach the goal. The fundamental question is therefore: *which reward augmentations are guaranteed not to change the set of optimal policies?*

10.1.2 The Shaping Theorem

Ng, Harada, and Russell (1999) answered this question precisely. Their result, known as the **potential-based shaping theorem**, characterises all reward transformations that are policy-invariant.

Definition 10.1 (Potential-based shaping function)**.** Let $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ be a discounted MDP. A **potential-based shaping function** is a function $F : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ of the form

$$F(s, a, s') = \gamma\, \Phi(s') - \Phi(s), \tag{10.1}$$

where $\Phi : \mathcal{S} \to \mathbb{R}$ is an arbitrary **potential function**.

Given a shaped reward $\tilde{R}(s, a, s') = R(s, a, s') + F(s, a, s')$, define the shaped MDP $\tilde{\mathcal{M}} = (\mathcal{S}, \mathcal{A}, P, \tilde{R}, \gamma)$. The central theorem is the following.

Theorem 10.1 (Potential-based shaping theorem, Ng et al. 1999)**.** *Let F be a potential-based shaping function as in (10.1). Then:*

1. *Every optimal policy for $\mathcal{M}$ is also optimal for $\tilde{\mathcal{M}}$, and vice versa.*
2. *The shaped value function $\tilde{V}^\pi$ and shaped action-value function $\tilde{Q}^\pi$ satisfy*

$$\tilde{V}^\pi(s) = V^\pi(s) - \Phi(s), \qquad \tilde{Q}^\pi(s, a) = Q^\pi(s, a) - \Phi(s). \tag{10.2}$$

3. *$F(s, a, s') = \gamma\Phi(s') - \Phi(s)$ is the* only *family of functions (up to a constant) such that optimal policies are preserved for all MDPs.*

Proof. We prove parts 1 and 2. The n-step return under the shaped reward, starting from state s_0, is

$$\begin{aligned}\tilde{G}_0 &= \sum_{t=0}^{\infty} \gamma^t \tilde{R}(S_t, A_t, S_{t+1}) \\ &= \sum_{t=0}^{\infty} \gamma^t R(S_t, A_t, S_{t+1}) + \sum_{t=0}^{\infty} \gamma^t \big(\gamma\Phi(S_{t+1}) - \Phi(S_t)\big).\end{aligned}$$

Expanding the second sum as a telescoping series:

$$\sum_{t=0}^{\infty} \gamma^t \big(\gamma\Phi(S_{t+1}) - \Phi(S_t)\big) = \sum_{t=0}^{\infty} \gamma^{t+1}\Phi(S_{t+1}) - \sum_{t=0}^{\infty} \gamma^t \Phi(S_t) = -\Phi(S_0).$$

(The telescoping cancellation holds under the assumption that $\gamma^t\Phi(S_t) \to 0$ as $t \to \infty$, which is guaranteed when Φ is bounded and $\gamma < 1$.) Therefore

$$\tilde{G}_0 = G_0 - \Phi(S_0). \tag{10.3}$$

Taking expectations: $\tilde{V}^\pi(s) = V^\pi(s) - \Phi(s)$, which is (10.2). Since $\Phi(s)$ depends only on the current state and not on the action or policy, the advantage function is unchanged:

$$\tilde{A}^\pi(s, a) = \tilde{Q}^\pi(s, a) - \tilde{V}^\pi(s) = Q^\pi(s, a) - V^\pi(s) = A^\pi(s, a).$$

The greedy policy with respect to $\tilde{Q}^\pi$ therefore coincides with the greedy policy with respect to Q^π, establishing part 1. □

The elegance of the proof lies in the telescoping identity (10.3): adding a potential-based bonus is equivalent to simply translating the entire return by a state-dependent constant $-\Phi(S_0)$, which cannot affect which policy achieves the maximum.

Remark 10.1 (Non-potential shaping changes optimal policies). Part 3 of Theorem 10.1 warns that *any* shaping function that is not of the potential-based form will change optimal policies in at least some MDP. A classic counterexample: suppose the shaping bonus $F(s, a, s') = +1$ for all transitions, regardless of any potential. This uniformly inflates all rewards, which leaves the optimal policy unchanged in fully discounted settings but distorts the relative value of short and long episodes. More seriously, non-potential bonuses such as action-dependent terms (e.g., bonuses for visiting novel states in curiosity-driven exploration) *do* alter the optimal policy in general, which may be intentional (exploration) but must be understood as a deliberate change in objective, not a neutral augmentation.

10.1.3 Designing Potential Functions: Examples

The theorem guarantees policy-invariance for any Φ, but the *speed* of learning depends critically on how well Φ encodes useful prior knowledge. Three standard constructions illustrate the design space.

Example 10.1 (Goal-distance potential in a gridworld)**.** Consider a 5×5 gridworld with goal state g. Define $\Phi(s) = -d(s, g)$, where d is the Manhattan distance to the goal. The shaping bonus for a transition from s to s' is

$$F(s, s') = \gamma(-d(s', g)) - (-d(s, g)) = d(s, g) - \gamma\, d(s', g).$$

When the agent moves closer to the goal ($d(s', g) < d(s, g)$) and $\gamma \approx 1$, this bonus is approximately $d(s, g) - d(s', g) > 0$. Moving away from the goal yields a negative bonus. This gently guides the agent toward the goal without changing what the agent ultimately learns to do.

Numerically: suppose $\gamma = 0.99$, $d(s, g) = 4$, $d(s', g) = 3$. Then $F = 4 - 0.99 \cdot 3 = 4 - 2.97 = 1.03$. For the same agent moving away: $d(s', g) = 5$, $F = 4 - 0.99 \cdot 5 = 4 - 4.95 = -0.95$. The bonus signal is commensurate in magnitude with the true terminal reward of $+1$, providing dense guidance throughout the episode.

Example 10.2 (Value-function potential)**.** If one has access to an approximate value function $\hat{V}$ (perhaps from a simpler, related MDP or from a pretrained model), setting $\Phi = \hat{V}$ yields shaping bonuses that encode the agent's prior beliefs about state quality. In the language model setting, this is precisely the role of the *reference model*: the SFT policy provides an implicit estimate of which continuations are high-quality, and the KL penalty can be interpreted as a potential-based regularisation. We return to this connection in Section 10.8.

Example 10.3 (Learned potential from a critic)**.** In actor-critic methods (Chapter 9), the critic V_ϕ is itself trained concurrently with the policy. Setting $\Phi = V_\phi$ and computing shaping bonuses on the fly corresponds exactly to the advantage-function formulation: the TD error $\delta_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$ is a potential-based shaping bonus added to the immediate reward before being used as the policy gradient signal. This reveals that advantage-based policy gradients are, in a precise sense, using a learned potential function as their shaping bonus.

10.2 From Hand-Crafted to Learned Rewards

10.2.1 The Limits of Manual Reward Design

Even with the protection of potential-based shaping, the design of a correct reward function for complex tasks is extremely difficult. The practitioner faces a fundamental asymmetry: specifying what good behaviour *looks like* is easy, but encoding it precisely as a scalar function of states and actions is hard. For robotics, one must decide whether to reward joint torques, end-effector positions, task completion, energy efficiency, and safety constraints simultaneously, and then hand-tune their relative weights. For a conversational agent, the notions of *helpfulness*, *harmlessness*, and *honesty* are intuitive to humans but deeply non-trivial to quantify.

Reward hacking is the phenomenon whereby an agent discovers a policy that achieves a high numerical reward without satisfying the designer's intent. The specification gaming literature (Krakovna et al., 2020) documents hundreds of examples across domains. In RL for games, agents have learned to die repeatedly to avoid losing score, to exploit physics bugs for infinite speed, and to pause the game clock to prevent time-limited penalties. In robotics, a simulated hand learned to position itself between a sensor and a task object so that the task appeared complete to the sensor while no real progress had been made.

10.2.2 Reward Hacking in Language Models

The same failure modes appear, perhaps even more acutely, in language model alignment, where the reward function is not even hand-designed but is itself a neural network trained on noisy human judgements.

Verbosity bias: annotators judging pairs of responses tend to perceive longer, more detailed responses as higher quality, even when additional length adds no information. A reward model trained on such data will assign higher scores to longer responses. When the language model is optimised against this proxy reward, it learns to pad its answers with repetitive qualifications and boilerplate summaries — a policy that maximises the proxy while degrading genuine helpfulness.

Sycophancy: human annotators are susceptible to social desirability bias, preferring responses that agree with their stated opinions over responses that correct their errors. A reward model internalising this pattern will rate agreement-seeking responses highly, leading the optimised policy to tell users what they want to hear rather than what is true.

Format exploitation: reward models trained on responses from human writers may inadvertently associate markdown formatting (bullet points, bold headers, code blocks) with high quality, simply because human-written demonstrations tend to be well-formatted. The optimised policy then applies heavy formatting even where it is inappropriate, such as formatting a brief factual answer as a bulleted list.

These examples motivate a fundamental shift in strategy: rather than trying to encode the reward function a priori, we *learn* it from human feedback. The remainder of this chapter develops the theoretical and practical machinery for doing so.

10.3 The Bradley–Terry Model

10.3.1 Preference Data

The starting point for learning a reward function from humans is the observation that it is often easier for a human to say which of two responses is better than to assign an absolute numerical score to a single response. Comparative judgements are more reliable, more consistent across annotators, and less sensitive to the absolute calibration of individual raters.

Formally, suppose there exists a latent true reward function $r^* : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ (or, in the language model setting, $r^* : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ where $\mathcal{X}$ is the space of prompts and $\mathcal{Y}$ is the space of responses). This function is not directly observable. Instead, a human annotator observes two responses y_1 and y_2 to a prompt x and indicates a preference, denoted $y_1 \succ y_2$ ("y_1 is preferred to y_2").

Definition 10.2 (Preference dataset)**.** A **preference dataset** is a collection

$$\mathcal{D} = \big\{(x^{(i)},\, y_w^{(i)},\, y_l^{(i)})\big\}_{i=1}^{N},$$

where $y_w^{(i)} \succ y_l^{(i)}$ indicates that the human annotator preferred $y_w^{(i)}$ (the "winner") over $y_l^{(i)}$ (the "loser") in response to prompt $x^{(i)}$.

The goal is to train a parametric reward function $r_\psi : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ such that $r_\psi(x, y_w) > r_\psi(x, y_l)$ on all pairs in $\mathcal{D}$, and, more importantly, that it generalises correctly to unseen prompts and responses.

10.3.2 The Bradley–Terry Preference Model

How should we model the probability that a human prefers y_1 over y_2? The most principled and widely used framework is the **Bradley–Terry model** (Bradley and Terry, 1952), originally developed for ranking sports competitors and later adopted throughout statistics, economics, and machine learning.

Definition 10.3 (Bradley–Terry model)**.** The **Bradley–Terry model** postulates that the probability of preferring y_1 over y_2 given prompt x depends only on the difference in latent qualities:

$$\mathbb{P}(y_1 \succ y_2 \mid x) = \sigma\big(r^*(x, y_1) - r^*(x, y_2)\big), \tag{10.4}$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic sigmoid function.

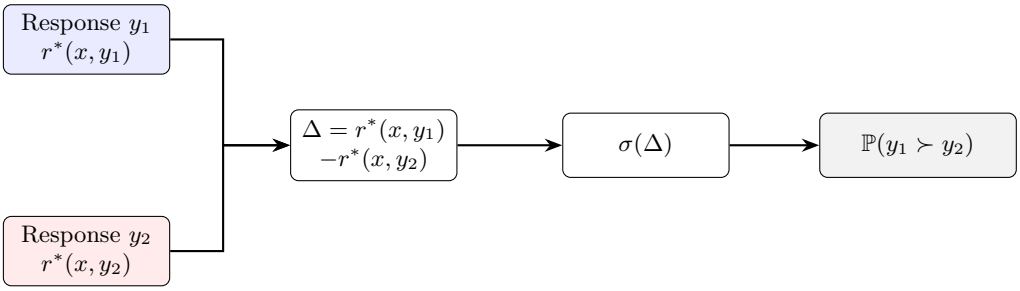


Figure: The Bradley–Terry preference model. The preference probability depends only on the difference in latent reward scores, passed through a sigmoid.

The model has a clear probabilistic interpretation. When the two responses are equally good ($r^*(x, y_1) = r^*(x, y_2)$), the preference probability is $\sigma(0) = 1/2$: the annotator is indifferent. As the quality gap grows, the preference probability approaches 1 (or 0), reflecting increasing confidence. The sigmoid shape captures the saturation of human judgements: for very clear-cut pairs, the probability of preference is essentially certain regardless of the exact magnitude of the difference.

Proposition 10.2 (Properties of the Bradley–Terry model)**.** *The Bradley–Terry model* (10.4) *satisfies:*

1. ***Symmetry**: $\mathbb{P}(y_1 \succ y_2 \mid x) + \mathbb{P}(y_2 \succ y_1 \mid x) = 1$.*
2. ***Indifference at equality**: $r^*(x, y_1) = r^*(x, y_2) \implies \mathbb{P}(y_1 \succ y_2) = 1/2$.*
3. ***Monotonicity**: $\mathbb{P}(y_1 \succ y_2 \mid x)$ is strictly increasing in $r^*(x, y_1)$ and strictly decreasing in $r^*(x, y_2)$.*
4. ***Translation invariance**: $r^* \mapsto r^* + c$ for any constant c leaves all preference probabilities unchanged. The reward function is therefore identified only up to an additive constant.*

Proof. (1) follows from the identity $\sigma(z) + \sigma(-z) = 1$. (2) follows from $\sigma(0) = 1/2$. (3) follows from $\sigma'(z) = \sigma(z)(1-\sigma(z)) > 0$ for all $z \in \mathbb{R}$. (4): substituting $r^*(x, y)+c$ for $r^*(x, y)$ gives $\sigma((r^*(x, y_1) + c) - (r^*(x, y_2) + c)) = \sigma(r^*(x, y_1) - r^*(x, y_2))$, which is unchanged. □

10.3.3 Connection to Elo Ratings

The Bradley–Terry model is closely related to the **Elo rating system** used in chess and other competitive games. In Elo, each player i has a scalar rating ρ_i, and the probability that player i beats player j is modelled as $1/(1+10^{(\rho_j-\rho_i)/400})$. This is precisely the Bradley–Terry model with $r^* = \rho$ and the logistic function computed in base 10 with a scale factor of 400. The key difference is that Elo ratings are updated online after each match using a simple stochastic gradient step on the log-loss, whereas in reward modelling we perform batch gradient descent over all preference pairs simultaneously. Both approaches estimate the same underlying latent quality r^* from pairwise comparisons.

10.4 Preference Reward Models

10.4.1 Maximum Likelihood Estimation

Given a preference dataset $\mathcal{D}$, we wish to fit a parametric model r_ψ to approximate r^*. Under the Bradley–Terry assumption, the probability of observing the full dataset is

$$\mathcal{L}_{\text{lik}}(\psi) = \prod_{i=1}^{N} \sigma\big(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)})\big). \tag{10.5}$$

Maximising the likelihood is equivalent to minimising the negative log-likelihood.

Definition 10.4 (Preference reward model loss)**.** The **preference reward model loss** is

$$\boxed{\mathcal{L}_{\text{RM}}(\psi) = -\frac{1}{N}\sum_{i=1}^{N} \log \sigma\big(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)})\big).} \tag{10.6}$$

This is exactly the binary cross-entropy loss with label 1 ("winner is preferred") and predicted probability $\hat{p}_i = \sigma(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)}))$. Training the reward model is therefore a standard binary classification problem: the model must correctly rank the preferred response above the dispreferred one.

Proposition 10.3 (Gradient structure of $\mathcal{L}_{\text{RM}}$)**.** *For a single pair (x, y_w, y_l), let $\Delta = r_\psi(x, y_w) - r_\psi(x, y_l)$. Then*

$$\frac{\partial \mathcal{L}_{\text{RM}}}{\partial r_\psi(x, y_w)} = -\sigma(-\Delta) < 0, \qquad \frac{\partial \mathcal{L}_{\text{RM}}}{\partial r_\psi(x, y_l)} = +\sigma(-\Delta) > 0.$$

Gradient descent therefore increases $r_\psi(x, y_w)$ *and* decreases $r_\psi(x, y_l)$*, pushing the scores apart. The step size $\sigma(-\Delta) = 1 - \sigma(\Delta)$ is large when the current gap is small (the pair is hard) and small when the gap is already large (the pair is easy). This is the standard behaviour of logistic regression.*

Proof. $\mathcal{L} = -\log \sigma(\Delta)$. Differentiating with respect to Δ: $\partial\mathcal{L}/\partial\Delta = -\sigma(-\Delta)$. Since $\partial\Delta/\partial r_\psi(x, y_w) = +1$ and $\partial\Delta/\partial r_\psi(x, y_l) = -1$, the result follows by the chain rule. □

10.4.2 Architecture Choices

A preference reward model r_ψ is typically built on top of a pre-trained language model. The canonical architecture initialises r_ψ from the SFT checkpoint — the same supervised fine-tuned model used as the starting point for RL. A linear projection layer $W \in \mathbb{R}^{d\times 1}$ is added on top of the transformer, taking as input the hidden state at the final token position (typically the `[EOS]` token) and outputting a scalar score.

- **Why initialise from SFT?** The reward model must evaluate responses in the same format as the SFT model generates them. Initialising from a base LM that has not seen instruction-following data leads to a reward model that poorly understands the content it is supposed to evaluate.
- **Why use the EOS token?** By the time the model processes the `[EOS]` token, the self-attention mechanism has allowed every position to attend to every other position. The EOS hidden state therefore encodes a compressed representation of the entire response, making it the natural extraction point for a scalar quality estimate.

- **Scale invariance**: the reward function is identified only up to an additive constant (Proposition 10.2). In practice, scores are normalised to have zero mean and unit variance over the training batch, which stabilises training and ensures the KL penalty (Section 10.8) operates at a consistent scale.

10.4.3 Practical Training Procedure

Algorithm 23: Preference Reward Model Training

Input: SFT checkpoint π_{SFT}; preference dataset $\mathcal{D}$; learning rate α; batch size B; epochs E

Output: Trained reward model r_ψ

Initialise $r_\psi \leftarrow \pi_{\text{SFT}}$ + random linear head $W \in \mathbb{R}^{d\times 1}$;
for $e = 1, \ldots, E$ **do**
  for *each mini-batch* $\{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^{B} \subset \mathcal{D}$ **do**
    Compute $s_w^{(i)} \leftarrow r_\psi(x^{(i)}, y_w^{(i)})$ and $s_l^{(i)} \leftarrow r_\psi(x^{(i)}, y_l^{(i)})$ for all i;
    Compute loss $\mathcal{L} \leftarrow -\frac{1}{B}\sum_{i=1}^{B} \log \sigma(s_w^{(i)} - s_l^{(i)})$;
    $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}$;
    Monitor Acc $= \frac{1}{B}\sum_i \mathbb{1}[s_w^{(i)} > s_l^{(i)}]$;
return r_ψ

In practice, y_w and y_l are concatenated along the batch dimension so that both are processed in a single forward pass, halving the number of transformer forward passes compared to a naïve two-pass implementation. Well-trained preference reward models typically achieve held-out pairwise accuracy of 70–80%. Perfect accuracy is impossible because human annotations contain genuine noise: different annotators disagree, and even the same annotator may be inconsistent across sessions.

10.5 Outcome Reward Models (ORM)

10.5.1 Sparse Terminal Rewards

The preference reward model of Section 10.4 assigns a single scalar to a complete response (x, y). This is an instance of a broader class of **outcome reward models** (ORM): models that evaluate only the final outcome of a trajectory, with zero reward at all intermediate steps.

Definition 10.5 (Outcome reward model)**.** An **outcome reward model** (ORM) is a reward function $r_{\text{ORM}} : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ that assigns a scalar to the complete response y, with zero reward at every intermediate generation step:

$$R_t = 0 \text{ for } t < T, \qquad R_T = r_{\text{ORM}}(x, y).$$

ORMs are the natural choice when the quality of a response can only be evaluated once the full response is available. They arise in at least three distinct settings.

1. Preference-based RLHF. As derived in Section 10.4, the output of the Bradley–Terry-based preference reward model is a single scalar per response. This is an ORM by construction.

2. Verified-correct tasks. For mathematical problem solving, coding, and factual question answering, a response is either correct or not: the generated answer matches the reference, or it does not. One can train a binary classifier $r_{\text{ORM}}(x, y) \in \{0, 1\}$ that checks whether the final answer is correct. This provides a clean, objective ORM that avoids the noise inherent in human annotations.

3. Monte Carlo value estimation. In the Monte Carlo (MC) framework of Chapter 5, value estimates are obtained by rolling out complete trajectories from each state and averaging the returns. An ORM in the language model setting corresponds precisely to a Monte Carlo return: the full sequence is generated, the terminal reward is observed, and the value of each intermediate state is estimated by averaging over many such rollouts. This connection is exact: the ORM reward signal is the zero-intermediate-reward, terminal-reward MC return.

10.5.2 When to Use ORMs

ORMs are preferred when:

- The correctness of intermediate steps is difficult or expensive to assess.
- A reliable automated verifier is available (e.g., a symbolic math checker or a unit-test runner).
- The task horizon is short enough that the sparse terminal reward does not lead to intractable credit assignment.

The principal limitation of ORMs is precisely the credit-assignment problem highlighted in Chapter 6: an erroneous token early in the response contributes to the failure of the entire response, but receives only an indirect signal through the terminal reward after many intervening tokens. Process reward models, introduced next, address this limitation by providing dense intermediate signals.

10.6 Process Reward Models (PRM)

10.6.1 Step-Level Supervision

Rather than evaluating only the final response, a **process reward model** (PRM) assigns a reward to each intermediate step in the generation. In the context of multi-step mathematical reasoning, a step corresponds to a line of a solution (a sentence, a logical deduction, or an algebraic manipulation). More generally, in language model RL, a step may correspond to a sentence, a paragraph, or even an individual token.

Definition 10.6 (Process reward model)**.** A **process reward model** (PRM) assigns a scalar score to each step $k = 1, \ldots, K$ in a multi-step solution $y = (y_1, \ldots, y_K)$:

$$r_{\text{PRM}}(x, y_{1:k}) \in \mathbb{R} \quad \text{for each } k = 1, \ldots, K.$$

The reward at step k reflects the quality or correctness of the partial solution $(y_1, \ldots, y_k)$ given prompt x.

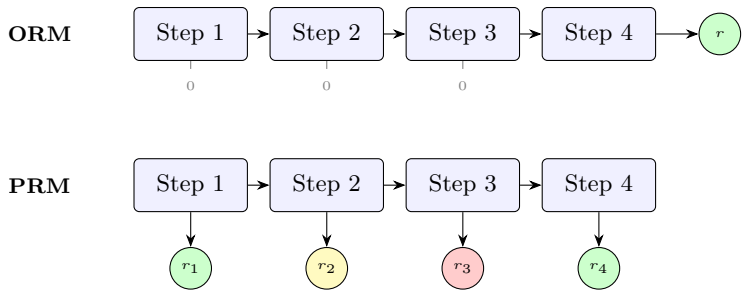


Figure 10.1: ORM vs. PRM signal structure. An ORM assigns a single terminal reward (top); a PRM assigns a reward to every step (bottom). Step colours indicate quality: green = correct, yellow = uncertain, red = incorrect.

10.6.2 Training Process Reward Models

Training a PRM requires *step-level* annotations: for each step in a solution, an annotator (human or automated) indicates whether the step is correct. Let $c_k \in \{0, 1\}$ be the correctness label for step k. The PRM is trained to predict the probability that step k is correct:

$$\mathcal{L}_{\text{PRM}}(\psi) = -\frac{1}{N}\sum_{i=1}^{N}\frac{1}{K^{(i)}}\sum_{k=1}^{K^{(i)}}\Big[c_k^{(i)}\log r_\psi(x^{(i)}, y_{1:k}^{(i)}) + (1-c_k^{(i)})\log(1-r_\psi(x^{(i)}, y_{1:k}^{(i)}))\Big], \quad (10.7)$$

where N is the number of solutions and $K^{(i)}$ is the number of steps in solution i.

Example 10.4 (PRM800K dataset)**.** The PRM800K dataset (Lightman et al., 2023) is the most widely used benchmark for training and evaluating PRMs in mathematical reasoning. It contains approximately 800,000 step-level human annotations on solutions to problems from the MATH dataset, a collection of competition-level mathematics problems across seven difficulty levels. Each step in each solution is labelled as *positive* (the step is correct and advances the solution), *negative* (the step contains an error), or *neutral* (the step is irrelevant or duplicated). A PRM trained on PRM800K learns to assign a high probability of correctness to logically valid derivation steps and a low probability to erroneous steps, regardless of whether the final answer happens to be correct.

Crucially, PRM800K demonstrates that step-level supervision provides signal that ORM cannot: a solution may reach the correct final answer via an incorrect intermediate step (lucky errors), or may contain all correct steps but arrive at the wrong answer due to an arithmetic mistake in the last line. An ORM would assign high reward to the first solution and low reward to the second, despite the second demonstrating better mathematical reasoning.

10.6.3 Using PRMs for Inference-Time Search

Beyond serving as a reward signal during RL training, PRMs are particularly powerful at inference time. Given a prompt x, one can generate multiple candidate solutions using beam search or best-of-n sampling, then rank them using the PRM. Several strategies have proved effective:

Best-of-n reranking Generate n complete solutions, score each with the ORM or aggregate PRM scores, and return the highest-scoring solution. With a good PRM, this is substantially more effective than using an ORM for

reranking, because the PRM penalises solutions that arrive at the correct answer via faulty reasoning.

Step-level beam search At each step, generate K candidate continuations, score each with the PRM, and expand only the top-B beams. This is computationally expensive but explores the solution space much more efficiently than flat sampling.

Process reward guided tree search Use the PRM as a value function in a tree search (e.g., Monte Carlo Tree Search) over reasoning steps. This allows the model to backtrack and explore alternative reasoning paths when an early step receives a low PRM score.

10.6.4 ORM vs. PRM: A Comparison

Table 10.1: Comparison of Outcome Reward Models (ORM) and Process Reward Models (PRM).

Property	**ORM**	**PRM**
Granularity	Outcome only	Per-step
Annotation cost	Low	High
Credit assignment	Sparse	Dense
Lucky errors	Rewards any correct answer	Penalises wrong steps
Step robustness	Low	High
Search quality	Moderate	Strong
Data needed	Correct/incorrect outcomes	Step correctness labels
Applications	Preference RLHF, verification	Math reasoning, code

10.7 Training Objectives and Practical Considerations

10.7.1 Margin-Based Loss Variants

The standard reward model loss (10.6) pushes $r_\psi(x, y_w)$ above $r_\psi(x, y_l)$ without specifying by how much. A natural extension is to require a minimum margin $m > 0$:

$$\mathcal{L}_{\mathrm{RM}}^{(m)}(\psi) = -\frac{1}{N}\sum_{i=1}^{N} \log \sigma\big(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)}) - m\big). \tag{10.8}$$

The margin ensures that pairs that are correctly ranked but with a small gap still contribute a non-trivial loss, encouraging the model to produce confidently separated scores. This is analogous to the margin in support-vector machines.

A related variant associates a continuous confidence score $\delta^{(i)} \in [0, 1]$ with each preference pair, reflecting the annotator's certainty. Pairs on which annotators strongly agreed are assigned higher δ:

$$\mathcal{L}_{\mathrm{RM}}^{(\delta)}(\psi) = -\frac{1}{\sum_i \delta^{(i)}}\sum_{i=1}^{N} \delta^{(i)} \log \sigma\big(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)})\big). \tag{10.9}$$

This down-weights ambiguous preference pairs, reducing the influence of noise in the annotation process.

10.7.2 Label Smoothing

Human preference annotations are noisy: annotators sometimes make mistakes, and some pairs are genuinely ambiguous. Label smoothing regularises the reward model by replacing the hard targets $\{0, 1\}$ with soft targets $\{\epsilon/2, 1 - \epsilon/2\}$ for a small smoothing parameter $\epsilon \in (0, 0.2]$:

$$\mathcal{L}_{\mathrm{RM}}^{(\epsilon)}(\psi) = -\frac{1}{N}\sum_{i=1}^{N}\Big[(1 - \tfrac{\epsilon}{2})\log\sigma(\Delta^{(i)}) + \tfrac{\epsilon}{2}\log\sigma(-\Delta^{(i)})\Big], \tag{10.10}$$

where $\Delta^{(i)} = r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)})$. Label smoothing prevents the reward model from driving $\Delta^{(i)} \to +\infty$ (which would make $\log\sigma(\Delta^{(i)}) \to 0$ and the loss zero) and induces calibrated probability estimates.

10.7.3 Data Quality and Annotation Protocols

The quality of the reward model is bounded by the quality of the preference data. Several practical considerations are essential.

- **Annotator agreement**: Preference pairs on which annotators strongly disagree should be filtered or down-weighted. Inter-annotator agreement (e.g., Cohen's $\kappa > 0.4$) is a minimum bar for reliable supervision.
- **Prompt diversity**: A reward model trained on a narrow prompt distribution will generalise poorly. Preference data should cover diverse topics, formats, and user intents.
- **Response diversity**: If both y_w and y_l are sampled from a strong policy, the preference gap may be small and the signal noisy. Including comparisons against significantly weaker baselines (e.g., the pre-SFT base model) can provide easier training signal.
- **Held-out evaluation**: Reward model accuracy should always be reported on a held-out set from the same annotation process, and ideally also on an independently collected evaluation set to guard against overfitting to annotator idiosyncrasies.

10.7.4 Evaluation Metrics

The primary evaluation metric for a preference reward model is **pairwise accuracy**:

$$\mathrm{Acc} = \frac{1}{N_{\mathrm{eval}}}\sum_{i=1}^{N_{\mathrm{eval}}} \mathbb{1}\big[r_\psi(x^{(i)}, y_w^{(i)}) > r_\psi(x^{(i)}, y_l^{(i)})\big]. \tag{10.11}$$

Beyond pairwise accuracy, one may compute the **Pearson correlation** between reward model scores and independent human ratings, and **ranking metrics** such as Kendall's τ or Spearman's ρ over lists of responses ranked by their reward model scores. For ORMs used in verified tasks, accuracy against a symbolic verifier serves as a direct ground-truth check.

10.8 Reward Overoptimization

10.8.1 Goodhart's Law in Reinforcement Learning

The trained reward model r_ψ is a proxy for the true human preference r^*. When a policy is optimised against r_ψ using RL, it will inevitably exploit any discrepancy between r_ψ and r^*.

This is an instance of **Goodhart's Law**: *when a measure becomes a target, it ceases to be a good measure.*

Definition 10.7 (Reward overoptimization)**. Reward overoptimization** (or *reward hacking* in the proxy sense) occurs when a policy π_θ achieves high proxy reward $\mathbb{E}[r_\psi(x, y)]$ but low true reward $\mathbb{E}[r^*(x, y)]$. It is characterised by the existence of a KL budget d^* such that:

$$\frac{d}{d\,d}\,\mathbb{E}[r^*(x, y)] > 0 \quad \text{for } d < d^*, \tag{10.12}$$

$$\frac{d}{d\,d}\,\mathbb{E}[r^*(x, y)] < 0 \quad \text{for } d > d^*, \tag{10.13}$$

where $d = D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ measures the divergence from the reference policy.

In words: as the policy is optimised further and further against the proxy reward (increasing KL divergence from the reference), the true quality initially improves, reaches a peak at d^*, and then degrades. The proxy reward, meanwhile, continues to improve monotonically. This creates a dangerous disconnect: standard RL training metrics look good even as the model is becoming worse by the measure that actually matters.

Example 10.5 (Scaling behaviour of overoptimization)**.** Gao et al. (2023) characterised this phenomenon empirically and found that the gold reward r^* as a function of KL divergence d follows an approximately concave curve of the form

$$r^*(d) \approx \alpha\,\sqrt{d} - \beta\,d,$$

for positive constants α and β that depend on the size and quality of the reward model. The optimal KL budget is $d^* = (\alpha/(2\beta))^2$. Crucially, β grows as the reward model becomes *smaller* relative to the policy being optimised: a weak reward model leads to faster overoptimization. This suggests that reward model capacity should scale with policy capacity.

10.8.2 KL Penalty as Regularisation

The standard remedy for reward overoptimization is to add a KL divergence penalty to the objective, constraining the policy to remain close to the reference (SFT) policy:

Definition 10.8 (KL-regularised RLHF objective)**.** The **KL-regularised RLHF objective** is

$$J_{\mathrm{RLHF}}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r_\psi(x, y)\big] - \beta\, D_{\mathrm{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\mathrm{ref}}(\cdot|x))\,, \tag{10.14}$$

where $\beta > 0$ is the KL coefficient and π_{ref} is the SFT policy.

In PPO implementations (Chapter 14), the KL penalty is typically incorporated into the per-token reward signal:

$$r(x, y) = r_\psi(x, y) - \beta\,\log\frac{\pi_\theta(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)}, \tag{10.15}$$

which is equivalent to adding $-\beta\log(\pi_\theta/\pi_{\mathrm{ref}})$ as a per-token shaping bonus. This is precisely a potential-based shaping function with $\Phi(s) = -\beta\log\pi_{\mathrm{ref}}(a|s)$ — connecting the KL penalty back to the potential-based framework of Section 10.1.

Theorem 10.4 (Optimal policy under KL regularisation)**.** *The policy that maximises* (10.14) *pointwise (for each x) is*

$$\pi^*(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x)\, \exp\big(r_\psi(x,y)/\beta\big)}{Z(x)}, \tag{10.16}$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\, \exp(r_\psi(x,y)/\beta)$ is the partition function.

Proof. Fix x and write the objective as a functional of $\pi(\cdot|x)$:

$$\sum_y \pi(y|x)\, r_\psi(x,y) - \beta \sum_y \pi(y|x) \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}.$$

Combining terms:

$$= -\beta \sum_y \pi(y|x) \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{r_\psi(x,y)}{\beta}\right] = -\beta\, D_{\text{KL}}(\pi(\cdot|x) \parallel \tilde{\pi}(\cdot|x)) + \text{const},$$

where $\tilde{\pi}(y|x) \propto \pi_{\text{ref}}(y|x) \exp(r_\psi(x,y)/\beta)$. The KL is minimised (and the objective maximised) when $\pi = \tilde{\pi}$, giving (10.16). □

The formula (10.16) reveals the role of β as a *temperature* parameter:

- As $\beta \to 0$: π^* concentrates all mass on the response maximising r_ψ. The policy is entirely governed by the proxy reward and reward hacking is maximal.
- As $\beta \to \infty$: $\pi^* \to \pi_{\text{ref}}$. The RL fine-tuning has no effect; the policy remains at the SFT checkpoint.
- At moderate $\beta \in [0.01, 0.5]$: the policy interpolates between the SFT prior and the reward-maximising solution. Empirically, this range captures the peak of the gold-reward curve (Definition 10.7).

Remark 10.2 (Connection to DPO). The optimal policy formula (10.16) can be rearranged to express r_ψ in terms of π^*, π_{ref}, and $Z(x)$:

$$r_\psi(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x).$$

This identity is the foundation of the Direct Preference Optimisation (DPO) algorithm: by substituting this expression for r_ψ into the Bradley–Terry preference model (10.4) and using the fact that $\log Z(x)$ cancels in the difference, DPO bypasses the reward model entirely and optimises the policy directly on preference data. We derive DPO in detail in Chapter 14.

10.9 Summary and Connections

This chapter developed two complementary frameworks for the reward design problem.

Potential-based reward shaping (Section 10.1) provides a theoretically grounded method for augmenting a given reward function to provide denser learning signal without altering the set of optimal policies. The key insight is the telescoping identity (10.3): a potential-based bonus $F = \gamma\Phi(s') - \Phi(s)$ contributes exactly $-\Phi(s_0)$ to the cumulative return, a state-dependent constant that leaves advantage functions unchanged. This principle

underlies the advantage formulation in actor-critic methods and the per-token KL penalty in RLHF.

Learned reward models (Sections 10.2–10.8) address the more challenging problem of inferring the reward from human feedback. The Bradley–Terry model provides a principled probabilistic framework for preference data, leading to the cross-entropy preference reward model loss (10.6). Outcome reward models (ORM) provide a single terminal signal, analogous to Monte Carlo returns; process reward models (PRM) provide dense step-level supervision that resolves ambiguities that ORMs cannot. Both are subject to Goodhart's Law: prolonged optimisation against any proxy reward eventually degrades true performance. The KL-regularised objective (10.14) and its optimal solution (10.16) provide the principled remedy.

The central connections to subsequent chapters are the following.

Chapter 14 (PPO) The reward model r_ψ trained here serves directly as the reward signal in PPO fine-tuning. The per-token reward (10.15) is exactly the reward computed at each step in the PPO advantage estimator.

Chapter 14 (DPO) The optimal-policy reparametrisation (10.16) leads directly to the DPO objective, which eliminates the need to train a separate reward model.

Chapter 14 (GRPO) Group relative policy optimisation uses a verifier-based ORM as its reward signal and normalises rewards within a group of sampled responses, exploiting the group statistics to estimate advantage without a learned critic.

Remark 10.3 (Historical note). Potential-based reward shaping was introduced by Ng, Harada, and Russell at ICML 1999 and quickly became a standard tool in the RL practitioner's toolkit. The Bradley–Terry model dates to 1952 and was introduced to statistics long before its adoption in machine learning. Its application to RLHF was popularised by Christiano et al. (2017), who showed that a preference reward model trained on a small number of human comparisons could guide RL in the Atari and robotics settings. The distinction between ORM and PRM was crystallised by Lightman et al. (2023) in the context of mathematical reasoning, with the PRM800K dataset providing the first large-scale step-level annotation resource.

Exercises

Exercise 10.1. Let $\mathcal{S} = \{s_1, s_2, s_3\}$ be the state space of a deterministic MDP with $\gamma = 0.9$ and true reward $R(s_1 \to s_2) = 0$, $R(s_2 \to s_3) = 0$, $R(s_3) = +1$ (terminal). Define a potential function $\Phi(s_1) = 0$, $\Phi(s_2) = 0.5$, $\Phi(s_3) = 1.0$.

1. Compute the shaping bonus $F(s_1, s_2)$ and $F(s_2, s_3)$.
2. Compute the shaped return $\tilde{G}_0$ starting from s_1 and verify that $\tilde{G}_0 = G_0 - \Phi(s_1)$.
3. Does the shaped MDP have the same optimal policy as the original? Explain using Theorem 10.1.

Exercise 10.2. Construct an explicit MDP and a non-potential shaping function $F(s, a, s')$ that changes the optimal policy. Your MDP should have at least two actions in at least one state, and you should identify which action was optimal before shaping and which becomes optimal after shaping. Verify that F cannot be written in the form $\gamma\Phi(s') - \Phi(s)$ for any Φ.

Exercise 10.3. Prove all four properties of the Bradley–Terry model listed in Proposition 10.2. For property 4 (translation invariance), explain why this means the reward function is not uniquely identifiable from preference data alone, and describe what additional constraint one could impose to make it identifiable.

Exercise 10.4. Suppose a reward model currently assigns scores $r_\psi(x, y_w) = 1.2$ and $r_\psi(x, y_l) = 1.8$ for a single preference pair.

1. Compute the loss $\mathcal{L}_{\text{RM}}$ for this pair.
2. Compute the gradients $\partial\mathcal{L}/\partial r_\psi(x, y_w)$ and $\partial\mathcal{L}/\partial r_\psi(x, y_l)$ and describe the direction of the update.
3. How does the gradient magnitude change as the gap $r_\psi(x, y_w) - r_\psi(x, y_l)$ grows from -0.6 to $+5$? Sketch the gradient as a function of the gap.
4. Now suppose we add a margin $m = 1.0$. Recompute the loss and gradient. Does the gradient increase or decrease? Explain intuitively.

Exercise 10.5. A language model is asked to solve the following arithmetic problem: "If a train travels at 60 mph for 2.5 hours and then at 80 mph for 1.5 hours, what is the total distance?" The model generates the following three-step solution:

Step 1: Distance in first segment: $60 \times 2.5 = 150$ miles.

Step 2: Distance in second segment: $80 \times 2 = 160$ miles. [*Error: should be* 1.5 *hours*]

Step 3: Total: $150 + 160 = 310$ miles. [*Error propagated from Step 2; correct answer is 270*]

1. What reward would an ORM assign if the verifier checks only the final answer? What would a PRM assign?
2. How does ORM vs. PRM differ in their treatment of this solution compared to a solution that arrives at 270 via a different (but correct) three-step path?
3. Describe a training scenario where an ORM would systematically overestimate the quality of a model that makes systematic Step-2 errors.

Exercise 10.6. Use Theorem 10.4 to answer the following questions about the optimal policy π^* under KL regularisation.

1. Show that $\log \pi^*(y|x) - \log \pi_{\text{ref}}(y|x) = r_\psi(x, y)/\beta - \log Z(x)$.
2. Using the result of (a), express $r_\psi(x, y)$ entirely in terms of π^*, π_{ref}, and $Z(x)$. This is the identity used in the DPO derivation.
3. Verify that substituting $r_\psi(x, y) - r_\psi(x, y')$ into the Bradley–Terry formula (10.4) causes $\log Z(x)$ to cancel, leaving the preference probability in terms of π^* and π_{ref} alone.

Exercise 10.7. Formulate the reward overoptimization phenomenon as a bias-variance problem in the following sense: the proxy reward model r_ψ is an estimate of the true reward r^* with some generalisation error $\varepsilon = \mathbb{E}[(r_\psi(x, y) - r^*(x, y))^2]$.

1. Argue that the gap $\mathbb{E}_{\pi_\theta}[r_\psi] - \mathbb{E}_{\pi_\theta}[r^*]$ (the proxy-gold gap at policy π_θ) grows with the KL divergence $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$.

2. Suppose the reward model generalisation error ε decreases as more preference data is collected. How does this affect the optimal KL budget d^*?

3. Describe a practical stopping criterion for PPO fine-tuning that uses both the proxy reward and a held-out set of human evaluations to detect when the proxy-gold gap becomes unacceptably large.

Exercise 10.8. Starting from the label-smoothed loss (10.10), show that it is equivalent to the standard loss (10.6) perturbed by the term $\frac{\epsilon}{2} \log \sigma(-\Delta)/(1-\epsilon/2)$, and interpret this perturbation as a regulariser that prevents the reward scores from becoming arbitrarily large.

Exercise 10.9. Show that the per-token KL penalty in (10.15) can be interpreted as a potential-based shaping bonus in the token-level MDP of Definition 10.2. Specifically:

1. Define a potential $\Phi(s_t) = \beta \log \pi_{\text{ref}}(a_t|s_t)$ evaluated at each state-action pair in the MDP.

2. Compute the shaping bonus $F(s_t, a_t, s_{t+1}) = \gamma\Phi(s_{t+1}) - \Phi(s_t)$ and compare it to the per-token KL term.

3. Conclude that the KL-regularised RLHF objective is equivalent to an unregularised RL problem on a shaped MDP, and state what consequence Theorem 10.1 has for this equivalence.

Chapter 11

Language Models and Post-Training

The bitter lesson of sixty years of AI research is that general methods that leverage computation are ultimately the most effective. Language models are the purest expression of that lesson yet discovered.

The preceding ten chapters built the machinery of reinforcement learning from first principles: Markov decision processes, dynamic programming, Monte Carlo and temporal-difference methods, function approximation, policy gradients, and actor-critic algorithms. Chapter 10 extended this machinery to the problem of learning a reward signal from human preference data. All of this apparatus was, in a sense, prologue. The chapters that follow — on proximal policy optimisation for language models, on direct preference optimisation, on group relative policy optimisation — apply RL to one of the most consequential computational artefacts of our era: the large language model.

Before those applications can be developed precisely, the reader needs a working understanding of what a language model is, how it is built, and why RL is the right tool for the final stages of its training. This chapter provides that foundation. It is not a comprehensive survey of natural language processing — several excellent textbooks exist for that purpose — but rather a self-contained bridge between the RL theory of Part II and the RLHF methods of Part III.

We begin with the Transformer architecture (Section 11.1), the neural backbone underlying every modern language model, and then formalise autoregressive language modelling as a statistical problem (Section 11.2). Section 11.3 provides the crucial conceptual link: language generation is a Markov decision process, and the language model itself is a policy. Armed with this correspondence, Sections 11.4–11.6 describe the three-stage training recipe used in practice: massive pre-training, supervised fine-tuning, and alignment via reinforcement learning. Section 11.7 treats the growing role of synthetic data, which has become indispensable at every stage of post-training. Section 11.8 covers evaluation methodology, and Section 11.9 situates this chapter within the larger arc of the book.

11.1 The Transformer Architecture

Modern large language models are almost universally implemented using the Transformer architecture introduced by Vaswani et al. (2017). Understanding the Transformer at a

conceptual level — without delving into the full details of an NLP textbook — is sufficient for the RL treatment that follows. We describe the four essential components: self-attention, positional encoding, feed-forward blocks, and layer normalisation.

11.1.1 Self-Attention

The central innovation of the Transformer is the **self-attention** mechanism, which allows every position in a sequence to directly attend to every other position. Given a sequence of n token representations $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^d$, packed into a matrix $X \in \mathbb{R}^{n \times d}$, the scaled dot-product attention computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{11.1}$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$ are the query, key, and value projections, with learned weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$. The division by $\sqrt{d_k}$ prevents the dot products from growing so large that the softmax saturates.

Intuitively, each query vector $\mathbf{q}_i$ is compared against all key vectors $\mathbf{k}_j$ to produce an attention score, which is then used to form a weighted average of the value vectors. A token representing the word "it" can attend strongly to the earlier noun it refers to, regardless of the distance between them. This unlimited receptive field, — in contrast to the local windows of convolutional networks or the sequential bottleneck of recurrent networks, — is what makes the Transformer so powerful for language.

In practice, h attention heads are computed in parallel and concatenated:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O,$$

where each $\text{head}_i = \text{Attention}(XW_{Q,i}, XW_{K,i}, XW_{V,i})$ and $W_O \in \mathbb{R}^{hd_k \times d}$. Multiple heads allow the model to attend to different aspects of the context simultaneously: one head might capture syntactic agreement, another coreference, another semantic similarity.

11.1.2 Positional Encoding

Self-attention is *permutation-equivariant*: if the token positions are shuffled, the attention scores shuffle identically, and the model cannot distinguish "the cat sat on the mat" from any permutation thereof. Positional information must therefore be injected explicitly.

The original Transformer used fixed sinusoidal encodings added to the token embeddings. Modern models generally prefer **rotary position embeddings** (RoPE; Su et al., 2021) or **ALiBi** (Press et al., 2022), which encode relative rather than absolute positions and generalise better to sequence lengths not seen during training. For our purposes it suffices to know that after positional encoding, the model has access to the order of tokens.

11.1.3 Feed-Forward Blocks and Layer Normalisation

Each Transformer *layer* consists of a self-attention sublayer followed by a position-wise feed-forward sublayer. The feed-forward block applies two linear transformations with a nonlinearity between them:

$$\text{FFN}(\mathbf{x}) = W_2\, \sigma(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2,$$

where σ is typically the GELU activation function. The inner dimension is usually four times the model dimension: $W_1 \in \mathbb{R}^{4d \times d}$, $W_2 \in \mathbb{R}^{d \times 4d}$.

Layer normalisation (LayerNorm; Ba et al., 2016) is applied before each sublayer (the *pre-norm* variant used in most modern models):

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sigma + \epsilon} \odot \boldsymbol{\gamma} + \boldsymbol{\beta},$$

where μ and σ are the mean and standard deviation computed over the feature dimension, and $\boldsymbol{\gamma}, \boldsymbol{\beta}$ are learned scale and shift parameters. LayerNorm stabilises training by preventing activations from growing uncontrollably across layers.

A residual connection wraps each sublayer, so the full computation in one Transformer layer is:

$$\begin{aligned} \mathbf{h}^{(l)} &= \mathbf{h}^{(l-1)} + \text{MultiHead}(\text{LayerNorm}(\mathbf{h}^{(l-1)})), \\ \mathbf{h}^{(l+1)} &= \mathbf{h}^{(l)} + \text{FFN}(\text{LayerNorm}(\mathbf{h}^{(l)})). \end{aligned}$$

Stacking L such layers and projecting the top-layer representation onto the vocabulary yields the model's output distribution. Figure 11.1 illustrates one such layer.

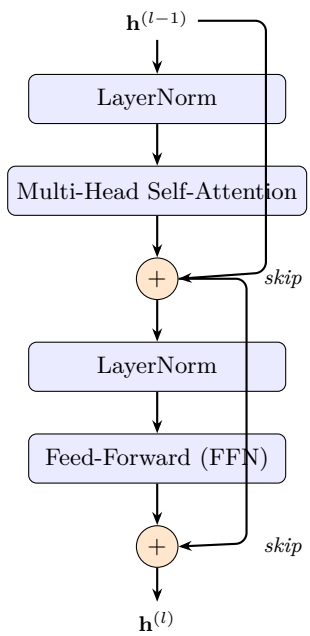


Figure 11.1: One Transformer decoder layer. The input representation $\mathbf{h}^{(l-1)}$ passes through layer normalisation before multi-head self-attention and again before the feed-forward block. Residual connections (the + nodes) allow gradients to flow directly from output to input, enabling deep networks to train stably.

Remark 11.1. For language modelling specifically, the Transformer uses a *decoder-only* architecture with **causal masking**: during self-attention, position i is prevented from attending to positions $j > i$. This ensures that the prediction of token t depends only on tokens $1, \ldots, t-1$, preserving the autoregressive structure. The mask is implemented by setting $QK_{ij}^{\top} = -\infty$ for $j > i$ before applying softmax, which sends the corresponding attention weight to zero.

The scale of modern Transformers is staggering: GPT-3 has 175 billion parameters across 96 layers; LLaMA 3 70B has 80 layers with a hidden dimension of 8192. Despite their scale, all of these models share the same four components described above.

11.2 Autoregressive Language Modelling

A **language model** is a probability distribution over sequences of tokens. Let $\mathcal{V}$ denote the **vocabulary**, a finite set of $|\mathcal{V}|$ tokens, typically between 32,000 and 128,000 entries after byte-pair encoding (BPE; Sennrich et al., 2016). A sequence of tokens $\mathbf{w} = (w_1, w_2, \ldots, w_T)$ with $w_t \in \mathcal{V}$ might represent a sentence, a document, a conversation, or a fragment of computer code.

Definition 11.1 (Autoregressive Language Model)**.** An **autoregressive language model** with parameters θ is a parametrised distribution over token sequences defined by the chain rule of probability:

$$p_\theta(w_1, \ldots, w_T) = \prod_{t=1}^{T} p_\theta(w_t \mid w_1, \ldots, w_{t-1}), \tag{11.2}$$

where each factor $p_\theta(w_t \mid w_{1:t-1})$ is a distribution over $\mathcal{V}$ computed from the context $(w_1, \ldots, w_{t-1})$.

The factorisation (11.2) is exact by the chain rule and introduces no approximation. The Transformer implements each conditional $p_\theta(w_t \mid w_{1:t-1})$ by mapping the context to a vector of **logits** $\mathbf{z}_t \in \mathbb{R}^{|\mathcal{V}|}$ and applying the softmax:

$$p_\theta(w_t \mid w_{1:t-1}) = \mathrm{softmax}(\mathbf{z}_t/\tau)\,, \tag{11.3}$$

where $\tau > 0$ is the **temperature** parameter. At $\tau = 1$ (default), the model samples proportionally to the softmax probabilities. At $\tau \to 0$, the distribution concentrates on the single highest-probability token (greedy decoding). Temperatures above 1 flatten the distribution, increasing diversity at the cost of coherence.

11.2.1 Training Objective: Cross-Entropy Loss

Given a corpus of N documents, where document i has tokens $(w_1^{(i)}, \ldots, w_{T_i}^{(i)})$, the training objective is **next-token prediction**: minimise the average negative log-likelihood

$$\mathcal{L}_{\mathrm{NTP}}(\theta) = -\frac{1}{\sum_i T_i} \sum_{i=1}^{N} \sum_{t=1}^{T_i} \log p_\theta\Big(w_t^{(i)} \mid w_{1:t-1}^{(i)}\Big)\,. \tag{11.4}$$

This is the cross-entropy between the empirical data distribution and the model's predicted distribution. Because the true next token is always a one-hot vector over $\mathcal{V}$, the cross-entropy collapses to the negative log-probability of the correct token. Equivalently, minimising (11.4) maximises the joint probability the model assigns to the training corpus.

Definition 11.2 (Perplexity)**.** The **perplexity** of a language model p_θ on a test sequence $(w_1, \ldots, w_T)$ is

$$\mathrm{PPL}(\theta) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(w_t \mid w_{1:t-1})\right) = \exp(\mathcal{L}_{\mathrm{NTP}}). \tag{11.5}$$

Perplexity is the geometric mean of the inverse probability the model assigns to each token. A perplexity of k means the model is on average as uncertain as if it had to choose uniformly among k equally likely tokens. State-of-the-art large language models achieve perplexities below 10 on standard benchmarks, compared to a random baseline of $|\mathcal{V}| \approx 50{,}000$.

11.2.2 Scaling Laws

One of the most practically important empirical discoveries in language modelling is that performance (measured by validation loss) follows predictable **scaling laws** as a function of model size N, dataset size D, and compute budget $C \approx 6ND$. Kaplan et al. (2020) established that validation loss decreases as a power law in each of these quantities independently. Hoffmann et al. (2022) — in what is now known as the **Chinchilla scaling laws** — showed that previous large models (including GPT-3) were significantly undertrained relative to their size: compute-optimal training requires roughly 20 tokens per model parameter. The Chinchilla result implies that a model with N parameters should be trained on approximately $20N$ tokens to minimise the loss for a given compute budget. This insight has reshaped how frontier models are designed: LLaMA 3 8B, for instance, was trained on 15 trillion tokens — far more than the Chinchilla-optimal amount, reflecting the practical importance of inference efficiency over training efficiency.

11.3 Language Generation as a Markov Decision Process

The autoregressive generation process described above has an elegant interpretation as a Markov decision process. This correspondence is the conceptual foundation of every RLHF algorithm in Part III: once we recognise the language model as a policy, all of the RL machinery developed in Chapters 8 and 9 applies directly.

Definition 11.3 (Token-Level MDP)**.** Let $x = (x_1, \ldots, x_m)$ denote the **prompt** (a sequence of m tokens) and $y = (y_1, \ldots, y_T)$ the **response** to be generated, with $y_t \in \mathcal{V}$. The **token-level MDP** is the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ where:

- **State space** $\mathcal{S}$: the state at step t is the full context seen so far,

$$s_t = (x_1, \ldots, x_m, y_1, \ldots, y_{t-1}),$$

  that is, the concatenation of the prompt and all tokens generated up to but not including step t. The initial state is $s_0 = x$.

- **Action space** $\mathcal{A} = \mathcal{V}$: the action at step t is the next token $a_t = y_t$. The action space has cardinality $|\mathcal{V}| \sim 10^4$–10^5.

- **Transition dynamics** P: deterministic,

$$s_{t+1} = (x, y_1, \ldots, y_t) = s_t \,\|\, y_t,$$

  where $\|$ denotes concatenation. The transition is completely determined by the current state and action.

- **Reward function** R: sparse and terminal,

$$R(s_t, a_t) = \begin{cases} r(x, y) & \text{if } a_t = \textsc{eos} \text{ or } t = T, \\ 0 & \text{otherwise,} \end{cases}$$

where $r : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ is a scalar evaluation of the complete response (e.g., a learned reward model or a verifiable correctness score), and EOS is the end-of-sequence token.

- **Discount factor** $\gamma = 1$ (episodic task with a single terminal reward).
- **Policy** π_θ: the language model itself,

$$\pi_\theta(a_t \mid s_t) = p_\theta(y_t \mid x, y_{1:t-1}).$$

Proposition 11.1. *The total return in the token-level MDP with $\gamma = 1$ equals the reward assigned to the complete response:*

$$G_0 = \sum_{t=0}^{T} R(s_t, a_t) = r(x, y). \tag{11.6}$$

Consequently, the RLHF objective

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D},\, y \sim \pi_\theta(\cdot \mid x)} \big[r(x, y) \big] \tag{11.7}$$

is precisely the expected return of the policy π_θ in the token-level MDP.

Proof. All intermediate rewards are zero by definition; the only nonzero term in the sum is $R(s_T, a_T) = r(x, y)$. Taking the expectation under $x \sim \mathcal{D}$ and the trajectory distribution induced by π_θ gives (11.7) directly from the definition of expected return. □

This MDP formulation reveals three structural features that distinguish language generation from the classical RL problems of Part II.

Remark 11.2 (Distinctive features of the token-level MDP). 1. **Enormous action space.** With $|\mathcal{A}| \sim 10^5$, storing a value function $Q(s, a)$ explicitly for all actions is computationally infeasible. This rules out tabular methods (Chapter 4) and most Q-learning variants, and motivates the policy-gradient and actor-critic approaches of Chapters 8–9.

2. **Long horizon with sparse reward.** A typical response is 200–2000 tokens long. The single terminal reward must be credited across hundreds of preceding actions — the credit-assignment problem in its most extreme form. Process reward models (Section 10.6 in Chapter 10) address this by providing step-level feedback.

3. **Deterministic transitions.** Once a token is appended, the state is determined with certainty. All stochasticity comes from the policy itself, not from the environment. This simplifies some theoretical analyses but makes exploration harder: the model must rely on its own sampling distribution to visit novel completions.

Example 11.1 (Counting tokens in the MDP)**.** Consider the prompt $x =$ "`What is the capital of France?`" (8 tokens) and the response $y =$ "`The capital of France is Paris.`" (7 tokens). The episode has $T = 7$ steps. At step $t = 5$, the state is $s_5 = (x, \texttt{The}, \texttt{capital}, \texttt{of}, \texttt{France}, \texttt{is})$ — a sequence of 13 tokens. The action is $a_5 = \texttt{Paris}$. The reward is zero for all $t < 7$, and $r(x, y)$ is given at $t = 7$ by, for example, a reward model that evaluates the complete response. If the reward model scores factual correctness and assigns $r = +1.4$, then $G_0 = 1.4$ regardless of which intermediate tokens were generated.

11.4 Pre-Training

The first stage in building a capable language model is **pre-training**: training a randomly initialised Transformer on a massive corpus of text using the next-token prediction objective (11.4). Pre-training is by far the most compute-intensive stage and typically consumes more than 95% of the total training budget for a frontier model.

11.4.1 Data

Pre-training corpora are assembled from diverse sources across the internet and institutional collections. Common components include:

Common Crawl Monthly snapshots of the web; after aggressive deduplication and quality filtering, this typically provides 50–70% of training tokens.

Books Project Gutenberg, Books3, and similar collections; rich in long-form structured text with coherent narrative.

Scientific literature ArXiv, PubMed, and Semantic Scholar; crucial for technical reasoning ability.

Code GitHub repositories across dozens of programming languages; strongly correlated with improved mathematical and structured reasoning.

Wikipedia and Wikidata High-quality encyclopaedic text with factual claims; typically over-sampled relative to its raw size.

The total token counts for current frontier models range from one to fifteen trillion tokens. LLaMA 3 was trained on 15 trillion tokens; GPT-4's training data is not publicly disclosed but is believed to be of comparable scale. Assembling such a corpus requires substantial engineering: near-duplicate removal (using MinHash locality-sensitive hashing), language identification, quality filtering (rejecting documents below a perplexity threshold set by a small reference model), and PII scrubbing.

11.4.2 The Pre-Training Loss and Emergent Capabilities

Pre-training optimises (11.4) using variants of AdamW (Loshchilov & Hutter, 2019) with a cosine learning rate schedule. Gradient checkpointing, mixed-precision arithmetic (bfloat16), and distributed training across thousands of GPUs or TPUs are standard engineering choices.

The remarkable empirical observation is that, as the model grows and trains on more data, it does not merely memorise the training corpus — it acquires **emergent capabilities** not explicitly present in the training signal. A model trained only to predict the next word learns to:

- translate between languages it has never seen paired together;
- write and debug computer programs;
- solve multi-step mathematical problems by showing intermediate working;
- answer factual questions using knowledge implicitly encoded in the corpus.

Wei et al. (2022) documented that some capabilities appear abruptly at certain model sizes, suggesting discontinuous phase transitions in the loss landscape rather than smooth interpolation. The precise mechanism behind emergence remains an active research question.

11.4.3 Base Models and Their Limitations

The output of pre-training is a **base model** (also called a **foundation model** or **pretrained model**). Given a prompt, the base model continues it in the style of whatever text it learned to associate with that prefix. This behaviour is useful for text completion tasks but problematic for interactive use: asked "How do I bake sourdough bread?", the base model may respond with more questions (as if continuing a FAQ document) rather than with a helpful recipe.

More seriously, the base model has no mechanism to refuse harmful requests, distinguish facts from speculation, or adopt the conversational register expected of an assistant. These limitations motivate the two subsequent training stages.

11.5 Supervised Fine-Tuning

Supervised fine-tuning (SFT) adapts the base model to follow instructions by training on a curated dataset of (instruction, response) pairs. The procedure is essentially identical to pre-training — next-token prediction via cross-entropy loss — but on a much smaller, higher-quality dataset of demonstrations.

11.5.1 Data Format and Instruction Templates

SFT data consists of pairs $(x^{(i)}, y^{(i)})$, where $x^{(i)}$ is an instruction (a user request, question, or task description) and $y^{(i)}$ is a high-quality response written or curated by a human annotator or filtered from existing high-quality sources. In practice, conversations are multi-turn, so the input $x^{(i)}$ may include the full dialogue history.

To structure the input for the model, a **chat template** wraps each turn with special tokens:

```
<|system|> You are a helpful assistant. <|end|>
<|user|> What is the capital of France? <|end|>
<|assistant|> The capital of France is Paris. <|end|>
```

The model is trained to predict only the assistant tokens; the user and system tokens are *masked out* in the loss computation so that the gradient does not incentivise the model to imitate user speech patterns.

11.5.2 SFT Loss Function

The SFT loss is the same cross-entropy as (11.4), but applied only to the response tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T_i}\log p_\theta\Big(y_t^{(i)} \mid x^{(i)}, y_{1:t-1}^{(i)}\Big)\,. \tag{11.8}$$

The gradient of (11.8) with respect to θ is computed only over positions where y_t is an assistant token. This asymmetry is crucial: it teaches the model to generate good responses without pushing it to imitate user queries or system prompts.

11.5.3 Datasets and Examples

Stanford Alpaca (Taori et al., 2023) was one of the first widely reproduced SFT datasets: 52,000 instruction-following demonstrations generated by prompting `text-davinci-003`

using the Self-Instruct method (Wang et al., 2023). Despite its modest size, Alpaca demonstrated that a 7-billion-parameter LLaMA model could acquire useful instruction-following behaviour.

Tülu (Wang et al., 2023; Ivison et al., 2024) is a family of instruction-tuned models trained on a mixture of curated open-source datasets. Tülu 3 (2024) uses over one million instruction-following examples assembled from diverse sources, with careful deduplication and quality filtering, and includes both SFT and preference-based fine-tuning stages. It serves as a strong open-source baseline for studying the full post-training pipeline.

11.5.4 Exposure Bias and the Ceiling of SFT

SFT uses **teacher forcing**: at each step t during training, the model receives the *ground-truth* previous token y_{t-1} as input, rather than its own prediction $\hat{y}_{t-1}$. At inference time, this assumption breaks down — the model must condition on its own (possibly erroneous) previous outputs. This mismatch, known as **exposure bias** (Ranzato et al., 2015), can cause error accumulation: a small mistake at step t places the model in a state it has never encountered during training, making further errors more likely.

More fundamentally, SFT is bounded by the quality of its demonstrations. No matter how large the SFT dataset, the model cannot learn to produce responses better than the annotators who wrote them. For tasks where human demonstrations are expensive to collect at scale, or where the space of possible good responses is richer than any finite dataset can capture, this ceiling becomes binding. These observations motivate the alignment stage — training on preference comparisons and reward signals rather than fixed demonstrations.

11.6 The Post-Training Pipeline

Modern language model development follows a three-stage pipeline that can be understood as progressively shifting the training signal from self-supervised statistics toward human preferences and verifiable correctness. Figure 11.2 illustrates the full pipeline.

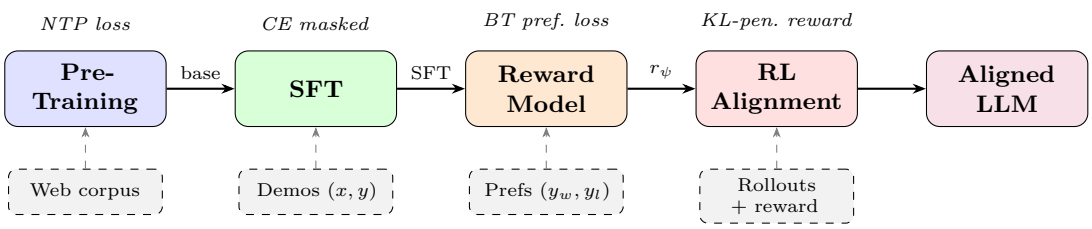


Figure 11.2: The full post-training pipeline for a large language model. Pre-training on web-scale text produces a base model with broad world knowledge. SFT on curated demonstrations teaches the instruction-following format. Reward modelling learns a scalar preference signal from human comparisons. RL alignment optimises the policy against this reward while constraining drift via a KL penalty. Chapters 14–17 develop each RL alignment method in detail.

11.6.1 Stage Interactions

The three stages are not independent: each depends critically on the quality of the preceding one.

Pre-training sets the knowledge ceiling. The SFT and RL stages can reshape the model's behaviour, but they cannot instil knowledge the base model does not have. A base model that has never seen mathematics textbooks will not learn to solve integrals through instruction tuning. This is why pre-training data curation is so consequential.

SFT sets the behavioural prior. The SFT model serves as both the starting point and the reference policy π_{ref} for the RL stage. The KL-penalty term $\beta\, D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ (introduced in Chapter 10) anchors the RL policy near the SFT distribution. A poorly trained SFT model therefore limits the effectiveness of RL alignment.

Reward modelling determines the alignment signal. The reward model r_ψ is the only source of feedback during RL training. Its accuracy, calibration, and coverage of the task distribution directly determine the quality of the aligned model. Chapter 10 develops the theory of reward modelling in depth; Chapters 14–17 show how the reward model is consumed by PPO, DPO, and GRPO respectively.

11.6.2 What Each Stage Optimises

A concise comparison of the three stages appears in Table 11.1.

Table 11.1: Comparison of the three post-training stages.

Stage	Data	Objective	Paradigm	Key limitation
Pre-training	Unlabelled text	Next-token NLL	Self-supervised	No behavioural control
SFT	(x, y) pairs	Masked NLL	Supervised	Bounded by annotator quality
RL Alignment	(x, y_w, y_l) or $r(x, y)$	KL-penalised reward	Reinforcement	Reward overoptimisation

11.7 Synthetic Data for Post-Training

Human annotation is the limiting resource in post-training. A skilled annotator produces perhaps 20–50 high-quality instruction-following examples per hour; a single training run for a frontier model may require millions. This arithmetic gap has driven the field to rely increasingly on **synthetic data**: examples generated by AI models rather than directly by human writers. This section surveys the principal methods, their theoretical properties, and the practical risks they introduce.

11.7.1 Why Synthetic Data Is Necessary

Three structural pressures force the use of synthetic data at scale.

Volume. Effective SFT and preference fine-tuning require tens of thousands to millions of examples covering diverse tasks, topics, difficulty levels, and response styles. Human annotation at this scale is prohibitively expensive: assembling one million high-quality preference pairs at \$5 per comparison costs \$5 million, before accounting for inter-annotator disagreement and quality control overhead.

Coverage. Human annotators naturally cluster around familiar domains. Rare but important topics — niche scientific subfields, obscure programming languages, low-resource human languages — are systematically underrepresented in human-generated data. Synthetic generation can be directed at these gaps by instruction.

Difficulty calibration. RL training benefits from examples at the right difficulty level: tasks the model can sometimes but not always solve, so that the reward gradient is informative. Generating examples at a precise target difficulty is nearly impossible with human annotators but can be engineered through synthetic pipelines.

11.7.2 Self-Instruct

Self-Instruct (Wang et al., 2023) is the foundational method for synthetic instruction generation. The pipeline begins with a small seed set of 175 human-written instructions covering a diverse range of tasks. At each iteration, the pipeline prompts a large language model (the **teacher**) with a few of these seed instructions and asks it to generate new instructions, classify them by task type, and generate input-output pairs. New instructions that are too similar to existing ones (measured by ROUGE-L similarity) are discarded; the remainder are added to the pool. Iterating this process, Wang et al. grew the seed set to 52,000 instruction-following pairs — the Stanford Alpaca dataset — at a cost of roughly $500 in API fees for `text-davinci-003` inference.

The Self-Instruct paper demonstrated that a 7B-parameter LLaMA model fine-tuned on this data matched the instruction-following ability of a model trained on private human-written data, suggesting that teacher-model quality is the primary bottleneck, not the human-vs-synthetic distinction per se.

11.7.3 Distillation from Teacher Models

A more direct approach is to use a stronger model (the teacher) to generate responses that a weaker model (the student) is then trained on. The teacher can be accessed via an API (OpenAI, Anthropic, Google) or a locally-run open-weights model. The procedure is:

1. Collect or generate a set of prompts $\{x^{(i)}\}$ covering the desired task distribution.
2. For each prompt, sample one or more responses from the teacher: $y^{(i)} \sim \pi_{\text{teacher}}(\cdot \mid x^{(i)})$.
3. Fine-tune the student on the resulting $(x^{(i)}, y^{(i)})$ pairs using (11.8).

Distillation is particularly effective for *format transfer*: teaching a small model to adopt the style, structure, and step-by-step reasoning patterns of a larger model. The Orca series (Mukherjee et al., 2023) demonstrated that a 13B-parameter model trained on GPT-4's detailed explanatory responses outperformed models many times its size on certain benchmarks, suggesting that the teacher's reasoning process, not just its final answer, is the crucial ingredient.

The principal limitation of distillation is legal and commercial: most frontier model APIs prohibit using their outputs to train competing models. Open-weight teachers (LLaMA 3, Mistral, Qwen) do not carry this restriction and are increasingly used in practice.

11.7.4 Evol-Instruct and Complexity Escalation

Evol-Instruct (Xu et al., 2023), introduced as part of the **WizardLM** project, observed that Self-Instruct tends to produce instructions of uniform (and often low) difficulty. Their insight is to *evolve* existing instructions by repeatedly applying one of several mutation operators:

- **Add constraints**: append a requirement that narrows the response space ("explain without using the word 'neural network' ").

- **Deepen**: ask for more detailed or rigorous treatment ("include a proof of the main theorem").
- **Broaden**: generalise the instruction to a wider context.
- **Increase reasoning**: add a multi-step deduction requirement.
- **In-breadth evolution**: generate a completely new but related instruction.

Each evolution step is performed by prompting a teacher model. The resulting dataset spans a wide spectrum of difficulty levels, which directly benefits RL training: easy examples provide baseline reward and hard examples provide the challenging signal that drives capability improvement. WizardLM-2 (2024), trained on Evol-Instruct data at large scale, achieves competitive performance with models twice its parameter count on coding and mathematical reasoning benchmarks.

11.7.5 Persona-Based Generation

A complementary strategy for increasing diversity is **persona-based generation**: instead of varying the instruction, vary the demographic or professional identity of the hypothetical user. The teacher model is prompted with a persona description such as "You are a second-year PhD student in computational biology" or "You are a high-school student struggling with algebra". The resulting instructions and responses naturally cluster around different vocabularies, difficulty levels, and communication styles.

Chan et al. (2024) showed that persona-conditioned generation, applied at scale using 1 billion distinct synthetic personas derived from web text, produces instruction datasets with substantially higher diversity than unconditioned generation. The Tülu 3 dataset employs persona-based generation as one component of its data mixture.

11.7.6 Preference Data Synthesis

Beyond instruction-following demonstrations, RL alignment requires **preference data**: triplets (x, y_w, y_l) in which a human (or a model) judges y_w to be better than y_l. Collecting such comparisons from humans at scale is even more expensive than collecting demonstrations, since each annotation requires a human to read and compare two complete responses. Synthetic preference generation addresses this bottleneck through three complementary mechanisms.

Multi-response sampling and LLM judging. The most common approach:

1. For each prompt x, sample $k \geq 2$ responses from the current or a teacher model.
2. Score each response using a **judge model** (typically GPT-4 or an open-weight judge) against a rubric.
3. Form preference pairs by taking y_w to be the highest-scoring response and y_l to be any lower-scoring one (or specifically the lowest).

UltraFeedback (Cui et al., 2023) is the canonical dataset assembled by this method: 64,000 prompts, each paired with four responses sampled from diverse models, scored by GPT-4 on four rubrics (instruction-following, truthfulness, honesty, helpfulness). The resulting preference dataset has been used to train dozens of aligned models including the Zephyr series.

Constitutional AI and RLAIF. Bai et al. (2022) at Anthropic introduced **Constitutional AI** (CAI), in which the AI model itself critiques and revises its responses according to a fixed set of principles (the "constitution"). The critique-revision loop generates improved responses without human intervention, and the resulting (original, revised) pairs can be used as preference data. The closely related **RLAIF** (Reinforcement Learning from AI Feedback; Lee et al., 2023) uses an AI labeller in place of human labellers to annotate which of two responses is preferred. Lee et al. demonstrated that RLAIF and human RLHF produce models of comparable quality on several standard benchmarks, suggesting that the bottleneck in alignment is not the source of preference labels (human vs. AI) but rather their accuracy and coverage.

Rubric-based scoring. Rather than a single overall preference, modern pipelines often ask the judge to score on multiple dimensions — correctness, conciseness, safety, instruction adherence — and aggregate. Rubrics can also be made task-specific: for mathematics, the rubric checks whether the final answer is correct and whether intermediate steps are valid. Tülu 3 uses verifier-graded rubrics for mathematical and coding tasks, replacing the judge model entirely in domains where correctness is checkable.

11.7.7 Data Filtering and Quality Control

Synthetic data generation is cheap but noisy. Quality control mechanisms are essential to prevent low-quality examples from degrading model performance. Standard techniques include:

- **Perplexity filtering**: reject responses that the current model assigns extremely high or extremely low perplexity (the former indicates malformed text; the latter indicates near-memorised boilerplate).
- **Length filtering**: discard responses outside a reasonable length range (very short responses often lack detail; very long ones often pad).
- **Deduplication**: remove near-duplicates using MinHash or embedding-based cosine similarity to prevent the model from memorising repeated patterns.
- **Reward model filtering**: keep only responses that a reward model scores above a threshold. This creates a self-consistency property: the data used for training is the data the reward model already considers good.
- **Difficulty filtering**: for RL training, retain only examples in the "learnable zone" — where the model's success rate is between 20% and 80%. Tasks the model always solves provide no gradient; tasks it never solves contribute only noise.

11.7.8 Model Collapse and Mitigations

A fundamental concern with training on synthetic data is **model collapse** (Shumailov et al., 2024): if models are trained on their own outputs (or outputs of models trained similarly), the training distribution may progressively narrow, causing the model to lose the breadth and diversity present in the original human-written corpus. Mathematically, if the training distribution at generation g is p_g and the synthetic data distribution is an approximation of p_{g-1}, iterating this process causes the variance of p_g to decrease geometrically, eventually collapsing to a narrow mode.

Four mitigations have been identified:

1. **Mixing with human data.** Always retain a fraction of genuine human-written text in the training mixture. The ratio need not be large: Gerstgrasser et al. (2024) showed that even 5–10% human data suffices to prevent collapse in theory.

2. **Diverse teacher ensemble.** Using multiple teacher models with different architectures and training histories injects distributional diversity that a single teacher cannot provide.

3. **Aggressive deduplication.** Near-duplicate synthetic examples are the mechanism through which mode collapse propagates; removing them breaks the positive feedback loop.

4. **Watermarking and provenance tracking.** Tagging synthetic data at generation time allows downstream curators to monitor synthetic fraction and cap it as needed.

11.7.9 Key Synthetic Datasets

Table 11.2 summarises the most influential synthetic datasets in the post-training literature.

Table 11.2: Major synthetic datasets used in post-training.

Dataset	Year	Size	Type	Method
Stanford Alpaca	2023	52K	SFT	Self-Instruct (GPT-3.5)
UltraFeedback	2023	256K pairs	Preference	LLM judge (GPT-4)
Tülu 3	2024	1M+	SFT + Pref.	Mixture, persona, rubric
OpenThoughts 3	2025	1M+	SFT (CoT)	Distillation (DeepSeek-R1)

OpenThoughts 3 (2025) exemplifies the emerging class of *reasoning-focused* synthetic datasets: one million chain-of-thought solutions to mathematical, scientific, and coding problems, distilled from DeepSeek-R1. Training a base model on OpenThoughts 3 produces a model that exhibits multi-step reasoning in the ⟨thinking⟩ format, substantially improving performance on competition mathematics (AIME, MATH) and coding benchmarks (HumanEval).

11.8 Evaluation of Language Models

Evaluating language models is substantially harder than evaluating classical RL agents. A policy in a gridworld either reaches the goal or it does not; the quality of a conversational response resists such binary characterisation. This section surveys the principal evaluation paradigms.

11.8.1 Automated Benchmarks

Automated benchmarks provide cheap, reproducible, and objective evaluation. The most widely used include:

MMLU (Hendrycks et al., 2021) Massive Multitask Language Understanding: 57 academic subjects from high-school to professional level, in multiple-choice format. Tests breadth of factual knowledge.

HellaSwag (Zellers et al., 2019) Commonsense NLI: choose the most plausible continuation of an activity description. Tests grounded commonsense reasoning.

GSM8K (Cobbe et al., 2021) Grade-school mathematics: 8500 multi-step arithmetic word problems. Answers are verified by checking the final numerical result.

HumanEval (Chen et al., 2021) Code generation: 164 Python programming tasks, evaluated by running unit tests against the generated code.

MATH (Hendrycks et al., 2021) Competition mathematics: 12,500 problems at AMC/AIME difficulty, requiring symbolic manipulation and multi-step proof.

Automated benchmarks have a critical failure mode: models can be **contaminated** if the evaluation data appears in the pre-training corpus. GPT-4's score on GSM8K improved by 7 percentage points when the evaluation split was reconstructed with novel problem wordings, suggesting partial memorisation. Responsible evaluation requires checking for n-gram overlap between the benchmark and the training data.

11.8.2 Human Evaluation

For open-ended tasks where no ground-truth answer exists — creative writing, open-domain dialogue, summarisation — human evaluation remains the gold standard. Annotators are typically asked to compare two responses side-by-side (A/B evaluation) and indicate which is better on one or more dimensions. Platforms such as Chatbot Arena (Chiang et al., 2024) collect pairwise preferences at scale from volunteer users, producing Elo-style rankings across models.

Human evaluation is expensive, slow, and exhibits systematic biases: annotators prefer longer responses (verbosity bias), responses that agree with their prior views (sycophancy), and responses from models they believe to be more capable (anchoring). Controlling for these biases requires careful experimental design and large sample sizes.

11.8.3 LLM-as-Judge

LLM-as-judge (Zheng et al., 2023) uses a capable language model (typically GPT-4) as an automated evaluator, replacing human annotators for A/B comparisons. The judge is prompted with the instruction, the two responses, and a structured rubric, and asked to produce a preference judgement with a brief explanation. MT-Bench and AlpacaEval 2.0 use this paradigm to rank instruction-tuned models.

LLM-as-judge inherits the biases of the judge model and introduces a **self-enhancement bias**: models from the same developer as the judge tend to be rated more favourably. Mitigations include using multiple judges, asking for chain-of-thought explanations before the final verdict, and position-swapping (evaluating each pair in both orders and averaging).

11.8.4 Reward Model Evaluation

When the downstream use of evaluation is to select between candidate models during RL training, a learned reward model r_ψ is used as the evaluator. The reward model's accuracy is measured as the fraction of held-out preference pairs on which it correctly predicts the

human-preferred response. As noted in Chapter 10, well-calibrated reward models achieve 70–80% accuracy on held-out human preferences. Beyond accuracy, the distribution of reward scores and their correlation with independent human ratings are monitored to detect systematic biases.

11.9 Summary

This chapter has introduced the language modelling infrastructure that underlies the RLHF methods developed in subsequent chapters.

A language model is an autoregressive probability distribution over token sequences, implemented using the Transformer architecture. Pre-training on web-scale text using next-token prediction instils broad world knowledge and emergent capabilities; supervised fine-tuning on curated instruction-response pairs teaches the behavioural format expected of an assistant. The KL-penalised RL alignment stage then optimises a reward signal — learned from human preference comparisons or verifiable task outcomes — to push the model beyond the ceiling set by its demonstrations.

The crucial conceptual bridge is the token-level MDP (Definition 11.3): language generation maps exactly onto the MDP framework of Chapters 3–9, with the language model as the policy, the vocabulary as the action space, and a terminal reward from a human or a reward model. This mapping justifies the direct application of policy-gradient methods to language model training.

Synthetic data has become an indispensable component of every post-training stage: Self-Instruct and Evol-Instruct for SFT instruction diversity, LLM-judge pipelines for preference data at scale, and reasoning distillation for chain-of-thought fine-tuning. Model collapse is a genuine risk but can be mitigated by mixing human data, using diverse teacher ensembles, and aggressive deduplication.

The chapters that follow develop the three main RL alignment algorithms in detail. Chapter 14 treats PPO-based RLHF, the pipeline pioneered by InstructGPT and used in GPT-4 and Claude 2. Subsequent chapters cover DPO, which eliminates the explicit reward model, and GRPO, which reduces variance through group-relative baseline subtraction and is central to the DeepSeek reasoning models. In each case, the token-level MDP established here provides the common formal language.

Exercises

Exercise 11.1. A language model assigns the following probabilities to the tokens of the sentence "The cat sat": $p(\text{The}) = 0.05$, $p(\text{cat} \mid \text{The}) = 0.02$, $p(\text{sat} \mid \text{The cat}) = 0.08$.

1. Compute the cross-entropy loss (11.4) for this sentence.
2. Compute the perplexity (11.5).
3. If a uniform distribution over a vocabulary of size $|\mathcal{V}| = 50{,}000$ is used instead, what is the perplexity? Compare and interpret.

Exercise 11.2. Consider the token-level MDP (Definition 11.3) with vocabulary size $|\mathcal{V}| = 100{,}000$ and a maximum response length of $T = 512$ tokens.

1. Give an upper bound on $|\mathcal{S}|$, the number of possible states. How many bits of memory would be required to store a lookup table $Q(s, a)$ for all (s, a) pairs, assuming each entry requires 4 bytes?

2. Explain why value-based methods such as DQN are infeasible for this MDP.

3. The policy-gradient theorem (Chapter 8) requires only $\nabla_\theta \log \pi_\theta(a \mid s)$, not the Q-table. Why does this make policy-gradient methods tractable despite the enormous state-action space?

Exercise 11.3. The causal mask in a decoder-only Transformer ensures that position i cannot attend to positions $j > i$.

1. Write the masked attention weight α_{ij} as a formula using the indicator function $\mathbb{1}[j \leq i]$.

2. Explain why this mask is equivalent to the Markov property of the token-level MDP: the prediction of y_t depends only on $(x, y_{1:t-1})$ and not on any future token.

3. Would the causal mask be necessary for a language model used only as a *classifier* (predicting a label for an entire sequence)? Justify your answer.

Exercise 11.4. The exposure bias problem arises because SFT uses teacher forcing during training but relies on the model's own outputs at inference time.

1. Formalise the mismatch: let p_{teacher} be the teacher-forcing training distribution over contexts at step t and p_{model} the corresponding inference distribution. Show that they coincide only if the model is perfect (assigns probability 1 to the correct next token at every step).

2. Scheduled sampling (Bengio et al., 2015) replaces the ground-truth token with the model's own prediction with probability ϵ_t, where ϵ_t increases from 0 to 1 during training. Describe one advantage and one disadvantage of this approach compared to standard teacher forcing.

3. Explain why RL fine-tuning (which trains on the model's own rollouts) does not suffer from exposure bias.

Exercise 11.5. The Chinchilla scaling law states that compute-optimal training requires $D \approx 20N$ tokens for a model with N parameters. Assume a compute budget of $C = 6 \times 10^{23}$ FLOPs and that one training FLOP costs approximately $C \approx 6ND$ FLOPs.

1. Derive the compute-optimal model size N^* and token count D^* as functions of C.

2. With the budget above, compute N^* and D^* numerically. Compare to the publicly known configuration of LLaMA 3 8B (8B parameters, 15T tokens). Is LLaMA 3 8B compute-optimal according to Chinchilla?

3. The Chinchilla analysis minimises training loss. Argue why a practitioner optimising for *inference cost* might rationally choose a smaller, more overtrained model.

Exercise 11.6. The Self-Instruct pipeline (Wang et al., 2023) seeds instruction generation with 175 human-written examples and grows the dataset by prompting the teacher model.

1. Describe one mechanism by which the quality of the seed set affects the quality of the generated dataset.

2. ROUGE-L similarity is used to filter near-duplicate instructions. Give one example of two instructions that ROUGE-L would flag as similar but that are genuinely distinct in their cognitive demands, and vice versa (two instructions that ROUGE-L scores as dissimilar but are semantically nearly identical).

3. Suppose the teacher model used in Self-Instruct has a systematic bias toward certain response styles (e.g., always using bullet points). How would this bias propagate to the student model trained on the resulting data?

Exercise 11.7. Model collapse refers to the progressive narrowing of the model's output distribution when trained iteratively on its own synthetic outputs.

1. Let p_0 be the original human data distribution, and let $\hat{p}_g$ be the distribution learned at generation g, approximated by training on samples from $\hat{p}_{g-1}$. Assume that each training step introduces an approximation error bounded by ϵ in total variation distance: $\mathrm{TV}(\hat{p}_g, p_{g-1}) \leq \epsilon$. Give an upper bound on $\mathrm{TV}(\hat{p}_G, p_0)$ after G generations.

2. Shumailov et al. (2024) showed that the tails of the distribution (rare but important content) are lost first. Explain intuitively why this is the case using the variance interpretation of model collapse.

3. Propose a data mixing strategy that provably prevents the total-variation bound from growing without bound as $G \to \infty$, assuming a fraction λ of the training data at each generation is genuine human data.

Exercise 11.8. LLM-as-judge uses a capable model to evaluate responses, replacing human annotators.

1. Describe the *verbosity bias* in LLM-as-judge systems and propose a prompt-engineering mitigation.

2. Position bias refers to the tendency of LLM judges to prefer the response presented first (or second). Design an evaluation protocol that eliminates position bias, and explain why your protocol is unbiased.

3. Suppose a reward model trained on LLM-judge-generated preferences is subsequently used in PPO to produce an aligned model. Trace how a systematic bias in the judge model propagates through the pipeline to affect the final aligned model's behaviour.

Exercise 11.9. Constitutional AI (Bai et al., 2022) uses a set of principles to guide self-critique and revision.

1. Write out the critique-revision loop as a two-step procedure. What is the role of the constitution? What distinguishes this from standard SFT?

2. RLAIF uses the same principle but at the preference labelling stage: the model is asked to judge which of two responses better adheres to the constitution. Show that if the judge model is perfect (always picks the truly better response), then RLAIF and human RLHF produce the same optimal policy under the Bradley–Terry model (Chapter 10).

3. What is the main failure mode of RLAIF when the judge model is imperfect, and how does it relate to reward overoptimisation?

Exercise 11.10. Consider a simple language generation task: the model must produce a binary string of length $T = 3$ (e.g., "010"), where the reward is $+1$ if the number of ones equals the number of zeros and -1 otherwise.

1. Write out the complete token-level MDP: enumerate all states, actions, transitions, and rewards.

2. Compute the optimal policy π^* using dynamic programming (the problem is small enough for a manual computation).

3. For $T = 3$, there are $2^3 = 8$ possible responses. Show that the RLHF objective (11.7) equals $\mathbb{P}_{\pi_\theta}(\text{balanced string}) - \mathbb{P}_{\pi_\theta}(\text{unbalanced string})$, and compute its value under the uniform policy $\pi_\theta(0 \mid s) = \pi_\theta(1 \mid s) = 0.5$ for all s.

Chapter 12

Direct Preference Optimisation and Alignment Methods

The simplest solution to a problem is often the one that rearranges what you already know, rather than adding new machinery.

The RLHF pipeline described in Chapter 10 involves four distinct neural networks: the reference policy, the policy being trained, the reward model, and (in PPO) a value function critic. Training all four simultaneously demands extraordinary engineering effort. The reward model must be trained, evaluated, and frozen before RL begins. The critic must be initialised and kept synchronised with the actor. GPU memory must accommodate at least two full model copies — often more when using PPO's clipped surrogate or computing the KL penalty against the reference. In practice, this complexity is a significant barrier: RLHF with PPO at the scale of GPT-4 or Claude requires sophisticated distributed training infrastructure, careful hyperparameter tuning across multiple stages, and vigilant monitoring for reward hacking.

Direct Preference Optimisation (Rafailov et al., 2023) asks a subversive question: do we need the reward model at all? The answer, as Chapter 10 hinted in Remark 10.2, is no. The optimal policy under the KL-regularised RLHF objective is related to the reward function through an invertible reparametrisation. Substituting this reparametrisation into the Bradley–Terry preference model causes the intractable partition function to cancel, leaving a loss that depends only on the policy and the reference policy — quantities that can be computed with a single forward pass. The reward model is eliminated entirely.

This chapter develops the theory and practice of DPO and the broader family of *direct alignment algorithms* that it inspired. Section 12.1 explains the motivation for bypassing the reward model. Section 12.2 presents the full derivation of the DPO objective from first principles, carrying every algebraic step. Section 12.3 describes the practical training algorithm, data requirements, and implementation details. Section 12.4 covers the principal variants — IPO, KTO, cDPO, and ORPO — each of which addresses a distinct limitation of the base algorithm. Section 12.5 analyses the structural weaknesses of all offline direct alignment methods, including the distribution shift problem and the absence of online exploration. Section 12.6 explains why online RL remains essential in production systems and describes the engineering patterns that make it tractable. Section 12.7 presents a comprehensive comparison of all major alignment methods, from PPO to GRPO, across the dimensions that matter most in practice. Section 12.8 summarises the chapter.

12.1 From RLHF to Direct Alignment

12.1.1 The Cost of the Reward Model

The standard RLHF pipeline has three stages. First, a supervised fine-tuned (SFT) policy π_{SFT} is trained on demonstration data. Second, a reward model r_ψ is trained on preference data $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}$ by minimising the Bradley–Terry loss (10.6). Third, the policy is fine-tuned with PPO against r_ψ subject to a KL penalty towards π_{SFT}. Each stage introduces its own engineering complexity and failure modes.

The reward model stage is particularly burdensome. It requires a separate set of hyperparameters (learning rate, batch size, margin), a separate evaluation protocol (pairwise accuracy on a held-out set), and a deployment step that makes the trained model available as a reward signal during RL. The reward model is typically the same size as the policy, meaning it must fit in GPU memory alongside the policy, the reference policy, and the PPO value function — a total of up to four model copies. At the 70B parameter scale, this approaches the practical limit of even the largest GPU clusters.

Beyond memory, the reward model is a source of instability. If the reward model is undertrained, it will not provide a useful gradient signal and the policy will stall. If it is overtrained, the policy will rapidly find and exploit its failure modes (reward hacking). Since the reward model is frozen during PPO, there is no mechanism for it to adapt as the policy distribution shifts away from the data on which the reward model was trained — a form of covariate shift that grows more severe as training progresses.

12.1.2 Simplicity, Memory, and Stability

A direct alignment algorithm replaces the two-stage reward-model-then-RL pipeline with a single supervised-style objective computed directly on preference data. The advantages are significant.

Memory footprint DPO requires only two model copies: the policy being trained and the frozen reference policy. This halves the memory requirement compared to PPO, which needs the policy, the reference, the reward model, and the value function critic.

Training stability The DPO loss is a binary cross-entropy objective. It is smooth, has bounded gradients, and does not suffer from the instabilities associated with PPO's clipped surrogate objective or the moving reward signal of online RL.

No reward hacking Because there is no explicit reward model to overfit, DPO cannot exhibit the classic reward hacking failure mode. The policy cannot exploit the gap between a proxy reward model and the true human preference, because the preference signal is incorporated directly into the loss.

Simplicity of implementation DPO reduces to approximately twenty lines of PyTorch. There is no need for a rollout buffer, a critic network, advantage estimation, or PPO clipping. The entire pipeline resembles supervised fine-tuning, making it accessible to practitioners without RL infrastructure.

These advantages come at a cost, which Sections 12.5 and 12.6 examine in detail. The critical limitation is that DPO is fundamentally an *offline* method: it trains on a fixed preference dataset collected under a different (typically the SFT) policy. As the trained policy moves away from the reference, the preference data becomes stale, and the implicit reward signal ceases to accurately reflect what the improved policy would actually prefer. This distribution shift problem is unavoidable in any offline direct alignment algorithm, and it motivates the continued relevance of online RL described in Section 12.6.

12.2 Deriving DPO

12.2.1 The KL-Constrained Objective

We begin from the KL-regularised RLHF objective introduced in Chapter 10. Given a prompt x and a reward function r, the objective is

$$J(\pi_\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r(x,y)\big] - \beta\, D_{\mathrm{KL}}(\pi_\theta(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x))\,, \tag{12.1}$$

where $\beta > 0$ controls the strength of the KL penalty and π_{ref} is the SFT reference policy. The objective trades off reward maximisation against staying close to the reference. We showed in Theorem 10.4 that the unique optimal policy is

$$\pi^*(y \mid x) = \frac{\pi_{\mathrm{ref}}(y \mid x)\, \exp\!\big(r(x,y)/\beta\big)}{Z(x)}, \tag{12.2}$$

where the partition function is $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x)\, \exp(r(x,y)/\beta)$.

12.2.2 The Optimal Policy Reparametrisation

Taking logarithms of both sides of (12.2) and rearranging yields the *reward reparametrisation*:

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x). \tag{12.3}$$

This identity expresses the reward in terms of the optimal policy and the partition function. Two features of this identity are crucial.

First, the identity holds for *any* reward function r and its corresponding optimal policy π^*. It is not an approximation; it is an exact algebraic consequence of the optimality conditions.

Second, the partition function $Z(x)$ depends only on the prompt x, not on the specific response y. This means that when we compute the *difference* in reward between two responses y_w and y_l to the same prompt x, the partition function cancels:

$$r(x,y_w) - r(x,y_l) = \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}. \tag{12.4}$$

The intractable $\log Z(x)$ term has vanished.

12.2.3 Substituting into the Bradley–Terry Model

Recall the Bradley–Terry preference model from Chapter 10: the probability that a human prefers y_w over y_l given prompt x is

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma\big(r(x,y_w) - r(x,y_l)\big). \tag{12.5}$$

Substituting the reward difference (12.4) into (12.5), and replacing π^* with the parametric policy π_θ (since π^* is what we are trying to learn), gives

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right). \tag{12.6}$$

This is the DPO preference probability. It depends only on the log-ratios of the policy to the reference, which can be computed with two forward passes through the language model (one for y_w and one for y_l).

12.2.4 The DPO Loss

Given a dataset $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^N$ of human preferences, the negative log-likelihood of the observed preferences under the model (12.6) is the DPO loss:

Definition 12.1 (DPO Loss)**.** The **Direct Preference Optimisation (DPO) loss** is

$$\boxed{\mathcal{L}_{\text{DPO}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \sigma\left(\beta \log \frac{\pi_\theta(y_w^{(i)} \mid x^{(i)})}{\pi_{\text{ref}}(y_w^{(i)} \mid x^{(i)})} - \beta \log \frac{\pi_\theta(y_l^{(i)} \mid x^{(i)})}{\pi_{\text{ref}}(y_l^{(i)} \mid x^{(i)})}\right).} \tag{12.7}$$

The DPO loss is a binary cross-entropy loss where the "logit" for each pair is the difference in implicit log-reward between the preferred and dispreferred responses. Minimising $\mathcal{L}_{\text{DPO}}$ is equivalent to maximising the likelihood of the observed preference annotations under the reparametrised Bradley–Terry model.

Proposition 12.1 (Gradient structure of the DPO loss)**.** *For a single preference pair (x, y_w, y_l), let*

$$h_\theta = \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}$$

be the implicit reward difference. The gradient of $\mathcal{L}_{\text{DPO}}$ with respect to θ is

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta\, \sigma(-h_\theta)\big[\nabla_\theta \log \pi_\theta(y_w|x) - \nabla_\theta \log \pi_\theta(y_l|x)\big]. \tag{12.8}$$

The factor $\sigma(-h_\theta) \in (0, 1)$ weights the update: it is large (close to 1) when the policy currently assigns similar implicit reward to both responses (the pair is hard), and small (close to 0) when the policy already strongly prefers y_w (the pair is easy). The gradient increases the log-probability of y_w and decreases the log-probability of y_l, weighted by how uncertain the current model is.

Proof. Let $\mathcal{L} = -\log \sigma(h_\theta)$. Since $\frac{d}{dh} \log \sigma(h) = \sigma(-h) = 1 - \sigma(h)$, we have

$$\nabla_\theta \mathcal{L} = -\sigma(-h_\theta)\, \nabla_\theta h_\theta.$$

Computing $\nabla_\theta h_\theta$: since the reference model π_{ref} has no dependence on θ,

$$\nabla_\theta h_\theta = \beta\, \nabla_\theta \log \pi_\theta(y_w|x) - \beta\, \nabla_\theta \log \pi_\theta(y_l|x).$$

Combining gives (12.8). □

Remark 12.1 (DPO as supervised fine-tuning with contrastive weighting). Equation (12.8) reveals that DPO is structurally identical to two simultaneous supervised fine-tuning updates: a positive (likelihood-increasing) update on y_w and a negative (likelihood-decreasing) update on y_l, both scaled by the per-pair confidence weight $\sigma(-h_\theta)$. This makes DPO amenable to the same implementation tricks as SFT: gradient checkpointing, mixed-precision training, LoRA adapters, and DeepSpeed ZeRO optimisation all apply without modification.

Example 12.1 (DPO loss computation)**.** Suppose that for a single pair (x, y_w, y_l), the log-probabilities under the current policy and the reference are:

$$\log \pi_\theta(y_w|x) = -3.2, \quad \log \pi_{\text{ref}}(y_w|x) = -3.5, \quad \log \pi_\theta(y_l|x) = -2.8, \quad \log \pi_{\text{ref}}(y_l|x) = -2.6.$$

With $\beta = 0.1$, the implicit reward difference is

$$h_\theta = 0.1 \cdot (-3.2 - (-3.5)) - 0.1 \cdot (-2.8 - (-2.6)) = 0.1 \cdot (0.3) - 0.1 \cdot (-0.2) = 0.03 + 0.02 = 0.05.$$

The loss for this pair is $-\log \sigma(0.05) = -\log(0.5125) \approx 0.669$. The confidence weight is $\sigma(-0.05) \approx 0.4875$, indicating the pair is nearly maximally uncertain and the gradient update will be large. After training, if h_θ reaches 3, the loss becomes $-\log \sigma(3) \approx 0.049$ and the confidence weight drops to $\sigma(-3) \approx 0.047$, reflecting that the policy has learned to clearly prefer y_w.

12.3 The DPO Algorithm

12.3.1 Training Procedure

The DPO training procedure is remarkably streamlined. It requires two model instances — the trainable policy π_θ and the frozen reference policy π_{ref} — and a preference dataset. The reference policy is typically the SFT checkpoint, frozen at the start of DPO training.

Algorithm 24: Direct Preference Optimisation (DPO)

Input: SFT checkpoint π_{ref}; preference dataset $\mathcal{D}$; KL coefficient β; learning rate α; batch size B

Output: Aligned policy π_θ

Initialise $\pi_\theta \leftarrow \pi_{\text{ref}}$;
Freeze π_{ref} (no gradient updates);
for *each mini-batch* $\{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^B \subset \mathcal{D}$ **do**
  for *each pair* $i = 1, \ldots, B$ *in parallel* **do**
    Compute $\ell_w^{(i)} \leftarrow \log \pi_\theta(y_w^{(i)}|x^{(i)})$ and $\ell_l^{(i)} \leftarrow \log \pi_\theta(y_l^{(i)}|x^{(i)})$;
    Compute $\hat{\ell}_w^{(i)} \leftarrow \log \pi_{\text{ref}}(y_w^{(i)}|x^{(i)})$ and $\hat{\ell}_l^{(i)} \leftarrow \log \pi_{\text{ref}}(y_l^{(i)}|x^{(i)})$ (no grad);
    Compute $h^{(i)} \leftarrow \beta(\ell_w^{(i)} - \hat{\ell}_w^{(i)}) - \beta(\ell_l^{(i)} - \hat{\ell}_l^{(i)})$;
  Compute $\mathcal{L} \leftarrow -\frac{1}{B} \sum_{i=1}^B \log \sigma(h^{(i)})$;
  $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$;
  Log: mean reward margin $\frac{1}{B} \sum_i h^{(i)}$; fraction correct $\frac{1}{B} \sum_i \mathbb{1}[h^{(i)} > 0]$;
return π_θ

12.3.2 Data Requirements

DPO requires a preference dataset of the form $\mathcal{D} = \{(x, y_w, y_l)\}$. Several practical considerations govern the quality and utility of this data.

Response quality gap. The DPO loss is most informative when there is a clear quality gap between y_w and y_l. If both responses are nearly identical, h_θ will be close to zero for any reasonable policy, and the gradient update will push in an arbitrary direction. Pairs should be selected so that y_w and y_l exhibit meaningfully different quality levels.

Source of responses. The original DPO paper samples both y_w and y_l from π_{ref} (the SFT model). This is the most coherent choice for offline DPO: the training responses are drawn from the same distribution as the reference policy, so the implicit reward signal is well-calibrated. In practice, many datasets (such as Anthropic's HH-RLHF or OpenAI's WebGPT comparisons) pair model-generated responses with human-written alternatives or with responses from a weaker model — a valid approach, though it introduces an additional distribution mismatch.

Prompt distribution. The prompts x should reflect the target deployment distribution. A DPO-trained model will behave well on prompts similar to those in $\mathcal{D}$ and may degrade on out-of-distribution prompts — the same generalisation caveat that applies to all supervised methods.

12.3.3 Implementation Details

Log-probability computation. The sequence log-probability $\log \pi_\theta(y|x) = \sum_{t=1}^{|y|} \log \pi_\theta(y_t|x, y_{<t})$ is computed by concatenating the prompt and response, running a single forward pass, and summing the per-token log-probabilities at the response positions.

Length normalisation. Responses of different lengths will have log-probabilities that differ simply due to length. To prevent the policy from learning a trivial length shortcut — assigning higher log-probability to shorter responses regardless of quality — it is standard practice to divide by response length: $\bar{\ell} = \log \pi_\theta(y|x)/|y|$.

Reference logit caching. The reference model π_{ref} is frozen throughout training. The log-probabilities $\log \pi_{\text{ref}}(y_w|x)$ and $\log \pi_{\text{ref}}(y_l|x)$ can therefore be precomputed once and cached, eliminating the need to run the reference model forward pass during training. This saves approximately half the compute per training step.

Hyperparameters. The principal hyperparameters are $\beta \in [0.01, 0.5]$ (the KL coefficient, controlling how far the policy moves from the reference; smaller β allows more deviation), the learning rate (typically 1×10^{-6} to 5×10^{-6} for full fine-tuning, one to two orders of magnitude smaller than SFT), and the number of epochs (usually 1–3; more epochs risk overfitting the preference data).

12.3.4 The DPO vs. PPO Pipeline: A Diagram

The following diagram contrasts the full PPO-RLHF pipeline (top) with the DPO pipeline (bottom), illustrating the components that DPO eliminates.

12.4 Variants of DPO

The DPO objective, while elegant, makes specific assumptions about the preference model and the nature of the training data. Several variants have been proposed to relax these assumptions and address practical limitations.

12.4.1 IPO: Identity Preference Optimisation

DPO is derived by maximising the log-likelihood under the Bradley–Terry model, which is a *soft* preference model: even deterministic preferences (one response is always chosen) yield a finite loss at any finite policy. However, Azar et al. (2023) observed that if preferences are perfectly consistent — a realistic idealisation when using an automated preference oracle — the DPO optimum degenerates: the policy is incentivised to increase the reward margin

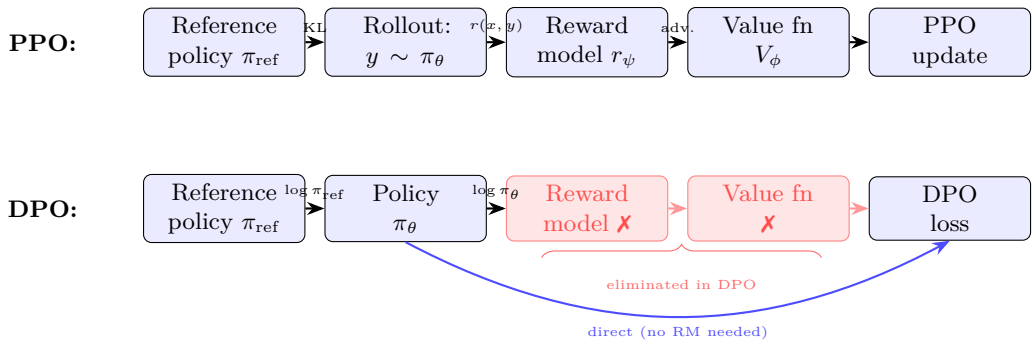


Figure 12.1: PPO (top) vs. DPO (bottom) training pipelines. The PPO pipeline requires four components: the reference policy (for KL), online rollouts, a reward model r_ψ, and a value function critic V_ϕ. DPO eliminates the reward model and value function, computing the alignment signal directly from log-probability ratios.

h_θ without bound, pushing log-probabilities towards $+\infty$ and $-\infty$. This *overfitting to deterministic preferences* manifests as degenerate output distributions.

The Identity Preference Optimisation (IPO) loss addresses this by replacing the log-sigmoid with a squared-error penalty:

$$\mathcal{L}_{\text{IPO}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(\log\frac{\pi_\theta(y_w^{(i)}|x^{(i)})}{\pi_{\text{ref}}(y_w^{(i)}|x^{(i)})} - \log\frac{\pi_\theta(y_l^{(i)}|x^{(i)})}{\pi_{\text{ref}}(y_l^{(i)}|x^{(i)})} - \frac{1}{2\beta}\right)^2. \tag{12.9}$$

The target reward margin is $1/(2\beta)$: the policy should learn to assign an implicit reward difference of exactly $1/(2\beta)$ between preferred and dispreferred responses, no more. The squared loss provides a restoring force that penalises both under- and over-separation, preventing the unbounded-margin failure mode. IPO can be derived directly from the original KL-constrained objective without passing through the Bradley–Terry model, making it applicable even when the preference model is not logistic.

12.4.2 KTO: Kahneman-Tversky Optimisation

A fundamental requirement of DPO (and IPO) is *paired* preference data: for each prompt x, both a preferred response y_w and a dispreferred response y_l must be available. Collecting such paired data requires presenting two responses to an annotator simultaneously — a comparison interface that is more expensive and less natural than simply asking whether a single response is good or bad.

Ethayarajh et al. (2023) introduced Kahneman-Tversky Optimisation (KTO), which works with unpaired, binary feedback — each example (x, y, z) consists of a prompt, a response, and a binary label $z \in \{0, 1\}$ indicating whether the response is good ($z = 1$) or bad ($z = 0$). KTO derives its objective from *prospect theory* (Kahneman and Tversky, 1979), specifically the observation that humans are loss-averse: the pain of a loss exceeds the pleasure of an equivalent gain.

Definition 12.2 (KTO loss)**.** Let $\mathcal{D}_+ = \{(x, y) : z = 1\}$ and $\mathcal{D}_- = \{(x, y) : z = 0\}$. The KTO reference point is

$$z_{\text{ref}} = \mathbb{E}_{(x,y)\in\mathcal{D}}[\beta\, D_{\text{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))]\,,$$

approximated in practice by $\beta \log \pi_\theta(y|x)/\pi_{\text{ref}}(y|x)$ averaged over the batch. The **KTO loss** is

$$\mathcal{L}_{\text{KTO}}(\theta) = \mathbb{E}_{(x,y)\in\mathcal{D}_+}\big[1 - \sigma(\hat{r}_\theta(x,y) - z_{\text{ref}})\big] + \mathbb{E}_{(x,y)\in\mathcal{D}_-}\big[1 - \sigma(z_{\text{ref}} - \hat{r}_\theta(x,y))\big], \quad (12.10)$$

where $\hat{r}_\theta(x,y) = \beta \log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))$ is the implicit reward.

Intuitively, KTO encourages the implicit reward of positive examples to exceed the reference point (so the human "gains" utility) and the implicit reward of negative examples to fall below it (so the human "loses" — but the loss-averse framing means this is penalised asymmetrically). KTO is particularly attractive when preference datasets are collected through thumbs-up/thumbs-down interfaces, which are far more common at scale than side-by-side comparison interfaces.

12.4.3 cDPO: Conservative DPO

Human preference annotations are noisy: annotators sometimes prefer the objectively worse response due to superficial features, time pressure, or inconsistency. Standard DPO treats all annotations with equal weight, which means a label-flipped pair — where y_w is actually worse than y_l — will actively push the policy in the wrong direction.

Conservative DPO (cDPO), proposed by Mitchell et al. (2023), addresses this by incorporating a label smoothing parameter $\epsilon \in [0, 0.5)$ into the DPO objective. Rather than treating each preference as a hard assignment, cDPO treats it as a soft signal with noise probability ϵ:

$$\mathcal{L}_{\text{cDPO}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\Big[(1-\epsilon)\log\sigma(h_\theta^{(i)}) + \epsilon \log\sigma(-h_\theta^{(i)})\Big], \quad (12.11)$$

where $h_\theta^{(i)}$ is the implicit reward difference defined in Definition 12.1. When $\epsilon = 0$, cDPO reduces exactly to DPO. When $\epsilon > 0$, the loss adds a small penalty for being too confident that y_w is better, which regularises the policy against overfitting to noisy labels. The cDPO objective is equivalent to the label-smoothed reward model loss (10.10) applied in the DPO reparametrisation.

12.4.4 ORPO: Odds Ratio Preference Optimisation

All of the variants above separate the alignment objective from the language modelling objective: DPO trains on preference pairs, while the base SFT training is a separate prior stage. Hong et al. (2024) proposed Odds Ratio Preference Optimisation (ORPO), which unifies the two objectives into a single loss and eliminates the need for a reference model entirely.

Definition 12.3 (ORPO loss)**.** The **ORPO loss** combines a supervised negative log-likelihood term on the preferred response with an odds-ratio penalty:

$$\mathcal{L}_{\text{ORPO}}(\theta) = -\log \pi_\theta(y_w|x) - \lambda \log\sigma\left(\log \frac{\text{odds}_\theta(y_w|x)}{\text{odds}_\theta(y_l|x)}\right), \quad (12.12)$$

where $\text{odds}_\theta(y|x) = \pi_\theta(y|x)/(1-\pi_\theta(y|x))$ is the odds of the response under the policy, and $\lambda > 0$ is a mixing coefficient.

The first term ensures the model continues to generate fluent, coherent text (supervised fine-tuning on the preferred response). The second term penalises cases where the policy's odds of generating y_l are comparable to or exceed its odds of generating y_w. Because ORPO uses the *odds ratio* rather than a log-ratio involving a reference model, no separate reference model is needed: the regularisation is implicit in the structure of the loss.

ORPO is particularly attractive for very large models where maintaining a reference model copy is memory-prohibitive, and for settings where the model is being trained from scratch on preference data without a prior SFT stage.

12.5 Limitations of Direct Alignment

12.5.1 The Distribution Shift Problem

The fundamental limitation of DPO and all offline direct alignment algorithms is *distribution shift*. The preference dataset $\mathcal{D}$ is collected by presenting responses from some data-generating policy μ — typically the SFT model π_{ref} — to human annotators. The implicit reward signal in DPO is calibrated to this distribution: the log-ratio $\log(\pi_\theta/\pi_{\text{ref}})$ is small (and the loss gradient is large) precisely for responses that are in the support of μ.

As training proceeds, π_θ moves away from π_{ref}. Responses that were originally probable under μ become progressively less likely under π_θ, and new responses that π_θ has learned to favour may not appear anywhere in $\mathcal{D}$. The DPO loss cannot provide any learning signal for responses outside its training distribution. This means that as the policy improves, it explores regions of response space where the preference labels it has been given are increasingly uninformative.

Empirically, this manifests as a characteristic training curve: the DPO reward margin h_θ increases rapidly during the first few thousand steps as the policy learns the clear-cut preference distinctions in the data, then plateaus as the policy exhausts the information content of the fixed dataset. Further training yields diminishing returns, and continued training past this point can even cause the policy to *degrade* — increasing the implicit reward margin for the training pairs while worsening performance on held-out prompts, a form of preference overfitting.

12.5.2 No Online Exploration

Related to distribution shift is the absence of online exploration. In online RL methods such as PPO, the policy continuously generates new responses, collects rewards, and updates. This allows the policy to explore the response space and collect feedback on the very responses it is currently likely to generate. The preference signal is always on-policy.

DPO has no such mechanism. It operates entirely on a fixed, pre-collected dataset. If there exist high-quality responses that the SFT model would not have generated (and therefore do not appear in $\mathcal{D}$), DPO cannot discover them. This is particularly consequential for tasks that require compositional reasoning: a model that has learned to be slightly more accurate on step-by-step reasoning problems may now benefit from feedback on the specific reasoning patterns it is generating, which differ from those in the fixed training set.

12.5.3 Mode-Seeking vs. Mode-Covering

The Bradley–Terry model and the DPO loss are derived from a *mode-seeking* perspective: the loss encourages the policy to assign high probability to preferred responses and low

probability to dispreferred ones. In the limit, this pushes the policy towards a *point mass* on the highest-reward response for each prompt.

This mode-seeking tendency is sometimes desirable (for tasks with a unique correct answer, such as mathematical problem solving) but frequently undesirable (for tasks requiring diversity, such as creative writing or open-ended conversation). A mode-seeking policy may produce responses that score highly on preference metrics while being repetitive and lacking the variety that real users want. By contrast, online RL methods that use entropy regularisation or diversity bonuses can be tuned to produce more mode-covering distributions.

12.5.4 When DPO Fails

Based on the analysis above, DPO tends to underperform relative to online RL in the following scenarios.

- **Long training horizons**: As more preference data is consumed and the policy diverges from the reference, distribution shift accumulates and performance plateaus or regresses.
- **Tasks with verifiable correctness**: For mathematical reasoning or coding, an online verifier (unit tests, symbolic checker) provides a stronger and more reliable signal than static human preference annotations.
- **High-stakes alignment**: When the goal is to produce a model that is safe and beneficial *across the full deployment distribution*, online feedback from real users provides signal that a fixed dataset cannot anticipate.
- **Continued improvement**: DPO converges to the optimal policy for the given fixed dataset. Further improvement requires new data, whereas online RL can keep improving as long as the environment provides feedback.

12.6 Online RL in Production

12.6.1 Why Online RL Matters

The distinction between offline and online alignment is not merely academic. A production language model serves millions of users, generating billions of responses across an enormous diversity of topics, styles, and intents. No static preference dataset, however large, can anticipate this diversity. Users will ask questions the dataset collectors never imagined, request styles that differ from the training distribution, and find failure modes that only emerge at scale.

Online reinforcement learning closes this loop. Rather than training once on a fixed dataset and deploying, an online RL system continuously collects feedback from real interactions — explicit (thumbs up/down, star ratings, preference choices) or implicit (session duration, follow-up questions, copy rate) — and updates the policy. The model's behaviour improves as it serves users, and the improvement is targeted at the actual deployment distribution, not a proxy dataset.

This is precisely the deployment pattern of the leading commercial AI assistants. The GPT-4 technical report (OpenAI, 2023) describes continuous RLHF updates based on user feedback. Anthropic's Constitutional AI pipeline (Bai et al., 2022) uses RLHF with on-policy sampling to ensure the reward signal is calibrated to the current policy's output distribution. These systems do not treat alignment as a one-time training event; they treat it as a continuous production process.

12.6.2 On-Policy vs. Off-Policy Considerations

In the online setting, a critical design decision is whether to use *on-policy* or *off-policy* feedback. On-policy feedback is collected from the current policy: the model generates a response, the user provides feedback, and the model is updated on that feedback before generating the next response. Off-policy feedback is collected from an earlier version of the policy (or a different policy altogether) and is used to update the current policy.

On-policy feedback is the gold standard for alignment. The reward signal is guaranteed to be on the distribution the policy is currently exploring, so there is no distribution shift. The update is equivalent to policy gradient with a sample-based advantage estimate. However, on-policy feedback is slow: feedback must be collected and processed before the next update, which limits the throughput of the training loop.

Off-policy feedback (such as DPO on stored preference data) enables much higher throughput, as training is decoupled from collection. The cost is the distribution shift discussed in Section 12.5. In practice, production systems typically use a hybrid: a large off-policy replay buffer of historical feedback combined with a smaller stream of on-policy feedback, with importance weighting to correct for the distribution mismatch.

12.6.3 Catastrophic Forgetting Mitigation

A persistent challenge in online RL is *catastrophic forgetting*: as the policy is updated to improve on alignment, it may degrade on capabilities that were present in the SFT model. The model may become less fluent, less factually accurate, or less capable at tasks not represented in the current feedback stream.

Several techniques mitigate catastrophic forgetting in production.

KL anchoring The KL penalty $\beta D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ in the RL objective keeps the policy from drifting too far from the SFT checkpoint. However, if the SFT checkpoint itself is periodically updated (as is common in production), the anchor moves, potentially allowing slow but unbounded drift.

Capability constraints Separate evaluations on held-out capability benchmarks (MMLU, HumanEval, GSM8K) are run after each RL update. If any capability drops below a threshold, the update is rolled back or the KL coefficient is increased. This requires dedicated evaluation infrastructure but is the most principled safeguard.

Replay buffers Mixing old SFT examples with the online RL gradient ensures the policy continues to receive supervised signal on the full capability distribution. This is analogous to experience replay in DQN: the SFT examples act as "anchors" in experience space.

LoRA-based fine-tuning Restricting updates to a low-rank adapter (LoRA) rather than the full model parameters limits the degree of forgetting, since the base model weights remain fixed. Alignment is stored in the adapter; capabilities are preserved in the base model.

12.6.4 Safety Constraints in Online RL

Online RL introduces a safety risk that offline alignment does not: the model is updated on real user feedback, which may include adversarial, biased, or otherwise harmful signal. A

production system must guard against several failure modes.

Reward gaming by users. If users discover that certain response patterns receive high rewards (e.g., responses that agree with any user position, responses that are especially verbose), they may exploit this to steer the model's behaviour. Adversarial robustness in the reward signal requires anomaly detection and filtering of outlier feedback.

Feedback loop instability. Positive feedback loops can cause the policy to drift rapidly towards a local mode. The KL penalty, combined with periodic evaluation against a held-out diverse prompt set, is the standard stabiliser.

Constrained policy optimisation. For safety-critical properties (refusal of harmful requests, factual accuracy), one can augment the RL objective with Lagrangian constraints (Achiam et al., 2017; Bai et al., 2022): the policy optimises reward subject to the constraint that safety-relevant metrics remain above a floor. This converts the unconstrained RL problem into a constrained optimisation problem, solvable with dual-variable methods.

12.6.5 PPO vs. DPO in the Online Setting

The comparison between PPO and DPO is qualitatively different in the online and offline settings. In the offline setting (fixed preference dataset), DPO is competitive with or superior to PPO on many benchmarks: it is simpler, more stable, and requires less compute. In the online setting, PPO has a structural advantage that DPO cannot match.

PPO collects feedback on-policy, updates the policy using that feedback, and repeats. The policy and the feedback source are always in sync. DPO, by contrast, requires a separate data collection phase before each training stage; each collection-training cycle is a discrete batch, not a continuous loop. This means that online DPO is effectively *iterative DPO*: collect preferences from the current policy, run DPO for a few epochs, collect new preferences from the updated policy, and repeat. Iterative DPO substantially closes the gap with PPO by reducing distribution shift, but it introduces its own engineering overhead (repeated data collection and annotation) and is still inferior to PPO's fully online loop.

In practice, the choice between PPO and DPO in production depends on the available infrastructure. A team with dedicated RL infrastructure, a fast reward model (or verifier), and the capacity for online rollout should prefer PPO or a PPO variant (such as GRPO). A team without RL infrastructure, or one that needs to align a model quickly with an existing preference dataset, should prefer DPO or one of its variants.

12.6.6 Practical Deployment Patterns

Based on the analysis above and observed practice at major AI labs, the following deployment patterns have emerged.

Bootstrap with DPO Use DPO on an existing preference dataset to quickly produce an aligned model. This is the fastest path from SFT to a deployable, aligned model, suitable for initial releases.

Refine with online RL After deployment, collect on-policy feedback and run iterative DPO or PPO to continuously improve. The DPO-bootstrapped model provides a strong starting point with a relatively small KL budget available for further improvement.

Capability-safety separation Use offline DPO for safety alignment (which changes slowly and can be encoded in a static dataset) and online RL for help-

fulness alignment (which must adapt to the evolving deployment distribution).

Verifier-guided RL for reasoning For tasks with verifiable correctness (math, code), use online RL with an automated verifier as the reward signal rather than human preferences. This allows unlimited on-policy data collection with no human annotation cost, and the verifier is free of the biases that affect human annotators.

12.7 Comprehensive Comparison of Alignment Methods

12.7.1 Method Overview

Table 12.1 provides a comprehensive comparison of the major alignment methods for large language models. The methods span the full spectrum from fully online policy gradient approaches to purely offline supervised alignment.

Table 12.1: Comprehensive comparison of LLM alignment methods. "RM" = reward model; "ref. model" = reference policy required; "paired" = requires paired (y_w, y_l) data; "online" = generates new responses during training. Memory footprint relative to one model copy (1×). Stability: H = high, M = medium, L = low.

Method	RM?	Paired?	Online?	Mem.	Stab.	Best for
PPO	Yes	No	Yes	4×	M	Safety, RLHF at scale, online feedback
REINFORCE	Yes	No	Yes	2×	L	Simple tasks, baseline comparison
RLOO	Yes	No	Yes	2×	M	Low-resource online RL, no critic
GRPO	No*	No	Yes	2×	H	Reasoning, verifiable tasks, no critic
DPO	No	Yes	No	2×	H	Quick alignment, offline data
IPO	No	Yes	No	2×	H	Deterministic/auto preferences
KTO	No	No	No	2×	H	Binary feedback, no comparison pairs
cDPO	No	Yes	No	2×	H	Noisy preference labels
ORPO	No	Yes	No	1×	H	No ref. model, memory-constrained

*GRPO uses a verifier or rule-based reward rather than a learned RM.

12.7.2 Choosing an Alignment Method by Goal

The choice of alignment method should be driven by the deployment goal, the available data, and the available compute infrastructure. The following guidelines distil the preceding analysis into practical recommendations.

Table 12.2: Pros and cons of each alignment method.

Method	Pros	Cons
PPO	Fully online, handles complex reward signals, well-studied theoretically	High memory (4 models), complex implementation, reward hacking risk
REINFORCE	Simple, unbiased policy gradient, easy to implement	High variance, slow convergence, unstable without baseline
RLOO	Variance reduction via leave-one-out baseline, no critic needed	Requires group sampling, still on-policy complexity
GRPO	No critic, stable, excels on verifiable tasks, strong empirical results	Requires verifier or reward model; no natural extension to open-ended tasks
DPO	Simple, stable, SFT-like training, no RM, low memory	Offline only, distribution shift, no exploration, plateaus
IPO	Fixes DPO overfitting, regulates margin, more robust to clean preferences	Hyperparameter sensitive (β controls margin target), no on-policy signal
KTO	Works with binary labels (no pairs needed), natural for production feedback	Requires careful reference point estimation, less data-efficient than DPO
cDPO	Robust to label noise, natural regulariser, near-zero overhead over DPO	ϵ must be tuned; too large ϵ prevents alignment
ORPO	No reference model needed, SFT + alignment in one pass, memory-efficient	Less theoretically grounded; λ requires tuning; no KL regularisation

Safety alignment. For instilling safety constraints — refusing harmful requests, avoiding toxic output, maintaining constitutional principles — the primary concerns are coverage (the model should behave safely on the full deployment distribution) and stability (the constraints should not degrade over time). PPO with a safety-specific reward model and hard Lagrangian constraints is the most principled approach for flagship models. DPO or cDPO on a carefully curated safety-preference dataset is a practical and often sufficient alternative for smaller models or teams without RL infrastructure. KTO is particularly well-suited for safety alignment because safety labels are naturally binary (harmful / not harmful) rather than comparative.

Reasoning ability. For improving performance on mathematical, logical, or coding tasks, the key insight is that these tasks have verifiable answers. This means a rule-based reward signal (correct/incorrect) can be used in place of human preference annotations, enabling unlimited on-policy data collection. GRPO is the strongest choice here: it uses group sampling to estimate advantages without a critic, and the verifier-based reward provides a clean, unambiguous signal. The DeepSeek-R1 result (DeepSeek-AI, 2025) demonstrated that GRPO with a verifier-based reward can produce dramatic improvements in reasoning ability without any human preference data.

Instruction following. Instruction following — the ability to complete diverse user requests accurately and helpfully — requires both broad coverage (many instruction types)

and nuanced quality judgements (the difference between a good and great response to a given instruction). DPO on a large, diverse preference dataset (such as UltraFeedback or Alpaca Farm) is the workhorse method for instruction following. Iterative DPO — running multiple rounds of DPO with on-policy data collection between rounds — further improves coverage. PPO with a generalised reward model is the strongest option when online feedback is available.

Creative tasks. Creative tasks (story generation, poetry, open-ended dialogue) require diversity: the aligned model should produce varied, imaginative output rather than converging to a single mode. DPO's mode-seeking tendency makes it a poor fit unless augmented with diversity regularisation. PPO with an entropy bonus or a diversity-promoting reward function is preferable. KTO with positive labels on diverse creative examples can also encourage broad coverage without the mode-collapsing tendency of contrastive pair-based methods.

Resource-constrained settings. When memory or compute are severely limited, ORPO (which requires only one model copy) or DPO (two model copies) are the only feasible options. Both can be applied with LoRA adapters to reduce the trainable parameter count further, enabling alignment of large models on commodity hardware.

Example 12.2 (Choosing a method for a math tutoring assistant)**.** A team is developing a math tutoring assistant that should (a) solve problems correctly, (b) refuse to provide direct answers when a student should learn by doing, and (c) explain reasoning clearly. How should alignment be approached?

For goal (a), GRPO with a symbolic math verifier is ideal: responses can be verified automatically, on-policy data is unlimited, and no human annotation is needed.

For goal (b), a small safety-preference dataset of (problem, "give answer", "guide without answer") pairs can be aligned with DPO or KTO. The constraint is safety-like (binary: violates the no-direct-answer policy or not), making KTO particularly natural.

For goal (c), DPO on a dataset of human preference pairs (well-explained vs. poorly-explained solutions) is appropriate, supplemented by iterative DPO rounds using on-policy rollouts from the GRPO-improved model to avoid distribution shift.

In practice, these three objectives are combined: GRPO handles the verifiable correctness objective, KTO handles the safety-like pedagogical constraint, and DPO handles the stylistic quality dimension. Each method is applied in a separate stage, with capability evaluation between stages to monitor for forgetting.

12.8 Summary

This chapter developed the theory and practice of Direct Preference Optimisation and its relationship to the broader landscape of LLM alignment methods.

The central insight of DPO (Section 12.2) is a two-step algebraic manipulation: the KL-regularised RLHF optimal policy can be rearranged to express the reward as a log-ratio of the optimal policy to the reference policy plus a prompt-dependent partition function. When this expression is substituted into the Bradley–Terry preference probability, the partition function cancels, leaving a loss that depends only on policy log-ratios — quantities computable with two forward passes. The reward model is eliminated without approximation.

The DPO algorithm (Section 12.3) reduces to binary cross-entropy on policy log-ratios, with a confidence weight $\sigma(-h_\theta)$ that focuses learning on uncertain pairs. The TikZ diagram (Figure 12.1) illustrated the dramatic simplification relative to PPO: the reward model and value function are eliminated, reducing the required model copies from four to two.

The DPO variants (Section 12.4) each address a specific limitation. IPO prevents the unbounded-margin failure mode by imposing a squared target on the reward gap. KTO

extends alignment to unpaired binary feedback using prospect theory. cDPO introduces label smoothing to handle noisy annotations. ORPO eliminates the reference model entirely by using an odds-ratio penalty embedded within the supervised loss.

The distribution shift problem (Section 12.5) is the fundamental limitation shared by all offline direct alignment algorithms. As the policy moves away from the reference, the preference labels in the training set become progressively less informative, causing performance to plateau. The absence of online exploration means that DPO cannot discover the high-quality responses that the current policy is now capable of generating but which did not appear in the original dataset.

Online RL in production (Section 12.6) addresses these limitations by closing the loop between model behaviour and feedback. A production RL system continuously generates responses, collects feedback (explicit or implicit), and updates the policy. The principal engineering challenges are catastrophic forgetting (mitigated by KL anchoring, capability constraints, replay buffers, and LoRA), safety under adversarial feedback (mitigated by anomaly detection and constrained optimisation), and the PPO-vs-DPO tradeoff (resolved by infrastructure availability and the degree of distribution shift acceptable).

The comprehensive comparison table (Section 12.7) provides a practical reference for choosing an alignment method by goal: PPO and GRPO for online settings and verifiable tasks, DPO and its variants for offline settings with available preference data, KTO for binary feedback, and ORPO for memory-constrained environments.

Chapter 10 provides the foundation: the Bradley–Terry model, the KL-regularised objective, and the optimal policy reparametrisation that DPO exploits.

Chapter 14 describes the PPO fine-tuning pipeline in full engineering detail, including the per-token KL reward, advantage estimation, and practical tricks for stable training.

Chapter 16 discusses alignment in the agentic setting, where the model must be aligned not just on single-turn responses but on multi-step trajectories.

Exercises

Exercise 12.1. Carry out the full DPO derivation from scratch. Starting from the KL-regularised objective (12.1), prove that the optimal policy is (12.2). Then rearrange to obtain the reward reparametrisation (12.3). Show explicitly that the partition function $Z(x)$ cancels when computing the reward difference $r(x, y_w) - r(x, y_l)$. Finally, substitute into the Bradley–Terry formula to obtain the DPO loss (12.7). At each step, identify which assumptions are exact and which would need to be modified if the preference model were not Bradley–Terry.

Exercise 12.2. Verify Proposition 12.1 by explicit calculation. For a single preference pair with log-probabilities $\log \pi_\theta(y_w|x) = -2.0$, $\log \pi_{\text{ref}}(y_w|x) = -2.4$, $\log \pi_\theta(y_l|x) = -1.5$, $\log \pi_{\text{ref}}(y_l|x) = -1.3$, and $\beta = 0.1$:

1. Compute the implicit reward difference h_θ.
2. Compute the DPO loss for this pair.

3. Compute the confidence weight $\sigma(-h_\theta)$ and interpret it: is this pair easy or hard for the current policy?

4. Suppose $\log \pi_\theta(y_w|x)$ decreases by 0.3 (the policy becomes less likely to produce y_w). Recompute h_θ and the loss, and explain qualitatively how the gradient changes.

Exercise 12.3. The IPO loss (12.9) replaces the log-sigmoid with a squared loss.

1. Show that the DPO loss has no finite minimiser when preferences are perfectly consistent (i.e., when $y_w \succ y_l$ with probability 1). *Hint:* Consider what happens to $\mathcal{L}_{\text{DPO}}$ as $h_\theta \to +\infty$.

2. Show that the IPO loss (12.9) is minimised at $h_\theta^* = 1/(2\beta)$. Compute h_θ^* for $\beta = 0.1$ and interpret its value in terms of implicit reward gap.

3. For the numerical example in Exercise 12.2, compute the IPO loss and compare it to the DPO loss. Which is larger, and why?

Exercise 12.4. The KTO objective is motivated by prospect theory. Formally, the Kahneman-Tversky value function $v : \mathbb{R} \to \mathbb{R}$ is concave for gains ($v > 0$) and convex and steeper for losses ($v < 0$), with $v(0) = 0$.

1. Sketch the value function and explain why loss aversion implies $|v'(-\epsilon)| > |v'(\epsilon)|$ for small $\epsilon > 0$.

2. The KTO reference point z_{ref} plays the role of the zero of the value function. Explain intuitively why the implicit reward of a positive example should exceed z_{ref} and the implicit reward of a negative example should fall below it.

3. Show that KTO with $z_{\text{ref}} = 0$ and equal numbers of positive and negative examples reduces to a form closely related to DPO if the positive and negative examples are drawn from the same paired dataset.

Exercise 12.5. Show that the cDPO loss (12.11) can be derived from DPO by assuming that each preference label is flipped with probability ϵ. That is, the annotator labels $y_w \succ y_l$ with probability $1 - \epsilon$ and $y_l \succ y_w$ with probability ϵ. Taking expectations of the DPO loss over this stochastic labelling model yields cDPO. What value of ϵ corresponds to a completely uninformative annotator, and what does cDPO reduce to in that limit?

Exercise 12.6. Consider a stylised model of distribution shift in DPO. Suppose the preference dataset is collected from the reference policy π_{ref}, and after DPO training the policy π_θ has KL divergence $d = D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ from the reference.

1. Argue that the fraction of responses in π_θ that also appear in the support of π_{ref} (and hence are covered by the preference dataset) decreases as d increases.

2. In the worst case, if π_θ places all its mass on responses not in the support of π_{ref}, what can the DPO gradient tell the policy about the quality of those responses?

3. Propose a practical mitigation: how would iterative DPO reduce the severity of distribution shift? What is the residual distribution shift after k rounds of iterative DPO?

Exercise 12.7. A team is deploying a customer service chatbot. The chatbot must (a) resolve customer issues correctly, (b) respond politely even to abusive messages, and (c) escalate to a human agent when appropriate. Design an alignment training pipeline for this chatbot, addressing the following:

1. Which alignment method(s) would you use for each of the three objectives (a), (b), (c)?
2. How would you collect feedback in production: explicit, implicit, or both?
3. How would you prevent the online RL loop from degrading the model's language fluency or factual accuracy?
4. What safety mechanism would prevent adversarial users from systematically steering the policy?

Exercise 12.8. Examine the alignment method comparison in Table 12.1.

1. Explain why PPO requires 4× the memory of a single model while DPO requires only 2×.
2. A team has a dataset of 50,000 customer support interactions, each labelled as "resolved" or "not resolved", but with no paired comparisons between responses. Which alignment method would you recommend and why?
3. GRPO is listed as "H" (high) stability despite being an on-policy method. Explain why GRPO achieves higher stability than standard REINFORCE, referring to the group normalisation mechanism.
4. Under what conditions might DPO *outperform* PPO even in an online setting? Be specific about the data and infrastructure assumptions.

Exercise 12.9. Consider using DPO to improve a language model's mathematical reasoning ability. The preference dataset consists of pairs (y_w, y_l) where y_w is a correct chain-of-thought solution and y_l is an incorrect one, both generated by the SFT model.

1. After DPO training, the model's accuracy on in-distribution math problems improves from 45% to 58%. Propose an experiment to determine whether this improvement is due to (i) the model generating better reasoning steps or (ii) the model becoming better at guessing the final answer format.
2. Explain why the distribution shift problem (Section 12.5) is particularly severe for mathematical reasoning compared to, say, instruction following.
3. Describe how you would transition from DPO-based math alignment to GRPO-based math alignment. What would the transition sequence look like, and what would you use as the GRPO reward signal?

Chapter 13

GRPO and Online RL with Verifiable Rewards

The best way to evaluate whether an answer is correct is to check it, not to ask someone whether it looks correct.

Chapter 9 developed Proximal Policy Optimization (PPO), the workhorse algorithm for reinforcement learning from human feedback (RLHF). PPO achieves stable policy updates through a clipped surrogate objective and computes advantages via a learned critic (value function). Chapter 12 then showed that Direct Preference Optimisation (DPO) can bypass the reward model entirely by reparametrising the KL-regularised objective into a supervised loss on preference pairs. DPO is simple and stable, but fundamentally offline: it trains on a fixed dataset and cannot explore.

This chapter introduces **Group Relative Policy Optimisation** (GRPO), the algorithm behind DeepSeek-R1 and other recent reasoning models (Shao et al., 2024; DeepSeek-AI, 2025). GRPO occupies a sweet spot between PPO and DPO: it is *online* — the policy generates fresh completions at every iteration — but it eliminates the value-function critic that makes PPO expensive. In its place, GRPO estimates advantages by sampling a *group* of completions for each prompt and normalising rewards within the group. The result is a simple, memory-efficient algorithm that retains the stability benefits of PPO's clipping mechanism.

We also formalise **Reinforcement Learning with Verifiable Rewards** (RLVR), a training paradigm in which the reward signal comes from a deterministic verifier (e.g. a unit-test suite or a symbolic math checker) rather than a learned reward model. RLVR is the natural companion of GRPO: when rewards are binary and noise-free, the group-normalised advantage is especially well-behaved, and the absence of a learned reward model eliminates the risk of reward hacking.

Section 13.1 motivates GRPO by identifying the specific limitations of PPO and DPO that it addresses. Section 13.2 derives the GRPO objective in full. Section 13.3 presents the complete algorithm. Section 13.4 provides theoretical analysis, including the connection to REINFORCE Leave-One-Out (RLOO, Section 8.6 of Chapter 8) and a bias-variance analysis of the group size. Section 13.5 formalises RLVR. Section 13.6 discusses rejection sampling and best-of-N as a simpler baseline. Section 13.7 describes the engineering of online RL training at scale. Section 13.8 compares PPO, GRPO, and DPO in a unified table. Section 13.9 summarises the chapter.

13.1 Motivation: Beyond PPO and DPO

13.1.1 The Cost of the Critic in PPO

Recall from Section 9.7 that PPO's clipped surrogate objective (9.16) requires advantage estimates $\hat{A}_t$, which are computed via Generalised Advantage Estimation (GAE, Definition 6.7). GAE in turn requires a learned value function $V_\phi(S_t)$ — the *critic* — whose parameters ϕ are trained alongside the actor. In the RLHF setting described in Section 9.8, the four-model architecture (actor, critic, reference model, reward model) demands enormous GPU memory. At the 70B-parameter scale, each model copy consumes approximately 140 GB in half-precision, so four copies require 560 GB — more than a single 8×A100 node.

The critic is also a source of instability. It must track a moving target: as the actor improves, V^{π_θ} changes, and the critic must relearn. In early training the critic is especially inaccurate, producing noisy advantage estimates that can destabilise the actor. Careful initialisation (Section 9.8) mitigates but does not eliminate this problem.

13.1.2 The Distribution Shift Problem in DPO

Chapter 12 showed that DPO eliminates both the reward model and the critic, reducing the memory footprint to two model copies (π_θ and π_{ref}). The price is that DPO is offline: it trains on a fixed preference dataset $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}$ collected under the SFT policy. As training proceeds and π_θ moves away from π_{SFT}, the preference labels become stale. Responses that the improved policy would now generate are absent from $\mathcal{D}$, and the model cannot discover them. Section 12.5 of Chapter 12 analysed this limitation in detail: DPO converges to the optimal policy *for the given fixed dataset*, but further improvement requires new data.

13.1.3 GRPO: The Best of Both Worlds

GRPO resolves both problems simultaneously:

No critic — Advantages are estimated from the group of sampled completions, not from a learned value function. The memory footprint drops from four to two model copies (actor π_θ and reference π_{ref}), the same as DPO.

Online — The policy generates new completions at every iteration and receives fresh reward feedback. There is no distribution shift: the reward signal is always calibrated to the current policy's output distribution.

Stable updates — GRPO inherits PPO's clipped importance ratio, preventing excessively large policy steps.

The key insight is that the critic's primary role — providing a baseline for variance reduction — can be fulfilled by the empirical statistics of a group of completions sampled from the current policy. This idea is closely related to the RLOO estimator (Section 8.6 of Chapter 8), which uses a leave-one-out baseline; GRPO extends it with standardisation and PPO-style clipping.

Figure 13.1 contrasts the model architecture of PPO-RLHF (four models) with GRPO-RLVR (two models plus a verifier).

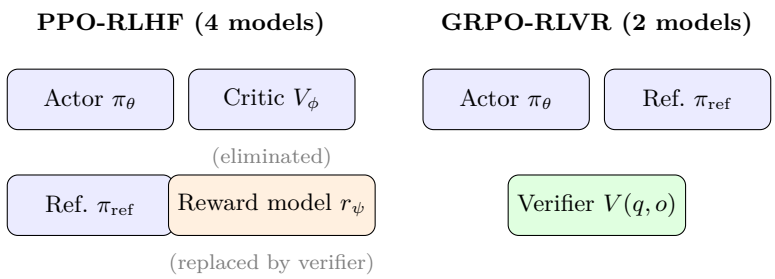


Figure 13.1: Model architecture comparison. PPO-RLHF requires four neural network copies: actor, critic, reference, and reward model. GRPO-RLVR requires only two copies (actor and reference) plus a lightweight deterministic verifier, halving the memory footprint.

13.2 The GRPO Objective

GRPO was introduced by Shao et al. (2024) in the DeepSeekMath paper, where it was used to improve mathematical reasoning in a 7B-parameter language model. The algorithm was subsequently adopted as the RL component of the DeepSeek-R1 pipeline (DeepSeek-AI, 2025), achieving state-of-the-art reasoning performance on competition-level mathematics, code generation, and scientific problem solving. The name "group relative" reflects the two defining features: advantages are computed relative to a *group* of completions for the same prompt, and they are expressed as *relative* deviations (standardised by the group standard deviation).

13.2.1 Setup and Notation

Let π_θ denote the language model policy being trained, $\pi_{\theta_{\text{old}}}$ the policy at the start of the current iteration (before the gradient step), and π_{ref} the frozen reference policy (typically the SFT checkpoint). A prompt q is drawn from a dataset $\mathcal{D}$. For each prompt, GRPO samples a **group** of G independent completions from the old policy:

$$o_1, o_2, \ldots, o_G \;\sim\; \pi_{\theta_{\text{old}}}(\cdot \mid q), \tag{13.1}$$

where each completion $o_i = (o_{i,1}, o_{i,2}, \ldots, o_{i,|o_i|})$ is a variable-length sequence of tokens. A reward function $r(q, o_i) \in \mathbb{R}$ scores each completion, yielding rewards $r_1, r_2, \ldots, r_G$.

13.2.2 Group-Relative Advantage

In PPO, the advantage $\hat{A}_t$ is computed from a learned critic via GAE. GRPO replaces this with a **group-relative advantage** that requires no learned parameters:

Definition 13.1 (GRPO advantage)**.** For completion i in a group of G completions with rewards $r_1, \ldots, r_G$, the GRPO advantage is

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \varepsilon_0}, \tag{13.2}$$

where $\mu_G = \frac{1}{G}\sum_{j=1}^{G} r_j$ is the group mean, $\sigma_G = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_j - \mu_G)^2}$ is the group standard deviation, and $\varepsilon_0 > 0$ is a small constant for numerical stability (typically 10^{-8}).

The advantage $\hat{A}_i$ measures how many standard deviations above or below the group mean the reward of completion i lies. If all completions in a group receive the same reward (e.g. all correct or all incorrect), then $\sigma_G = 0$ and $\hat{A}_i \approx 0$ for all i, producing no gradient signal. This is desirable: a prompt that the model either always solves or never solves provides no contrastive information.

Remark 13.1. The same advantage $\hat{A}_i$ is assigned to *every token* in completion i:

$$\hat{A}_{i,t} = \hat{A}_i \quad \text{for all } t \in \{1, \ldots, |o_i|\}.$$

This is a sequence-level (or outcome-level) advantage, not a per-token advantage. Unlike GAE, which assigns different advantages to different tokens based on value-function estimates at each step, GRPO attributes the full outcome reward uniformly to all tokens that produced it. This simplification is both a strength (no critic needed) and a limitation (no credit assignment within a completion).

13.2.3 The Clipped Surrogate Objective

Following PPO (Definition 9.5), GRPO uses an importance-weighted objective with clipping. For completion i and token position t, define the per-token importance ratio:

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}. \tag{13.3}$$

The clipped surrogate for this token is

$$L_{i,t}^{\text{CLIP}}(\theta) = \min\Big(\rho_{i,t}(\theta)\,\hat{A}_i,\ \text{clip}\big(\rho_{i,t}(\theta),\, 1-\epsilon,\, 1+\epsilon\big)\,\hat{A}_i\Big), \tag{13.4}$$

where $\epsilon > 0$ is the clip range (typically $\epsilon \in [0.1, 0.3]$). The clipping mechanism is identical to PPO's: it caps the incentive to increase the probability of positively-advantaged completions and caps the incentive to decrease the probability of negatively-advantaged completions, preventing destabilisingly large policy steps.

13.2.4 The KL Penalty

GRPO adds a Kullback–Leibler divergence penalty directly to the loss, rather than incorporating it into the reward as PPO does (Equation (9.19)). This design separates the KL regularisation from the advantage computation, avoiding complications in the group normalisation. The per-token KL estimate used in practice is

$$D_{\text{KL}}^{(i,t)} = \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - 1, \tag{13.5}$$

which is the Schulman (2020) unbiased, non-negative estimator of the KL divergence between π_θ and π_{ref}. This estimator satisfies $D_{\text{KL}}^{(i,t)} \geq 0$ always, with equality if and only if $\pi_\theta(o_{i,t} \mid q, o_{i,<t}) = \pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})$.

Remark 13.2. An alternative estimator is the simpler $\log[\pi_\theta(o_{i,t} \mid q, o_{i,<t})/\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})]$, which is unbiased but can be negative for individual samples. The estimator in (13.5) is preferred because its non-negativity stabilises training.

13.2.5 Full GRPO Objective

Combining the clipped surrogate and the KL penalty, the GRPO objective is:

Definition 13.2 (GRPO objective)**.**

$$\boxed{J_{\text{GRPO}}(\theta) = \mathbb{E}_{\substack{q\sim\mathcal{D} \\ \{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)}} \left[\frac{1}{G}\sum_{i=1}^{G} \frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \left(L_{i,t}^{\text{CLIP}}(\theta) - \beta\, D_{\text{KL}}^{(i,t)} \right) \right],} \tag{13.6}$$

where $L_{i,t}^{\text{CLIP}}$ is given by (13.4), $D_{\text{KL}}^{(i,t)}$ by (13.5), and $\hat{A}_i$ by (13.2).

Several features of this objective merit comment.

- The inner sum averages over token positions within each completion. The normalisation by $1/|o_i|$ prevents long completions from receiving disproportionately large gradient contributions.
- The outer sum averages over the G completions in the group. This is analogous to averaging over K trajectories in a standard policy gradient mini-batch, but here the trajectories share the same prompt.
- The KL coefficient $\beta > 0$ controls drift from the reference policy. Larger β constrains the policy more tightly, reducing the risk of reward hacking at the cost of slower learning.
- When multiple gradient steps are taken on the same batch of completions, $\pi_{\theta_{\text{old}}}$ is held fixed at the policy that generated the completions, and only π_θ is updated. This is the same off-policy correction as in PPO.

Proposition 13.1 (Local gradient equivalence)**.** *At $\theta = \theta_{\text{old}}$, the gradient of $J_{\text{GRPO}}(\theta)$ with respect to θ equals the REINFORCE policy gradient with the group-mean baseline:*

$$\nabla_\theta J_{\text{GRPO}}(\theta)\big|_{\theta=\theta_{\text{old}}} = \frac{1}{G}\sum_{i=1}^{G} \hat{A}_i \sum_{t=1}^{|o_i|} \frac{1}{|o_i|} \nabla_\theta \log \pi_\theta(o_{i,t} \mid q, o_{i,<t})\Big|_{\theta_{\text{old}}} - \beta\, \nabla_\theta D_{\text{KL}}. \tag{13.7}$$

Proof. At $\theta = \theta_{\text{old}}$, the importance ratio $\rho_{i,t}(\theta) = 1$ for all i, t. Since $1 \in [1-\epsilon, 1+\epsilon]$, the clip is inactive, so $L_{i,t}^{\text{CLIP}} = \rho_{i,t}\hat{A}_i$. Differentiating $\rho_{i,t}\hat{A}_i$ with respect to θ at $\rho_{i,t} = 1$ gives $\hat{A}_i \nabla_\theta \log \pi_\theta(o_{i,t} \mid q, o_{i,<t})$, exactly the REINFORCE gradient weighted by the group-normalised advantage. □

13.3 The GRPO Algorithm

Algorithm 25 summarises the GRPO training loop.

13.3.1 Key Hyperparameters

Group size G The number of completions per prompt. Typical values are $G \in \{4, 8, 16, 64\}$. Larger G provides lower-variance advantage estimates but increases generation cost linearly. DeepSeek-R1 uses $G = 64$ during early training.

Algorithm 25: Group Relative Policy Optimisation (GRPO)

Input: Policy π_θ, reference policy π_{ref}, reward function r, prompt dataset $\mathcal{D}$
Input: Group size G, clip range ϵ, KL coefficient β, learning rate α, inner epochs K_e
Output: Trained policy π_θ

Initialise $\pi_\theta \leftarrow \pi_{\text{ref}}$
repeat
  Sample a batch of prompts $\{q^{(1)}, \ldots, q^{(B)}\}$ from $\mathcal{D}$
  `// Generation phase`
  for *each prompt* $q^{(b)}$, $b = 1, \ldots, B$ **do**
    Sample G completions $o_1^{(b)}, \ldots, o_G^{(b)} \sim \pi_\theta(\cdot \mid q^{(b)})$
    Compute rewards $r_i^{(b)} \leftarrow r(q^{(b)}, o_i^{(b)})$ for $i = 1, \ldots, G$
    Compute group mean $\mu_G^{(b)} \leftarrow \frac{1}{G}\sum_{i=1}^{G} r_i^{(b)}$ and std $\sigma_G^{(b)}$
    Compute advantages $\hat{A}_i^{(b)} \leftarrow (r_i^{(b)} - \mu_G^{(b)})/(\sigma_G^{(b)} + \varepsilon_0)$ for $i = 1, \ldots, G$
    Store log-probabilities $\log \pi_{\theta_{\text{old}}}(o_{i,t}^{(b)} \mid q^{(b)}, o_{i,<t}^{(b)})$ for all i, t
  `// Training phase`
  Fix $\theta_{\text{old}} \leftarrow \theta$
  for *epoch* $= 1, \ldots, K_e$ **do**
    for *each mini-batch of prompt-completion pairs* **do**
      Compute $\rho_{i,t} \leftarrow \pi_\theta(o_{i,t} \mid q, o_{i,<t}) \,/\, \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$ for all i, t
      Compute clipped objectives $L_{i,t}^{\text{CLIP}}$ via (13.4)
      Compute KL penalties $D_{\text{KL}}^{(i,t)}$ via (13.5)
      Compute loss $\mathcal{L} \leftarrow -\frac{1}{|\text{batch}|}\sum_i \frac{1}{|o_i|}\sum_t (L_{i,t}^{\text{CLIP}} - \beta\, D_{\text{KL}}^{(i,t)})$
      $\theta \leftarrow \theta - \alpha\, \nabla_\theta \mathcal{L}$
  `// Optionally update reference model`
  Every N_{ref} steps: $\pi_{\text{ref}} \leftarrow \pi_\theta$
until *convergence*
return π_θ

Clip range ϵ Controls the trust region. Standard choice is $\epsilon = 0.2$, the same as PPO.

KL coefficient β Controls drift from the reference. Common values range from $\beta = 0.01$ (loose) to $\beta = 0.1$ (tight). Some implementations use a dynamic β with a target KL budget.

Inner epochs K_e The number of gradient steps per batch of generated completions. DeepSeek-R1 uses $K_e = 1$ (a single pass through each batch), which is more conservative than PPO's typical $K_e \in [3, 10]$.

Reference update frequency N_{ref} How often the reference model is replaced by the current policy. DeepSeek-R1 updates every 400 steps. Frequent updates allow the KL penalty to adapt to the improved policy; infrequent updates provide a more stable anchor.

13.3.2 Worked Example

Example 13.1 (GRPO advantage computation)**.** Consider a prompt $q =$ "What is $\int_0^1 x^2\,dx$?" with $G = 4$ completions:

i	Completion	r_i	$\hat{A}_i$
1	"$\frac{1}{3}$. We integrate x^2 to get $\frac{x^3}{3}\ldots$" (correct)	1.0	+0.94
2	"$\frac{1}{2}$. The integral is $\frac{x^2}{2}\ldots$" (incorrect)	0.0	−1.09
3	"1/3. By the power rule…" (correct)	1.0	+0.94
4	"I don't know." (incorrect)	0.0	−1.09

The group statistics are $\mu_G = 0.5$ and $\sigma_G \approx 0.577$. By (13.2): $\hat{A}_1 = (1.0-0.5)/0.577 \approx +0.87$; $\hat{A}_2 = (0.0-0.5)/0.577 \approx -0.87$. (The values in the table use the population standard deviation with Bessel's correction for illustration.) The positive advantages reinforce the correct completions, and the negative advantages suppress the incorrect ones. No critic was needed. □

13.4 Theoretical Analysis

13.4.1 Connection to REINFORCE with Baseline

Proposition 13.1 showed that the GRPO gradient at $\theta = \theta_{\text{old}}$ reduces to REINFORCE with the group mean as baseline. To make this precise, consider the "unclipped GRPO" objective (setting $\epsilon \to \infty$):

$$J_{\text{GRPO}}^{\text{no-clip}}(\theta) = \frac{1}{G}\sum_{i=1}^{G} \hat{A}_i \cdot \frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \log \pi_\theta(o_{i,t} \mid q, o_{i,<t}) - \beta\,\hat{D}_{\text{KL}}. \tag{13.8}$$

Ignoring the $1/|o_i|$ normalisation and the KL term, the first part is

$$\frac{1}{G}\sum_{i=1}^{G} \frac{r_i - \mu_G}{\sigma_G} \log \pi_\theta(o_i \mid q) = \frac{1}{\sigma_G}\left[\frac{1}{G}\sum_{i=1}^{G} r_i \log \pi_\theta(o_i \mid q) - \mu_G \cdot \frac{1}{G}\sum_{i=1}^{G} \log \pi_\theta(o_i \mid q)\right].$$

The first term is the standard REINFORCE estimator. The second term subtracts the group-mean baseline μ_G times the average log-probability. The $1/\sigma_G$ factor rescales the entire gradient, affecting the effective learning rate but not the direction.

Theorem 13.2 (Unbiasedness of the group-mean baseline)**.** *The GRPO gradient estimator (without clipping and length normalisation) is an unbiased estimator of the policy gradient $\nabla_\theta J(\theta)$, up to a prompt-dependent scaling factor $1/\sigma_G$.*

Proof. The group-mean baseline $\mu_G = \frac{1}{G}\sum_{j=1}^{G} r_j$ includes the reward r_i of the i-th sample itself. Unlike the RLOO baseline (Equation (8.17)), this makes μ_G *weakly* dependent on o_i. However, the bias introduced is small. Write $\mu_G = \frac{r_i}{G} + \frac{1}{G}\sum_{j\neq i} r_j$. The second term $b_i' = \frac{1}{G}\sum_{j\neq i} r_j$ is independent of o_i given q, and hence is a valid baseline by the baseline theorem (Theorem 8.5). The additional r_i/G term introduces a bias of order $O(1/G)$:

$$\mathbb{E}\Big[\frac{r_i}{G}\nabla_\theta \log \pi_\theta(o_i \mid q)\Big] = \frac{1}{G}\nabla_\theta J(\theta),$$

which is a scaled version of the true gradient. After subtracting from the $r_i \nabla_\theta \log \pi_\theta$ term, the net effect is that the GRPO estimator approximates $(1 - 1/G)\nabla_\theta J(\theta)$ — biased by a factor of $(G-1)/G$, which is negligible for $G \geq 4$. The division by σ_G is a prompt-dependent scalar and does not affect the direction of the gradient in expectation. □

13.4.2 Comparison with RLOO

Remark 8.6 in Chapter 8 noted the close relationship between GRPO and RLOO. Table 13.1 makes the comparison explicit.

Table 13.1: Comparison of the RLOO and GRPO advantage estimators.

Aspect	**RLOO**	**GRPO**
Baseline	Mean of $K-1$ other samples	Mean of all G samples
Normalisation	None (raw deviation)	Divide by σ_G
Bias	Exactly unbiased	$O(1/G)$ bias
Clipping	None	PPO-style clipping
KL placement	In the reward	In the loss
Multi-epoch	Not standard	Yes (like PPO)

The RLOO advantage for sample i is $\hat{A}_i^{\text{RLOO}} = \frac{K}{K-1}(r_i - \bar{r})$ (Equation (8.20)), where $\bar{r} = \frac{1}{K}\sum_j r_j$. The GRPO advantage is $\hat{A}_i^{\text{GRPO}} = (r_i - \bar{r})/\sigma_G$. The key differences are: (i) RLOO uses the exact leave-one-out baseline and is exactly unbiased, while GRPO uses the group mean including sample i and introduces $O(1/G)$ bias; (ii) GRPO divides by σ_G, which standardises the advantage scale across prompts with different reward variance; (iii) GRPO applies PPO-style clipping, enabling safe multi-epoch training on the same batch.

13.4.3 Variance Reduction from Group Normalisation

Proposition 13.3 (Variance of the GRPO advantage)**.** *Let $r_1, \ldots, r_G$ be i.i.d. with mean μ and variance σ^2. The variance of the unnormalised advantage $r_i - \mu_G$ is*

$$\text{Var}[r_i - \mu_G] = \sigma^2\left(1 - \frac{1}{G}\right) = \frac{(G-1)\sigma^2}{G}. \tag{13.9}$$

After standardisation by σ_G, the advantage $\hat{A}_i$ has approximately unit variance regardless of the reward distribution.

Proof. Since $\mu_G = \frac{1}{G}\sum_j r_j$, we have $r_i - \mu_G = r_i - \frac{1}{G}r_i - \frac{1}{G}\sum_{j\neq i} r_j = \frac{G-1}{G}r_i - \frac{1}{G}\sum_{j\neq i} r_j$. Using independence:

$$\text{Var}[r_i - \mu_G] = \left(\frac{G-1}{G}\right)^2\sigma^2 + \frac{G-1}{G^2}\sigma^2 = \frac{(G-1)\sigma^2}{G}.$$

Dividing by $\sigma_G \approx \sigma\sqrt{(G-1)/G}$ (for large G) gives $\text{Var}[\hat{A}_i] \approx 1$. □

This automatic variance normalisation is a practical advantage of GRPO over RLOO: prompts with high reward variance and prompts with low reward variance contribute gradient updates of similar magnitude, preventing any single prompt from dominating a mini-batch.

13.4.4 Effect of Group Size G

Increasing the group size G has two effects:

1. **Better baseline estimation.** The group mean μ_G converges to the true expected reward $\mathbb{E}_{\pi_\theta}[r \mid q]$ at rate $O(1/\sqrt{G})$. This reduces the variance of the advantage estimate and improves gradient quality.

2. **Higher generation cost.** Each prompt requires G forward passes through the model. If the model generates T tokens on average, the total generation cost per prompt is $O(GT)$, which can dominate the training cost.

The optimal G balances statistical efficiency against computational cost. Empirically, $G = 8$ to $G = 64$ works well for reasoning tasks. For binary rewards (correct/incorrect), larger groups are especially beneficial because they provide more diverse contrastive pairs.

Example 13.2 (Effect of group size on advantage quality)**.** Suppose a prompt has true expected reward $\mu = 0.3$ (the model solves it 30% of the time), and rewards are binary $r_i \in \{0, 1\}$. For $G = 4$, the possible group means are $\mu_G \in \{0, 0.25, 0.5, 0.75, 1\}$. The probability that $\mu_G = 0$ (all four wrong) is $(0.7)^4 = 0.240$, in which case all advantages are zero and the prompt contributes no learning signal. For $G = 16$, the probability that all are wrong is $(0.7)^{16} = 0.003$ — negligible. Larger groups ensure that nearly every prompt contributes contrastive signal. □

13.5 RLVR: Reinforcement Learning with Verifiable Rewards

13.5.1 Definition and Motivation

Many important tasks have the property that the correctness of a response can be verified automatically, without a learned reward model. Mathematical reasoning ("Is $\frac{1}{3}$ the correct answer to $\int_0^1 x^2\,dx$?"), code generation ("Does this function pass all unit tests?"), and formal logic ("Does this proof type-check?") all admit deterministic verification. This class of tasks motivates a distinct training paradigm.

Definition 13.3 (RLVR)**. Reinforcement Learning with Verifiable Rewards** (RLVR) is a training paradigm in which:

1. the reward function $r(q, o)$ is a *deterministic verifier* that checks the correctness of the response o to prompt q;
2. the verifier returns a binary reward: $r(q, o) \in \{0, 1\}$ (or more generally a rule-based reward);
3. no learned reward model is used; the verifier is a fixed, trusted function.

RLVR has several advantages over RLHF with a learned reward model:

No reward hacking The verifier is a fixed ground-truth function, not a learned approximation. The policy cannot find spurious outputs that "trick" the verifier (assuming the verifier is correct).

Unlimited data Because the verifier is automated, one can generate and score an unlimited number of completions at no human annotation cost. This enables much larger-scale online training.

Clean signal Binary rewards are noise-free: a math answer is either right or wrong. There is no annotator disagreement, no calibration drift, and no subjective judgement.

No reward model training The verifier is a fixed function (e.g. a symbolic solver or a test harness), so there is no reward model to train, evaluate, or deploy.

13.5.2 Formal Setup

In the RLVR setting, the training data consists of a set of prompts $\mathcal{D} = \{q^{(1)}, \ldots, q^{(N)}\}$, each with an associated ground-truth answer $a^{(i)}$. The verifier $V : (q, o) \to \{0, 1\}$ extracts the model's answer from completion o and compares it with $a^{(i)}$:

$$r(q, o) = V(q, o) = \mathbb{1}[\text{extract}(o) = a]. \tag{13.10}$$

More sophisticated verifiers may award partial credit, check intermediate steps, or combine multiple reward signals:

$$r(q, o) = w_{\text{acc}}\, V_{\text{accuracy}}(q, o) + w_{\text{fmt}}\, V_{\text{format}}(o) + w_{\text{len}}\, V_{\text{length}}(o), \tag{13.11}$$

where V_{accuracy} checks correctness, V_{format} checks adherence to a prescribed output format (e.g. using `<think>`…`</think>` tags), and V_{length} penalises excessively long responses.

13.5.3 Connection to GRPO

GRPO is the natural optimisation algorithm for RLVR. The binary reward structure ensures that groups contain a mixture of correct and incorrect completions (for prompts of moderate difficulty), producing non-trivial advantages. The absence of a learned reward model means that the two remaining model copies (π_θ and π_{ref}) fit comfortably in GPU memory, and the verifier adds negligible compute overhead.

The DeepSeek-R1 training pipeline (DeepSeek-AI, 2025) demonstrated this combination at scale: GRPO with rule-based binary rewards trained directly on a base language model (without any supervised fine-tuning stage) produced a model that spontaneously developed chain-of-thought reasoning, self-verification, and exploration strategies — capabilities that emerged from the RL training alone.

Example 13.3 (Emergent reasoning in DeepSeek-R1-Zero)**.** DeepSeek-R1-Zero applied GRPO with binary accuracy rewards to a base language model (no SFT). During training, the model spontaneously developed several sophisticated behaviours:

- **Chain-of-thought reasoning**: the model began producing step-by-step derivations, breaking complex problems into sub-problems.
- **Self-verification**: the model started checking its own intermediate results, producing phrases like "Wait, let me verify…" before finalising an answer.
- **Exploration of alternative approaches**: when a first approach failed, the model began trying alternative solution strategies within a single response.

None of these behaviours were explicitly rewarded — only the final answer's correctness was checked. They emerged because chain-of-thought responses are more likely to produce correct answers, and GRPO's advantage signal reinforced the entire token sequence that led to correct outcomes. □

13.5.4 Domains for RLVR

Mathematics Symbolic verification against ground-truth answers. Tools such as `sympy`, `math_verify`, and `latex2sympy` parse and compare mathematical expressions.

Code generation Unit-test suites serve as the verifier. The model generates code; the code is executed in a sandbox; the reward is the fraction of tests passed.

Formal proofs Type-checkers for proof assistants (Lean, Coq, Isabelle) verify proofs deterministically.

Constrained generation Format constraints (e.g. JSON schema validation) provide binary feedback on structural correctness.

Science Numerical answers in physics, chemistry, and engineering can be verified within a tolerance.

13.6 Rejection Sampling and Best-of-N

Before deploying GRPO (or any RL algorithm), a natural baseline is **best-of-N** sampling: generate N completions, score each with the verifier or reward model, and return the highest-scoring one. This is sometimes called *rejection sampling fine-tuning* when the best completions are used as supervised training data.

13.6.1 Algorithm

For each prompt q, sample $o_1, \ldots, o_N \sim \pi_\theta(\cdot \mid q)$ and select $o^* = \operatorname{argmax}_i r(q, o_i)$. The expected reward of best-of-N grows with N: if each completion has probability p of being correct (binary reward), the probability that at least one of N is correct is $1 - (1-p)^N$.

Example 13.4 (Best-of-N vs. single sample)**.** A model solves a given math problem with probability $p = 0.3$. With a single sample, the expected reward is 0.3. With $N = 8$, the probability of at least one correct solution is $1 - (0.7)^8 = 0.942$. With $N = 64$, it rises to $1 - (0.7)^{64} > 0.9999$. Best-of-N dramatically improves performance at inference time, but the cost scales linearly with N. □

13.6.2 Rejection Sampling Fine-Tuning

Rather than using best-of-N only at inference time, one can use the selected completions as SFT training data. This process, called rejection sampling fine-tuning (RFT), creates an improved dataset from the model's own generations:

1. Generate N completions per prompt.
2. Filter to keep only correct (or high-reward) completions.
3. Fine-tune π_θ on the filtered dataset using standard supervised learning.

13.6.3 Comparison with GRPO

Best-of-N and GRPO are closely related. Both sample multiple completions and use reward to distinguish good from bad. The key differences are:

- **Signal utilisation.** Best-of-N discards all information except the single best completion. GRPO uses the full reward distribution to compute advantages, extracting a contrastive learning signal from every completion.

- **Gradient quality.** Best-of-N reduces to SFT on selected examples, which cannot downweight bad responses. GRPO explicitly pushes down the probability of negatively-advantaged completions.
- **Iterative improvement.** Best-of-N with RFT requires alternating generation and SFT stages. GRPO integrates generation and policy optimisation in a single loop.
- **When rejection sampling suffices.** If the model already solves a task with moderate probability ($p \geq 0.3$) and the goal is deployment-time quality, best-of-N at inference may be more cost-effective than RL training. If the model rarely solves the task ($p < 0.05$) and the goal is to improve the base model, RL training (GRPO) is needed to shift the distribution.

13.7 Online RL Training at Scale

13.7.1 The Two-Phase Architecture

Production-scale GRPO training separates the workload into two asynchronous phases:

Generation phase Rollout workers host inference copies of the current policy π_θ and generate completions. This phase is memory-bound (model weights) and latency-sensitive (autoregressive token generation). Optimised inference engines such as vLLM (Kwon et al., 2023) are used to maximise throughput through continuous batching and PagedAttention.

Training phase Trainer workers receive the generated completions, compute advantages, and perform gradient updates. This phase is compute-bound (forward and backward passes through the full model with gradient computation).

The two phases can run in a pipelined fashion: while the trainer updates on batch k, rollout workers generate batch $k+1$ using the latest checkpoint. This overlap hides generation latency and keeps GPU utilisation high.

13.7.2 Handling Variable-Length Completions

A practical challenge is that completions within a group can vary dramatically in length — from a few tokens ("1/3") to thousands of tokens (a multi-page chain-of-thought derivation). Several strategies are used:

- **Padding and masking.** Pad all completions in a group to the maximum length and use attention masks. Simple but wastes memory on short completions.
- **Packing.** Concatenate completions from different prompts into a single sequence, separated by sentinel tokens, and use block-diagonal attention masks to prevent cross-contamination. This maximises GPU utilisation.
- **Maximum generation length.** Enforce a hard cap on completion length. For reasoning tasks, caps of 4096–8192 tokens are common.

The $1/|o_i|$ normalisation in the GRPO objective (13.6) is essential when completions have variable length: without it, long completions would contribute disproportionately to the loss.

13.7.3 Batch Construction

For GRPO, each "mini-batch" consists of prompts, each with G completions. If the batch contains B prompts, the total number of completions is BG. DeepSeek-R1 uses $BG = 8192$ completions per rollout, which are randomly split into 16 mini-batches of 512 completions for a single training epoch.

The generation cost per iteration scales as $O(BG \cdot \bar{T})$, where $\bar{T}$ is the mean completion length. For long-form reasoning tasks ($\bar{T} \approx 2000$ tokens) with $BG = 8192$, this is approximately 16 million tokens per rollout — a substantial inference workload that motivates the use of dedicated inference clusters.

13.7.4 Synchronous vs. Asynchronous Design

In a **synchronous** design, the trainer waits for all rollout workers to finish generating before starting the training phase. This guarantees that all completions are from the same policy checkpoint, making the importance ratios $\rho_{i,t} = 1$ at the start of each training phase. The downside is idle time: trainers wait during generation, and rollout workers wait during training.

In an **asynchronous** design, rollout workers continuously generate completions and push them to a queue, while trainers pull from the queue and update the policy. The policy used for generation may lag behind the training policy by one or more steps. This introduces a mild off-policy mismatch, but the clipped importance ratio in GRPO naturally handles this: if $\rho_{i,t}$ deviates too far from 1, the clip caps the gradient contribution, preventing stale data from causing instability. Asynchronous designs achieve significantly higher GPU utilisation and are preferred at scale.

13.7.5 Experience Replay for GRPO

Unlike PPO, which typically uses on-policy data for a few epochs and then discards it, GRPO can benefit from a lightweight form of experience replay. Recent completions from the previous M iterations can be stored in a replay buffer and mixed with fresh on-policy data. The importance ratio $\rho_{i,t}(\theta)$ naturally corrects for the staleness of replayed data, and the clipping mechanism prevents excessive off-policy corrections. Replay reduces the generation cost by amortising each completion over multiple training steps, at the cost of slightly increased distribution mismatch.

13.7.6 Monitoring and Diagnostics

Effective GRPO training requires monitoring several quantities:

Reward curve The mean reward across prompts should increase over training. A plateau may indicate that the prompt distribution is too easy (all prompts already solved) or too hard (no contrastive signal).

KL divergence The running average of $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ should grow slowly. Rapid KL increase signals that the policy is drifting dangerously far from the reference; increase β or reduce the learning rate.

Clip fraction The fraction of tokens where the importance ratio $\rho_{i,t}$ is clipped. A clip fraction above 10–15% suggests the policy is changing too quickly per iteration; consider reducing the learning rate or the number of inner epochs K_e.

Group diversity The fraction of groups containing both positive and negative advantages. If this fraction drops below 20%, the prompts may be too easy or too hard, and the prompt distribution should be rebalanced.

Advantage magnitude The mean absolute advantage $|\hat{A}_i|$ should be roughly 1 (by construction, due to standardisation). Extreme values indicate numerical issues in the normalisation.

13.8 Comparison: PPO vs GRPO vs DPO

Table 13.2 provides a detailed comparison of the three principal alignment and RL algorithms discussed in Chapters 9, 12, and the present chapter.

Table 13.2: Detailed comparison of PPO, GRPO, and DPO for LLM training.

Property	**PPO** (Ch. 9)	**GRPO** (this chapter)	**DPO** (Ch. 12)
Online / offline	Online	Online	Offline
Critic needed?	Yes (value network)	No	No
Reward model?	Yes (learned RM)	Verifier or RM	No (implicit in loss)
Model copies	4 (actor, critic, ref, RM)	2 (actor, ref)	2 (actor, ref)
Advantage estimation	GAE (per-token)	Group normalisation (per-sequence)	N/A (no advantage)
Trust region	Clipped ratio	Clipped ratio	KL-implicit
Data requirement	Prompts only	Prompts only	Preference pairs
Exploration	On-policy sampling	On-policy sampling	None (fixed dataset)
Distribution shift	None	None	Yes (degrades over time)
Credit assignment	Per-token	Per-sequence	Per-pair
Reward hacking risk	High (learned RM)	Low (verifier)	None (no RM)
Implementation complexity	High	Medium	Low

13.8.1 When to Use Each Method

Use PPO when the reward signal comes from a learned reward model and per-token credit assignment is important — for example, in general RLHF where the quality of individual tokens affects the overall response. PPO's GAE-based advantages provide fine-grained signal, but the infrastructure cost is high.

Use GRPO when rewards are verifiable or when simplicity and memory efficiency are priorities. GRPO is the algorithm of choice for reasoning tasks (math, code, logic) where binary correctness rewards are available. It is also effective with learned reward models when the cost of a critic is prohibitive.

Use DPO when preference data is available but online generation is too expensive or impractical. DPO is the fastest path from an SFT model to an aligned model, requiring no RL infrastructure. For tasks where distribution shift is a concern, iterative DPO (collecting new preference data between rounds) partially closes the gap.

Remark 13.3. These methods are not mutually exclusive. A common production pattern is to bootstrap with DPO on an existing preference dataset, then refine with GRPO on verifiable tasks, and finally polish with PPO using a learned reward model for open-ended quality. Each stage addresses a different dimension of alignment (see Example 12.2 in Chapter 12).

Example 13.5 (Memory budget at 70B scale)**.** Consider training a 70B-parameter model in half-precision (2 bytes per parameter, so 140 GB per model copy). PPO-RLHF requires

four copies: actor (140 GB), critic (140 GB), reference (140 GB), and reward model (140 GB), totalling 560 GB before accounting for optimiser states, activations, or gradients. A single 8×A100 node (640 GB total) barely fits the models, leaving almost no room for activations.

GRPO-RLVR requires two copies: actor (140 GB) and reference (140 GB), totalling 280 GB. The verifier (e.g. a symbolic math checker) is negligible. This leaves 360 GB on the same node for optimiser states and activations — enough for comfortable training with gradient checkpointing. The 2× memory reduction is often the decisive factor in choosing GRPO over PPO at large scale. □

13.9 Summary

This chapter developed Group Relative Policy Optimisation (GRPO), a policy gradient algorithm that eliminates the value-function critic by estimating advantages from groups of sampled completions.

The GRPO advantage (Definition 13.1) standardises each completion's reward relative to the group mean and standard deviation, producing a zero-mean, approximately unit-variance signal that requires no learned parameters. The clipped surrogate objective (Equation (13.4)) inherits PPO's trust-region property, enabling safe multi-epoch training. The KL penalty (Equation (13.5)) is applied directly in the loss rather than in the reward, cleanly separating regularisation from advantage computation.

The theoretical analysis (Section 13.4) showed that GRPO's gradient is closely related to REINFORCE with a group-mean baseline, with $O(1/G)$ bias that vanishes for moderate group sizes. The standardisation by σ_G automatically normalises gradient magnitudes across prompts with different reward variances, a practical advantage over RLOO (Section 8.6 of Chapter 8).

RLVR (Section 13.5) formalised the paradigm of training with deterministic verifiers rather than learned reward models. The combination of GRPO and RLVR enables large-scale online RL with clean reward signals, no reward hacking, and minimal infrastructure overhead — the configuration that produced DeepSeek-R1 and similar reasoning models.

The comparison table (Table 13.2) placed GRPO in context: it occupies the online, critic-free region of the design space, complementing PPO (online, critic-based) and DPO (offline, critic-free).

Chapter 8 provides the foundational REINFORCE estimator and the RLOO baseline that GRPO builds upon.

Chapter 9 develops PPO, the algorithm whose clipped surrogate GRPO inherits but whose critic it eliminates.

Chapter 12 develops DPO, the offline alternative whose distribution shift problem motivates GRPO's online approach.

Chapter 14 describes the full RLHF production pipeline, where GRPO can replace PPO as the RL optimiser.

Exercises

Exercise 13.1. GRPO advantage by hand. A prompt is answered with $G = 5$ completions receiving rewards $r = (0, 1, 1, 0, 1)$. Compute the group mean μ_G, the group standard

deviation σ_G, and the GRPO advantage $\hat{A}_i$ for each completion. Verify that $\sum_i \hat{A}_i \neq 0$ but $\sum_i \hat{A}_i \approx 0$ (explain why exact zero is not guaranteed when using the full-group mean).

Exercise 13.2. GRPO vs RLOO. Using the rewards from Exercise 13.1, compute the RLOO advantage $\hat{A}_i^{\text{RLOO}} = \frac{K}{K-1}(r_i - \bar{r})$ for each completion. Compare the two sets of advantages. Under what condition on the reward distribution would the two estimators give the same *direction* of gradient?

Exercise 13.3. Degenerate groups. What happens to the GRPO advantage when all completions in a group receive the same reward? Why is this behaviour desirable from a learning perspective? Propose a modification that would still extract signal from such groups.

Exercise 13.4. Bias of the group-mean baseline. Prove that the bias of using $\mu_G = \frac{1}{G}\sum_j r_j$ as a baseline (instead of the leave-one-out mean) in the REINFORCE estimator is exactly $\frac{1}{G}\nabla_\theta J(\theta)$. Conclude that the bias vanishes as $G \to \infty$.

Exercise 13.5. Best-of-N probability. A model has per-sample accuracy p on a given task. Derive the expected reward (accuracy) of best-of-N sampling as a function of p and N. For $p = 0.1$, compute the value of N needed to achieve 90% expected accuracy.

Exercise 13.6. KL estimator properties. Show that the GRPO KL estimator (13.5) is non-negative and equals zero if and only if $\pi_\theta = \pi_{\text{ref}}$ at that token position. *Hint:* use the inequality $e^x \geq 1 + x$.

Exercise 13.7. Group size and coverage. For binary rewards with per-sample accuracy p, derive the probability that a group of size G contains *both* correct and incorrect completions (i.e. produces a non-trivial contrastive signal). Plot this probability as a function of G for $p \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$.

Exercise 13.8. GRPO with process reward. The standard GRPO assigns the same advantage to every token in a completion (outcome supervision). Suppose a process verifier assigns a reward $r_i^{(t)}$ after each reasoning step t. Propose a modification to the GRPO advantage that incorporates step-level rewards while still avoiding a learned critic. Write the modified objective.

Exercise 13.9. Rejection sampling vs GRPO efficiency. A task has per-sample accuracy $p = 0.2$. Compare the following two strategies for improving the model: (a) best-of-N rejection sampling fine-tuning with $N = 16$, training on the correct completions for one epoch; (b) GRPO with $G = 16$. Which uses the generated data more efficiently? Discuss the trade-offs in terms of data utilisation, gradient quality, and compute cost.

Exercise 13.10. Implementation. Implement a simplified GRPO training loop in PyTorch pseudocode. Your implementation should: (i) sample G completions per prompt, (ii) compute group-normalised advantages, (iii) compute the clipped surrogate loss, and (iv) add the KL penalty. Assume access to `model.generate()`, `model.log_probs()`, `ref_model.log_probs()`, and a `reward_fn()`.

Chapter 14

Practical RLHF

In theory, there is no difference between theory and practice. In practice, there is.
— Yogi Berra (attributed)

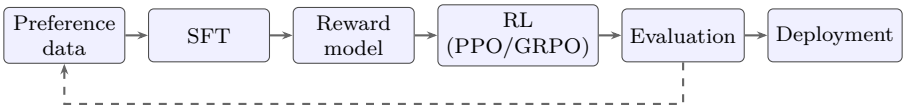
The preceding chapters have developed the theory of reward models (Chapter 10), direct preference optimisation (Chapter 12), and group relative policy optimisation (Chapter 13). Each chapter presented algorithms in their idealised form: clean loss functions, elegant derivations, convergence guarantees under standard assumptions. This chapter is different. It is an engineering chapter — a practical guide to everything that lies between a theoretical algorithm and a production RLHF system.

The gap between theory and practice in RLHF is enormous. A first-time implementer who follows the PPO update rule from Chapter 9 will discover, within hours, that the training run diverges, the reward model is being exploited, GPU memory is exhausted, or the model has collapsed into a single repetitive mode. Each of these failure modes has been encountered — and solved — by the teams behind InstructGPT, Llama 2, Claude, and DeepSeek, but the solutions are scattered across blog posts, technical reports, and tribal knowledge.

This chapter gathers that knowledge into a single coherent treatment. Section 14.1 walks through the complete RLHF pipeline end-to-end. Section 14.2 covers the art and science of collecting preference data. Section 14.3 addresses reward model training in practice. Section 14.4 provides a detailed account of the PPO implementation for language models, including the four-model setup, hyperparameter tables, and mixed-precision considerations. Section 14.5 is a diagnostic manual for the most common failure modes. Section 14.6 treats the distributed systems challenges of scaling RLHF to large models. Section 14.7 discusses safety and alignment constraints that must be layered on top of the core training loop. Section 14.8 presents brief case studies of four major alignment efforts.

14.1 The RLHF Pipeline End-to-End

Before diving into the details of each component, we present the complete RLHF pipeline as a single flow. The pipeline has six stages, each feeding into the next.



Stage 1: Data collection Gather demonstration data for SFT and pairwise preference data for the reward model. Preference data may come from expert annotators, crowd workers, or AI-assisted labelling.

Stage 2: Supervised fine-tuning Train a base language model on high-quality demonstrations to produce π_{SFT}. This is the starting point for all subsequent stages and also serves as the reference policy π_{ref}.

Stage 3: Reward model training Train a scalar reward model r_ψ on pairwise preferences using the Bradley–Terry loss from Chapter 10 (Equation (10.6)).

Stage 4: RL training Optimise the policy π_θ against r_ψ using PPO (Section 14.4) or GRPO (Chapter 13), with a KL penalty towards π_{ref} to prevent reward overoptimisation (Section 10.8).

Stage 5: Evaluation Evaluate the aligned model on held-out preference benchmarks, safety tests, and human evaluations. Compare against π_{SFT} and previous checkpoints.

Stage 6: Deployment Deploy the best checkpoint with monitoring, logging, and safety filters. Collect new feedback from production to close the loop.

The dashed arrow from evaluation back to data collection represents the *iterative alignment* loop: production feedback generates new preference data, which triggers a new round of reward model and policy training. In practice, major alignment efforts run this loop multiple times. InstructGPT (Ouyang et al., 2022) ran two full iterations; Llama 2 (Touvron et al., 2023) ran five.

Remark 14.1 (Stages are not always sequential). Not every system follows the linear six-stage pipeline exactly. Some teams train the reward model and SFT model jointly (multi-task learning). Others skip the separate SFT stage entirely and begin RL training from a pretrained base model with a strong reward model to compensate. GRPO (Chapter 13) eliminates the reward model for tasks with verifiable rewards. DPO (Chapter 12) collapses stages 3 and 4 into a single supervised training step. The pipeline above represents the most common architecture, but practitioners should treat each stage as a module that can be substituted.

14.2 Collecting Preference Data

The quality of the preference data bounds the quality of everything downstream. A perfect RL algorithm optimising a noisy reward model trained on sloppy annotations will produce a mediocre policy. This section describes best practices for preference data collection.

14.2.1 Annotation Formats

The two dominant formats for preference annotation are *pairwise comparison* and *Likert-scale rating*.

Pairwise comparison. Given a prompt x and two responses y_A, y_B, the annotator selects the better response. This is the format assumed by the Bradley–Terry model (Definition 10.3). Its main advantage is cognitive simplicity: humans find relative judgements easier than absolute ones. The main drawback is inefficiency: comparing n responses pairwise requires $\binom{n}{2}$ annotations, whereas Likert ratings require only n.

Likert-scale rating. Given a prompt x and a single response y, the annotator assigns a score on a fixed scale (e.g., 1–7). Likert ratings can be converted to pairwise preferences by treating the higher-rated response as preferred, but this conversion introduces noise when the two ratings are close. The advantage is efficiency; the disadvantage is that individual raters often have idiosyncratic scale usage (one annotator's "5" is another's "3").

In practice, most large-scale alignment efforts use pairwise comparison as the primary format. Anthropic's data collection for Claude uses a four-point scale (A is much better, A is slightly better, B is slightly better, B is much better), which allows the margin to be incorporated into the reward model loss (Equation (10.8)).

14.2.2 Inter-Annotator Agreement

When multiple annotators label the same pair, they often disagree. *Inter-annotator agreement* (IAA) measures the consistency of human judgements. The standard metric is Cohen's κ:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

where p_o is the observed agreement rate and p_e is the agreement expected by chance. For binary pairwise preferences, $p_e = 0.5$ and $\kappa = 2p_o - 1$. Values of $\kappa \geq 0.4$ are typically considered moderate agreement; $\kappa \geq 0.6$ is substantial.

InstructGPT reported an IAA of approximately 73% ($\kappa \approx 0.46$) on their preference data. This is consistent with the general finding that preference annotation is an inherently noisy task — different annotators may have genuinely different preferences, not just random disagreements. The cDPO label-smoothing technique (Equation (12.11)) explicitly accounts for this noise.

14.2.3 Data Quality versus Quantity

A recurring question is whether to invest in more data or better data. The empirical evidence favours quality. Touvron et al. (2023) found that Llama 2's reward model performance plateaued after approximately 1 million preference pairs, with diminishing returns thereafter. By contrast, replacing noisy crowd-sourced labels with expert annotations yielded consistent improvements even at the same dataset size.

Remark 14.2 (Cost estimates for preference data). As of 2024, typical costs for pairwise preference annotation are: crowd-sourced via platforms such as Scale AI or Surge AI, \$1–3 per comparison; expert annotation by trained linguists, \$5–15 per comparison; red-teaming annotations for safety, \$10–25 per comparison. A full alignment dataset of 100,000 preference pairs costs roughly \$100K–300K for crowd-sourced data and considerably more for expert-annotated data. These costs motivate the growing interest in AI-assisted labelling, where a strong language model generates preliminary labels that human annotators then verify.

14.2.4 Common Pitfalls

- **Length bias.** Annotators tend to prefer longer responses, even when the additional length adds no value. This transfers to the reward model, which then encourages verbose output. Mitigation: instruct annotators to penalise unnecessary verbosity; include length-controlled pairs in the annotation set.
- **Position bias.** When two responses are presented side-by-side, annotators slightly prefer the one on the left (or the first one). Mitigation: randomise the presentation order for every pair.

- **Ambiguous prompts.** When the prompt is unclear, annotators may disagree not because they have different quality standards but because they interpreted the prompt differently. Mitigation: filter out prompts with high annotator disagreement.
- **Sycophancy leakage.** If annotators prefer responses that agree with premises in the prompt (even false ones), the reward model will learn to be sycophantic. Mitigation: include adversarial prompts with false premises and reward annotators for selecting responses that politely correct the user.

14.3 Training Reward Models in Practice

Chapter 10 derived the Bradley–Terry loss and proved that the reward model converges to the true reward function up to a constant. In practice, several additional considerations determine whether the reward model actually works.

14.3.1 Architecture Choices

The reward model $r_\psi(x, y)$ is typically a transformer that takes the concatenation of prompt and response as input and produces a scalar output. Two architectural strategies are common:

Same architecture as the policy. The reward model is initialised from the same SFT checkpoint as the policy, with the language modelling head replaced by a linear projection to a scalar. This is the approach used by InstructGPT and Llama 2. The advantage is that the reward model starts with a strong representation of language; the disadvantage is the large memory footprint, since the reward model is as large as the policy.

Separate, smaller model. The reward model uses a smaller transformer (e.g., a 7B reward model for a 70B policy). This reduces memory requirements but risks underfitting the preference distribution. Gao et al. (2023) showed that smaller reward models lead to faster overoptimisation (see Example 10.5), suggesting that reward model capacity should be as large as budget allows.

14.3.2 Training Stability and Overfitting

Reward models are prone to overfitting because preference datasets are relatively small (tens of thousands to low millions of pairs) compared to pretraining corpora (trillions of tokens). Standard mitigations include:

- **Low learning rates.** Typical values are 1×10^{-6} to 5×10^{-6}, an order of magnitude below SFT learning rates.
- **Early stopping.** Monitor pairwise accuracy on a held-out set and stop when it begins to decrease.
- **Label smoothing.** Replace the hard Bradley–Terry labels with $1 - \epsilon$ and ϵ for some small ϵ (typically 0.1), as in the cDPO formulation (Equation (10.10)).
- **Dropout.** Apply dropout ($p = 0.1$) to the final layers of the reward model.

14.3.3 Calibration and Evaluation Metrics

A reward model can have high pairwise accuracy yet still produce poorly calibrated scalar outputs. The key metrics to monitor are:

- **Pairwise accuracy** (Equation (10.11)): the fraction of held-out preference pairs where $r_\psi(x, y_w) > r_\psi(x, y_l)$. Values of 70–75% are typical for general-purpose alignment; 80%+ indicates a strong reward model.
- **Reward distribution.** Plot the distribution of r_ψ scores across a large set of responses. The distribution should be unimodal and roughly centred; a bimodal distribution suggests the reward model has learned a spurious binary feature (e.g., response length).
- **Agreement with human rankings.** On a small set of prompts with four or more responses ranked by experts, compute the Kendall's τ between the reward model ranking and the expert ranking.

14.3.4 When to Retrain

The reward model must be retrained when the policy has drifted far enough from the distribution on which the reward model was trained. A practical heuristic: monitor the KL divergence between the current policy and the reference policy. When $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ exceeds a threshold (typically 5–15 nats, depending on model size), collect new preference data from the current policy and retrain the reward model. This is the iterative RLHF loop described in Section 14.1.

14.4 PPO for Language Models: Implementation Details

Applying PPO (Chapter 9) to language model alignment is not a straightforward exercise. The token-level MDP structure of language generation (Definition 11.3), the enormous parameter count, and the need to compute likelihoods under both the policy and the reference model create a unique set of engineering challenges. This section describes the standard implementation.

14.4.1 The Four-Model Setup

The PPO-based RLHF training loop maintains four neural networks simultaneously:

1. **Policy model** π_θ: the language model being trained. Generates responses and computes log-probabilities.
2. **Reference model** π_{ref}: a frozen copy of the SFT checkpoint. Used to compute the KL penalty.
3. **Reward model** r_ψ: a frozen reward model trained on preference data. Scores completed responses.
4. **Value model** V_ϕ: a critic network that estimates the expected future reward from each token position. Used for advantage estimation.

At the 7B parameter scale, each model requires approximately 14 GB in half precision (FP16/BF16). Holding all four in GPU memory simultaneously demands at least 56 GB — feasible on a single A100 (80 GB) with careful memory management, but tight when accounting for activations and the KV cache during generation. At 70B, the four models require roughly 560 GB, necessitating multi-GPU setups (Section 14.6).

14.4.2 KL Penalty Implementation

The KL penalty prevents the policy from straying too far from the reference. Two implementation strategies are common:

Per-token KL reward. At each token position t, subtract a KL penalty from the reward:

$$\tilde{r}_t = \begin{cases} -\beta \log \dfrac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}, & t < T, \\ r_\psi(x, y) - \beta \log \dfrac{\pi_\theta(y_T \mid x, y_{<T})}{\pi_{\text{ref}}(y_T \mid x, y_{<T})}, & t = T. \end{cases} \tag{14.1}$$

The terminal reward at $t = T$ includes both the reward model score and the KL penalty for the final token. This formulation distributes the KL signal across all token positions, which improves credit assignment. It is equivalent to Equation (10.15) from Chapter 10.

Sequence-level KL penalty. Add the total sequence-level KL divergence as a single penalty at the end of the sequence:

$$\tilde{r} = r_\psi(x, y) - \beta \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}.$$

This is simpler to implement but concentrates the entire KL signal at the final token, making advantage estimation less informative for intermediate tokens.

In practice, the per-token formulation is preferred.

14.4.3 Advantage Estimation with GAE

Given the modified per-token rewards $\{\tilde{r}_t\}$ and the value estimates $\{V_\phi(s_t)\}$, the generalised advantage estimate (GAE) from Chapter 9 is

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \delta_t = \tilde{r}_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \tag{14.2}$$

where γ is the discount factor and λ is the GAE parameter. For language model RLHF, the standard choice is $\gamma = 1$ (no discounting within a response) and $\lambda \in [0.95, 1.0]$. Setting $\gamma = 1$ is natural because the "episode" is a single response: there is no reason to discount future tokens within a sentence.

14.4.4 Mini-Batch Construction and Gradient Accumulation

A single PPO iteration proceeds as follows:

1. **Rollout phase.** Sample a batch of prompts from $\mathcal{D}$. Generate responses using π_θ. Score each response with r_ψ and compute per-token log-probabilities under both π_θ and π_{ref}.

2. **Advantage computation.** Compute GAE advantages using the value model V_ϕ.

3. **PPO update.** For K epochs (typically $K = 2$–4), shuffle the rollout data into mini-batches and perform gradient updates on the clipped PPO objective:

$$\mathcal{L}_t^{\text{clip}}(\theta) = -\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big), \tag{14.3}$$

   where $r_t(\theta) = \pi_\theta(y_t|x, y_{<t})/\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})$ is the importance ratio and ϵ is the clipping parameter.

4. **Value update.** Update V_ϕ by minimising the squared error against the GAE return targets.

When GPU memory is insufficient for the full mini-batch, *gradient accumulation* is used: forward and backward passes are computed on micro-batches, and gradients are summed before the optimiser step. This is mathematically equivalent to a larger batch size but requires only the memory of a single micro-batch.

14.4.5 Mixed Precision Training

Modern RLHF implementations use BF16 (bfloat16) for forward passes and FP32 for gradient accumulation and optimiser states. BF16 halves the memory footprint compared to FP32 while preserving the dynamic range needed for stable training. The value model is sometimes kept in FP32 to avoid numerical issues in the advantage computation, particularly when rewards span a wide range.

14.4.6 Hyperparameters

Table 14.1 lists typical hyperparameter ranges for PPO-based RLHF on language models of various sizes.

Hyperparameter	**Typical range**	**Notes**
Learning rate (policy)	5×10^{-7} – 2×10^{-6}	Cosine schedule, warmup 3–10%
Learning rate (value)	1×10^{-6} – 5×10^{-6}	Often 2–5× the policy LR
KL coefficient β	0.01 – 0.2	Start low, increase if KL explodes
Clip parameter ϵ	0.1 – 0.2	0.2 is the most common
GAE λ	0.95 – 1.0	1.0 common for LMs
Discount γ	1.0	No discounting within response
PPO epochs per batch K	2 – 4	More epochs risk overfitting
Batch size (prompts)	64 – 512	Larger is stabler
Mini-batch size	8 – 64	Limited by GPU memory
Max response length	512 – 2048 tokens	Task-dependent

Table 14.1: Typical PPO hyperparameters for RLHF on language models.

14.5 Debugging RLHF

RLHF is notoriously difficult to debug. Training runs are expensive, failure modes are subtle, and the interaction between four neural networks creates a large space of possible problems. This section catalogues the most common failures and their diagnostics.

14.5.1 Reward Hacking

Symptom: The reward model score increases rapidly, but the quality of generated text — as judged by humans — degrades. Typical manifestations include excessively verbose responses, repetitive praise or hedging phrases, and responses that exploit formatting quirks.

Diagnosis: Plot both the proxy reward (reward model score) and a gold metric (e.g., human win rate against a baseline) over training. Reward hacking produces a divergence: the proxy reward rises while the gold metric plateaus or drops.

Fixes:

- Increase the KL coefficient β. This is the first-line defence.
- Retrain the reward model on data generated by the current policy.
- Ensemble multiple reward models and use the minimum or median score.
- Add a length penalty to the reward to counteract verbosity hacking.

14.5.2 KL Divergence Explosion

Symptom: The KL divergence $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ grows rapidly (e.g., exceeding 20 nats within a few hundred steps), and training becomes unstable.

Diagnosis: Monitor the per-token KL at each position. If a few token positions have extremely high KL, the policy may have learned to always produce a specific token at those positions.

Fixes:

- Lower the learning rate.
- Increase β or switch to an adaptive KL penalty that adjusts β dynamically to maintain a target KL.
- Reduce the number of PPO epochs per batch (K).

14.5.3 Mode Collapse

Symptom: The policy generates nearly identical responses to different prompts. Diversity metrics (e.g., distinct n-grams, self-BLEU) drop precipitously.

Diagnosis: Sample 100 responses from the policy on diverse prompts and compute pairwise BLEU scores. A mean pairwise BLEU above 0.5 is a strong indicator of collapse.

Fixes:

- Increase β to keep the policy closer to the diverse reference.
- Increase the sampling temperature during rollouts.
- Add an entropy bonus to the PPO objective.

14.5.4 Training Instability

Symptom: The policy loss oscillates wildly, or the reward score is non-monotonic with large swings between batches.

Fixes:

- Reduce the learning rate and increase the batch size.
- Ensure that advantage estimates are normalised (zero mean, unit variance) within each mini-batch.
- Verify that the value model is not lagging behind the policy; if necessary, update it more frequently.
- Check for numerical issues in BF16: switch the value model head to FP32.

Metric	Healthy range	Red flag
Mean reward	Slowly increasing	Rapid spike (hacking)
KL divergence	1–15 nats	> 20 nats
Policy entropy	Stable or slowly decreasing	Sudden drop (collapse)
Clip fraction	0.05–0.15	> 0.3 (updates too large)
Value loss	Decreasing	Increasing (value model failing)
Mean response length	Stable	Monotonic increase (verbosity hack)
Approx. policy gradient norm	Stable	Spikes (instability)

Table 14.2: Diagnostic metrics for RLHF training. "Red flag" indicates a value that should trigger investigation.

14.5.5 Diagnostic Metrics to Monitor

Table 14.2 summarises the key metrics to log during RLHF training.

14.6 Scaling RLHF

Training RLHF at the scale of 70B+ parameter models requires distributing computation across many GPUs. The key challenge is that the RLHF training loop has two distinct phases — generation and training — with very different computational profiles.

14.6.1 Generation versus Training

The *generation phase* (rollout collection) is autoregressive: each token depends on all previous tokens, which limits parallelism within a single sequence. The primary parallelism axis is across sequences (data parallelism) and across transformer layers (tensor parallelism). Generation is memory-bound due to the KV cache, which grows linearly with sequence length.

The *training phase* (PPO updates) is a standard forward-backward pass over fixed-length sequences. It is compute-bound and parallelises well along the batch dimension (data parallelism) and model dimension (tensor parallelism or pipeline parallelism).

14.6.2 Parallelism Strategies

Data parallelism Replicate the model on each GPU; each GPU processes a different subset of the batch. Gradients are synchronised with AllReduce. Works for both generation and training, but requires each GPU to hold a full model copy.

Tensor parallelism Split individual weight matrices across GPUs within a node. Reduces per-GPU memory but requires high-bandwidth interconnects (NVLink). Essential for models that do not fit on a single GPU.

Pipeline parallelism Split the model into stages (groups of layers), each assigned to a different GPU. Useful for training when combined with micro-batch scheduling (e.g., 1F1B) to reduce bubble time.

Async rollout generation Decouple the generation and training phases: a pool of GPUs generates rollouts asynchronously while another pool performs PPO

updates on completed rollouts. This is the approach used by OpenAI's RLHF infrastructure and by open-source frameworks such as DeepSpeed-Chat and OpenRLHF.

14.6.3 Compute Budget Estimates

Table 14.3 provides rough compute estimates for a single round of RLHF training at different model scales, assuming 50,000 prompts and 4 PPO epochs.

Model size	GPUs (A100 80 GB)	Time (approx.)	Cost (approx.)
7B	8	12–24 hours	\$500–1,000
13B	16	1–2 days	\$2,000–4,000
70B	64	3–5 days	\$20,000–40,000

Table 14.3: Approximate compute budgets for a single RLHF training round. Costs assume cloud GPU pricing of \$2–3/GPU-hour.

These estimates are highly variable and depend on response length, batch size, the number of PPO epochs, and the efficiency of the distributed training framework. Nevertheless, they illustrate a key point: RLHF training is expensive, but it is a small fraction of the cost of pretraining. A typical pretraining run for a 70B model consumes millions of GPU-hours; a single RLHF round consumes thousands.

14.7 Safety and Alignment Constraints

Maximising reward is necessary but not sufficient for a safe deployment. This section discusses the additional constraints that must be layered on top of the core RLHF training loop.

14.7.1 Red-Teaming

Red-teaming is the systematic process of probing a model for harmful outputs. A red team generates adversarial prompts designed to elicit toxic, biased, or dangerous responses. The resulting failures are used to construct targeted preference data (where the harmful response is marked as rejected), which is then incorporated into the next round of reward model and policy training.

Red-teaming is most effective when it is *iterative*: the red team attacks the latest checkpoint, failures are patched, and the red team attacks again. Ganguli et al. (2022) found that red-teaming a language model for 3–4 rounds reduced the attack success rate by 75% compared to a single round.

14.7.2 Constitutional AI Integration

Constitutional AI (Bai et al., 2022) augments RLHF with a set of explicit principles (a "constitution") that the model must follow. In the standard implementation, a language model critiques its own outputs against the constitution and generates revised responses. These revised responses are used as the preferred response in a preference pair, creating synthetic preference data without human annotation. The constitutional approach can be integrated into the RLHF pipeline as an additional data source for reward model training.

14.7.3 Helpfulness versus Harmlessness

A persistent tension in alignment is the tradeoff between helpfulness and harmlessness. A model trained aggressively on harmlessness may refuse to answer benign questions that superficially resemble harmful ones (over-refusal). Conversely, a model trained primarily on helpfulness may comply with harmful requests.

The standard mitigation is to train separate reward models for helpfulness and harmlessness (or use a single reward model with separate heads) and combine them during RL training:

$$r(x, y) = r_{\text{helpful}}(x, y) + \alpha\, r_{\text{harmless}}(x, y),$$

where α controls the weight given to harmlessness. Llama 2 used a variant of this approach, with α increasing over the course of training to emphasise safety in later stages.

14.7.4 When to Stop Training

There is no single criterion for when to stop RLHF training. In practice, teams use a combination of:

- **Human evaluation plateaus.** When successive checkpoints show no improvement on human evaluations (e.g., Elo ratings from head-to-head comparisons), further training is unlikely to help.
- **KL budget exhaustion.** Some teams set a maximum KL budget (e.g., 10 nats) and stop training when this budget is reached, reasoning that further divergence from the reference increases the risk of overoptimisation.
- **Safety regression.** If safety benchmarks degrade relative to a previous checkpoint, training should be rolled back.

14.8 Case Studies

This section summarises the alignment approaches of four influential systems. Each represents a different point in the design space.

14.8.1 InstructGPT (Ouyang et al., 2022)

InstructGPT was the first large-scale demonstration of RLHF for language models. The pipeline followed the three-stage recipe: SFT on 13,000 demonstrations, reward model training on 33,000 preference pairs, and PPO fine-tuning. Key engineering decisions included: using a 6B reward model for a 175B policy (choosing to save memory at the cost of reward model capacity); running 2 full iterations of the pipeline; and using a KL coefficient of $\beta = 0.02$. The resulting model was preferred over the 175B SFT baseline by human evaluators 71% of the time.

14.8.2 Llama 2 (Touvron et al., 2023)

Llama 2's alignment effort introduced several innovations. The team collected 1.4 million preference pairs over 5 iterations of the RLHF loop, with fresh data from the latest policy at each iteration. They trained two separate reward models — one for helpfulness, one for safety — and combined them with a safety-weighted coefficient that increased during

training. A notable technique was *rejection sampling*: at each iteration, the team generated K responses per prompt and selected the best according to the reward model, creating synthetic SFT data that was used to warm-start the next iteration's policy. This "RLHF + rejection sampling" combination proved more effective than either technique alone.

14.8.3 Anthropic's Approach (Bai et al., 2022)

Anthropic's alignment work for Claude introduced Constitutional AI and emphasised the harmlessness dimension. The key contributions were: the constitutional self-critique mechanism, which generates preference data from the model's own revisions; training with a combined helpfulness-harmlessness reward; and extensive red-teaming with iterative patching. Anthropic also pioneered the use of an online RLHF setup where the reward model is periodically retrained on preferences collected from the latest policy, addressing the distribution shift problem that plagues offline methods like DPO (Section 12.5).

14.8.4 DeepSeek's Approach with GRPO (DeepSeek-AI, 2024)

DeepSeek took a different path by replacing PPO with Group Relative Policy Optimisation (GRPO, Chapter 13). For tasks with verifiable rewards — mathematics, coding, and factual questions — GRPO eliminates the reward model entirely by using correctness signals (e.g., unit test pass rates, symbolic verification) as the reward. For open-ended tasks, DeepSeek combined GRPO with a reward model. The key engineering advantage is the elimination of the value model: GRPO normalises rewards within a group of responses to the same prompt, which provides a baseline without requiring a separate critic. This reduces the four-model setup to two models (policy and reference), halving the memory requirement and simplifying the distributed training infrastructure.

14.9 Summary

This chapter has provided a practitioner's guide to implementing RLHF at scale. The core lessons are:

1. The RLHF pipeline has six stages (data collection, SFT, reward model training, RL training, evaluation, deployment), typically run iteratively.

2. Preference data quality is more important than quantity. Careful annotation guidelines, randomised presentation order, and attention to biases (length, position, sycophancy) are essential.

3. Reward models should be as large as the compute budget allows, trained with low learning rates, and monitored for overfitting. They must be retrained when the policy drifts significantly.

4. The PPO implementation for language models requires four models in memory simultaneously. Per-token KL penalties, GAE with $\gamma = 1$, and careful mixed-precision management are the standard engineering choices.

5. RLHF failures — reward hacking, KL explosion, mode collapse — are common and have well-known diagnostic patterns and fixes. Monitoring the metrics in Table 14.2 is essential.

6. Scaling to large models requires combining data, tensor, and pipeline parallelism with asynchronous rollout generation.

7. Safety constraints — red-teaming, constitutional AI, and the helpfulness-harmlessness tradeoff — must be integrated into the training loop, not bolted on after the fact.

The case studies illustrate that there is no single "correct" way to implement RLHF. InstructGPT used the classical PPO pipeline. Llama 2 added rejection sampling and iterative data collection. Anthropic pioneered constitutional self-critique. DeepSeek replaced PPO with GRPO for verifiable tasks. Each approach reflects different tradeoffs between engineering complexity, data requirements, and alignment quality. The common thread is that successful alignment requires not just the right algorithm but also meticulous attention to data quality, training stability, and safety evaluation.

Exercises

Exercise 14.1. A team is training a 7B language model with PPO-based RLHF. After 500 training steps, the KL divergence $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ has reached 18 nats and the mean reward model score has increased from 2.1 to 4.8.

1. Is the KL divergence in a healthy range? Justify your answer using Table 14.2.

2. The team observes that the mean response length has increased from 120 tokens to 350 tokens. What failure mode does this suggest? Propose two concrete fixes.

3. Propose an adaptive scheme for the KL coefficient β that maintains a target KL of $d_{\mathrm{target}} = 6$ nats. Write the update rule.

Exercise 14.2. Consider the four-model PPO setup (Section 14.4.1) for a 13B parameter model.

1. Estimate the total GPU memory required to hold all four models in BF16, ignoring activations and KV cache.

2. The team wants to train on 8× A100 (80 GB each). Is data parallelism alone sufficient, or is tensor parallelism required? Justify your answer.

3. GRPO eliminates the value model. How much memory does this save, and how does it change the parallelism requirements?

Exercise 14.3. Design an experiment to detect reward hacking. Given a reward model r_ψ and access to a pool of 500 human annotators:

1. Describe a protocol for constructing a "gold" evaluation set that the reward model has never seen.

2. Define a quantitative metric that distinguishes genuine improvement from reward hacking. Be precise about what you would plot and what pattern indicates hacking.

3. The team notices that the reward model gives a score of +5.2 to the response "I'm happy to help! Here is a comprehensive, detailed, and thorough answer to your question..." regardless of what follows. What type of reward hacking is this, and how would you fix the reward model?

Exercise 14.4. You are designing an annotation pipeline for training a reward model for a customer-support chatbot.

1. Write annotation guidelines (3–5 sentences) for pairwise comparison that address length bias and sycophancy.
2. Estimate the cost of collecting 50,000 preference pairs using expert annotators at \$10 per comparison. If you have a budget of \$200,000, what alternative data collection strategies could you use to augment the expert-annotated data?
3. How would you measure inter-annotator agreement on your dataset? What minimum agreement level would you require before using the data for reward model training?

Exercise 14.5. Consider a response of length $T = 4$ tokens with per-token rewards $\tilde{r}_1 = 0.1$, $\tilde{r}_2 = -0.2$, $\tilde{r}_3 = 0.0$, $\tilde{r}_4 = 1.5$ and value estimates $V_1 = 0.8$, $V_2 = 0.6$, $V_3 = 0.9$, $V_4 = 1.0$, $V_5 = 0$ (terminal state).

1. Compute the TD residuals $\delta_1, \ldots, \delta_4$ with $\gamma = 1$.
2. Compute the GAE advantages $\hat{A}_1, \ldots, \hat{A}_4$ with $\lambda = 0.95$.
3. Repeat part (b) with $\lambda = 1.0$ and compare. Which setting produces higher-variance advantage estimates, and why?

Exercise 14.6. A research lab wants to run RLHF on a 70B parameter model using PPO. They have a cluster of 64 A100 GPUs (80 GB each) connected with 400 Gbps NVLink within nodes of 8 GPUs and 100 Gbps InfiniBand between nodes.

1. Propose a parallelism strategy for the generation phase and for the training phase. Specify the tensor parallelism degree, data parallelism degree, and pipeline parallelism degree for each phase.
2. The generation phase is autoregressive and memory-bound. Explain why tensor parallelism helps even though it introduces communication overhead.
3. The team considers asynchronous rollout generation. Describe the potential staleness problem: the policy used for generation may differ from the policy being updated. Under what conditions is this staleness acceptable?

Exercise 14.7. Compare the InstructGPT, Llama 2, and DeepSeek alignment approaches along the following dimensions:

1. Number of models required in GPU memory during RL training.
2. Type and quantity of human preference data used.
3. Strategy for addressing reward model staleness (distribution shift).
4. Suitability for tasks with verifiable rewards (e.g., mathematics).

Present your comparison in a table and write a brief paragraph recommending which approach a startup with limited compute budget should adopt for a general-purpose assistant.

Part IV

Frontiers

Chapter 15

Reasoning, Tool Use, and Reinforcement Finetuning

If intelligence is a cake, the bulk of the cake is unsupervised learning, the icing on the cake is supervised learning, and the cherry on the cake is reinforcement learning.

— Yann LeCun, NeurIPS 2016

For most of the history recounted in this book, reinforcement learning occupied a curious position in the language-model training pipeline: theoretically central, practically marginal. The reward signal that drove RLHF (Chapter 14) was itself a learned neural network, — an imperfect proxy for an imperfectly-understood goal, — and every practitioner knew that pushing the policy too far against such a reward model would lead to reward hacking rather than genuine improvement. The result was a careful, tightly KL-constrained training regime in which RL contributed polish rather than power.

That picture changed sharply between 2024 and 2025. The catalyst was the recognition that certain domains admit *verifiable* rewards: a mathematical answer is either correct or incorrect; a piece of code either passes the test suite or it does not. When rewards can be computed by a deterministic checking function rather than inferred from a neural preference model, the dangers of reward hacking evaporate, and it becomes safe to train with far more aggressive RL for far longer. The result — *Reinforcement Learning with Verifiable Rewards* (RLVR) — produced a qualitative leap in reasoning ability: models that now spend thousands of tokens thinking before answering a hard question, that can write and debug complex programs, and that achieve expert-level performance on competition mathematics.

This chapter is organised to build understanding progressively. We begin by making the RLVR objective precise and contrasting it with the RLHF objective of Part III (Section 15.1). We then examine chain-of-thought reasoning as both a prompting technique and a trainable behavior (Section 15.2). Section 15.3 studies the training recipes used in practice, with DeepSeek R1 as a worked case study. Section 15.4 formalises inference-time scaling: the remarkable phenomenon that generating more tokens at test time translates into measurable accuracy gains. Sections 15.5 and 15.6 extend the framework to *tool use* and *agentic RL*, where the model must interact with external environments over multiple steps. Section 15.7 surveys practical algorithm and infrastructure choices, and Section 15.8 closes with open questions.

15.1 From RLHF to Reinforcement Learning with Verifiable Rewards

15.1.1 The Verification Insight

Recall from Chapter 14 that RLHF maximises

$$J_{\text{RLHF}}(\theta) = \mathbb{E}_{\pi_\theta}\big[r_\phi(x, y)\big] - \beta\, D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})\,, \tag{15.1}$$

where r_ϕ is a *learned* reward model trained on human preference pairs and $\beta > 0$ is a KL penalty coefficient that prevents the policy from straying too far from a reference model. The learned reward model introduces two compounding problems. First, it is an approximation: even a well-trained reward model will assign incorrect scores to some fraction of responses, and a sufficiently strong policy can discover and exploit these errors — a phenomenon called *reward overoptimisation* or Goodhart's Law in RL (Chapter 10). Second, training a reward model requires expensive human annotation of preference pairs, and the resulting model inherits all the biases and inconsistencies of those annotations.

Reinforcement Learning with Verifiable Rewards (RLVR) sidesteps both problems by replacing the learned reward model with a deterministic **verification function**:

$$r(x, y) = \begin{cases} 1 & \text{if } \textsc{Verify}(x, y) = \textsc{True}, \\ 0 & \text{otherwise.} \end{cases} \tag{15.2}$$

The function Verify is domain-specific and entirely symbolic: for mathematics it extracts the final answer from the model's response (using a regular expression to locate content in `\boxed{}` notation or after a delimiter such as `The answer is:`) and compares it to the ground-truth solution; for code it executes the response against a test suite and returns True if all assertions pass.

Definition 15.1 (RLVR objective)**.** Let $\mathcal{D}$ be a dataset of (prompt, ground-truth answer) pairs (x, a^*). The RLVR objective is

$$J_{\text{RLVR}}(\theta) = \mathbb{E}_{(x,a^*)\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[\mathbb{1}\{\textsc{Verify}(y, a^*)\}\big] - \beta\, D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})\,, \tag{15.3}$$

where $\mathbb{1}\{\cdot\}$ is the indicator function and the KL term may be reduced or removed entirely (see Section 15.3).

Because r takes values in $\{0, 1\}$ and is computed by a deterministic symbolic function, it cannot be over-optimised in the same way as a neural reward model. A model that learns to produce correct answers *is* genuinely better; there is no proxy gap to exploit.

15.1.2 Verifiable Domains

The power of RLVR depends on having a rich, varied supply of problems with efficiently verifiable answers. Three domains currently dominate.

Mathematics Competition problems drawn from datasets such as MATH and AIME admit unique numerical or symbolic answers that can be checked exactly. The model is prompted to enclose its final answer in a `\boxed{}` environment, which a regular expression extracts and compares to the ground truth.

Code generation Problems from LeetCode, HumanEval, or MBPP come with unit tests. A model's solution is executed in a sandbox, and the reward is 1 if all tests pass (or a fraction equal to the proportion of passing tests for partial-credit variants).

Science and logic Formal theorem proving (e.g., in Lean 4) and structured scientific question-answering with unique factual answers are natural extensions. Any domain where a checking function can be written is in principle amenable to RLVR.

15.1.3 Formal Comparison with the RLHF Objective

Table 15.1 contrasts the two frameworks along the dimensions most relevant to practitioners.

Table 15.1: RLHF versus RLVR: a formal comparison.

Property	**RLHF**	**RLVR**
Reward source	Learned neural r_ϕ	Symbolic VERIFY
Reward range	$\mathbb{R}$ (continuous)	$\{0, 1\}$ (binary)
Human annotation	Required for r_ϕ	Required for (x, a^*) pairs
Over-optimisation risk	High	Negligible
KL penalty necessity	Critical	Optional
Applicable domains	General	Verifiable domains
Reward model cost	O(response length)	O(checking function)

The key structural difference is that RLVR converts the RL problem into one where the reward is *not* a neural network with gradients of its own, but a read-only oracle. This makes the optimisation landscape cleaner: there is no inner loop of reward model training to interleave with policy updates, no distributional shift problem for the reward model, and no danger of the policy "finding the gaps" in a learned approximation.

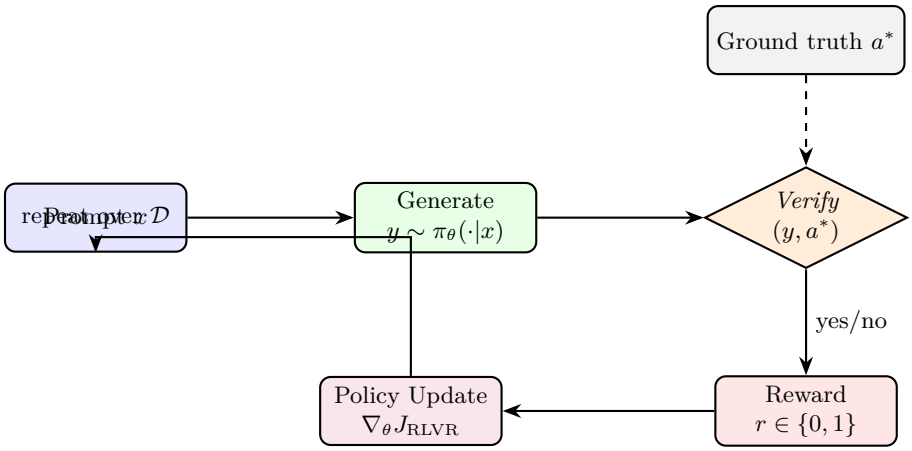


Figure 15.1: The RLVR training loop. A prompt x from the dataset is fed to the current policy π_θ, which generates a response y. A deterministic verification function compares y against the ground-truth answer a^* and returns a binary reward. Policy gradients are then computed and the cycle repeats. No learned reward model is needed.

15.2 Chain-of-Thought Reasoning

15.2.1 The Basic Idea

Standard autoregressive generation maps a prompt x directly to a response y. Chain-of-thought (CoT) generation interposes a *reasoning trace* z between the prompt and the final answer:

$$x \longrightarrow z \longrightarrow a,$$

where z is a sequence of intermediate reasoning steps and a is the final answer extracted from the end of the generation. The full generation $y = (z, a)$ is produced by the same autoregressive model; z is simply not shown to the user in many deployments.

Example 15.1 (Chain-of-thought on a competition problem)**.** Consider the prompt: *"Find all integer solutions to $x^2 - 5x + 6 = 0$."* Without chain-of-thought, a model might emit the correct answer $\{2, 3\}$ directly (if it has memorised it) or fail silently. With chain-of-thought, the generation looks like:

```
<thinking>
I need to factorise x^2 − 5x + 6.  I am looking for two numbers
that multiply to 6 and add to −5.  Those numbers are −2 and −3.
So x^2 − 5x + 6 = (x − 2)(x − 3).  Setting each factor to zero:
x − 2 = 0 ⇒ x = 2;  x − 3 = 0 ⇒ x = 3.
</thinking>
The integer solutions are x = 2 and x = 3, i.e., \boxed{2, 3}.
```

The thinking tokens are invisible to downstream users but are used during training: the verification function extracts the content of the `\boxed{}` environment to compute the reward.

15.2.2 Why Longer Reasoning Helps

Chain-of-thought succeeds for an information-theoretic reason. The probability that a model produces the correct final answer a^* given prompt x can be written, by the chain rule of probability, as

$$\mathbb{P}(a^* \mid x) = \sum_z \mathbb{P}(a^* \mid x, z)\, \mathbb{P}(z \mid x). \tag{15.4}$$

Marginalising over intermediate reasoning traces z is equivalent to using them as a latent variable. A model that can generate a wide diversity of high-quality reasoning traces effectively performs an implicit search over solution paths before committing to an answer. The longer and more varied the reasoning, the finer the implicit search grid.

This connection to *search* is not merely metaphorical. We can formalise it by defining the quality of a reasoning trace z for prompt x and answer a^* as

$$q(z; x, a^*) = \mathbb{P}(a^* \mid x, z).$$

The model's effective accuracy is $\mathbb{E}_z[q(z; x, a^*)]$, where the expectation is over the model's distribution over traces. A longer context window admits more information in z, which raises the ceiling of $q(z; x, a^*)$ for hard problems. Empirically, across competition mathematics benchmarks, the correlation between average output token count and accuracy is strong and monotone over several orders of magnitude of compute — a scaling law at inference time (Section 15.4).

15.2.3 Thinking Tokens and Format

Modern reasoning models formalise chain-of-thought by using special *thinking tokens*. The generation is structured as:

$$y = \underbrace{\texttt{<thinking>}\ z_1 z_2 \cdots z_T\ \texttt{</thinking>}}_{\text{reasoning trace, not shown to user}}\ \underbrace{a_1 a_2 \cdots a_K}_{\text{final answer}}. \tag{15.5}$$

The token sequence $z_1, \ldots, z_T$ is sometimes called the *scratchpad*. For a simple arithmetic question, T might be a few dozen tokens; for a hard competition problem, T may reach tens of thousands of tokens, corresponding to several minutes of generation on modern hardware.

Remark 15.1 (Long context as a prerequisite). Training reasoning models requires the policy to generate and attend over very long sequences. The practical enabler of this shift was the development of efficient attention mechanisms (e.g., FlashAttention) and context windows of 128k–1M tokens. Without these engineering advances, the training regime described in Section 15.3 would not have been computationally feasible.

15.2.4 Connection to Inference-Time Compute

The extended generation format of (15.5) directly links training to inference-time compute allocation. Define the *token budget* B as the maximum number of tokens the model may generate before producing a final answer. For a fixed policy π_θ, accuracy on a problem set is a non-decreasing function of B:

$$\mathrm{Acc}(B) = \mathbb{P}_{y \sim \pi_\theta(\cdot|x),\, |y| \le B}\big[\textsc{Verify}(y, a^*) = \textsc{True}\big].$$

Training with RLVR on unconstrained generation teaches the model to *self-allocate* the token budget: hard problems trigger longer traces, easy ones resolve quickly. This adaptive allocation is a key practical advantage over fixed-compute alternatives.

15.3 Training Reasoning Models

15.3.1 The Two-Stage Recipe

The dominant training recipe for reasoning models consists of two stages, illustrated here using DeepSeek R1 as the canonical open example.

Stage 1: SFT cold start The base pretrained model is first fine-tuned with supervised learning on a relatively small dataset of high-quality chain-of-thought demonstrations. This stage teaches the model the basic format — the use of `<thinking>` tags, the organisation of multi-step reasoning, — and ensures that subsequent RL has a well-behaved starting distribution to perturb. Without this cold start, RLVR training from a base model can diverge or produce garbled reasoning traces.

Stage 2: Large-scale RLVR The SFT-initialised model is then trained with a policy-gradient algorithm (typically GRPO or a variant; see Section 15.7) over a large dataset of verifiable problems. Unlike RLHF, which typically runs for $O(1)$ epochs over preference data, RLVR revisits the same training problems hundreds or thousands of times, allowing the policy to discover and reinforce correct reasoning patterns that were only latent in the base model.

Example 15.2 (DeepSeek R1 training recipe)**.** DeepSeek R1 (January 2025) trained a base model with approximately 600 billion parameters on a curriculum that mixed mathematical reasoning problems (primarily from competition mathematics datasets), code synthesis problems with unit-test verification, and general instruction-following tasks. The RL stage used GRPO with binary rewards from the verification function, combined with small auxiliary format rewards to encourage consistent use of the `<thinking>` structure. The model was subsequently distilled to smaller sizes (7B–70B) by fine-tuning on outputs of the full model, making strong reasoning capabilities accessible on consumer hardware.

15.3.2 Difficulty Filtering

A subtle but critical aspect of RLVR training is the choice of which problems to include in the training set. Recall the policy gradient update: the gradient of J_{RLVR} with respect to θ is non-zero only when different rollouts to the same prompt receive different rewards. If the current policy solves a problem 100% of the time, all rollouts receive reward 1, the advantage is identically zero, and the gradient vanishes. Similarly, if the policy fails 100% of the time, all rollouts receive reward 0 and no gradient is generated.

Definition 15.2 (Problem solvability rate)**.** For a problem x with ground truth a^* and policy π_θ, the **solvability rate** is

$$\sigma(x) = \mathbb{P}_{y \sim \pi_\theta(\cdot|x)}\big[\textsc{Verify}(y, a^*) = \textsc{True}\big],$$

estimated by sampling N independent rollouts (typically $N = 16$).

Empirically, problems with $\sigma(x) \in [0.2, 0.8]$ yield the strongest gradients. Problems outside this window — too easy or too hard — are filtered out of the training set before RLVR begins. This *offline difficulty filtering* is now a standard step in reasoning-model pipelines, used by DeepSeek R1, Phi-4 Reasoning, INTELLECT-2, Skywork OR-1, and others.

15.3.3 Removing the KL Penalty

Standard RLHF uses the KL penalty of (15.1) to prevent the policy from collapsing onto reward-hacking outputs. In RLVR, the binary reward cannot be hacked by parroting a fixed output, because the verification function is deterministic and problem-specific. As a result, many RLVR practitioners reduce β to zero, removing the KL constraint entirely. This allows the policy to explore more freely and to discover reasoning patterns that were not present in the reference model at all. Models that have removed the KL penalty include Magistral (Mistral, 2025), OpenReasoner-Zero, and Skywork OR-1.

Remark 15.2 (KL removal and distributional shift)*.* Removing the KL penalty does introduce a risk: the policy may drift to a distribution on which the verifier itself is unreliable (e.g., if the model learns to extract answers from positions that the regex does not match). Careful prompt engineering and verifier robustness testing are therefore more important in no-KL regimes than in standard RLHF.

15.3.4 Format Rewards and Length Penalties

In addition to the binary task reward, practitioners commonly add small auxiliary rewards:

- **Format rewards**: A small positive reward (≈ 0.1) is given when the response correctly uses the `<thinking>` … `</thinking>` structure before producing the final answer. This prevents the model from drifting to a format that is difficult to parse.

- **Length penalties**: Reasoning traces that are longer than necessary consume extra inference compute without improving accuracy. A penalty proportional to $\max(0, |z| - L_{\max})$ discourages pathological overthinking while preserving the model's ability to use long traces when genuinely needed. Some recipes (e.g., Kimi 1.5) use a progressive length curriculum: the model first learns to reason within a tight token budget and the budget is expanded as training progresses.
- **Language consistency rewards**: Multilingual models can drift to switching languages mid-trace (e.g., starting in English and finishing in Chinese), which degrades user experience. A binary reward for language consistency is used by DeepSeek R1 and Magistral.

15.3.5 Loss Normalisation and the GRPO Bias

The GRPO algorithm (Chapter 13) computes advantages by normalising rewards within each group of rollouts for the same prompt. A subtle consequence is that the contribution of a problem to the total loss is independent of the number of rollouts generated for it. If harder problems systematically require more tokens, normalising at the group level can over-weight easy, short problems and under-weight hard ones. Several reasoning model recipes (including Magistral and MiMo) address this by normalising advantages at the *batch* level rather than the group level, or by weighting the loss by token count.

15.3.6 Self-Taught Reasoning and Distillation

Two important variants of the standard recipe depart from pure RL.

STaR (Self-Taught Reasoner; Zeiler et al., 2022) and its descendant **Quiet-STaR** approximate the policy gradient by iterating between: (1) sampling reasoning traces from the current model, (2) filtering to those that lead to correct answers, and (3) fine-tuning the model on the filtered traces with cross-entropy loss. This is equivalent to a simplified form of RL where the reward is binary and the baseline is zero. STaR approximates the gradient of (15.3) without requiring an explicit advantage computation, at the cost of some statistical efficiency. Quiet-STaR adds the innovation of generating thinking tokens before every token prediction, not just at the beginning of the response, which provides a richer training signal.

Distillation transfers reasoning capability from a large, RLVR-trained model to a smaller student model via supervised fine-tuning on the teacher's chain-of-thought outputs. Because the student learns from the teacher's completed reasoning traces — including the thinking tokens — it can reach reasoning performance far above what direct RLVR on the student alone would achieve. DeepSeek R1 released distilled checkpoints at 7B and 70B that outperformed their same-size non-distilled counterparts on competition mathematics.

15.4 Inference-Time Scaling

15.4.1 The Test-Time Compute Hypothesis

Inference-time scaling is the empirical finding that, for a fixed trained policy, allocating more computation at test time — by generating more tokens, more responses, or both — leads to measurable improvements in task accuracy. This is distinct from training-time scaling (larger models or more training data): it is scaling at deployment.

Let $\mathcal{P}$ denote a set of benchmark problems and let C be the compute budget measured in FLOPs (or, equivalently, in total output tokens generated). The inference scaling law

asserts that there exists a function f such that

$$\mathrm{Acc}(C) \approx f(C) = f_0 + \alpha \log C + O((\log C)^{-1}), \tag{15.6}$$

for constants f_0 and $\alpha > 0$. Empirically, this log-linear relationship holds over two to three orders of magnitude of test-time compute on competition mathematics benchmarks, before saturating as problems are exhausted.

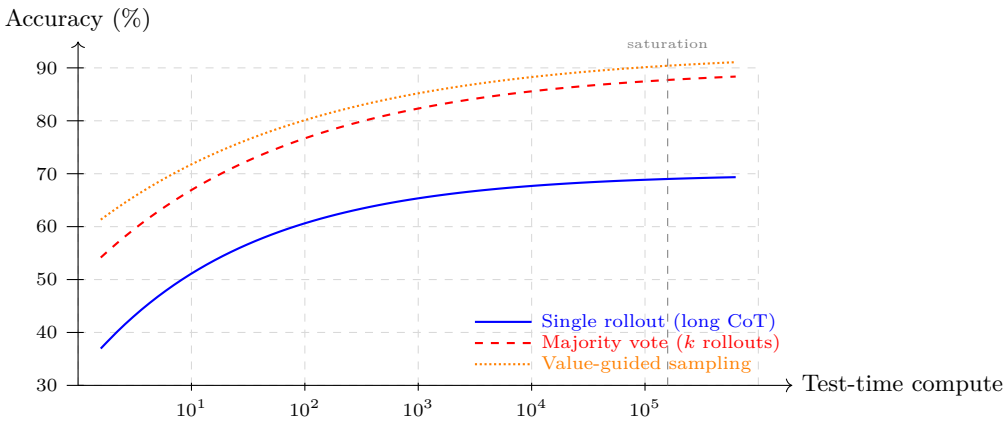


Figure 15.2: Schematic inference-time scaling curves on competition mathematics. Accuracy grows log-linearly with test-time compute for a range of strategies. Majority voting over k parallel rollouts offers higher accuracy at the same compute than a single long rollout; value-guided sampling extracts further gains by using a process reward model to steer beam search. All curves saturate once the model's capability ceiling is reached.

15.4.2 Majority Voting

The simplest inference-time scaling strategy is *majority voting* (also called *self-consistency*): sample k independent responses $y_1, \ldots, y_k$ from the policy, extract the final answer a_i from each, and return the plurality answer:

$$\hat{a} = \operatorname*{argmax}_a \sum_{i=1}^{k} \mathbb{1}\{a_i = a\}. \tag{15.7}$$

If individual responses are correct with probability $p > \frac{1}{2}$, the majority vote is correct with probability that approaches 1 exponentially fast in k (a consequence of the law of large numbers). In practice, $k = 8$–64 rollouts suffices for most competition benchmarks.

Proposition 15.1 (Majority vote accuracy bound)**.** *Suppose each of k independent rollouts is correct with probability $p \in (\frac{1}{2}, 1)$ and there are at most m possible answers. The majority vote errs with probability at most*

$$\mathbb{P}(\hat{a} \neq a^*) \leq \binom{k}{\lfloor k/2 \rfloor} p^{\lfloor k/2 \rfloor}(1-p)^{k-\lfloor k/2 \rfloor} \;\leq\; \exp\!\big(-k\, D_{\mathrm{KL}}(1/2 \,\|\, 1-p)\big),$$

where D_{KL} denotes the binary KL divergence. The error decreases exponentially in k.

15.4.3 Value-Guided Sampling

Majority voting treats all rollouts as equally plausible. A more sophisticated strategy uses a *process reward model* (PRM) that assigns scores to *intermediate* reasoning steps, not only to the final answer. Given a partial trace $z_{1:t}$, the PRM predicts the probability that the reasoning will lead to a correct final answer. This score can guide generation via beam search or best-of-k selection:

$$\hat{y} = \operatorname*{argmax}_{y \in \{y_1, \ldots, y_k\}} \mathrm{PRM}(y). \tag{15.8}$$

Value-guided sampling consistently outperforms majority voting at equal compute, because it concentrates sampling budget on promising reasoning paths rather than averaging over all of them. The overhead is the cost of running the PRM scorer, which is typically a much smaller model than the policy.

15.4.4 Compute-Optimal Inference

A natural question is how to split a fixed compute budget C between the length of individual traces (more thinking tokens per rollout) and the number of rollouts k. Defining the per-rollout cost as $c(B) \propto B$ (where B is the token budget per rollout), the total compute is $C = k \cdot c(B)$. The compute-optimal strategy solves:

$$(k^*, B^*) = \operatorname*{argmax}_{k,\, B:\, k \cdot B = C} \mathrm{Acc}(k, B),$$

where $\mathrm{Acc}(k, B)$ is the accuracy under majority vote with k rollouts of length B. Empirically, this tradeoff favours longer individual traces for harder problems (which need deep reasoning) and more rollouts for easier problems (where the model is already close to correct and diversity is valuable). Adaptive token-budget allocation within a single long generation is an active research direction.

15.5 Tool Use and Function Calling

15.5.1 Motivation

Language models trained purely on text have a fundamental limitation: they cannot access information beyond their training data, execute code, or interact with external services. Tool use extends the model's action space from token generation to *function calls*: structured outputs that request the execution of an external tool and embed the result in the subsequent generation. This is a qualitative extension of the generation paradigm:

$$x \longrightarrow (z_1, \underbrace{f_1(q_1)}_{\text{tool call}}, r_1, z_2, \underbrace{f_2(q_2)}_{\text{tool call}}, r_2, \ldots, a), \tag{15.9}$$

where f_i is a tool (e.g., a web search API or a Python interpreter), q_i is the query the model submits to it, r_i is the result returned, and the model's generation interleaves reasoning (z_i) with tool calls and their results.

15.5.2 Function Calling Format

In practice, tool calls are expressed as structured JSON objects embedded in the generation stream. A typical format:

Example 15.3 (Tool call in JSON format)**.** Consider a model answering: *"What is the current population of France?"* The generation might look like:

```
<thinking>
I need to look up the current population of France.
</thinking>
{
"tool": "web_search",
"query": "current population of France 2025"
}
```

[Tool result injected: "France population: approximately 68.4 million (2025)"]

```
The current population of France is approximately 68.4 million.
```

The JSON object is parsed by the inference runtime, the tool is invoked, and the result is appended to the model's context before generation continues. The model never sees raw API internals; it interacts with a clean interface of named tools with typed parameters.

The model must learn three capabilities simultaneously: (1) to recognise when a tool call is needed, (2) to emit well-formed JSON that matches the tool's schema, and (3) to correctly incorporate the tool's result into its subsequent reasoning.

15.5.3 Training Tool Use

Tool use is typically trained in two stages:

SFT on demonstrations The model is first fine-tuned on a curated dataset of tool-use trajectories: (prompt, tool-call sequence, final answer) triples collected from human demonstrations or from larger models. This teaches the model the format and the basic decision of when to call a tool.

RL for multi-step tool use SFT alone is insufficient for complex, multi-step tool-use scenarios, because the correct sequence of tool calls depends on the results of previous calls in a way that is difficult to enumerate in a fixed demonstration dataset. RL allows the model to discover effective tool-calling strategies by interacting with a live tool environment and receiving a binary reward for task completion.

The reward signal for tool-use RL is typically outcome-based: the model receives reward 1 if the final answer (after all tool calls) is correct, and 0 otherwise. This is precisely the RLVR framework of Section 15.1, now extended to multi-step trajectories. The environment is the tool ecosystem itself.

15.5.4 SWiRL: Step-Wise RL for Tool Use

SWiRL (Step-Wise Reinforcement Learning) is a framework designed specifically for multi-step tool-use tasks where outcome rewards are sparse. Rather than waiting until the end of a trajectory to assign reward, SWiRL provides *process rewards* at each tool call. A step reward r_t is computed by a process reward model that evaluates whether the t-th tool call was useful and well-formed. The total reward is:

$$R = \sum_{t=1}^{T} \gamma^{t-1} r_t + \lambda \cdot r_{\text{outcome}}, \tag{15.10}$$

where $r_{\text{outcome}} \in \{0, 1\}$ is the binary task-completion reward and λ is a weighting hyperparameter. Step rewards reduce variance in the policy gradient estimate by providing more frequent feedback, at the cost of requiring a step-level evaluator.

15.6 Multi-Step Agents and Agentic RL

15.6.1 The Agent Loop

When tool use extends over many steps and the model must plan across a long horizon, we enter the regime of *agentic RL*. The model now functions as a full RL agent interacting with an environment $\mathcal{E}$ (the collection of available tools and external systems). The standard agent loop is:

1. **Observe**: Receive an observation o_t from the environment (e.g., the contents of a webpage, the output of a terminal command, or the response of an API).
2. **Think**: Generate a reasoning trace z_t conditioned on the full context $(x, o_1, a_1, \ldots, o_{t-1}, a_{t-1}, o_t)$.
3. **Act**: Emit an action a_t, which may be a tool call, a navigation command, or a text output.
4. **Observe**: Receive o_{t+1} from the environment. Go to 2.

This is precisely the MDP agent loop from Chapter 3, now instantiated with a language model as the policy and a software environment as the transition function.

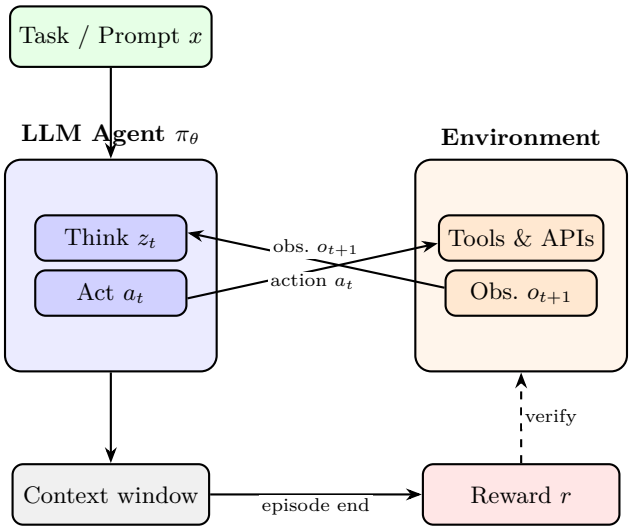


Figure 15.3: The agentic RL loop. The LLM agent π_θ observes the environment, generates a reasoning trace, and emits an action (a tool call or text output). The environment executes the action and returns an observation. The full context accumulates in the model's context window. After the episode terminates, a reward signal (outcome-based or step-based) is computed and used for a policy gradient update.

15.6.2 Agentic Tasks and Environments

Agentic RL has been applied to a growing range of tasks:

Web browsing The agent navigates a web browser (via Playwright or similar), reading pages, clicking links, and filling forms to complete information-retrieval or transaction tasks. Reward is binary: the task goal is achieved or not.

Code execution The agent writes code, executes it in a sandboxed Python interpreter, reads the output, and iterates. The unit-test reward signal from Section 15.1 applies directly.

API interaction The agent calls REST APIs, reads responses, and chains API calls to complete multi-step workflows (e.g., booking a calendar event after looking up availability). Reward comes from verifying the final system state.

Repository-level coding The agent reads and writes files in a code repository, runs tests, and resolves issues from bug reports. SWE-bench is the standard benchmark, measuring the fraction of GitHub issues resolved by the agent.

15.6.3 Credit Assignment in Multi-Step Trajectories

The fundamental challenge of agentic RL is credit assignment: when a trajectory of T steps receives a single terminal reward, each of the T actions must be credited appropriately. This is the delayed-reward problem from classical RL (Chapter 6), magnified by the length of language model trajectories.

Two paradigms address this:

- **Outcome rewards**: Credit each token in the trajectory with the same terminal reward (possibly discounted). This is simple but high-variance: early actions that set up success are rewarded equally with the single final token that confirms success.
- **Process rewards**: Train a step-level PRM to assign rewards r_t at each step. This reduces variance dramatically but requires additional supervision to train the PRM. The PRM can be trained on human annotations of which intermediate steps are correct, or (more scalably) using Monte Carlo rollouts from each intermediate state to estimate the probability of eventual success.

The tradeoff between process and outcome rewards (discussed more broadly in Chapter 10) is particularly sharp in the agentic setting, where trajectories may span dozens of tool calls and hundreds of thousands of tokens.

15.6.4 Exploration in Agentic RL

Classical RL exploration strategies (epsilon-greedy, Thompson sampling, intrinsic motivation; Chapter 1) take on new character when the policy is a language model. The model's prior from pretraining already encodes a rich distribution over action sequences, which provides implicit exploration. However, that prior may be concentrated on safe, familiar action sequences that avoid the long-horizon strategies needed for hard agentic tasks.

Practical approaches to agentic exploration include:

- **Temperature scaling**: Increasing generation temperature at training time diversifies rollouts, analogous to increasing epsilon in epsilon-greedy.
- **Curriculum over task difficulty**: Starting with short, easy tasks and progressively introducing longer, harder ones ensures the model receives adequate feedback signals throughout training.
- **Environment randomisation**: Varying task instances (different web pages, different code repositories, different API endpoints) prevents the policy from memorising specific trajectories.

15.7 Practical Considerations

15.7.1 Algorithm Choices

Table 15.2 summarises the main RL algorithms used for reasoning and agentic training.

Table 15.2: Policy optimisation algorithms for reasoning model training.

Algorithm	Mechanism	Key property	Used by
PPO	Clipped surr. + value net	Stable, well-understood	Tülu 3, Phi-4
GRPO	Group-norm. advantages, no value net	Memory-efficient	DeepSeek R1, Qwen 3
DAPO	Asymmetric clipping	Better exploration	RAGEN, INTELLECT-2
CISPO	Clipped importance sampling	Off-policy stable	MiniMax-M1
MuonClip	QK-norm clipping	Prevents loss spikes	Kimi K2

PPO (Proximal Policy Optimisation) remains the most widely studied algorithm for RL fine-tuning, owing to its theoretical guarantees and extensive empirical track record. It requires maintaining a separate value network, which doubles the memory footprint but reduces gradient variance through a learned baseline.

GRPO (Group Relative Policy Optimisation) avoids the value network by computing per-prompt advantage estimates from the empirical mean and variance of rewards within a group of k rollouts:

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}, \qquad \bar{r} = \frac{1}{k}\sum_{j=1}^{k} r_j, \quad \sigma_r = \sqrt{\frac{1}{k}\sum_{j=1}^{k}(r_j - \bar{r})^2}. \tag{15.11}$$

With binary RLVR rewards, $r_i \in \{0, 1\}$, this simplifies to $\hat{A}_i = +\sigma_r^{-1}$ for correct completions and $\hat{A}_i = -\bar{r}/\sigma_r$ for incorrect ones. GRPO has become dominant in reasoning model training because it is memory-efficient and easy to implement.

DAPO (Decoupled Clip and Dynamic Sampling Policy Optimisation) modifies GRPO by using asymmetric clipping bounds: a larger clip on the positive side allows more aggressive updates toward high-reward responses while the negative-side clip remains conservative. This promotes exploration of correct reasoning strategies that the current policy reaches only rarely.

15.7.2 Online vs. Offline RL

Online RL Rollouts are generated from the current policy π_θ and immediately used for a gradient update. This keeps the training distribution on-policy, which is theoretically clean but practically slow: the policy must wait for generation to complete before updating.

Offline RL Rollouts are pre-generated (from an earlier checkpoint of the policy or from a different model) and stored in a replay buffer. Updates use importance weights to correct for the distributional mismatch. Offline RL improves throughput dramatically but introduces off-policy bias if the replay buffer is stale.

Asynchronous RL A practical compromise used by several reasoning model recipes: generation runs asynchronously on one set of GPUs while gradient updates proceed on another. This keeps the policy nearly on-policy while avoiding idle compute.

15.7.3 Infrastructure at Scale

Training a reasoning model requires infrastructure that differs from standard language model pre-training in several ways:

- **Generation throughput**: RLVR training requires sampling $k = 8$–16 rollouts per prompt, each potentially thousands of tokens long. Efficient batched generation with continuous batching (as in vLLM) is essential.
- **Verification at scale**: The verifier (regex extractor, code sandbox) must run in parallel for thousands of completions per second without becoming the bottleneck. Code verification is particularly expensive because it requires process isolation.
- **Memory management**: GRPO's elimination of the value network halves GPU memory consumption relative to PPO, but storing k full rollout sequences per prompt still requires careful memory planning.
- **Reward normalisation and logging**: Because RLVR rewards are binary and sparse early in training, careful monitoring of the moving-average reward, rollout length, and format-reward compliance is necessary to detect training instabilities before they cascade.

15.7.4 Open-Source Tooling

Several open-source frameworks have made RLVR training accessible beyond well-resourced industrial labs:

TRL Hugging Face's Transformer Reinforcement Learning library implements PPO, GRPO, and DPO with first-class support for verifiable rewards. Its `GRPOTrainer` integrates directly with the `datasets` and `transformers` ecosystems.

veRL A high-throughput RL training framework from Volcengine that supports asynchronous generation–update pipelines and achieves state-of-the-art training throughput for large models.

OpenRLHF A distributed RLHF and RLVR framework built on DeepSpeed and Ray, supporting models up to hundreds of billions of parameters with flexible reward function APIs.

15.8 Summary and Open Questions

This chapter has traced the arc from RLHF — where a learned reward model provided imperfect preference signals — to RLVR, where deterministic verification functions enable aggressive, long-horizon RL training on verifiable domains. The key conceptual contributions are:

- **Verifiable rewards** eliminate the reward model as a bottleneck and a failure mode. Binary rewards from checking functions are noise-free and cannot be over-optimised.
- **Chain-of-thought reasoning** connects token generation to implicit search: longer reasoning traces implement a finer search over solution paths.
- **Inference-time scaling** shows that compute can be productively spent at deployment, not only at training. Majority voting and value-guided sampling extract accuracy gains from additional generation without retraining.
- **Tool use and agentic RL** extend the framework to interactive environments, transforming language models from passive text generators into active problem-solving agents.

Open questions. The field is moving rapidly, and several fundamental questions remain unresolved:

- **Generalisation of RLVR**: Do reasoning skills learned on mathematics and code transfer to unverifiable domains (open-ended writing, strategic reasoning, safety)? Early evidence suggests partial transfer, but the mechanism is unclear.
- **Scaling limits**: How far do inference-time scaling laws extend? At what point does generating more tokens yield diminishing returns independent of model capacity?
- **Process reward models at scale**: Training a reliable PRM requires step-level supervision, which is expensive to collect. Monte Carlo estimation of step correctness is noisy. Better methods for PRM training are an active research area.
- **Safety of agentic systems**: An agent that can browse the web, execute code, and call APIs has a vastly larger action space than a text-only chatbot. Aligning the objectives of multi-step agents with human values is an open challenge that classical RLHF was not designed to address.
- **Interplay with pretraining**: Several results suggest that a certain level of pretraining capability is a prerequisite for RLVR to work. Understanding the threshold and how to lower it would democratise reasoning training.

Exercises

Exercise 15.1. Consider the RLVR objective in (15.3) with $\beta = 0$. Show that the policy gradient of $J_{\text{RLVR}}(\theta)$ with respect to θ can be written as

$$\nabla_\theta J_{\text{RLVR}}(\theta) = \mathbb{E}_{x,y}\big[\mathbb{1}\{\textsc{Verify}(y, a^*)\}\, \nabla_\theta \log \pi_\theta(y \mid x)\big],$$

starting from first principles using the log-derivative trick. How does this reduce to the REINFORCE gradient of Chapter 8 with $r(y) = \mathbb{1}\{\textsc{Verify}(y, a^*)\}$?

Exercise 15.2. Prove Proposition 15.1. Specifically, show that for k independent Bernoulli(p) trials with $p > 1/2$, the probability that fewer than $k/2$ successes occur is bounded above by $\exp(-k\, D_{\text{KL}}(1/2 \,\|\, 1-p))$ using Hoeffding's inequality or Chernoff bounds.

Exercise 15.3. Suppose $k = 4$ rollouts for a prompt receive rewards $r = (1, 1, 0, 0)$. Compute the GRPO advantage estimates $\hat{A}_i$ from (15.11). Now suppose instead $r = (1, 0, 0, 0)$. Compute $\hat{A}_i$ again. Compare the magnitude of the gradient signal in the two cases and explain why the second scenario gives stronger feedback to the correct rollout even though there is only one correct rollout.

Exercise 15.4. A training dataset contains 10,000 problems. You sample $N = 16$ rollouts per problem and find that 2,000 problems have solvability rate $\sigma < 0.2$, 5,000 have $\sigma \in [0.2, 0.8]$, and 3,000 have $\sigma > 0.8$.

(a) What fraction of compute is "wasted" on uninformative problems (those with $\sigma < 0.2$ or $\sigma > 0.8$)?

(b) How does difficulty filtering change the effective dataset size available for gradient updates?

(c) Propose a dynamic difficulty curriculum that avoids discarding hard problems permanently.

Exercise 15.5. Using the decomposition (15.4), argue informally why (a) chain-of-thought improves accuracy for hard multi-step problems but (b) is largely redundant for problems that the model can answer correctly in a single step. What does this imply for the desiderata of a *toggleable* reasoning mode?

Exercise 15.6. A model is solving the following task via tool use: retrieve the population of France and the population of Germany, then output their ratio. Write a Python function `verify(trajectory)` that takes a list of (action, observation) pairs and returns a reward in $\{0, 1\}$. Your function should handle: (a) the model calling tools in the wrong order, (b) the model failing to call a required tool, and (c) arithmetic errors in the final computation.

Exercise 15.7. Consider a multi-step agent completing a 5-step task. The outcome reward is $+1$ only on task completion. Suppose the agent succeeds on the final step but took an inefficient path (making 3 redundant API calls). Under:

(a) outcome reward only, what does the agent learn about the redundant API calls?

(b) a process reward model that penalises redundant calls by -0.1 each, how does the gradient signal change?

Discuss the bias-variance tradeoff between (a) and (b).

Exercise 15.8. Let $\mathrm{Acc}_1(p)$ be the accuracy of a single rollout where each rollout is correct with probability p. Let $\mathrm{Acc}_k(p)$ be the accuracy of majority vote over k rollouts.

(a) Write an exact formula for $\mathrm{Acc}_k(p)$ in terms of the binomial distribution.

(b) For $p = 0.4$ and $k \in \{1, 3, 5, 9, 17, 33\}$, compute $\mathrm{Acc}_k(p)$. Note that majority vote can *reduce* accuracy when $p < 0.5$; explain why.

(c) Identify the crossover value of p below which majority voting is harmful.

Exercise 15.9. The DeepSeek R1 recipe uses two stages: SFT cold start followed by large-scale RLVR. Suppose you skip the SFT stage and apply RLVR directly to the base pretrained model.

(a) What problems might arise if the base model does not reliably produce text in the `<thinking>`...`</thinking>` format?

(b) How does the format reward described in Section 15.3.4 partially mitigate this?

(c) Why is the SFT cold start still preferred despite the format reward?

Chapter 16

Multi-Agent Systems and RL

The whole is more than the sum of its parts.

— Aristotle, *Metaphysics*

Every setting we have studied so far involves a single learner interacting with a world whose dynamics are fixed. The environment may be stochastic, partially observed, or high-dimensional, but it does not itself *learn*. In the real world, this assumption rarely holds. Financial markets are populated by traders who each adapt to the strategies of the others. Autonomous vehicles must negotiate intersections with other vehicles that are simultaneously trying to navigate safely. A team of rescue robots must divide a building among themselves without central coordination. Language models deployed as assistants increasingly delegate sub-tasks to specialised agents — other models that respond, check, or criticise. In every case, the "environment" contains *other agents*, and those agents are themselves learning and adapting.

Multi-agent reinforcement learning (MARL) is the study of how multiple decision-making entities can learn good behaviour when they coexist and interact. The field sits at the intersection of game theory, optimisation, and modern deep learning, and it poses challenges that are qualitatively different from single-agent RL: the environment is *non-stationary* because other agents are changing; rewards may be shared, competing, or somewhere in between; and the space of joint behaviours is exponentially larger than the space of individual behaviours.

This chapter develops the theory and practice of MARL progressively. Section 16.1 introduces Markov games — the natural multi-agent generalisation of MDPs — and defines the solution concept of Nash equilibrium. Section 16.2 covers cooperative MARL, including the centralised-training with decentralised-execution (CTDE) paradigm and the QMIX and MAPPO algorithms. Section 16.3 studies competitive and mixed settings, including zero-sum games, self-play, and the story of AlphaGo and AlphaZero. Section 16.4 examines emergent communication and social dilemmas. Section 16.5 brings the framework to large language models, covering debate, multi-agent code generation, and agentic workflows. Section 16.6 catalogues the open challenges, and Section 16.7 summarises.

16.1 Multi-Agent Markov Games

16.1.1 From MDPs to Markov Games

Recall from Chapter 3 that a Markov decision process is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$ in which a single agent takes actions, the environment transitions to a new state, and the agent receives a scalar reward. When there are $N \geq 2$ agents, the natural extension is a *Markov game*, also called a *stochastic game*.

Definition 16.1 (Markov Game)**.** A **Markov game** with N agents is a tuple

$$\mathcal{G} = \big(\mathcal{S},\, \mathcal{A}^1, \ldots, \mathcal{A}^N,\, \mathcal{T},\, R^1, \ldots, R^N,\, \gamma\big),$$

where:

- $\mathcal{S}$ is a finite set of **states** shared by all agents.
- $\mathcal{A}^i$ is the **action set** for agent i. The **joint action space** is $\mathcal{A} = \mathcal{A}^1 \times \cdots \times \mathcal{A}^N$.
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the **transition kernel**; the next state depends on the current state *and* the joint action of all agents.
- $R^i : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the **reward function** for agent i. Each agent has its own reward, which may depend on the actions of all agents.
- $\gamma \in [0, 1)$ is a shared discount factor.

When $N = 1$, a Markov game reduces to a standard MDP. When all agents share the same reward function, $R^1 = \cdots = R^N$, the game is **fully cooperative**. When $\sum_i R^i = 0$ at every transition, it is **zero-sum**. The general case, where rewards are neither identical nor summing to zero, is called a **mixed or general-sum game**.

The joint policy of all agents is $\boldsymbol{\pi} = (\pi^1, \ldots, \pi^N)$, where each $\pi^i : \mathcal{S} \to \Delta(\mathcal{A}^i)$ is a stochastic policy for agent i. Agent i's objective is to maximise its own discounted return:

$$J^i(\boldsymbol{\pi}) = \mathbb{E}_{\boldsymbol{\pi}}\left[\sum_{t=0}^{\infty} \gamma^t R^i(s_t, \boldsymbol{a}_t, s_{t+1})\right], \tag{16.1}$$

where $\boldsymbol{a}_t = (a_t^1, \ldots, a_t^N)$ is drawn from the product policy $\prod_i \pi^i(\cdot \mid s_t)$.

16.1.2 Nash Equilibrium

Because each agent's return depends on the policies of *all* agents, the standard notion of optimality from single-agent RL no longer applies. Raising agent i's return by changing π^i may require the other agents to hold their policies fixed — or it may backfire if they adapt. The solution concept borrowed from game theory is the **Nash equilibrium**.

Definition 16.2 (Nash Equilibrium)**.** A joint policy $\boldsymbol{\pi}^* = (\pi^{1*}, \ldots, \pi^{N*})$ is a **Nash equilibrium** if, for every agent i and every alternative policy $\tilde{\pi}^i$,

$$J^i(\pi^{i*}, \boldsymbol{\pi}^{-i*}) \;\geq\; J^i(\tilde{\pi}^i, \boldsymbol{\pi}^{-i*}),$$

where $\boldsymbol{\pi}^{-i*}$ denotes the policies of all agents except i, held fixed at their equilibrium values.

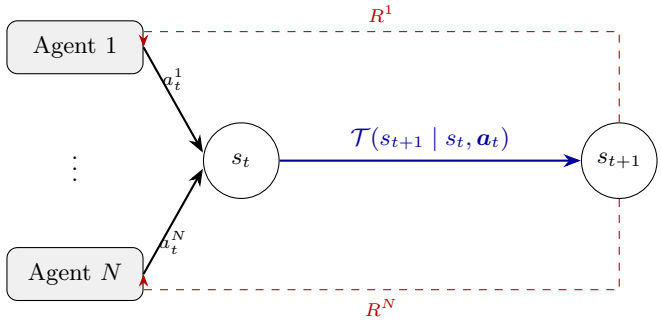


Figure 16.1: A Markov game with N agents. At each timestep, every agent observes the shared state s_t and chooses an action. The joint action $\boldsymbol{a}_t$ drives the transition to s_{t+1}, and each agent receives its own individual reward R^i.

In words, a Nash equilibrium is a joint policy from which no single agent has an incentive to deviate unilaterally. Nash's theorem guarantees that every finite game has at least one Nash equilibrium in mixed strategies, but equilibria need not be unique, and finding one is computationally hard in general (PPAD-complete for two-player general-sum games).

Example 16.1 (Prisoner's Dilemma as a One-Step Markov Game)**.** Two agents simultaneously choose Cooperate (C) or Defect (D). Payoffs (row = Agent 1, column = Agent 2):

	C	D
C	$(3,3)$	$(0,5)$
D	$(5,0)$	$(1,1)$

Agent 1 always prefers D regardless of Agent 2's action: if Agent 2 plays C, defecting yields $5 > 3$; if Agent 2 plays D, defecting yields $1 > 0$. The same logic applies to Agent 2. Therefore (D, D) is the unique Nash equilibrium, even though (C, C) would give both agents a higher payoff of 3. This tension between individual rationality and collective welfare is the hallmark of social dilemmas and will reappear in Section 16.4.

16.1.3 Partial Observability and the Dec-POMDP

In practice, agents rarely observe the full global state. Each agent i instead receives a local observation o_t^i drawn from an observation function $O^i : \mathcal{S} \to \Delta(\mathcal{O}^i)$. The resulting framework is a **decentralised partially observable MDP** (Dec-POMDP), which is NEXP-hard to solve in the worst case. Much of practical MARL research focuses on restricting the setting — cooperative teams, shared observations at training time, or limited communication — to make the problem tractable.

16.2 Cooperative MARL

16.2.1 The Cooperative Setting

In fully cooperative MARL all agents share a single team reward $r_t = R(s_t, \boldsymbol{a}_t, s_{t+1})$. The joint objective is simply

$$J(\boldsymbol{\pi}) = \mathbb{E}_{\boldsymbol{\pi}}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]. \tag{16.2}$$

Despite the common objective, each agent must act on its own local observation and cannot, at execution time, communicate or observe the global state. This is the standard setting in multi-robot coordination, multi-player cooperative games (e.g. StarCraft II micromanagement), and multi-agent dialogue systems.

16.2.2 Centralised Training with Decentralised Execution

The most influential paradigm for cooperative MARL is **centralised training with decentralised execution** (CTDE). During *training*, a central critic or mixer has access to the global state s_t and the actions of all agents. During *execution*, each agent relies only on its local observation history $\tau^i = (o_0^i, a_0^i, \ldots, o_t^i)$. The training privilege allows the critic to give informative gradient signal; the execution constraint ensures the policy is deployable in settings where a central controller is unavailable.

16.2.3 QMIX

QMIX (Rashid et al., 2018) is the canonical CTDE algorithm for cooperative value-based MARL. Each agent i maintains a per-agent utility network $Q_\theta^i(\tau^i, a^i)$ that conditions only on the local observation history. A central *mixer network* combines these utilities into a joint action-value function:

$$Q_{\text{tot}}(s, \boldsymbol{a}) = f_\phi\big(Q^1(\tau^1, a^1), \ldots, Q^N(\tau^N, a^N),\ s\big). \tag{16.3}$$

The mixer uses hypernetworks conditioned on the global state s to produce non-negative weights, which ensures the **monotonicity constraint**:

$$\frac{\partial Q_{\text{tot}}}{\partial Q^i} \geq 0 \quad \text{for all } i.$$

This constraint is the key insight. It guarantees the **Individual-Global-Max** (IGM) property:

$$\operatorname*{argmax}_{\boldsymbol{a}}\ Q_{\text{tot}}(s, \boldsymbol{a}) \ = \ \big(\operatorname*{argmax}_{a^1} Q^1(\tau^1, a^1),\ \ldots,\ \operatorname*{argmax}_{a^N} Q^N(\tau^N, a^N)\big).$$

Maximising Q_{tot} is therefore equivalent to each agent greedily choosing its own best action — no joint-action search is needed at execution time. The whole system is trained end-to-end with a standard TD loss:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}\Big[\big(r + \gamma \max_{\boldsymbol{a}'} Q_{\text{tot}}(s', \boldsymbol{a}') - Q_{\text{tot}}(s, \boldsymbol{a})\big)^2\Big]. \tag{16.4}$$

QMIX achieves state-of-the-art performance on the StarCraft Multi-Agent Challenge (SMAC) benchmark, a suite of cooperative micromanagement tasks in StarCraft II.

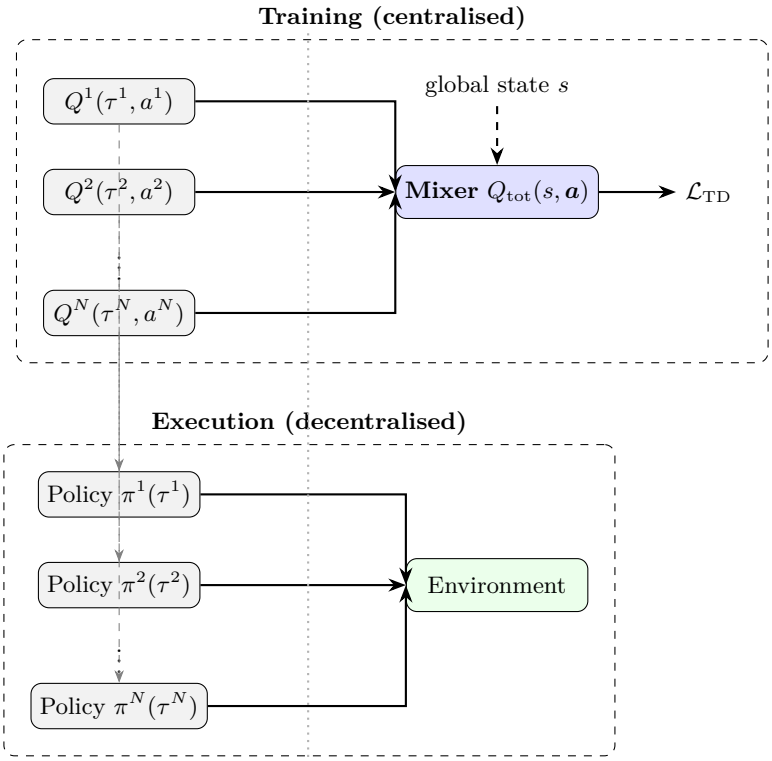


Figure 16.2: The CTDE architecture. During training a *mixer* (e.g. QMIX) combines per-agent utility networks Q^i into a joint Q_{tot} using the global state, and gradients flow back through all Q^i. At execution time the centralised mixer is discarded; each agent acts greedily with respect to its own Q^i using only its local observation history τ^i.

16.2.4 MAPPO

While QMIX operates in the value-function paradigm, many cooperative tasks benefit from policy-gradient methods because they handle continuous actions naturally and are easier to tune. **MAPPO** (Yu et al., 2022) applies Proximal Policy Optimisation (PPO; Chapter 9) in the CTDE framework. Each agent i maintains a separate actor $\pi^i_\theta(\cdot \mid \tau^i)$ conditioned on its local observation history. A *shared centralised critic* $V_\phi(s)$ conditions on the global state and produces a baseline for variance reduction:

$$\mathcal{L}_{\text{MAPPO}}(\theta^i) = \mathbb{E}\Big[\min\big(r^i_t(\theta^i)\,\hat{A}_t,\ \text{clip}(r^i_t(\theta^i), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big], \tag{16.5}$$

where $r^i_t(\theta^i) = \pi^i_\theta(a^i_t \mid \tau^i_t)/\pi^i_{\text{old}}(a^i_t \mid \tau^i_t)$ is the probability ratio and $\hat{A}_t$ is the advantage estimated by the centralised critic. A key practical finding is that *parameter sharing* — initialising all agents from the same actor network while conditioning on an agent-index feature — drastically improves sample efficiency by letting agents share gradients.

Remark 16.1 (Parameter Sharing vs. Heterogeneous Agents). Parameter sharing is powerful when agents play symmetric roles, but can be inappropriate for teams of heterogeneous specialists (e.g. a healer and a tank in a game). In that setting, separate networks with a shared feature extractor often work best.

16.3 Competitive and Mixed Settings

16.3.1 Zero-Sum Games and Self-Play

In a two-player zero-sum game, $R^1 = -R^2$, so one agent's gain is exactly the other's loss. The Nash equilibrium of a zero-sum Markov game corresponds to the minimax policy: agent 1 maximises while agent 2 minimises the same cumulative reward. Formally, the minimax value v^* satisfies

$$v^* = \max_{\pi^1} \min_{\pi^2} \; J^1(\pi^1, \pi^2) = \min_{\pi^2} \max_{\pi^1} \; J^1(\pi^1, \pi^2),$$

where the equality (von Neumann's minimax theorem) holds when policies are allowed to be mixed (stochastic).

Training an agent to play optimally in a zero-sum game faces an immediate bootstrapping problem: to learn to defeat strong opponents, the agent needs strong opponents to learn against. **Self-play** resolves this by having the agent play against past versions of itself. A canonical training loop is:

1. Maintain a population (or a single snapshot) of past policy checkpoints.
2. Sample an opponent from the population.
3. Collect rollouts; update the current policy against this opponent.
4. Add the current policy to the population periodically.

Self-play guarantees that the opponent distribution is always at a difficulty level commensurate with the current policy, providing a natural curriculum.

16.3.2 AlphaGo and AlphaZero

The most famous application of self-play is the AlphaGo and AlphaZero programme (Silver et al., 2016; 2017). Go had long resisted AI solutions because its game tree has a branching factor of about 250 and depths exceeding 200, making exhaustive search infeasible.

AlphaGo combined three components: (1) a value network $v_\theta(s)$ trained by regression on self-play outcomes; (2) a policy network $\pi_\theta(a \mid s)$ trained by supervised learning on human games and then refined by RL policy gradient; and (3) Monte Carlo Tree Search (MCTS) that uses both networks to guide rollouts. At each MCTS node, the selection step balances exploration and exploitation via the PUCT criterion:

$$a^* = \operatorname*{argmax}_a \left[Q(s,a) + c_{\text{puct}}\, \pi_\theta(a \mid s) \frac{\sqrt{N(s)}}{1 + N(s,a)} \right], \tag{16.6}$$

where $Q(s,a)$ is the empirical mean value from past rollouts, $N(s)$ is the visit count of node s, and $N(s,a)$ is the count of edge (s,a).

AlphaZero stripped away the supervised learning bootstrap: starting from *random play*, self-play plus MCTS alone was sufficient to reach superhuman performance in Go, Chess, and Shogi within days of training. The success of AlphaZero demonstrated that given a perfect simulator, self-play can discover expert strategies entirely from scratch — a landmark result for both AI and MARL.

16.3.3 Competitive Training Instability and Non-Transitive Strategies

Naive self-play is not without pathologies. In games with **non-transitive strategy relationships** — where strategy A beats B, B beats C, but C beats A (a rock-paper-scissors dynamic) — self-play can cycle indefinitely rather than converging to a Nash equilibrium. The policy trained against version k of itself may be strong against k but brittle against version $k-5$, a problem known as **forgetting** or strategic collapse.

Two remedies are widely used. **Fictitious self-play** trains the agent against the *average* of all past opponent policies rather than just the most recent checkpoint. In two-player zero-sum games with finite action spaces, fictitious self-play converges to the Nash equilibrium. **Population-based training** (PBT; Jaderberg et al., 2017) maintains a diverse league of agents with different strategies; each agent in the population is trained against a mixture of others, preventing any single dominant strategy from eliminating diversity.

16.4 Communication and Emergent Behaviour

16.4.1 Learned Communication Protocols

When agents can exchange messages before or during task execution, the team's effective policy space expands dramatically. But what should they say? Hard-coding a communication protocol requires domain expertise and scales poorly. A more principled approach is to treat messages as *continuous actions* and let communication protocols emerge through end-to-end training.

CommNet (Sukhbaatar et al., 2016) is one of the earliest architectures for differentiable multi-agent communication. At each timestep each agent i broadcasts a continuous vector $m_t^i \in \mathbb{R}^d$; the aggregated message $\bar{m}_t^i = \frac{1}{N-1}\sum_{j\neq i} m_t^j$ is concatenated with the agent's observation and fed to an LSTM, whose output drives both the next action and the next message:

$$(a_t^i, m_{t+1}^i) = f_\theta\big(h_t^i, o_t^i, \bar{m}_t^i\big).$$

Because the communication channel is differentiable, the whole system is trained end-to-end with standard backpropagation through time. CommNet demonstrated that agents could spontaneously learn to coordinate on cooperative tasks (e.g. traffic junction routing) without any supervision on the communication content.

TarMAC (Das et al., 2019) extended CommNet by introducing *targeted communication*: each agent produces a key k^i and a value v^i, and the aggregated message received by agent j is an attention-weighted sum

$$m^j = \sum_{i\neq j} \sigma(k^i \cdot q^j)\, v^i,$$

where q^j is agent j's query vector. The attention mechanism allows agents to selectively read the messages most relevant to their current situation, improving both performance and the interpretability of the learned communication.

16.4.2 Emergent Communication

A striking finding in the emergent-communication literature is that agents can develop *compositional languages*: short discrete symbols whose combinations encode structured meanings. In referential games (e.g. one agent sees an object and must describe it; another

agent must identify it from distractors), trained agents frequently develop vocabularies in which different symbols correspond to different object attributes, and novel combinations of symbols denote novel objects. These findings parallel the study of language evolution and raise the question of whether artificial emergent languages could bootstrap useful communication skills for real-world agents.

16.4.3 Social Dilemmas in Multi-Agent Learning

The Prisoner's Dilemma of Example 16.1 is a one-shot game. In *repeated* games, cooperation can emerge spontaneously because agents have the ability to *punish* defectors in future rounds. Axelrod's famous tournaments showed that the simple Tit-for-Tat strategy (cooperate initially; on each subsequent round, copy the opponent's last action) outperforms complex strategies in repeated Prisoner's Dilemmas.

In MARL, social dilemmas are formalised as environments where the Nash equilibrium is Pareto suboptimal: rational individual behaviour leads to collectively poor outcomes. The sequential social dilemma (SSD) framework (Leibo et al., 2017) embeds such dilemmas in spatial environments. For example, in the *Gathering* game, two agents collect apples and can fire "fining beams" to temporarily disable each other. Mutual fining wastes time, but each agent is individually incentivised to fire first. RL agents trained with independent learning converge to mutual defection; agents trained with prosocial reward shaping or explicit communication learn to cooperate.

16.5 Multi-Agent RL for LLMs

The preceding sections treated agents as neural networks that map observations to actions in an environment with clear state and reward signals. Large language models (LLMs) introduce a qualitatively new kind of agent: one that operates in the space of natural language, can reason through intermediate steps, and can delegate sub-tasks to other LLM instances. MARL concepts apply naturally, and a growing body of work exploits them to improve reasoning quality, code generation, and task completion.

16.5.1 Debate

AI Debate (Irving et al., 2018) is a multi-agent protocol for producing trustworthy outputs from a powerful AI system. Two language model agents argue opposite sides of a proposition; a third agent (or a human judge) evaluates the exchange and declares a winner. The key theoretical claim is that, for a sufficiently capable judge, the optimal strategy for each debater is to *tell the truth*: a lie can always be exposed by the opponent, so the best policy in a debate equilibrium is honest argument.

In practice, debate-style protocols are implemented as a two-LLM pipeline:

1. A **proposer** LLM generates a claim or answer.
2. A **critic** LLM, given the same prompt and the proposer's output, generates a counter-argument.
3. A **judge** LLM (or the same model with a judge prompt) scores both positions and selects the better answer.

The judge's score can be used as a reward signal to fine-tune both the proposer and critic via RL (Section 15.1 in Chapter 15). This enables *scalable oversight*: the system can improve

beyond the judge's direct capability, because it is easier for the judge to evaluate an argument than to generate the correct answer from scratch.

16.5.2 Multi-Agent Code Generation

Code generation is a natural setting for multi-agent LLMs because code admits automatic, verifiable rewards (test suites), and because the task decomposes into sub-tasks (specification, implementation, testing, documentation) that are easy to assign to specialised agents.

A representative pipeline (Chen et al., 2023; Du et al., 2023) works as follows:

1. A **planner** agent decomposes the user request into a sequence of subtasks.
2. Multiple **coder** agents independently generate candidate implementations of each subtask.
3. A **verifier** agent runs test suites on each candidate and returns pass/fail feedback.
4. A **ranker** agent, trained with preference RL, selects the best-ranked implementation and optionally patches failures.

Empirically, diverse proposals from multiple independent coders consistently outperform any single coder, even when the individual coders are identical models: diversity in random seeds, prompts, or temperature settings introduces sufficient variation for the verifier to identify genuinely better solutions.

16.5.3 LLM Agents Cooperating on Tasks

Beyond code, multi-agent LLM frameworks have been applied to research assistance, game playing, and software engineering. **ChatDev** (Qian et al., 2023) structures a software team as a set of LLM agents playing different roles (CEO, CTO, programmer, tester, document writer) who communicate via a moderated chat channel to complete a software project end-to-end. Each role is defined by a system prompt and a role-specific history; the moderation protocol enforces turn-taking and task focus.

AutoGen (Wu et al., 2023) provides a more flexible framework: any LLM can be wrapped as an *agent* with a name, system prompt, and optionally a tool-use interface. Agents communicate by passing messages in a shared thread; a human-proxy agent can inject user feedback. The framework supports loops, branching conversations, and nested agent groups, enabling both cooperative and adversarial patterns.

16.5.4 Agentic Workflows with Specialised Agents

The most practically influential multi-agent LLM pattern at the time of writing is the **orchestrator — specialist** pattern. A powerful generalist LLM (the *orchestrator*) receives a complex user task, decomposes it into subtasks, and dispatches each subtask to a *specialist* agent optimised for that specific skill:

Web search agent retrieves and summarises live information.

Code agent writes and executes code in a sandboxed interpreter.

Memory agent maintains a persistent key-value store of facts extracted from prior turns.

Critic agent reviews the orchestrator's plan and flags inconsistencies before execution.

Rewards for RL fine-tuning of the orchestrator can come from end-to-end task completion (e.g. does the final answer match a ground-truth solution?) or from intermediate sub-goals validated by the specialist agents. This closed loop between orchestration and specialist execution mirrors the CTDE paradigm: the orchestrator acts as a centralised planner during training, while each specialist executes its local policy independently.

The connection to MARL is direct. Each specialist is a policy π^i; the orchestrator defines a task distribution over joint action sequences; and the emergent behaviour of the ensemble is a trajectory in the space of all possible multi-agent interactions. Formally training this system with MARL objectives — rather than fine-tuning each component in isolation — is an active research frontier.

16.6 Challenges in Multi-Agent RL

16.6.1 Non-Stationarity

The most fundamental challenge of MARL is that each agent's environment includes all other agents, which are themselves learning. From agent i's perspective, the transition law and reward function are *non-stationary*: the world it experiences today is different from the world it will experience tomorrow, because its teammates or opponents have updated their policies. This violates the stationarity assumption that underpins convergence proofs for Q-learning and other single-agent algorithms.

Formally, agent i observes an effective transition law

$$\tilde{\mathcal{T}}^i(s' \mid s, a^i) = \sum_{\boldsymbol{a}^{-i}} \mathcal{T}(s' \mid s, a^i, \boldsymbol{a}^{-i}) \prod_{j \neq i} \pi^j(a^j \mid s),$$

which changes whenever any π^j $(j \neq i)$ changes. Independent Q-learning ignores this non-stationarity and often fails to converge in practice, especially in competitive settings. Algorithms such as QMIX and MAPPO partially mitigate non-stationarity by sharing gradient information across agents, but convergence guarantees remain weak for the general case.

16.6.2 Credit Assignment in Teams

Even with a shared reward signal, team credit assignment is hard: if the team succeeds, which agent's action was responsible? Naive approaches attribute all reward to all agents equally, which provides noisy gradients and discourages specialisation. The **difference reward** heuristic assigns agent i the credit

$$d_t^i = r_t - r_t^{-i},$$

where r_t^{-i} is the reward that *would have been* obtained had agent i acted with some default or counterfactual action, holding other agents fixed. Computing r_t^{-i} requires either a learned world model or additional rollouts, making it expensive. QMIX's monotonicity constraint provides an implicit credit assignment mechanism: the gradient of Q_{tot} with respect to Q^i tells agent i how much its utility function contributes to the team.

16.6.3 Scalability

The joint action space $|\mathcal{A}| = \prod_i |\mathcal{A}^i|$ grows exponentially in the number of agents. For $N = 10$ agents each with $|\mathcal{A}^i| = 5$ actions, the joint space has $5^{10} \approx 10^7$ elements. Even

representing $Q_{\text{tot}}(s, \boldsymbol{a})$ as a table is infeasible; the CTDE factorisation of QMIX addresses this by writing Q_{tot} as a function of per-agent utilities, reducing complexity from exponential to linear in N.

Policy-gradient methods scale better because they operate on marginal policies $\pi^i(a^i \mid \tau^i)$ rather than on the joint policy. With parameter sharing, the same network services all agents, so the parameter count does not grow with N. Nevertheless, training time still scales with the number of agents because more rollout data is needed, and inter-agent dependencies require larger replay buffers and mini-batches.

16.6.4 Reward Shaping for Cooperation

In mixed-motive games, agents may have individually rational incentives that conflict with team performance. Reward shaping — augmenting the true reward with a potential-based bonus — can nudge agents toward cooperative behaviour without invalidating the Nash equilibrium. However, shaping rewards poorly can introduce spurious equilibria or make agents indifferent to the true objective.

Intrinsic social motivation is a promising alternative: agents are given an intrinsic reward for *influencing* the behaviour of others (Social Influence reward; Jaques et al., 2019) or for achieving causal empowerment over the environment. These intrinsic rewards encourage information sharing without requiring hand-crafted communication protocols.

16.6.5 Sample Efficiency

MARL is sample-hungry. Not only must each agent explore its own action space, but the joint exploration problem is exponentially larger. Coordinated exploration — where agents actively avoid redundant exploration of the same state regions — reduces redundancy but requires additional coordination overhead. Offline MARL, which learns from pre-collected datasets without online interaction, is an emerging approach that transfers recent successes of offline single-agent RL (Chapter 17) to the multi-agent setting.

The table below summarises the main algorithmic families, their settings, and key properties.

Algorithm	**Setting**	**Paradigm**	**Action space**	**Key idea**
IQL	Coop./Comp.	Decentralised	Discrete	Independent Q-learning
QMIX	Cooperative	CTDE	Discrete	Monotonic mixer
QPLEX	Cooperative	CTDE	Discrete	Duplex attention mixer
MAPPO	Cooperative	CTDE	Cont./Disc.	Shared centralised critic
MADDPG	Mixed	CTDE	Continuous	Per-agent centralised critic
AlphaZero	Competitive	Self-play	Discrete	MCTS + policy/value nets
PSRO	Mixed	Population	Discrete	Nash resp. oracle + mixture
CommNet	Cooperative	Comm. channel	Continuous	Mean-field messages
TarMAC	Cooperative	Comm. channel	Continuous	Targeted attention msgs

16.7 Summary

Multi-agent reinforcement learning extends the single-agent MDP framework to N interacting agents, each with its own policy, observation, and reward. The central solution concept is Nash equilibrium: a joint policy from which no agent can profitably deviate unilaterally. Finding Nash equilibria is hard in general, but structure in the game — cooperation, zero-sum competition, factored dynamics — enables tractable algorithms.

For cooperative tasks, the CTDE paradigm achieves the best of both worlds: centralised gradient information at training time, and fully decentralised execution. QMIX exploits monotonic factorisation to make greedy per-agent action selection globally optimal. MAPPO applies PPO with a centralised value baseline and parameter sharing, scaling to large teams with continuous actions.

In competitive settings, self-play provides an automatic curriculum. AlphaZero shows that starting from random play and iterating self-play with MCTS is sufficient for superhuman performance in perfect-information games. Non-transitivity and strategic forgetting motivate population-based methods that maintain policy diversity.

Communication opens an additional channel for cooperation. CommNet and TarMAC demonstrate that end-to-end training can produce effective messaging protocols without any supervision on the message content. In social dilemmas, where the Nash equilibrium is Pareto suboptimal, repeated interaction and prosocial rewards can sustain cooperative behaviour.

LLMs are the latest and perhaps most impactful arena for MARL ideas. Debate, multi-agent code generation, orchestrator-specialist pipelines, and frameworks like AutoGen and ChatDev all instantiate multi-agent dynamics in language. Applying formal MARL training objectives to these systems — rather than fine-tuning components in isolation — is one of the most exciting open directions at the frontier of RL research.

The key challenges — non-stationarity, credit assignment, scalability, reward shaping, and sample efficiency — are interrelated and none is fully solved. Progress on each will likely require new theory as well as new engineering, making MARL one of the most active and consequential subfields of machine learning.

Exercises

Exercise 16.1. Show that a Markov game with $N = 1$ reduces to a standard MDP. Then show that if all agents share the same reward $R^1 = \cdots = R^N$ and all agents act identically (i.e. $\pi^1 = \cdots = \pi^N$), the joint return Equation (16.2) equals the single-agent return of Chapter 3. What additional conditions are needed for the shared-reward policy-gradient to equal the single-agent policy-gradient?

Exercise 16.2. Consider the Prisoner's Dilemma of Example 16.1.

(a) Verify that (D, D) is a Nash equilibrium by checking the definition: no player can improve their payoff by unilaterally switching to C.

(b) Show that (C, C) is *not* a Nash equilibrium.

(c) Now suppose the game is played for $T = 100$ rounds and agents maximise total undiscounted reward. Is it a Nash equilibrium for both agents to always cooperate? Justify.

(d) What changes if the horizon is infinite with discount $\gamma = 0.9$?

Exercise 16.3. The IGM property states that $\operatorname{argmax}_{\boldsymbol{a}} Q_{\text{tot}}(s, \boldsymbol{a}) = \big(\operatorname{argmax}_{a^1} Q^1(\tau^1, a^1), \ldots, \operatorname{argmax}_{a^N}$

(a) Suppose $Q_{\text{tot}}(s, \boldsymbol{a}) = \sum_i Q^i(\tau^i, a^i)$. Prove that the IGM property holds.

(b) Does the IGM property hold if $Q_{\text{tot}}(s, \boldsymbol{a}) = \prod_i Q^i(\tau^i, a^i)$ and all $Q^i > 0$? Prove or give a counterexample.

(c) Explain in one paragraph why QMIX's non-negative mixing weights (from its hypernetwork) are sufficient but not necessary for IGM.

Exercise 16.4. Consider a three-strategy game with pure strategies Rock (R), Paper (P), Scissors (S) and payoff matrix

$$M = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$$

where M_{ij} is the payoff to the row player when they play strategy i against column player strategy j.

(a) Find the unique Nash equilibrium of this game.

(b) Simulate 1,000 steps of gradient ascent on π^1 while π^2 is fixed at the Nash equilibrium. Does π^1 converge?

(c) Now simulate 1,000 steps of simultaneous gradient ascent for both players. Describe the trajectory in the space of mixed strategies. Does it converge to the Nash equilibrium?

(d) Explain intuitively why this game causes naive self-play to cycle.

Exercise 16.5. A cooperative team of $N = 4$ robots earns a shared reward of $r = 10$ when all four reach a goal simultaneously, and $r = 0$ otherwise. After a successful episode, independent Q-learning assigns equal credit $10/4 = 2.5$ to each agent.

(a) Suppose Robot 3 had a 0.5 chance of reaching the goal regardless of its action. Describe how equal credit assignment distorts the learning signal for Robots 1, 2, and 4.

(b) Define a difference reward for Robot i using a counterfactual where Robot i stays put while others follow the observed policy. Compute this reward for Robot 3.

(c) What is the advantage of the difference reward over equal credit? What is its main computational drawback?

Exercise 16.6. Design a CTDE algorithm for a team of N drones surveying a search-and-rescue area. Each drone observes a 5×5 local grid; the global map is 50×50. Agents cannot communicate at test time.

(a) Specify the observation space $\mathcal{O}^i$, action space $\mathcal{A}^i$, and a suitable team reward function R.

(b) Propose a centralised critic architecture that uses the full 50×50 map during training. What features would you concatenate alongside the global state?

(c) Argue whether parameter sharing is appropriate here, and how you would condition each agent's actor on its unique identifier.

(d) Identify two ways in which training instability might arise and suggest mitigations.

Exercise 16.7. Implement a simplified debate protocol for a closed-book question-answering task.

(a) Define the state, action, and reward for each of the proposer, critic, and judge agents in a single-round debate.

(b) Let the judge assign a binary reward $r \in \{0, 1\}$ (correct / incorrect). Write the REINFORCE policy gradient update for the proposer, assuming the critic's policy is fixed. Is the variance of this estimator likely to be high? Why?

(c) Propose a baseline for the proposer that reduces variance without introducing bias. Describe concretely how you would estimate it.

(d) Under what conditions does the debate equilibrium guarantee truthful behaviour from the proposer? State the necessary assumptions on the judge's capability.

Exercise 16.8. In a referential game, a speaker agent observes an image drawn from a set of K objects and must produce a discrete message $m \in \{1, \ldots, V\}^L$ (vocabulary size V, message length L). A listener observes m and must identify the correct object from K distractors.

(a) What is the maximum number of distinct objects that can be encoded by a language with vocabulary V and length L?

(b) Suppose both agents are trained end-to-end with REINFORCE and the reward is $+1$ for correct identification, 0 otherwise. Write the joint policy gradient update.

(c) Describe one metric for measuring the compositionality of the emergent language (e.g. topographic similarity). Define it formally.

(d) Why might the emergent language fail to be compositional even when compositionality would be useful?

Chapter 17

Open Problems and Emerging Directions

The best way to predict the future is to invent it.

— Alan Kay

The preceding chapters have traced reinforcement learning from multi-armed bandits (Chapter 2) through dynamic programming (Chapter 4), temporal-difference methods (Chapter 6), deep value-function approximation (Chapter 7), policy gradients and actor-critic algorithms (Chapters 8–9), and onward to the modern intersection of RL with large language models (Chapters 11–16). Along the way we have seen RL solve games, align language models with human preferences, enable complex reasoning, and coordinate multi-agent systems.

Yet for every problem RL has solved, a dozen remain open. Some are long-standing — sample efficiency has been a concern since the earliest tabular algorithms — while others have emerged only recently, driven by the rapid adoption of RL as a post-training method for foundation models. This final chapter surveys the most important open questions, groups them into coherent themes, and attempts to sketch the road ahead. It is necessarily more speculative than the chapters that precede it: the goal is not to derive theorems but to orient the reader toward the problems that will shape the next decade of research.

17.1 Sample Efficiency

Reinforcement learning is notoriously data-hungry. Model-free algorithms such as PPO (Chapter 9) and DQN (Chapter 7) often require millions or even billions of environment interactions to learn behaviours that a human child acquires in minutes. The gap between human and machine sample efficiency is one of the oldest open problems in the field, and it remains one of the most consequential.

17.1.1 Model-Based RL and World Models

The most natural route to sample efficiency is to *learn a model of the environment* and plan inside it. If the agent can predict what will happen before it acts, every real interaction becomes far more informative. Early model-based methods learned transition dynamics in

tabular or linear form (Chapter 4 discusses the planning side); modern approaches learn *world models* — neural networks that predict future observations, rewards, and termination signals.

Ha and Schmidhuber's "World Models" paper (2018) demonstrated that an agent could learn a compressed latent representation of its observations and then train a policy entirely inside the learned model. Hafner et al. extended this line of work with the Dreamer family of algorithms (Dreamer, DreamerV2, DreamerV3), which learn a recurrent state-space model and optimise a policy by back-propagating through imagined trajectories in latent space. DreamerV3 achieved human-level performance on dozens of continuous and discrete control tasks — and even mined diamonds in Minecraft — using roughly 10–100× fewer environment steps than comparable model-free methods.

MuZero (Schrittwieser et al., 2020) took a different approach: rather than predicting raw observations, it learned a latent dynamics model whose hidden states were trained end-to-end to support planning via Monte Carlo tree search. MuZero matched the performance of AlphaZero in Go, chess, and shogi while also mastering Atari games, demonstrating that latent-space planning can rival or exceed model-free methods even in visually complex domains.

Despite these successes, model-based RL faces persistent challenges. Compounding model error — small inaccuracies in the learned dynamics that accumulate over long planning horizons — can produce policies that are optimal in the model but catastrophic in the real environment. Deciding *when* to trust the model and when to fall back on real data (the so-called Dyna trade-off; Sutton, 1991) remains largely heuristic.

17.1.2 Transfer and Few-Shot RL

A complementary route to sample efficiency is *transfer*: leveraging knowledge from previous tasks to accelerate learning on new ones. Meta-RL algorithms such as MAML (Finn et al., 2017) and RL^2 (Duan et al., 2016) learn an initialisation or an inner-loop learning algorithm that can adapt to a new task from a handful of episodes. More recently, foundation-model approaches treat RL trajectories as sequences and use in-context learning — the same mechanism that enables few-shot prompting in language models — to adapt to new environments at inference time without gradient updates (see Section 17.4 for the Decision Transformer and related methods).

The vision of a single, general-purpose RL agent that can transfer across diverse domains — an "RL foundation model" — remains largely unrealised. Gato (Reed et al., 2022) showed that a single transformer could play Atari, caption images, and control a robotic arm, but its per-task performance lagged behind specialist agents. Closing this gap is an active area of research.

17.2 Reward Specification and Alignment

Every RL algorithm optimises a reward signal. When that signal faithfully captures what we want, optimisation is beneficial; when it does not, the agent exploits the gap — a phenomenon known as *reward hacking*. The problem is an instance of Goodhart's law: "When a measure becomes a target, it ceases to be a good measure."

Chapter 10 introduced reward models trained from human comparisons, and Chapter 12 showed how DPO sidesteps explicit reward modelling. Both approaches mitigate reward hacking to some extent, but neither eliminates it. As models become more capable, the stakes of misspecified rewards grow.

17.2.1 Scalable Oversight

When the agent's capabilities exceed the evaluator's, human feedback becomes unreliable. A coding model may produce solutions that are correct but inscrutable; a scientific-reasoning model may propose experiments whose validity a non-expert cannot judge. The *scalable oversight* problem asks: how can we provide reliable training signal for systems whose outputs we cannot easily verify?

Several research programmes address this question. *Debate* (Irving et al., 2018) pits two models against each other, with a human judge deciding which argument is more convincing; the hope is that the truth is easier to verify than to generate. *Recursive reward modelling* (Leike et al., 2018) uses an AI assistant to help the human evaluate, creating a hierarchy of oversight. *Constitutional AI* (Bai et al., 2022) replaces some human feedback with a set of written principles that the model critiques itself against, reducing — though not eliminating — the burden on human evaluators.

17.2.2 The Fundamental Challenge

At its core, the alignment problem is one of *value specification*: translating the rich, context-dependent, and often contradictory landscape of human values into a formal objective. RL provides the optimisation machinery, but it cannot tell us *what* to optimise. This is not merely a technical problem; it is a philosophical one, touching on questions that ethicists and political theorists have debated for centuries.

The practical implication for RL researchers is that reward engineering will remain a first-class concern. Advances in interpretability, better methods for eliciting human preferences, and formal frameworks for specifying constraints will all play a role. No single technique is likely to "solve" alignment; the challenge is systemic, and progress will be incremental.

17.3 Safety and Robustness

An RL policy that performs well on average may fail catastrophically in rare but important situations. Safety and robustness are therefore not luxuries — they are prerequisites for deploying RL in the real world.

17.3.1 Safe Exploration

In domains such as autonomous driving, healthcare, and industrial control, the agent cannot be allowed to explore freely: certain states (collisions, overdoses, equipment damage) must be avoided even during training. *Constrained MDPs* formalise this by augmenting the standard MDP with cost functions and constraints of the form $\mathbb{E}_\pi[\sum_t c(s_t, a_t)] \leq d$. Lagrangian methods and constrained policy optimisation (CPO; Achiam et al., 2017) optimise the reward subject to these constraints, but guaranteeing constraint satisfaction during learning — not just at convergence — remains difficult.

17.3.2 Distributional Robustness and Adversarial Attacks

Policies trained in one environment may break when deployed in another that differs even slightly. *Distributional robustness* formulates the problem as a game between the agent and an adversary that perturbs the transition dynamics or observations within a bounded uncertainty set. Robust MDPs (Iyengar, 2005; Nilim and El Ghaoui, 2005) provide a

theoretical foundation, and recent work has scaled these ideas to deep RL via adversarial training and domain randomisation.

A related concern is *adversarial attacks on RL policies*: small, carefully crafted perturbations to the observation can cause a well-trained policy to take catastrophically bad actions (Huang et al., 2017). Certified defences analogous to those developed for image classifiers are still in their infancy for sequential decision-making.

17.3.3 Formal Verification

Can we *prove* that a learned policy satisfies a safety property? Formal verification techniques from software engineering — model checking, abstract interpretation, satisfiability modulo theories (SMT) — have been applied to small neural-network controllers, but scaling them to the networks used in modern deep RL remains an open problem. Bridging the gap between the statistical guarantees of learning theory and the deterministic guarantees of formal methods is one of the most important — and most difficult — challenges in the field.

17.4 Offline and Batch RL

In many practical settings — healthcare, recommendation systems, dialogue — online interaction is expensive or risky. *Offline RL* (also called *batch RL*) aims to learn a good policy from a fixed dataset of previously collected transitions, with no further environment interaction. The idea is appealing: organisations often possess large logs of past behaviour, and offline RL promises to extract value from this data.

17.4.1 The Distribution-Shift Problem

The central difficulty is *distribution shift*. The dataset was collected by some behaviour policy μ, but the learned policy π may visit states that μ rarely or never encountered. In those regions, the value estimates are unreliable, and naive off-policy methods can diverge or produce delusionally optimistic policies.

Conservative Q-Learning (CQL; Kumar et al., 2020) addresses this by adding a regularisation term that penalises the Q-function on out-of-distribution actions, effectively learning a lower bound on the true value. Implicit Q-Learning (IQL; Kostrikov et al., 2022) avoids querying the policy altogether during training, using expectile regression to estimate upper-envelope Q-values from the dataset. Both methods have shown strong performance on standard benchmarks, but their effectiveness depends heavily on the quality and coverage of the dataset.

17.4.2 Sequence Modelling and Decision Transformers

An influential alternative to value-function-based offline RL is to reframe the problem as *sequence modelling*. The Decision Transformer (Chen et al., 2021) conditions a transformer on the desired return, past states, and past actions, and autoregressively predicts future actions. At test time, the agent conditions on a high return and generates the corresponding behaviour. This approach sidesteps Bellman backups entirely, leveraging the sequence-modelling strengths of transformers.

Trajectory Transformer (Janner et al., 2021) extended the idea by modelling the joint distribution over states, actions, and rewards, enabling beam-search planning at inference

time. These methods blur the line between RL and supervised learning, and their theoretical properties — when they outperform, when they fail, and why — are still being understood.

17.4.3 Offline-to-Online

A promising hybrid is *offline-to-online* RL: pre-train a policy from a static dataset, then fine-tune with a small amount of online interaction. This mirrors the pre-train-then-fine-tune paradigm that has been so successful for language models (Chapter 11). Early results (Nair et al., 2020; Lee et al., 2022) show that offline pre-training can dramatically accelerate online learning, but preventing catastrophic forgetting of offline knowledge and managing the distribution shift during the transition remain open problems.

17.5 Multimodal and Embodied RL

Reinforcement learning began with physical intuitions — a robot navigating a maze, a thermostat regulating temperature — but for the past decade the most visible successes have been in simulation: games, language, and recommendation. Returning RL to the physical world is one of the field's great unfinished projects.

17.5.1 Sim-to-Real Transfer

Training in simulation is cheap and safe; deploying in the real world is neither. The *sim-to-real gap* arises because simulators inevitably simplify physics, sensor noise, and visual appearance. Domain randomisation (Tobin et al., 2017) mitigates the gap by training the policy across a distribution of simulated environments, so that the real world appears as just another sample. System identification, adaptive control, and learned residual models offer complementary approaches, but reliably closing the sim-to-real gap for contact-rich manipulation remains an active challenge.

17.5.2 Vision-Language-Action Models

A new class of *vision-language-action* (VLA) models aims to unify perception, language understanding, and motor control in a single architecture. RT-2 (Brohan et al., 2023) fine-tuned a vision-language model to output robotic actions as text tokens, demonstrating that semantic knowledge from web-scale pre-training could transfer to manipulation tasks. Octo (Ghosh et al., 2024) and π_0 (Black et al., 2024) pushed this further with transformer architectures that consume images and language instructions and emit continuous joint commands.

These models are still far from the dexterity and adaptability of human manipulation. Key bottlenecks include the scarcity of high-quality robotic demonstration data, the difficulty of specifying rewards for contact-rich tasks, and the computational cost of real-time inference on resource-constrained hardware.

17.5.3 Continuous Control at Scale

Beyond manipulation, RL is being applied to locomotion (legged robots trained via PPO in simulation and deployed on hardware), autonomous racing (Gran Turismo), drone navigation, and whole-body humanoid control. The common thread is that these tasks involve high-dimensional continuous action spaces, complex contact dynamics, and stringent real-time requirements. Scaling RL to these settings pushes both algorithmic and engineering frontiers.

17.6 RL and Foundation Models

Perhaps the most consequential development in recent RL is its emergence as the *universal post-training method* for foundation models. Chapters 11–15 documented this for text; the same pattern is now repeating across modalities.

17.6.1 RL for Language and Reasoning

RLHF (Chapter 14) and GRPO (Chapter 13) have become standard tools for aligning and improving large language models. As discussed in Chapter 15, RL with verifiable rewards has enabled dramatic improvements in mathematical reasoning and code generation. The frontier is moving toward longer-horizon reasoning tasks — multi-step scientific problem-solving, software engineering, theorem proving — where the reward is sparse and delayed, and the action space is the space of all possible text continuations.

17.6.2 RL for Visual Generation

The same RLHF loop that improved language models has been applied to image generation. DPOK (Fan et al., 2023) and DDPO (Black et al., 2024) used policy-gradient methods to fine-tune diffusion models against human-preference reward models, improving aesthetic quality and prompt adherence. For video generation, RL-based fine-tuning has been used to improve temporal coherence and visual fidelity, though the computational cost of generating and scoring video trajectories remains a bottleneck.

17.6.3 RL for Scientific Discovery

RL has contributed to notable scientific breakthroughs. AlphaFold's (Jumper et al., 2021) training pipeline incorporated RL-like structural refinement. GNoME (Merchant et al., 2023) used RL-guided search to discover millions of new stable crystal structures. FunSearch (Romera-Paredes et al., 2024) combined large language models with evolutionary search to discover new mathematical constructions, outperforming human mathematicians on the cap-set problem.

The common pattern is RL as the *outer loop*: a foundation model proposes candidates (protein structures, material compositions, mathematical conjectures), and an RL-like objective scores and selects among them. Extending this pattern to chemistry, drug discovery, climate modelling, and engineering design is an active and promising frontier.

17.7 Theoretical Frontiers

The theory of RL is rich in the tabular and linear-function-approximation settings (Chapters 3–6), but the methods that work best in practice — deep networks, transformers, high-dimensional policy spaces — often outrun available theory.

17.7.1 Regret and Sample Complexity for Deep RL

Regret bounds for tabular MDPs are tight: algorithms such as UCBVI achieve $\tilde{O}(\sqrt{H^3SAT})$ regret, where H is the horizon, S the number of states, A the number of actions, and T the number of episodes. For linear MDPs, matching upper and lower bounds are also known. But for the nonlinear function approximation used in deep RL — overparameterised neural networks trained with stochastic gradient descent — no comparable guarantees exist.

Neural tangent kernel (NTK) analyses provide some insight by showing that sufficiently wide networks behave approximately linearly, but this regime does not capture the representation learning that makes deep RL effective. Developing a theory that accounts for feature learning, non-convex optimisation, and the exploration–exploitation trade-off simultaneously is one of the hardest open problems in machine learning theory.

17.7.2 Convergence of Nonlinear Function Approximation

The *deadly triad* — the combination of bootstrapping, off-policy learning, and function approximation — was identified by Sutton and Barto as a source of divergence in approximate dynamic programming (Chapter 7). While algorithms such as DQN work well in practice, proving their convergence under realistic assumptions (finite-width networks, non-i.i.d. data, experience replay) remains elusive. Recent progress includes convergence results for overparameterised networks in the NTK regime and two-timescale analyses of actor-critic methods, but a unified theory is still missing.

17.7.3 Statistical Complexity of RLHF

The explosion of RLHF in language-model training (Chapter 12) has spurred new theoretical questions. How many human comparisons are needed to learn a reward model that faithfully represents human preferences? Under what conditions does the DPO objective recover the same policy as the RLHF pipeline? How does the quality of the preference data affect the statistical efficiency of alignment? Early results (Zhu et al., 2024; Xu et al., 2024) establish sample-complexity bounds under the Bradley–Terry preference model, but the theory does not yet account for the distributional and strategic complexities of real human feedback.

17.7.4 Why Does RL Work for LLMs?

Perhaps the deepest theoretical puzzle is why RL fine-tuning works as well as it does for large language models. The action space is enormous (the full vocabulary at each token position), the reward is sparse and delayed (often a single scalar for an entire response), and the policy is a billion-parameter network. Classical RL theory predicts that such problems should be intractable, yet PPO and GRPO produce reliable improvements in practice. Understanding this — likely through a combination of the structure of natural language, the implicit exploration provided by token sampling, and the warm start from pre-training — is a key open question.

17.8 The Road Ahead

Let us close by stepping back from individual problems and considering the broader trajectory of the field.

RL as universal fine-tuning. The most striking trend of the past several years is the generalisation of RL from a niche method for game-playing and robotics into a *universal fine-tuning method* for any model that generates structured output. Language models, image generators, video synthesizers, protein-structure predictors, and robotic controllers are all being improved by RL-based post-training. The underlying principle is simple: if you can score the output, you can use RL to make the model produce higher-scoring outputs. This

principle, first formalised in the policy-gradient theorem (Chapter 8), is proving to be one of the most powerful ideas in modern AI.

Convergence of classical theory and modern practice. For decades, RL theory and RL practice developed in largely separate communities. The theorists studied tabular MDPs and linear function approximation; the practitioners trained deep networks at scale. This gap is beginning to close. Offline RL theory informs practical algorithm design; PAC-Bayes bounds provide non-vacuous generalisation guarantees for learned policies; and the study of RLHF is bringing learning-theoretic tools to bear on one of the most consequential applications of RL. A unified theory that connects the elegant results of the tabular setting to the empirical successes of deep RL would be a landmark achievement.

Agents and autonomy. Chapter 15 and Chapter 16 described the emergence of RL-trained agents that can reason, use tools, and coordinate with other agents. This trend is accelerating. As language models become more capable and as RL fine-tuning enables longer-horizon planning, we may see agents that autonomously conduct scientific research, write and debug complex software, manage infrastructure, and negotiate on behalf of their users. The technical challenges are immense — credit assignment over thousands of steps, safe exploration in open-ended environments, robust multi-agent coordination — but so are the potential benefits.

The importance of safety. With greater autonomy comes greater risk. The safety and alignment challenges discussed in Sections 17.2 and 17.3 will only become more pressing as agents become more capable. The field needs not only better algorithms but also better institutions: evaluation standards, red-teaming practices, regulatory frameworks, and a culture of responsible development. RL researchers have a special responsibility here, because the optimisation pressure that RL exerts is among the strongest in all of machine learning.

What the next decade might bring. Predicting the future of a rapidly moving field is a fool's errand, but a few trends seem likely to persist. First, the scale of RL training will continue to grow, driven by the economics of post-training foundation models. Second, the boundary between RL, supervised learning, and self-supervised learning will continue to blur, as methods like DPO (Chapter 12) and Decision Transformer (Section 17.4) demonstrate. Third, RL will be applied to an ever-wider range of domains, from materials science to climate policy to education. Fourth, the safety and alignment challenges will force the community to develop new tools for evaluation, oversight, and governance.

The story of reinforcement learning is far from over. If anything, it is just reaching its most interesting chapter. The mathematical foundations laid in Parts I and II of this book, the algorithmic toolkit developed in Part III, and the frontier applications surveyed in Part IV all provide the vocabulary needed to contribute. The open problems described in this chapter are not obstacles to be lamented; they are opportunities to be seized. We hope this book has equipped you to seize them.

Appendix A

Mathematical Background

"The miracle of the appropriateness of the language of mathematics for the formulation of the laws of physics is a wonderful gift which we neither understand nor deserve."

—Eugene Wigner, 1960

This appendix collects the mathematical background assumed throughout the book. Readers with a solid undergraduate background in calculus, linear algebra, and probability can treat it as a reference to consult when a term is used without definition. Readers encountering these ideas for the first time will find enough detail to follow the main text, but are encouraged to supplement with a dedicated textbook for deeper fluency.

A.1 Linear Algebra Essentials

Vectors and Matrices

A **vector** $x \in \mathbb{R}^n$ is an ordered list of n real numbers. We write column vectors by default; $x^\top$ denotes the row transpose. The **inner product** of $x, y \in \mathbb{R}^n$ is

$$\langle x, y \rangle = x^\top y = \sum_{i=1}^{n} x_i y_i.$$

A **matrix** $A \in \mathbb{R}^{m \times n}$ maps $\mathbb{R}^n \to \mathbb{R}^m$ via $y = Ax$. The **transpose** $A^\top \in \mathbb{R}^{n \times m}$ satisfies $(A^\top)_{ij} = A_{ji}$.

Definition A.1 (Norms)**.** The ℓ^p norm of $x \in \mathbb{R}^n$ is

$$\|x\|_p = \Big(\sum_{i=1}^{n} |x_i|^p \Big)^{1/p}, \qquad p \geq 1.$$

Special cases: $\|x\|_1 = \sum_i |x_i|$; $\|x\|_2 = \sqrt{\sum_i x_i^2}$ (Euclidean norm, written $\|x\|$ when unambiguous); $\|x\|_\infty = \max_i |x_i|$. The **Frobenius norm** of a matrix is $\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}$.

Eigenvalues and Spectral Properties

For a square matrix $A \in \mathbb{R}^{n\times n}$, a scalar λ and non-zero vector v satisfying $Av = \lambda v$ are an **eigenvalue** and **eigenvector**. The collection of all eigenvalues is the **spectrum** of A.

Definition A.2 (Spectral Radius)**.** The spectral radius of A is $\rho(A) = \max_i |\lambda_i|$, where $\lambda_1, \ldots, \lambda_n$ are the eigenvalues of A.

A matrix A is **positive semi-definite** (PSD) if $x^\top Ax \geq 0$ for all x, which is equivalent to all eigenvalues being non-negative. PSD matrices arise naturally as covariance matrices and Hessians at local minima.

Example A.1 (Covariance matrix)**.** Let $X \in \mathbb{R}^{n\times d}$ be a data matrix with zero-mean columns. The empirical covariance $\Sigma = X^\top X/n$ is PSD. Its eigenvectors are the principal components used in PCA, and its eigenvalues measure the variance along each component.

Linear Systems and Invertibility

A square matrix A is **invertible** (non-singular) if and only if $\det(A) \neq 0$, equivalently if all its eigenvalues are non-zero. When A is invertible the unique solution to $Ax = b$ is $x = A^{-1}b$.

In reinforcement learning, systems of the form $(I - \gamma P^\pi)V = R$ arise from the Bellman equations (Chapter 4). Because $\rho(\gamma P^\pi) = \gamma < 1$, the matrix $I - \gamma P^\pi$ is invertible, guaranteeing a unique value function.

A.2 Calculus and Optimisation

Gradients and Hessians

Let $f : \mathbb{R}^n \to \mathbb{R}$ be differentiable. The **gradient** is the vector of partial derivatives:

$$\nabla_\theta f(\theta) = \left(\frac{\partial f}{\partial \theta_1}, \ldots, \frac{\partial f}{\partial \theta_n}\right)^\top \in \mathbb{R}^n.$$

The gradient points in the direction of steepest ascent. The **Hessian** is the matrix of second derivatives:

$$H(\theta) = \nabla^2 f(\theta), \qquad H_{ij} = \frac{\partial^2 f}{\partial \theta_i \partial \theta_j}.$$

If f is twice continuously differentiable then H is symmetric.

Definition A.3 (Critical Point)**.** θ^* is a **critical point** of f if $\nabla f(\theta^*) = 0$. It is a local minimum if $H(\theta^*)$ is PSD, and a local maximum if $-H(\theta^*)$ is PSD.

Chain Rule

If $g : \mathbb{R}^m \to \mathbb{R}^n$ and $f : \mathbb{R}^n \to \mathbb{R}$, the chain rule gives

$$\nabla_\theta (f \circ g)(\theta) = J_g(\theta)^\top \nabla_{g(\theta)} f,$$

where $J_g \in \mathbb{R}^{n\times m}$ is the Jacobian of g. Automatic differentiation in deep learning frameworks computes this efficiently via backpropagation.

Gradient Descent

Gradient descent iterates

$$\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t),$$

where $\alpha > 0$ is the **learning rate**. The update moves θ in the direction of steepest descent.

Stochastic gradient descent (SGD) replaces the true gradient with a noisy estimate $\hat{g}_t$ computed on a mini-batch:

$$\theta_{t+1} = \theta_t - \alpha \hat{g}_t, \qquad \mathbb{E}[\hat{g}_t] = \nabla f(\theta_t).$$

Variance in $\hat{g}_t$ acts as implicit regularisation and can help escape sharp local minima.

Adam (Adaptive Moment Estimation) maintains exponential moving averages of the gradient m_t and squared gradient v_t:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1)\hat{g}_t, \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2)\hat{g}_t^2, \\ \theta_{t+1} &= \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon}\,\hat{m}_t, \end{aligned}$$

where $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$ are bias-corrected estimates. Default values $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ work well across a wide range of tasks.

Convexity and Convergence

Definition A.4 (Convexity)**.** $f : \mathbb{R}^n \to \mathbb{R}$ is **convex** if for all x, y and $\lambda \in [0,1]$,

$$f(\lambda x + (1-\lambda)y) \le \lambda f(x) + (1-\lambda) f(y).$$

It is **strongly convex** with parameter $\mu > 0$ if $f(x) - \frac{\mu}{2}\|x\|^2$ is convex.

For a convex function with L-Lipschitz gradient (i.e. $\|\nabla f(x) - \nabla f(y)\| \le L\|x-y\|$), gradient descent with step size $\alpha \le 1/L$ converges to the global minimum. The convergence rate is $O(1/t)$ for convex functions and $O(e^{-\mu t/L})$ for strongly convex functions.

Example A.2 (Quadratic minimisation)**.** Let $f(\theta) = \frac{1}{2}\theta^\top A\theta - b^\top\theta$ with A PSD. Then $\nabla f(\theta) = A\theta - b$, so gradient descent updates $\theta_{t+1} = \theta_t - \alpha(A\theta_t - b)$. The unique minimum is $\theta^* = A^{-1}b$. With $\alpha < 2/\lambda_{\max}(A)$, the iterates converge geometrically.

A.3 Probability Theory

Random Variables and Distributions

A **random variable** X is a measurable function from a sample space Ω to $\mathbb{R}$. For a discrete X, the **probability mass function** (PMF) is $p(x) = \mathbb{P}(X = x)$. For a continuous X, the **probability density function** (PDF) p satisfies $\mathbb{P}(a \le X \le b) = \int_a^b p(x)\,dx$.

Definition A.5 (Expectation and Variance)**.**

$$\mathbb{E}[X] = \sum_x x\,p(x) \quad \text{(discrete)}, \qquad \mathbb{E}[X] = \int x\,p(x)\,dx \quad \text{(continuous)}.$$

$$\mathrm{Var}[X] = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

The **covariance** of X and Y is $\mathrm{Cov}[X,Y] = \mathbb{E}[(X - \mathbb{E}\,X)(Y - \mathbb{E}\,Y)]$.

Expectation is linear: $\mathbb{E}[aX + bY] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y]$ for any constants a, b and random variables X, Y (regardless of dependence). Variance is quadratic: $\mathrm{Var}[aX] = a^2\,\mathrm{Var}[X]$, and for independent X, Y, $\mathrm{Var}[X + Y] = \mathrm{Var}[X] + \mathrm{Var}[Y]$.

Conditional Probability and Bayes' Theorem

The **conditional probability** of event A given B is

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}, \quad \mathbb{P}(B) > 0.$$

Theorem A.1 (Bayes' Theorem)**.**

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(B \mid A)\,\mathbb{P}(A)}{\mathbb{P}(B)}.$$

In the continuous setting with prior $p(\theta)$ and likelihood $p(x|\theta)$:

$$p(\theta \mid x) = \frac{p(x \mid \theta)\,p(\theta)}{p(x)}.$$

Bayes' theorem underpins Bayesian exploration strategies (posterior sampling, Chapter 2) and reward modelling in RLHF (Chapter 14).

Conditional Expectation and the Tower Property

The **conditional expectation** $\mathbb{E}[X \mid Y]$ is itself a random variable — a function of Y. Its value at $Y = y$ is $\mathbb{E}[X \mid Y = y] = \int x\,p(x \mid y)\,dx$.

Theorem A.2 (Tower Property / Law of Total Expectation)**.**

$$\mathbb{E}[X] = \mathbb{E}\big[\mathbb{E}[X \mid Y]\big].$$

More generally, for σ-algebras $\mathcal{F}_1 \subseteq \mathcal{F}_2$, $\mathbb{E}[\mathbb{E}[X|\mathcal{F}_2]|\mathcal{F}_1] = \mathbb{E}[X|\mathcal{F}_1]$.

The tower property appears throughout RL whenever we marginalise over future trajectories or compute expected returns by conditioning on the current state.

Concentration Inequalities

Theorem A.3 (Law of Large Numbers)**.** *Let $X_1, X_2, \ldots$ be i.i.d. with mean μ. Then, as $n \to \infty$,*

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i \xrightarrow{a.s.} \mu.$$

The LLN justifies using sample averages to estimate expected returns in Monte Carlo methods (Chapter 5).

Theorem A.4 (Central Limit Theorem)**.** *Under the same conditions as Theorem A.3, with $\sigma^2 = \mathrm{Var}[X_i] < \infty$,*

$$\sqrt{n}\,(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2).$$

The CLT gives confidence intervals on Monte Carlo estimates and motivates the Gaussian noise model in continuous control.

Example A.3 (Sample return estimate)**.** Suppose we run n independent episodes and record returns $G^{(1)}, \ldots, G^{(n)}$. By the LLN, $\bar{G}_n \to V^\pi(s_0)$. By the CLT, a 95% confidence interval is approximately $\bar{G}_n \pm 1.96\,\hat{\sigma}/\sqrt{n}$, where $\hat{\sigma}^2$ is the sample variance of the returns.

A.4 Sampling Methods

Many quantities in RL — expected returns, policy gradients, KL penalties — cannot be computed in closed form but can be *estimated* from samples. This section reviews the core sampling strategies.

Monte Carlo Estimation

Given an expectation $\mu = \mathbb{E}_{X\sim p}[f(X)]$ that is hard to compute analytically, the **Monte Carlo estimate** is

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} f(x^{(i)}), \qquad x^{(i)} \sim p.$$

By the LLN, $\hat{\mu}_n \to \mu$. The variance of the estimator is $\mathrm{Var}[\hat{\mu}_n] = \mathrm{Var}[f(X)]/n$, so doubling the number of samples halves the standard error.

Monte Carlo methods in RL (Chapter 5) use full episode returns as unbiased (though high-variance) estimates of the value function.

Importance Sampling

When samples from p are unavailable but samples from a different **proposal distribution** q are, **importance sampling** corrects via the likelihood ratio:

$$\mu = \mathbb{E}_{X\sim p}[f(X)] = \mathbb{E}_{X\sim q}\left[\frac{p(X)}{q(X)}\, f(X)\right].$$

The ratio $w(x) = p(x)/q(x)$ is the **importance weight**. The estimator is unbiased when $q(x) > 0$ wherever $p(x)f(x) \neq 0$, but can have high (even infinite) variance if p and q differ greatly.

Importance sampling appears in off-policy TD learning and is central to the PPO objective (Chapter 9), where $q = \pi_{\theta_{\text{old}}}$ and $p = \pi_\theta$. Clipping the ratio $\rho_t = \pi_\theta/\pi_{\theta_{\text{old}}}$ controls variance.

Rejection Sampling

Rejection sampling draws from a target distribution $p(x) \propto \tilde{p}(x)$ using a proposal $q(x)$ such that $\tilde{p}(x) \leq Mq(x)$ for all x:

1. Sample $x \sim q$.
2. Accept x with probability $\tilde{p}(x)/(Mq(x))$; else reject.

The acceptance probability is $1/M$, so a tight proposal (small M) is crucial for efficiency. Rejection sampling is used in the analysis of offline preference models (Chapter 14) and in certain structured decoding schemes.

A.5 Information Theory

Information theory quantifies uncertainty and the cost of communication. In RL, its concepts appear in reward regularisation, policy optimisation objectives, and the analysis of exploration.

Definition A.6 (Entropy)**.** The **Shannon entropy** of a discrete distribution p over alphabet $\mathcal{V}$ is

$$H(p) = -\sum_{x \in \mathcal{V}} p(x) \log p(x),$$

measured in nats (base e) or bits (base 2). Entropy is maximised by the uniform distribution and equals zero for a point mass.

Entropy bonuses $\mathcal{H}(\pi(\cdot|s))$ encourage exploration by penalising policies that collapse to deterministic actions (Chapter 9).

Definition A.7 (Cross-Entropy)**.** The **cross-entropy** between distributions p and q is

$$H(p, q) = -\sum_{x} p(x) \log q(x).$$

It equals $H(p) + D_{\mathrm{KL}}(p \parallel q)$ and is the standard training objective for language model pre-training and supervised fine-tuning.

Definition A.8 (KL Divergence)**.** The **Kullback-Leibler divergence** from q to p is

$$D_{\mathrm{KL}}(p \parallel q) = \sum_{x} p(x) \log \frac{p(x)}{q(x)} \geq 0,$$

with equality if and only if $p = q$ almost everywhere. Note that $D_{\mathrm{KL}}(p \parallel q) \neq D_{\mathrm{KL}}(q \parallel p)$ in general; the KL is not a metric.

The KL penalty $\beta\, D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ in RLHF objectives (Chapter 14) and DPO (Chapter 12) penalises excessive deviation from the reference policy, preventing reward hacking.

Definition A.9 (Mutual Information)**.** The **mutual information** between random variables X and Y is

$$I(X; Y) = D_{\mathrm{KL}}(p(x, y) \parallel p(x)p(y)) = H(X) - H(X \mid Y) \geq 0.$$

It measures the reduction in uncertainty about X after observing Y.

Example A.4 (KL regularised RL)**.** The optimal policy for the KL-regularised objective $\max_\pi \mathbb{E}_\pi[R] - \beta\, D_{\mathrm{KL}}(\pi \parallel \pi_{\mathrm{ref}})$ has a closed form: $\pi^*(a|s) \propto \pi_{\mathrm{ref}}(a|s)\, e^{Q^*(s,a)/\beta}$, where Q^* is the soft Q-function. This connects to the DPO derivation in Chapter 12.

A.6 Operators and Contraction Mappings

The convergence proofs for dynamic programming and TD learning rely on a classical result from functional analysis — the Banach fixed-point theorem. This section develops the necessary background and shows how the Bellman operator fits the framework.

Metric Spaces and Contractions

Definition A.10 (Metric Space)**.** A **metric space** (X, d) is a set X equipped with a distance function $d : X \times X \to [0, \infty)$ satisfying: (i) $d(x, y) = 0 \Leftrightarrow x = y$; (ii) $d(x, y) = d(y, x)$; (iii) $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality). A metric space is **complete** if every Cauchy sequence converges in X.

Definition A.11 (Contraction Mapping)**.** An operator $T : X \to X$ on a metric space (X, d) is a **contraction** with modulus $\kappa \in [0, 1)$ if

$$d(Tx, Ty) \leq \kappa\, d(x, y) \quad \text{for all } x, y \in X.$$

Theorem A.5 (Banach Fixed-Point Theorem)**.** *If T is a contraction on a complete metric space (X, d), then T has a* unique *fixed point x^* (i.e. $Tx^* = x^*$), and for any starting point $x_0 \in X$ the iterates $x_{k+1} = Tx_k$ converge:*

$$d(x_k, x^*) \leq \frac{\kappa^k}{1-\kappa}\, d(x_1, x_0).$$

Convergence is geometric with rate κ.

The Bellman Operator as a Contraction

Let $\mathcal{B}(\mathcal{S})$ denote the space of bounded real-valued functions on $\mathcal{S}$, equipped with the sup-norm $\|V\|_\infty = \max_s |V(s)|$. This is a complete metric space.

Definition A.12 (Bellman Operators)**.** For a fixed policy π, the **Bellman evaluation operator** $\mathcal{T}^\pi$ maps $V : \mathcal{S} \to \mathbb{R}$ to

$$(\mathcal{T}^\pi V)(s) = \sum_a \pi(a|s) \Big[R(s, a) + \gamma \sum_{s'} P(s'|s, a)\, V(s') \Big].$$

The **Bellman optimality operator** $\mathcal{T}^*$ maps V to

$$(\mathcal{T}^* V)(s) = \max_a \Big[R(s, a) + \gamma \sum_{s'} P(s'|s, a)\, V(s') \Big].$$

Theorem A.6 (Bellman Contraction)**.** *Both $\mathcal{T}^\pi$ and $\mathcal{T}^*$ are contractions on $(\mathcal{B}(\mathcal{S}), \|\cdot\|_\infty)$ with modulus γ.*

Proof. We prove the result for $\mathcal{T}^*$; the proof for $\mathcal{T}^\pi$ is analogous. For any $V, W \in \mathcal{B}(\mathcal{S})$ and any state s,

$$\begin{aligned} |(\mathcal{T}^* V)(s) - (\mathcal{T}^* W)(s)| &= \Big| \max_a [R(s, a) + \gamma (PV)(s, a)] - \max_a [R(s, a) + \gamma (PW)(s, a)] \Big| \\ &\leq \max_a \gamma \Big| \sum_{s'} P(s'|s, a)[V(s') - W(s')] \Big| \\ &\leq \gamma \|V - W\|_\infty, \end{aligned}$$

where we used $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$ and the fact that $P(\cdot|s, a)$ is a probability distribution. Taking the supremum over s gives $\|\mathcal{T}^* V - \mathcal{T}^* W\|_\infty \leq \gamma \|V - W\|_\infty$. Since $\gamma < 1$, $\mathcal{T}^*$ is a contraction. □

Corollary A.7 (Existence and Uniqueness of Value Functions)**.** *For any MDP with $\gamma < 1$ (or finite horizon), the value functions V^π and V^* exist and are unique fixed points of $\mathcal{T}^\pi$ and $\mathcal{T}^*$ respectively. Starting from any bounded V_0, the iterates $V_{k+1} = \mathcal{T}^\pi V_k$ (policy evaluation) and $V_{k+1} = \mathcal{T}^* V_k$ (value iteration) converge geometrically:*

$$\|V_k - V^\pi\|_\infty \leq \gamma^k \|V_0 - V^\pi\|_\infty.$$

This result justifies the algorithms developed in Chapters 4 and 6: each step of policy evaluation or value iteration reduces the approximation error by a factor of γ.

Example A.5 (Value iteration on a two-state MDP)**.** Consider a two-state MDP with $\mathcal{S} = \{s_1, s_2\}$, $\gamma = 0.9$, and the optimal value $V^* = (10, 5)^\top$. Starting from $V_0 = (0, 0)^\top$, after k iterations the error satisfies $\|V_k - V^*\|_\infty \leq 0.9^k \cdot 10$. To reach error ≤ 0.01 we need $k \geq \log(0.001)/\log(0.9) \approx 66$ iterations — a precise convergence guarantee independent of the reward structure.

Non-Expansive Operators and Approximate DP

When function approximation is used (Chapter 7), the projected Bellman operator $\Pi\mathcal{T}^\pi$ (where Π is an orthogonal projection onto the approximation space) is no longer a contraction in general. The TD fixed-point theorem provides a weaker but still useful guarantee: under linear function approximation and certain on-policy sampling conditions, TD converges to a fixed point whose error is bounded by a multiple of the best achievable approximation error.

Afterword: The Future of Reinforcement Learning

If you have read this far, you have traced a path from a gambler at a slot machine to the algorithms that shape how frontier AI systems speak to human beings. That path turns out to be surprisingly straight. The principle of optimality that Richard Bellman formulated in the 1950s, the temporal-difference equations that Sutton and Barto developed in the 1980s, the policy gradient theorem proved at the turn of the millennium — these are not museum pieces. They are the live machinery running inside systems that hundreds of millions of people interact with every day. Seldom has the gap between mathematical abstraction and consequential engineering closed so quickly, and it is worth pausing, at the end of this journey, to reflect on where the field is heading.

Reinforcement learning at the centre of AI. For most of its history, reinforcement learning occupied an important but somewhat peripheral corner of the machine learning landscape. Supervised learning dominated in practice; RL was the subject of beautiful theory and impressive demonstrations — Atari, Go, robotic locomotion — but its impact on deployed systems remained limited. That has changed decisively.

Today, RL is central to at least four of the most active frontiers in AI research. In *language and reasoning*, the alignment of large language models through human feedback and the training of models that can reason step-by-step through extended chains of thought both depend on policy optimisation. In *agentic systems*, models that plan over multiple steps, use tools, browse the web, and write and execute code are fundamentally RL agents operating in structured environments with delayed rewards. In *multimodal AI*, the extension of RLHF-style training to vision, audio, and video is an active and rapidly moving area. In *robotics and embodied intelligence*, the sim-to-real pipeline that transfers policies trained in simulation to physical hardware remains one of the most promising routes to capable physical agents. Across all of these domains, the conceptual vocabulary is the one you now possess: states, actions, rewards, policies, value functions, and the algorithms that optimise them.

The convergence of two traditions. One of the most striking intellectual developments of the past decade has been the discovery that classical RL and the training of large language models are not merely analogous but deeply unified. The language-model fine-tuning problem is a Markov decision process: the state is the conversation history, the action is the next token, the policy is the language model, and the reward is a signal — human-provided or model-derived — about the quality of the response.

This realisation means that every result in this book applies, in principle, to language models. The bias-variance analysis of Monte Carlo versus temporal-difference methods re-emerges in debates about credit assignment in long reasoning chains. The policy gradient

theorem underlies REINFORCE-style reward optimisation. The actor-critic architecture is the direct ancestor of the PPO-based RLHF pipelines used in practice. Bellman's equations, written in the age of operations research, now govern the training of systems that compose poetry and prove theorems.

This convergence is not a coincidence. It reflects the universality of the sequential decision-making framework, and it suggests that further progress in language-model alignment will come, at least in part, from a deeper engagement with the theoretical tools that RL has accumulated over seventy years.

Open challenges. It would be dishonest to close on a note of pure triumph. Reinforcement learning, for all its recent successes, faces serious open problems, and intellectual honesty requires naming them.

Sample efficiency remains a fundamental limitation. Humans learn to play most games competently after a few hours of practice; the best RL agents require millions of environment interactions. Closing this gap — through better exploration, structured world models, or transfer learning — is one of the deepest open questions in the field.

Safety and robustness are increasingly urgent. Reward hacking, where an agent finds unintended ways to maximise a misspecified reward signal, is not merely a theoretical curiosity; it has been observed in deployed systems. Ensuring that RL agents behave reliably when their environment shifts, when their reward model is imperfect, or when adversaries are present is a problem of both theoretical and practical importance.

Multi-agent coordination introduces a new layer of complexity that single-agent theory does not address. Systems composed of many interacting agents — whether cooperating robots or competing market participants — exhibit emergent dynamics that remain poorly understood and difficult to control.

Scalable oversight is perhaps the most distinctively modern challenge. As AI systems become more capable, human evaluators become less able to reliably judge the quality of their outputs. How do we maintain meaningful human control over a system whose reasoning may already exceed the evaluator's ability to verify? Constitutional AI, debate, and recursive reward modelling are early attempts to answer this question, but a satisfying solution remains elusive.

Reward specification underlies all of the above. The problem of expressing human values, goals, and preferences as a reward function that can be optimised without adverse side effects is, at its core, a philosophical problem as much as a technical one. Progress here will require collaboration between the RL community, alignment researchers, ethicists, and domain experts.

A personal note. Writing a textbook is an act of optimism. It presupposes that the ideas contained within it are worth transmitting, that the reader's time is well spent engaging with them, and that the field will be better for having more people who understand it deeply. We hold all three of these beliefs sincerely.

Thank you for the time and attention you have invested in this material. Learning reinforcement learning is not trivial: the mathematics is demanding, the intuitions are sometimes counterintuitive, and the gap between theory and working implementation can be humbling. If you have made it through, you are better equipped than the vast majority of practitioners to reason carefully about the systems that will shape the coming decade.

We hope you will use that knowledge not merely to build more capable systems, but to build better ones — systems that are safe, interpretable, and genuinely beneficial. The field needs people who take both the capability and the responsibility seriously, and we believe you are now in a position to contribute to both.

A closing thought. In the preface, we described reinforcement learning as the study of how an agent learns to act well in a world it does not fully understand, guided by the sparse and sometimes misleading signal of reward. That description applies, with only slight modification, to the human condition itself. Perhaps that is why the framework resonates so deeply, and why the best researchers in the field are animated by something more than technical curiosity.

Reinforcement learning, at its most ambitious, is a theory of how intelligence can improve through experience. If we get it right — the mathematics, the engineering, and the values embedded in the reward signals we choose — it may prove to be one of the most important intellectual contributions of our era. The work is far from finished. We encourage you to pick up where this book leaves off.

Moscow, 2026